Velocity Vault Source Code

Updated build: v66

Includes gameplay, styling, WebGL, phone assets, PWA manifest, service worker, icon, Pages marker, and README/how-to-play notes.

Generated on May 15, 2026 from D:/laptop/create-a-top-of-the-line.

index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">
  <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'; style-src 'self'; img-src
 'self' data:; connect-src 'self'; manifest-src 'self'; worker-src 'self'; object-src 'none'; base-uri 'none'; form-
action 'none'; frame-ancestors 'none'; upgrade-insecure-requests">
  <meta name="referrer" content="no-referrer">
  <meta name="theme-color" content="#09100f">
  <meta name="mobile-web-app-capable" content="yes">
  <meta name="apple-mobile-web-app-capable" content="yes">
  <meta name="apple-mobile-web-app-title" content="Velocity Vault">
  <title>Velocity Vault</title>
  <link rel="manifest" href="manifest.webmanifest">
  <link rel="icon" href="app-icon.svg" type="image/svg+xml">
  <link rel="apple-touch-icon" href="app-icon.svg">
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <main class="shell">
    <section class="stage" aria-label="Velocity Vault race game">
      <canvas id="glCanvas" width="1280" height="720" aria-label="WebGL 3D racing renderer"></canvas>
      <canvas id="gameCanvas" width="1280" height="720" aria-label="Racing game canvas"></canvas>
      <div class="scanline" aria-hidden="true"></div>
      <div id="hud" class="hud">
        <div class="hud-left">
          <div class="brand-mark">Velocity Vault</div>
          <div class="chips">
            <span id="profileChip" class="chip">No Driver</span>
            <span id="modeChip" class="chip">Select Profile</span>
            <span id="missionChip" class="chip">Mission Ready</span>
            <span id="controllerChip" class="chip">Controller: Off</span>
          </div>
        </div>
        <div class="hud-stats" aria-live="polite">
          <div><span>Speed</span><strong id="speedStat">0</strong></div>
          <div><span>Focus</span><strong id="focusStat">100</strong></div>
          <div><span>Damage</span><strong id="damageStat">0%</strong></div>
          <div><span>Coins</span><strong id="coinStat">0</strong></div>
          <div><span id="repLabel">Rep</span><strong id="repStat">0</strong></div>
        </div>
      </div>
      <div id="toast" class="toast" role="status" aria-live="polite"></div>
      <div class="mobile-drive-controls" aria-label="Mobile driving controls">
        <button class="mobile-control control-size" data-control="size" type="button" aria-label="Switch compact
controls" aria-pressed="true">
          <span>Size</span>
          <strong id="touchSizeLabel">Full</strong>
        </button>
        <button class="mobile-control control-mode" data-control="mode" type="button" aria-label="Switch touch control
 mode" aria-pressed="false">
          <span>Controls</span>
          <strong id="touchModeLabel">Hold</strong>
        </button>
        <div class="drive-stick" aria-label="Compact drive control">
          <button class="mobile-control stick-up" data-control="gas" type="button" aria-
label="Accelerate">&uarr;</button>
          <button class="mobile-control stick-left" data-control="left" type="button" aria-label="Steer
left">&larr;</button>
          <button class="mobile-control stick-center" data-control="neutral" type="button" aria-label="Center
steering">&bull;</button>
          <button class="mobile-control stick-right" data-control="right" type="button" aria-label="Steer
right">&rarr;</button>
          <button class="mobile-control stick-down" data-control="brake" type="button" aria-label="Brake or
reverse">&darr;</button>
        </div>
        <div class="direction-pad" aria-label="Directional steering pad">
          <button class="mobile-control steer-left" data-control="left" type="button" aria-label="Steer
left">&larr;</button>
          <button class="mobile-control steer-neutral" data-control="neutral" type="button" aria-label="Center
steering">Center</button>
          <button class="mobile-control steer-right" data-control="right" type="button" aria-label="Steer
right">&rarr;</button>
        </div>
        <div class="pedal-pad" aria-label="Throttle and brake controls">
          <button class="mobile-control brake" data-control="brake" type="button" aria-label="Brake or
reverse">Brake</button>
          <button class="mobile-control gas" data-control="gas" type="button" aria-label="Accelerate">Gas</button>
          <button class="mobile-control boost" data-control="boost" type="button" aria-label="Boost">Boost</button>
        </div>
      </div>
```

```
    </section>

    <aside class="panel" aria-label="Game controls">
      <div id="loginView" class="view active">
        <h1>Driver Gate</h1>
        <p class="subtle">Profiles stay on this device. Choose a driver band, create a short callsign, and race.</p>
        <label class="field">
          <span>Driver callsign</span>
          <input id="driverName" type="text" maxlength="16" autocomplete="off" spellcheck="false"
placeholder="TurboAce">
        </label>
        <label class="field">
          <span>Local PIN</span>
          <input id="driverPin" type="password" inputmode="numeric" maxlength="6" autocomplete="off"
placeholder="Optional">
        </label>
        <div class="age-grid" role="radiogroup" aria-label="Age group">
          <button class="age-card selected" data-age="rookie" type="button">
            <strong>Rookie</strong>
            <span>5-8</span>
          </button>
          <button class="age-card" data-age="prodigy" type="button">
            <strong>Prodigy</strong>
            <span>9-12</span>
          </button>
          <button class="age-card" data-age="elite" type="button">
            <strong>Elite</strong>
            <span>13+</span>
          </button>
          <button class="age-card" data-age="family" type="button">
            <strong>Family</strong>
            <span>All</span>
          </button>
        </div>
        <button id="quickPlay" class="primary" type="button">Play</button>
        <button id="startProfile" class="ghost" type="button">Enter Garage</button>
        <button id="installApp" class="ghost" type="button">Install / Add to Phone</button>
        <button id="resetProfiles" class="ghost danger" type="button">Clear Local Profiles</button>
      </div>

      <div id="garageView" class="view">
        <div class="section-head">
          <div>
            <h2>Race Hub</h2>
            <p id="driverSummary" class="subtle"></p>
          </div>
          <button id="logoutBtn" class="icon-btn" type="button" aria-label="Switch driver" title="Switch
driver">Back</button>
        </div>

        <div class="tabs" role="tablist" aria-label="Garage tabs">
          <button class="tab active" data-tab="races" type="button">Races</button>
          <button class="tab" data-tab="vehicles" type="button">Vehicles</button>
          <button class="tab" data-tab="upgrades" type="button">Upgrades</button>
          <button class="tab" data-tab="missions" type="button">Missions</button>
          <button class="tab" data-tab="ai" type="button">AI</button>
        </div>

        <div id="racesTab" class="tab-panel active">
          <div id="raceList" class="race-list"></div>
          <div id="scenarioList" class="scenario-list" aria-label="Multiplayer scenarios"></div>
          <button id="launchRace" class="primary" type="button">Launch Race</button>
          <button id="resumeRace" class="ghost" type="button">Resume Saved Race</button>
        </div>

        <div id="upgradesTab" class="tab-panel">
          <div id="upgradeList" class="upgrade-list"></div>
        </div>

        <div id="vehiclesTab" class="tab-panel">
          <div id="vehicleList" class="vehicle-list"></div>
        </div>

        <div id="missionsTab" class="tab-panel">
          <div id="missionList" class="mission-list"></div>
        </div>

        <div id="aiTab" class="tab-panel">
          <div id="directorCard" class="director-card"></div>
          <button id="refreshDirector" class="ghost" type="button">Refresh AI Tuning</button>
        </div>
      </div>

      <div id="raceView" class="view race-controls">
```

```
        <div class="section-head">
          <div>
            <h2 id="raceTitle">Race Live</h2>
            <p id="raceBrief" class="subtle"></p>
          </div>
          <button id="pauseBtn" class="icon-btn" type="button" aria-label="Pause race" title="Pause
race">Pause</button>
        </div>
        <button id="saveRace" class="ghost save-race" type="button">Save Race</button>
        <button id="resetCar" class="ghost reset-car" type="button">Reset Car</button>
        <div class="control-grid">
          <button id="leftBtn" type="button" aria-label="Steer left">Left</button>
          <button id="gasBtn" type="button" aria-label="Accelerate">Gas</button>
          <button id="brakeBtn" type="button" aria-label="Brake or reverse">Brake / Reverse</button>
          <button id="boostBtn" type="button" aria-label="Boost">Boost</button>
          <button id="rightBtn" type="button" aria-label="Steer right">Right</button>
        </div>
        <div class="camera-grid" role="group" aria-label="Driver view">
          <button class="camera-btn active" data-camera="chase" type="button">Full Car</button>
          <button class="camera-btn" data-camera="hood" type="button">Half Car</button>
          <button class="camera-btn" data-camera="cockpit" type="button">Windshield</button>
        </div>
        <div class="renderer-grid" role="group" aria-label="Graphics renderer">
          <button class="renderer-btn active" data-renderer="webgl" type="button">WebGL 3D</button>
          <button class="renderer-btn" data-renderer="canvas" type="button">Classic 2D</button>
        </div>
        <div class="controller-card">
          <strong id="controllerName">Bluetooth controller ready</strong>
          <span>Pair a controller with your device, press any controller button, then steer with the left stick or
D-pad. RT / A accelerates, LT brakes or reverses, B boosts. Y resets the vehicle. Start pauses.</span>
        </div>
        <div class="progress-card distance-progress">
          <span>Distance</span>
          <div class="bar"><i id="distanceBar"></i></div>
        </div>
        <div class="progress-card mission-progress">
          <span>Mission</span>
          <div class="bar"><i id="missionBar"></i></div>
        </div>
        <div class="progress-card damage-progress">
          <span>Damage</span>
          <div class="bar damage-bar"><i id="damageBar"></i></div>
        </div>
        <button id="endRace" class="ghost" type="button">Return to Hub</button>
      </div>

      <div id="finishView" class="view">
        <h2 id="finishTitle">Race Complete</h2>
        <div id="finishStats" class="finish-stats"></div>
        <button id="nextRace" class="primary" type="button">Next Challenge</button>
        <button id="toGarage" class="ghost" type="button">Garage</button>
      </div>
    </aside>
  </main>
  <script src="phone-asset-pipeline.js" defer></script>
  <script src="webgl-pipeline.js" defer></script>
  <script src="game.js" defer></script>
</body>
</html>
```

styles.css

```
:root {
  color-scheme: dark;
  --bg: #09100f;
  --panel: #111817;
  --panel-2: #172220;
  --line: rgba(255, 255, 255, 0.14);
  --text: #f4fbf8;
  --muted: #a9bbb5;
  --lime: #bbf24a;
  --cyan: #46d9ff;
  --red: #ff5b6b;
  --gold: #ffd166;
  --road: #252927;
  --ink: #050807;
  --shadow: 0 24px 80px rgba(0, 0, 0, 0.42);
}

* {
  box-sizing: border-box;
}

html,
body {
  height: 100%;
}

html {
  min-height: 100%;
}

body {
  margin: 0;
  min-width: 320px;
  overflow: hidden;
  background:
    radial-gradient(circle at 18% 20%, rgba(70, 217, 255, 0.16), transparent 30%),
    linear-gradient(130deg, #070a09 0%, #0a1714 48%, #17140a 100%);
  color: var(--text);
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
}

button,
input {
  font: inherit;
}

button {
  border: 0;
  color: inherit;
  cursor: pointer;
}

.shell {
  display: grid;
  grid-template-columns: minmax(0, 1fr) 390px;
  gap: 18px;
  height: 100vh;
  padding: 18px;
}

.stage {
  position: relative;
  min-width: 0;
  overflow: hidden;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #050807;
  box-shadow: var(--shadow);
  touch-action: none;
}

.stage canvas {
  position: absolute;
  inset: 0;
  display: block;
  width: 100%;
  height: 100%;
}

#glCanvas {
  z-index: 1;
  opacity: 0;
```

```
}

#gameCanvas {
  z-index: 2;
  background: #050807;
}

.stage.webgl #glCanvas {
  opacity: 1;
}

.stage.webgl #gameCanvas {
  background: transparent;
}

.scanline {
  position: absolute;
  inset: 0;
  z-index: 3;
  pointer-events: none;
  background: linear-gradient(180deg, rgba(255,255,255,0.035), transparent 22%, transparent 78%, rgba(0,0,0,0.22));
  mix-blend-mode: soft-light;
}

.hud {
  position: absolute;
  z-index: 4;
  top: 16px;
  left: 16px;
  right: 16px;
  display: flex;
  justify-content: space-between;
  gap: 16px;
  pointer-events: none;
}

.brand-mark {
  width: fit-content;
  padding: 8px 10px;
  border: 1px solid rgba(187, 242, 74, 0.38);
  border-radius: 6px;
  background: rgba(5, 8, 7, 0.72);
  color: var(--lime);
  font-weight: 900;
  letter-spacing: 0;
  text-transform: uppercase;
}

.chips {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  margin-top: 10px;
}

.chip,
.hud-stats div,
.toast {
  border: 1px solid var(--line);
  border-radius: 6px;
  background: rgba(5, 8, 7, 0.72);
  backdrop-filter: blur(10px);
}

.chip {
  padding: 6px 9px;
  color: var(--muted);
  font-size: 12px;
  font-weight: 800;
}

.hud-stats {
  display: grid;
  grid-template-columns: repeat(5, minmax(70px, 1fr));
  gap: 8px;
}

.hud-stats div {
  padding: 8px 10px;
  text-align: right;
}

.hud-stats span {
  display: block;
```

```
  color: var(--muted);
  font-size: 11px;
  text-transform: uppercase;
}

.hud-stats strong {
  color: var(--text);
  font-size: 20px;
}

.toast {
  position: absolute;
  z-index: 5;
  left: 50%;
  bottom: 24px;
  max-width: min(520px, calc(100% - 48px));
  padding: 12px 14px;
  opacity: 0;
  transform: translateX(-50%) translateY(16px);
  transition: opacity 180ms ease, transform 180ms ease;
  color: var(--text);
  font-weight: 800;
}

.toast.show {
  opacity: 1;
  transform: translateX(-50%) translateY(0);
}

.mobile-drive-controls {
  display: none;
  touch-action: none;
}

.mobile-control {
  touch-action: none;
  user-select: none;
  -webkit-user-select: none;
  -webkit-touch-callout: none;
}

.drive-stick,
.direction-pad,
.pedal-pad {
  display: none;
  touch-action: none;
}

.panel {
  overflow: hidden auto;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: linear-gradient(180deg, rgba(17, 24, 23, 0.98), rgba(11, 16, 15, 0.98));
  box-shadow: var(--shadow);
}

.view {
  display: none;
  padding: 22px;
}

.view.active {
  display: block;
}

h1,
h2 {
  margin: 0;
  line-height: 1.05;
  letter-spacing: 0;
}

h1 {
  font-size: 36px;
}

h2 {
  font-size: 26px;
}

.subtle {
  margin: 8px 0 18px;
  color: var(--muted);
  line-height: 1.45;
```

```
}

.field {
  display: grid;
  gap: 7px;
  margin-bottom: 14px;
}

.field span {
  color: var(--muted);
  font-size: 13px;
  font-weight: 800;
}

input {
  width: 100%;
  min-height: 46px;
  border: 1px solid var(--line);
  border-radius: 6px;
  outline: none;
  background: #0b1110;
  color: var(--text);
  padding: 0 12px;
}

input:focus {
  border-color: var(--cyan);
  box-shadow: 0 0 0 3px rgba(70, 217, 255, 0.16);
}

.age-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 10px;
  margin: 18px 0;
}

.age-card,
.race-card,
.upgrade-card,
.mission-card,
.progress-card {
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel-2);
}

.age-card {
  min-height: 78px;
  padding: 12px;
  text-align: left;
}

.age-card strong,
.age-card span {
  display: block;
}

.age-card span {
  margin-top: 5px;
  color: var(--muted);
}

.age-card.selected {
  border-color: var(--lime);
  background: linear-gradient(135deg, rgba(187, 242, 74, 0.18), rgba(70, 217, 255, 0.08));
}

.primary,
.ghost {
  width: 100%;
  min-height: 48px;
  border-radius: 6px;
  font-weight: 900;
}

.primary {
  background: linear-gradient(135deg, var(--lime), var(--cyan));
  color: #06100e;
}

.ghost {
  margin-top: 10px;
  border: 1px solid var(--line);
```

```
  background: transparent;
  color: var(--text);
}

.ghost:disabled {
  cursor: not-allowed;
  opacity: 0.45;
}

.danger {
  color: #ffc5cc;
}

.section-head {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 14px;
}

.icon-btn {
  flex: 0 0 44px;
  width: 44px;
  height: 44px;
  border: 1px solid var(--line);
  border-radius: 6px;
  background: #0b1110;
  font-size: 18px;
  font-weight: 900;
}

.tabs {
  display: grid;
  grid-template-columns: repeat(5, 1fr);
  gap: 6px;
  margin: 14px 0 16px;
  padding: 4px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #0b1110;
}

.tab {
  min-height: 38px;
  border-radius: 6px;
  background: transparent;
  color: var(--muted);
  font-weight: 900;
}

.tab.active {
  background: var(--panel-2);
  color: var(--text);
}

.tab-panel {
  display: none;
}

.tab-panel.active {
  display: block;
}

.race-list,
.scenario-list,
.upgrade-list,
.vehicle-list,
.mission-list {
  display: grid;
  gap: 10px;
  margin-bottom: 14px;
}

.race-card,
.scenario-card,
.upgrade-card,
.vehicle-card,
.mission-card {
  padding: 13px;
}

.race-card,
.scenario-card,
```

```
.vehicle-card {
  cursor: pointer;
}

.race-card.selected,
.scenario-card.selected,
.vehicle-card.selected {
  border-color: var(--cyan);
  box-shadow: inset 0 0 0 1px rgba(70, 217, 255, 0.4);
}

.race-card:disabled,
.vehicle-card.locked {
  cursor: not-allowed;
  opacity: 0.58;
}

.scenario-list {
  margin-top: 2px;
}

.scenario-card {
  border: 1px solid var(--line);
  border-radius: 8px;
  background: linear-gradient(135deg, rgba(70, 217, 255, 0.08), rgba(187, 242, 74, 0.06));
  text-align: left;
}

.vehicle-card {
  display: grid;
  grid-template-columns: 76px 1fr;
  gap: 12px;
  width: 100%;
  text-align: left;
  align-items: center;
}

.vehicle-card .card-top,
.vehicle-card .tiny {
  grid-column: 2;
}

.vehicle-preview {
  grid-row: 1 / span 2;
  width: 74px;
  height: 52px;
  position: relative;
  border-radius: 7px;
  border: 1px solid rgba(244, 251, 248, 0.14);
  background:
    linear-gradient(135deg, rgba(255,255,255,0.1), rgba(5,8,7,0.34)),
    #101817;
  overflow: hidden;
}

.vehicle-preview-canvas {
  position: absolute;
  inset: 0;
  z-index: 1;
  display: block;
  width: 100%;
  height: 100%;
}

.vehicle-preview.asset-preview::before,
.vehicle-preview.asset-preview::after {
  display: none;
}

.vehicle-preview::before {
  content: "";
  position: absolute;
  left: 15px;
  right: 15px;
  top: 11px;
  bottom: 10px;
  border-radius: 10px 10px 5px 5px;
  background: linear-gradient(135deg, color-mix(in srgb, var(--vehicle-color), white 24%), var(--vehicle-color) 52%,
color-mix(in srgb, var(--vehicle-color), black 42%));
  box-shadow: 0 10px 18px rgba(0,0,0,0.36);
}

.vehicle-preview::after {
  content: "";
```

```
  position: absolute;
  left: 9px;
  right: 9px;
  bottom: 8px;
  height: 8px;
  border-radius: 99px;
  background: #050807;
}

.vehicle-preview[data-type="f1"]::before,
.vehicle-preview[data-type="prototype"]::before {
  clip-path: polygon(50% 0, 84% 22%, 72% 92%, 28% 92%, 16% 22%);
}

.vehicle-preview[data-type="truck"]::before,
.vehicle-preview[data-type="semi"]::before,
.vehicle-preview[data-type="tractor"]::before,
.vehicle-preview[data-type="monster"]::before,
.vehicle-preview[data-type="tank"]::before {
  left: 11px;
  right: 11px;
  border-radius: 5px;
}

.vehicle-preview[data-type="semi"]::before {
  left: 7px;
  right: 7px;
  top: 9px;
  bottom: 8px;
}

.vehicle-preview[data-type="tractor"]::before {
  left: 18px;
  right: 18px;
  top: 8px;
  bottom: 14px;
}

.vehicle-preview[data-type="tractor"]::after {
  left: 12px;
  right: 12px;
  height: 14px;
}

.vehicle-preview[data-type="monster"]::after {
  height: 13px;
}

.vehicle-preview[data-type="boat"]::before,
.vehicle-preview[data-type="snowmobile"]::before,
.vehicle-preview[data-type="airplane"]::before {
  clip-path: polygon(50% 0, 91% 70%, 70% 100%, 30% 100%, 9% 70%);
}

.vehicle-preview[data-type="helicopter"]::after {
  left: 12px;
  right: 12px;
  top: 8px;
  bottom: auto;
  height: 3px;
  background: rgba(244,251,248,0.75);
}

.card-top {
  display: flex;
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 8px;
}

.card-top strong {
  font-size: 16px;
}

.badge {
  align-self: flex-start;
  white-space: nowrap;
  border-radius: 999px;
  background: rgba(255, 209, 102, 0.18);
  color: var(--gold);
  padding: 4px 8px;
  font-size: 12px;
  font-weight: 900;
}
```

```
.tiny {
  color: var(--muted);
  font-size: 12px;
  line-height: 1.35;
}

.upgrade-action {
  display: grid;
  grid-template-columns: 1fr 96px;
  gap: 8px;
  align-items: center;
  margin-top: 10px;
}

.upgrade-action button {
  min-height: 36px;
  border-radius: 6px;
  background: var(--lime);
  color: var(--ink);
  font-weight: 900;
}

.upgrade-action button:disabled {
  cursor: not-allowed;
  opacity: 0.45;
}

.control-grid {
  display: grid;
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: 8px;
  margin: 18px 0;
}

.control-grid button,
.camera-grid button,
.renderer-grid button {
  min-height: 72px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel-2);
  font-size: 18px;
  font-weight: 900;
  touch-action: manipulation;
  user-select: none;
}

.camera-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 8px;
  margin-bottom: 10px;
}

.renderer-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 8px;
  margin-bottom: 10px;
}

.camera-grid button,
.renderer-grid button {
  min-height: 46px;
  color: var(--muted);
  font-size: 13px;
}

.camera-grid button.active,
.renderer-grid button.active {
  border-color: var(--cyan);
  color: var(--text);
  background: linear-gradient(135deg, rgba(70, 217, 255, 0.18), rgba(187, 242, 74, 0.1));
}

#boostBtn {
  background: linear-gradient(135deg, rgba(255, 209, 102, 0.96), rgba(255, 91, 107, 0.88));
  color: #160a05;
}

#gasBtn {
  background: linear-gradient(135deg, rgba(187, 242, 74, 0.92), rgba(70, 217, 255, 0.74));
  color: #06100e;
```

```
}

#brakeBtn {
  background: linear-gradient(135deg, rgba(244, 251, 248, 0.14), rgba(255, 91, 107, 0.28));
}

.control-grid button.pressed {
  transform: translateY(1px);
  filter: brightness(1.18);
  box-shadow: inset 0 0 0 2px rgba(244, 251, 248, 0.22);
}

.progress-card {
  padding: 12px;
  margin-bottom: 10px;
}

.controller-card,
.director-card {
  border: 1px solid var(--line);
  border-radius: 8px;
  background: #0b1110;
  padding: 12px;
  margin-bottom: 10px;
}

.controller-card strong,
.controller-card span,
.director-card strong,
.director-card span {
  display: block;
}

.controller-card strong,
.director-card strong {
  margin-bottom: 6px;
  color: var(--cyan);
}

.controller-card span,
.director-card span {
  color: var(--muted);
  font-size: 12px;
  line-height: 1.42;
}

.director-card ul {
  margin: 10px 0 0;
  padding-left: 18px;
  color: var(--muted);
  font-size: 12px;
  line-height: 1.45;
}

.director-card li + li {
  margin-top: 6px;
}

.progress-card span {
  display: block;
  margin-bottom: 8px;
  color: var(--muted);
  font-weight: 800;
}

.bar {
  height: 12px;
  overflow: hidden;
  border-radius: 999px;
  background: #080d0c;
}

.bar i {
  display: block;
  width: 0%;
  height: 100%;
  border-radius: inherit;
  background: linear-gradient(90deg, var(--lime), var(--cyan));
}

.damage-bar i {
  background: linear-gradient(90deg, var(--lime), var(--amber) 48%, var(--red));
}
```

```
.finish-stats {
  display: grid;
  gap: 10px;
  margin: 18px 0;
}

.finish-stats div {
  display: flex;
  justify-content: space-between;
  border-bottom: 1px solid var(--line);
  padding-bottom: 9px;
}

@media (min-width: 981px) {
  body.race-live .shell {
    grid-template-columns: minmax(0, 1fr) clamp(280px, 28vw, 360px);
    gap: 12px;
    padding: 12px;
  }

  body.race-live .panel {
    min-width: 0;
    max-height: calc(100dvh - 24px);
  }

  body.race-live .view {
    padding: clamp(12px, 1.4vw, 18px);
  }

  body.race-live .section-head {
    gap: 8px;
    margin-bottom: 10px;
  }

  body.race-live .section-head h2 {
    font-size: clamp(18px, 1.9vw, 24px);
  }

  body.race-live .subtle {
    margin-bottom: 10px;
    font-size: 12px;
  }

  body.race-live .control-grid {
    grid-template-columns: repeat(5, minmax(42px, 1fr));
    gap: 6px;
    margin: 10px 0;
  }

  body.race-live .control-grid button {
    min-height: clamp(46px, 8vh, 64px);
    padding: 0 4px;
    overflow: hidden;
    font-size: clamp(12px, 1.2vw, 16px);
    text-overflow: ellipsis;
  }

  body.race-live .camera-grid,
  body.race-live .renderer-grid {
    gap: 6px;
  }

  body.race-live .camera-grid button,
  body.race-live .renderer-grid button {
    min-height: 38px;
    padding: 0 4px;
    overflow: hidden;
    text-overflow: ellipsis;
  }

  body.race-live #pauseBtn,
  body.race-live #saveRace,
  body.race-live #resetCar,
  body.race-live #endRace {
    min-height: 42px;
  }

  body.race-live .mobile-drive-controls,
  body.race-live .drive-stick,
  body.race-live .direction-pad,
  body.race-live .pedal-pad,
  body.race-live .mobile-control.control-mode,
  body.race-live .mobile-control.control-size {
    display: none !important;
```

```
    pointer-events: none;
  }
}

@media (max-width: 980px) {
  body {
    overflow: auto;
  }

  .shell {
    grid-template-columns: 1fr;
    height: auto;
    min-height: 100vh;
  }

  .stage {
    min-height: 58vh;
  }

  .hud {
    position: absolute;
    display: grid;
  }

  .hud-stats {
    grid-template-columns: repeat(2, minmax(82px, 1fr));
  }
}

@media (max-width: 560px) {
  .shell {
    padding: 10px;
    gap: 10px;
  }

  .stage {
    min-height: 50vh;
  }

  .hud {
    inset: 10px 10px auto;
  }

  .brand-mark,
  .chip {
    font-size: 11px;
  }

  .hud-stats strong {
    font-size: 17px;
  }

  .view {
    padding: 16px;
  }
}

@media (max-width: 980px), (pointer: coarse) {
  body.race-live {
    position: fixed;
    inset: 0;
    width: var(--vvw, 100dvw);
    height: var(--vvh, 100dvh);
    overflow: hidden;
    overscroll-behavior: none;
  }

  body.race-live .shell {
    position: fixed;
    inset: 0;
    display: block;
    width: var(--vvw, 100dvw);
    height: var(--vvh, 100dvh);
    min-height: 0;
    padding: 0;
    overflow: hidden;
  }

  body.race-live .stage {
    width: 100%;
    height: 100%;
    min-height: 0;
    border: 0;
    border-radius: 0;
```

```
  touch-action: none;
}

body.race-live .panel {
  position: fixed;
  inset: 0;
  z-index: 10;
  overflow: visible;
  pointer-events: none;
  border: 0;
  border-radius: 0;
  background: transparent;
  box-shadow: none;
}

body.race-live #raceView {
  display: block;
  padding: 0;
  pointer-events: none;
}

body.race-live .control-grid,
body.race-live .controller-card,
body.race-live .progress-card,
body.race-live .section-head div {
  display: none;
}

body.race-live #pauseBtn,
body.race-live #saveRace,
body.race-live #resetCar,
body.race-live #endRace,
body.race-live .mobile-control {
  pointer-events: auto;
  backdrop-filter: blur(10px);
}

body.race-live #pauseBtn,
body.race-live #saveRace,
body.race-live #resetCar,
body.race-live #endRace {
  position: fixed;
  right: max(8px, env(safe-area-inset-right));
  z-index: 32;
  width: 76px;
  min-height: 34px;
  margin: 0;
  font-size: 11px;
  background: rgba(5, 8, 7, 0.68);
}

body.race-live #pauseBtn {
  top: max(8px, env(safe-area-inset-top));
}

body.race-live #saveRace {
  top: calc(max(8px, env(safe-area-inset-top)) + 40px);
}

body.race-live #resetCar {
  top: calc(max(8px, env(safe-area-inset-top)) + 80px);
  background: rgba(255, 91, 107, 0.34);
}

body.race-live #endRace {
  top: calc(max(8px, env(safe-area-inset-top)) + 120px);
}

body.race-live .mobile-drive-controls {
  position: fixed;
  inset: 0;
  z-index: 30;
  display: block;
  width: var(--vvw, 100dvw);
  height: var(--vvh, 100dvh);
  pointer-events: auto;
}

body.race-live .direction-pad,
body.race-live .pedal-pad {
  position: fixed;
  z-index: 31;
  display: grid;
  pointer-events: auto;
```

```
}

body.race-live .direction-pad {
  left: max(8px, env(safe-area-inset-left));
  bottom: max(12px, env(safe-area-inset-bottom));
  grid-template-columns: repeat(3, clamp(42px, 12vw, 62px));
  gap: 6px;
}

body.race-live .pedal-pad {
  right: max(8px, env(safe-area-inset-right));
  bottom: max(12px, env(safe-area-inset-bottom));
  grid-template-columns: clamp(54px, 15vw, 76px) clamp(62px, 17vw, 88px);
  grid-template-rows: clamp(42px, 11vw, 56px) clamp(50px, 13vw, 68px);
  gap: 6px;
}

body.race-live .mobile-control {
  min-width: 0;
  min-height: 0;
  border: 1px solid rgba(244, 251, 248, 0.28);
  border-radius: 13px;
  background:
    linear-gradient(145deg, rgba(255,255,255,0.14), rgba(5,8,7,0.36)),
    rgba(5, 8, 7, 0.58);
  color: #f4fbf8;
  font-size: clamp(10px, 3vw, 13px);
  font-weight: 900;
  opacity: 0.94;
  box-shadow: 0 10px 30px rgba(0,0,0,0.26);
}

body.race-live .mobile-control.steer-left,
body.race-live .mobile-control.steer-neutral,
body.race-live .mobile-control.steer-right {
  width: 100%;
  height: clamp(42px, 12vw, 62px);
}

body.race-live .mobile-control.gas {
  grid-column: 2;
  grid-row: 1 / span 2;
  background: linear-gradient(135deg, rgba(187, 242, 74, 0.74), rgba(70, 217, 255, 0.6));
  color: #06100e;
}

body.race-live .mobile-control.brake {
  grid-column: 1;
  grid-row: 2;
}

body.race-live .mobile-control.boost {
  grid-column: 1;
  grid-row: 1;
  background: linear-gradient(135deg, rgba(255, 209, 102, 0.78), rgba(255, 91, 107, 0.72));
  color: #160a05;
}

body.race-live .mobile-control.control-mode {
  position: fixed;
  left: 50%;
  bottom: max(12px, env(safe-area-inset-bottom));
  z-index: 31;
  width: clamp(74px, 18vw, 112px);
  height: clamp(32px, 8vw, 44px);
  transform: translateX(-50%);
  pointer-events: auto;
  background: rgba(5, 8, 7, 0.62);
}

body.race-live .mobile-control.control-mode span,
body.race-live .mobile-control.control-mode strong {
  display: block;
  line-height: 1.05;
}

body.race-live .mobile-control.control-mode span {
  color: rgba(244, 251, 248, 0.72);
  font-size: 9px;
  text-transform: uppercase;
}

body.race-live .mobile-control.control-mode strong {
  font-size: 12px;
```

```
  }

  body.race-live.touch-toggle .mobile-control.control-mode {
    border-color: rgba(187, 242, 74, 0.72);
    background: linear-gradient(135deg, rgba(187, 242, 74, 0.72), rgba(70, 217, 255, 0.5));
    color: #06100e;
  }
}

@media (orientation: portrait) and (max-width: 980px) {
  body.race-live .hud {
    top: max(8px, env(safe-area-inset-top));
    left: 8px;
    right: 90px;
    display: block;
  }

  body.race-live .brand-mark,
  body.race-live .chips {
    display: none;
  }

  body.race-live .hud-stats {
    grid-template-columns: repeat(3, minmax(48px, 1fr));
    gap: 5px;
  }

  body.race-live .hud-stats div:nth-child(n+4),
  body.race-live #saveRace,
  body.race-live #endRace,
  body.race-live .camera-grid,
  body.race-live .renderer-grid {
    display: none;
  }

  body.race-live #resetCar {
    top: calc(max(8px, env(safe-area-inset-top)) + 42px);
  }

  body.race-live .direction-pad {
    bottom: calc(max(12px, env(safe-area-inset-bottom)) + 2px);
    grid-template-columns: repeat(3, clamp(40px, 13vw, 54px));
  }

  body.race-live .pedal-pad {
    bottom: calc(max(12px, env(safe-area-inset-bottom)) + 2px);
    grid-template-columns: clamp(50px, 14vw, 62px) clamp(58px, 16vw, 74px);
    grid-template-rows: clamp(40px, 11vw, 52px) clamp(46px, 13vw, 60px);
  }

  body.race-live .mobile-control.steer-left,
  body.race-live .mobile-control.steer-neutral,
  body.race-live .mobile-control.steer-right {
    height: clamp(40px, 13vw, 54px);
  }

  body.race-live .mobile-control.control-mode {
    bottom: calc(max(12px, env(safe-area-inset-bottom)) + 68px);
  }
}

@media (orientation: landscape) and (max-width: 980px) and (max-height: 760px), (orientation: landscape) and (pointer:
 coarse) {
  body.race-live {
    overflow: hidden;
  }

  body:not(.race-live) {
    overflow: auto;
  }

  body:not(.race-live) .stage {
    touch-action: pan-y;
  }

  body:not(.race-live) .shell {
    height: auto;
    min-height: 100dvh;
  }

  body.race-live .shell {
    position: fixed;
    inset: 0;
    display: block;
```

```
  width: var(--vvw, 100dvw);
  height: var(--vvh, 100dvh);
  min-height: 0;
  padding: 0;
  overflow: hidden;
}

body.race-live .stage {
  width: 100%;
  height: 100%;
  min-height: 0;
  border-radius: 0;
  border: 0;
}

body.race-live .hud {
  top: 6px;
  left: 8px;
  right: 8px;
  display: flex;
  align-items: flex-start;
  gap: 8px;
}

body.race-live .brand-mark {
  padding: 6px 8px;
  font-size: 12px;
}

body.race-live .chips {
  display: none;
}

body.race-live .hud-stats {
  grid-template-columns: repeat(5, minmax(44px, 54px));
  gap: 5px;
  margin-left: auto;
  margin-right: 78px;
}

body.race-live .hud-stats div {
  padding: 5px 6px;
}

body.race-live .hud-stats span {
  font-size: 8px;
}

body.race-live .hud-stats strong {
  font-size: 14px;
}

body.race-live .panel {
  position: fixed;
  inset: 0;
  z-index: 10;
  overflow: visible;
  pointer-events: none;
  border: 0;
  border-radius: 0;
  background: transparent;
  box-shadow: none;
}

body.race-live #raceView {
  display: block;
  padding: 0;
  pointer-events: none;
}

body.race-live .section-head {
  position: fixed;
  top: 8px;
  right: 8px;
  z-index: 6;
  display: block;
  margin: 0;
  pointer-events: none;
}

body.race-live .section-head div {
  display: none;
}
```

```
body.race-live #pauseBtn,
body.race-live #saveRace,
body.race-live #resetCar,
body.race-live #endRace,
body.race-live .camera-grid button,
body.race-live .renderer-grid button,
body.race-live .mobile-control {
  pointer-events: auto;
  backdrop-filter: blur(10px);
}

body.race-live #pauseBtn {
  position: fixed;
  top: 8px;
  right: 8px;
  z-index: 12;
  width: 68px;
  height: 38px;
  font-size: 12px;
  background: rgba(5, 8, 7, 0.72);
}

body.race-live #saveRace {
  position: fixed;
  top: 52px;
  right: 8px;
  z-index: 12;
  width: 88px;
  min-height: 36px;
  margin: 0;
  font-size: 12px;
  background: rgba(5, 8, 7, 0.58);
}

body.race-live #endRace {
  position: fixed;
  top: 140px;
  right: 8px;
  z-index: 12;
  width: 88px;
  min-height: 36px;
  margin: 0;
  font-size: 12px;
  background: rgba(5, 8, 7, 0.58);
}

body.race-live #resetCar {
  position: fixed;
  top: 96px;
  right: 8px;
  z-index: 12;
  width: 88px;
  min-height: 36px;
  margin: 0;
  font-size: 12px;
  background: rgba(255, 91, 107, 0.28);
}

body.race-live .control-grid {
  display: none;
}

body.race-live .mobile-drive-controls {
  position: absolute;
  inset: 0;
  z-index: 8;
  display: block;
  pointer-events: auto;
}

body.race-live .direction-pad,
body.race-live .pedal-pad {
  position: absolute;
  z-index: 9;
  display: grid;
  pointer-events: auto;
}

body.race-live .direction-pad {
  left: max(10px, env(safe-area-inset-left));
  bottom: max(12px, env(safe-area-inset-bottom));
  grid-template-columns: repeat(3, clamp(44px, 11dvh, 66px));
  gap: 6px;
}
```

```
body.race-live .pedal-pad {
  right: max(10px, env(safe-area-inset-right));
  bottom: max(12px, env(safe-area-inset-bottom));
  grid-template-columns: clamp(58px, 14dvh, 86px) clamp(66px, 16dvh, 100px);
  grid-template-rows: clamp(42px, 9dvh, 58px) clamp(48px, 11dvh, 68px);
  gap: 6px;
}

body.race-live .mobile-control {
  min-width: 0;
  min-height: 0;
  border: 1px solid rgba(244, 251, 248, 0.26);
  border-radius: 14px;
  background:
    linear-gradient(145deg, rgba(255,255,255,0.12), rgba(5,8,7,0.34)),
    rgba(5, 8, 7, 0.54);
  color: #f4fbf8;
  font-size: clamp(10px, 2.8dvh, 13px);
  font-weight: 900;
  opacity: 0.92;
  box-shadow: 0 10px 30px rgba(0,0,0,0.24);
}

body.race-live .mobile-control.pressed {
  filter: brightness(1.25);
  box-shadow: inset 0 0 0 2px rgba(244, 251, 248, 0.28), 0 10px 30px rgba(0,0,0,0.24);
}

body.race-live .mobile-control.steer-left,
body.race-live .mobile-control.steer-neutral,
body.race-live .mobile-control.steer-right {
  width: 100%;
  height: clamp(44px, 11dvh, 66px);
}

body.race-live .mobile-control.gas {
  grid-column: 2;
  grid-row: 1 / span 2;
  background: linear-gradient(135deg, rgba(187, 242, 74, 0.72), rgba(70, 217, 255, 0.58));
  color: #06100e;
}

body.race-live .mobile-control.brake {
  grid-column: 1;
  grid-row: 2;
}

body.race-live .mobile-control.boost {
  grid-column: 1;
  grid-row: 1;
  background: linear-gradient(135deg, rgba(255, 209, 102, 0.78), rgba(255, 91, 107, 0.72));
  color: #160a05;
}

body.race-live .mobile-control.control-mode {
  position: absolute;
  left: 50%;
  bottom: max(10px, env(safe-area-inset-bottom));
  z-index: 9;
  width: clamp(70px, 17dvh, 104px);
  height: clamp(32px, 7dvh, 42px);
  transform: translateX(-50%);
  pointer-events: auto;
  background: rgba(5, 8, 7, 0.58);
}

body.race-live .mobile-control.control-mode span,
body.race-live .mobile-control.control-mode strong {
  display: block;
  line-height: 1.05;
}

body.race-live .mobile-control.control-mode span {
  color: rgba(244, 251, 248, 0.72);
  font-size: 9px;
  text-transform: uppercase;
}

body.race-live .mobile-control.control-mode strong {
  font-size: 12px;
}

body.race-live.touch-toggle .mobile-control.control-mode {
```

```
    border-color: rgba(187, 242, 74, 0.72);
    background: linear-gradient(135deg, rgba(187, 242, 74, 0.72), rgba(70, 217, 255, 0.5));
    color: #06100e;
  }

  body.race-live .camera-grid {
    position: fixed;
    left: 50%;
    top: 8px;
    bottom: auto;
    z-index: 4;
    width: min(280px, 38vw);
    transform: translateX(-50%);
    gap: 5px;
    margin: 0;
    pointer-events: none;
  }

  body.race-live .camera-grid button {
    min-height: 32px;
    padding: 0 5px;
    overflow: hidden;
    background: rgba(5, 8, 7, 0.48);
    font-size: 10px;
    text-overflow: ellipsis;
  }

  body.race-live .renderer-grid {
    position: fixed;
    left: 50%;
    top: 46px;
    bottom: auto;
    z-index: 4;
    width: min(210px, 30vw);
    transform: translateX(-50%);
    gap: 5px;
    margin: 0;
    pointer-events: none;
  }

  body.race-live .renderer-grid button {
    min-height: 30px;
    padding: 0 5px;
    overflow: hidden;
    background: rgba(5, 8, 7, 0.48);
    font-size: 10px;
    text-overflow: ellipsis;
  }

  body.race-live .controller-card {
    display: none;
  }

  body.race-live .progress-card {
    display: none;
  }
}

@media (orientation: landscape) and (max-width: 980px) and (max-height: 430px) {
  body.race-live .hud-stats {
    grid-template-columns: repeat(3, 46px);
    margin-right: 64px;
  }

  body.race-live .brand-mark,
  body.race-live .hud-stats div:nth-child(n+4) {
    display: none;
  }

  body.race-live #saveRace,
  body.race-live #endRace {
    display: none;
  }

  body.race-live #pauseBtn,
  body.race-live #resetCar {
    width: 62px;
    min-height: 32px;
    height: 32px;
    font-size: 11px;
  }

  body.race-live #resetCar {
    top: 46px;
```

```
  }

  body.race-live .camera-grid,
  body.race-live .renderer-grid {
    display: none;
  }

  body.race-live .direction-pad {
    grid-template-columns: repeat(3, clamp(40px, 11dvh, 52px));
  }

  body.race-live .pedal-pad {
    grid-template-columns: clamp(52px, 14dvh, 68px) clamp(58px, 16dvh, 76px);
    grid-template-rows: clamp(38px, 9dvh, 48px) clamp(44px, 11dvh, 56px);
  }

  body.race-live .mobile-control.steer-left,
  body.race-live .mobile-control.steer-neutral,
  body.race-live .mobile-control.steer-right {
    height: clamp(40px, 11dvh, 52px);
  }
}

@media (max-width: 980px), (pointer: coarse) {
  body.race-live .control-size {
    position: fixed;
    left: 50%;
    bottom: max(12px, env(safe-area-inset-bottom));
    z-index: 34;
    width: 58px;
    height: 32px;
    transform: translateX(calc(-50% - 72px));
    pointer-events: auto;
    border-radius: 999px;
    background: rgba(5, 8, 7, 0.72);
  }

  body.race-live .control-size span,
  body.race-live .control-size strong {
    display: block;
    line-height: 1;
  }

  body.race-live .control-size span {
    color: rgba(244, 251, 248, 0.68);
    font-size: 8px;
    text-transform: uppercase;
  }

  body.race-live .control-size strong {
    font-size: 11px;
  }

  body.race-live.control-mini .control-mode {
    display: none;
  }

  body.race-live.control-mini .drive-stick {
    position: fixed;
    left: max(10px, env(safe-area-inset-left));
    bottom: max(10px, env(safe-area-inset-bottom));
    z-index: 34;
    display: grid;
    grid-template-columns: repeat(3, 42px);
    grid-template-rows: repeat(3, 42px);
    gap: 4px;
    width: 138px;
    height: 138px;
    padding: 4px;
    pointer-events: auto;
    border: 1px solid rgba(244, 251, 248, 0.22);
    border-radius: 999px;
    background:
      radial-gradient(circle at 50% 50%, rgba(244, 251, 248, 0.12), rgba(5, 8, 7, 0.52) 66%),
      rgba(5, 8, 7, 0.34);
    box-shadow: 0 14px 34px rgba(0, 0, 0, 0.34);
    backdrop-filter: blur(10px);
    -webkit-backdrop-filter: blur(10px);
  }

  body.race-live.control-mini .drive-stick .mobile-control {
    width: 42px;
    height: 42px;
    border-radius: 999px;
```

```
  font-size: 18px;
  line-height: 1;
}

body.race-live.control-mini .stick-up {
  grid-column: 2;
  grid-row: 1;
}

body.race-live.control-mini .stick-left {
  grid-column: 1;
  grid-row: 2;
}

body.race-live.control-mini .stick-center {
  grid-column: 2;
  grid-row: 2;
  opacity: 0.58;
}

body.race-live.control-mini .stick-right {
  grid-column: 3;
  grid-row: 2;
}

body.race-live.control-mini .stick-down {
  grid-column: 2;
  grid-row: 3;
}

body.race-live.control-mini .direction-pad,
body.race-live.control-mini .pedal-pad {
  display: none;
}

body.race-live.control-mini .direction-pad {
  left: max(8px, env(safe-area-inset-left));
  bottom: max(10px, env(safe-area-inset-bottom));
  grid-template-columns: repeat(2, clamp(34px, 9vw, 46px));
  gap: 5px;
}

body.race-live.control-mini .mobile-control.steer-neutral {
  display: none;
}

body.race-live.control-mini .mobile-control.steer-left,
body.race-live.control-mini .mobile-control.steer-right {
  width: clamp(34px, 9vw, 46px);
  height: clamp(34px, 9vw, 46px);
  border-radius: 999px;
  font-size: 0;
}

body.race-live.control-mini .mobile-control.steer-left::after,
body.race-live.control-mini .mobile-control.steer-right::after {
  font-size: clamp(17px, 4.5vw, 22px);
  line-height: 1;
}

body.race-live.control-mini .mobile-control.steer-left::after {
  content: "<";
}

body.race-live.control-mini .mobile-control.steer-right::after {
  content: ">";
}

body.race-live.control-mini .pedal-pad {
  right: max(8px, env(safe-area-inset-right));
  bottom: max(10px, env(safe-area-inset-bottom));
  grid-template-columns: clamp(36px, 10vw, 48px) clamp(42px, 12vw, 58px);
  grid-template-rows: clamp(30px, 8vw, 40px) clamp(34px, 9vw, 46px);
  gap: 5px;
}

body.race-live.control-mini .mobile-control.gas {
  width: 100%;
  height: 100%;
  border-radius: 999px;
  font-size: clamp(10px, 2.7vw, 12px);
}

body.race-live.control-mini .mobile-control.brake,
```

```
  body.race-live.control-mini .mobile-control.boost {
    border-radius: 999px;
    font-size: 0;
  }

  body.race-live.control-mini .mobile-control.brake::after,
  body.race-live.control-mini .mobile-control.boost::after {
    font-size: clamp(9px, 2.7vw, 11px);
    line-height: 1;
  }

  body.race-live.control-mini .mobile-control.brake::after {
    content: "Brake";
  }

  body.race-live.control-mini .mobile-control.boost::after {
    content: "Boost";
  }

  body.race-live.control-mini .control-size {
    left: max(13px, env(safe-area-inset-left));
    bottom: calc(max(10px, env(safe-area-inset-bottom)) + 146px);
    width: 52px;
    height: 30px;
    transform: none;
    opacity: 0.88;
  }
}

@media (orientation: landscape) and (max-width: 980px) and (max-height: 760px), (orientation: landscape) and (pointer:
 coarse) {
  body.race-live.control-mini .drive-stick {
    grid-template-columns: repeat(3, 36px);
    grid-template-rows: repeat(3, 36px);
    width: 120px;
    height: 120px;
  }

  body.race-live.control-mini .drive-stick .mobile-control {
    width: 36px;
    height: 36px;
    font-size: 16px;
  }

  body.race-live.control-mini .control-size {
    bottom: calc(max(10px, env(safe-area-inset-bottom)) + 128px);
  }

  body.race-live.control-mini .direction-pad {
    grid-template-columns: repeat(2, clamp(34px, 10dvh, 44px));
  }

  body.race-live.control-mini .mobile-control.steer-left,
  body.race-live.control-mini .mobile-control.steer-right {
    width: clamp(34px, 10dvh, 44px);
    height: clamp(34px, 10dvh, 44px);
  }

  body.race-live.control-mini .pedal-pad {
    grid-template-columns: clamp(36px, 10dvh, 48px) clamp(42px, 12dvh, 56px);
    grid-template-rows: clamp(30px, 8dvh, 38px) clamp(34px, 10dvh, 46px);
  }
}

@media (orientation: portrait) and (max-width: 980px) {
  body.race-live.control-mini .direction-pad {
    bottom: max(10px, env(safe-area-inset-bottom));
  }

  body.race-live.control-mini .pedal-pad {
    bottom: max(10px, env(safe-area-inset-bottom));
  }

  body.race-live.control-mini .control-size {
    bottom: calc(max(10px, env(safe-area-inset-bottom)) + 146px);
  }
}

@media (max-width: 980px), (pointer: coarse) {
  body.race-live.control-mini .drive-stick {
    --stick-origin-x: 68px;
    --stick-origin-y: calc(var(--vvh, 100dvh) - 68px);
    --stick-knob-x: 0px;
    --stick-knob-y: 0px;
```

```
    display: block;
    width: 112px;
    height: 112px;
    padding: 0;
    left: max(12px, env(safe-area-inset-left));
    bottom: max(12px, env(safe-area-inset-bottom));
    grid-template-columns: none;
    grid-template-rows: none;
    overflow: visible;
    opacity: 0.78;
    transition: opacity 140ms ease, filter 140ms ease;
  }

  body.race-live.control-mini .drive-stick.floating-stick-active {
    left: var(--stick-origin-x);
    top: var(--stick-origin-y);
    bottom: auto;
    transform: translate(-50%, -50%);
    opacity: 0.96;
    filter: brightness(1.12);
  }

  body.race-live.control-mini .drive-stick .mobile-control {
    position: absolute;
    inset: 0;
    width: 100%;
    height: 100%;
    opacity: 0;
    pointer-events: none;
  }

  body.race-live.control-mini .drive-stick::before,
  body.race-live.control-mini .drive-stick::after {
    position: absolute;
    content: "";
    pointer-events: none;
    border-radius: 999px;
  }

  body.race-live.control-mini .drive-stick::before {
    inset: 0;
    border: 1px solid rgba(244, 251, 248, 0.28);
    background:
      radial-gradient(circle at 50% 50%, rgba(187, 242, 74, 0.18), rgba(70, 217, 255, 0.1) 28%, rgba(5, 8, 7, 0.5)
66%),
      linear-gradient(135deg, rgba(255,255,255,0.12), rgba(5,8,7,0.18));
    box-shadow: 0 16px 34px rgba(0, 0, 0, 0.36), inset 0 0 26px rgba(70, 217, 255, 0.12);
  }

  body.race-live.control-mini .drive-stick::after {
    left: 50%;
    top: 50%;
    width: 46px;
    height: 46px;
    transform: translate(calc(-50% + var(--stick-knob-x)), calc(-50% + var(--stick-knob-y)));
    border: 1px solid rgba(244, 251, 248, 0.42);
    background:
      radial-gradient(circle at 36% 28%, rgba(255,255,255,0.72), rgba(255,255,255,0.14) 28%, rgba(187,242,74,0.72)
54%, rgba(70,217,255,0.72));
    box-shadow: 0 8px 22px rgba(0, 0, 0, 0.36), 0 0 18px rgba(70, 217, 255, 0.24);
  }

  body.race-live.control-mini .control-size {
    bottom: calc(max(10px, env(safe-area-inset-bottom)) + 120px);
  }
}

@media (orientation: landscape) and (max-width: 980px) and (max-height: 760px), (orientation: landscape) and (pointer:
 coarse) {
  body.race-live.control-mini .drive-stick {
    width: 96px;
    height: 96px;
    left: max(10px, env(safe-area-inset-left));
    bottom: max(10px, env(safe-area-inset-bottom));
  }

  body.race-live.control-mini .drive-stick::after {
    width: 38px;
    height: 38px;
  }

  body.race-live.control-mini .control-size {
    bottom: calc(max(10px, env(safe-area-inset-bottom)) + 104px);
  }
```

```
}
```

game.js

```
"use strict";

const $ = (selector) => document.querySelector(selector);
const $$ = (selector) => Array.from(document.querySelectorAll(selector));

const canvas = $("#gameCanvas");
const ctx = canvas.getContext("2d");
const glCanvas = $("#glCanvas");

const storeKey = "velocityVaultProfilesV1";
const saveKey = "velocityVaultSavedRaceV1";
const starterGarageVersion = 66;
const raceDistanceMultiplier = 7.4;
const minimumRaceSeconds = 72;
const ageBands = {
  rookie: { label: "Rookie 5-8", help: "Wide lanes, bigger coin streaks, cheerful missions.", speed: 0.86, traffic:
0.72, rewards: 1.18 },
  prodigy: { label: "Prodigy 9-12", help: "Sharper goals, more traffic, better rewards.", speed: 1, traffic: 1,
rewards: 1 },
  elite: { label: "Elite 13+", help: "Fast pace, tighter windows, premium rep bonuses.", speed: 1.17, traffic: 1.25,
rewards: 1.15 },
  family: { label: "Family All", help: "Balanced settings made for pass-and-play.", speed: 0.96, traffic: 0.9,
rewards: 1.05 }
};

const races = [
  { id: "neon", name: "Pacific Coast Highway Sprint", length: 3600, target: "Collect 18 coins", type: "coins", goal:
18, reward: 150, rep: 16, theme: ["#0c1a1b", "#46d9ff", "#bbf24a"], place: "coast", sign: "Pacific Coast", mood:
"golden coast" },
  { id: "skyline", name: "Chicago Skyline Night Run", length: 4300, target: "Keep focus above 60", type: "focus",
goal: 60, reward: 180, rep: 20, theme: ["#12151d", "#ffd166", "#46d9ff"], place: "city", sign: "Lakefront Loop", mood:
 "wet downtown" },
  { id: "canyon", name: "Sedona Red Rock Clash", length: 5000, target: "Dodge 30 rivals", type: "dodges", goal: 30,
reward: 220, rep: 28, theme: ["#15110c", "#ff5b6b", "#ffd166"], place: "canyon", sign: "Sedona Route", mood: "desert
chase" },
  { id: "vault", name: "Rocky Mountain Grand Prix", length: 6200, target: "Score 5000 points", type: "score", goal:
5000, reward: 320, rep: 42, theme: ["#09100f", "#bbf24a", "#46d9ff"], place: "alpine", sign: "Mountain Pass", mood:
"storm pass" },
  { id: "harbor", name: "Miami Harbor Boat Rush", length: 4200, target: "Score 3500 points", type: "score", goal:
3500, reward: 240, rep: 30, theme: ["#07161b", "#46d9ff", "#ffd166"], place: "harbor", sign: "Harbor Run", mood:
"water sprint" },
  { id: "snowfield", name: "Aspen Snowmobile Dash", length: 3900, target: "Keep focus above 55", type: "focus", goal:
55, reward: 210, rep: 26, theme: ["#0b1519", "#f4fbf8", "#46d9ff"], place: "snow", sign: "Snow Pass", mood: "ice grip"
 },
  { id: "airstrip", name: "Nevada Airfield Scramble", length: 5200, target: "Dodge 28 rivals", type: "dodges", goal:
28, reward: 280, rep: 36, theme: ["#11131a", "#ffd166", "#ff5b6b"], place: "airfield", sign: "Runway 4", mood: "air
chase" },
  { id: "freight", name: "Texas Big Rig Freight Run", length: 5600, target: "Score 4200 points", type: "score", goal:
4200, reward: 270, rep: 34, theme: ["#10140f", "#ffd166", "#46d9ff"], place: "freight", sign: "Texas Freightway",
mood: "big rig draft", unlock: 0 },
  { id: "farmrally", name: "Iowa Tractor Rally", length: 3300, target: "Collect 16 route markers", type: "coins",
goal: 16, reward: 190, rep: 22, theme: ["#10180e", "#bbf24a", "#ffd166"], place: "farm", sign: "Iowa Backroads", mood:
 "field sprint", unlock: 0 },
  { id: "tokyo", name: "Tokyo Neon Expressway", length: 5700, target: "Keep focus above 62", type: "focus", goal: 62,
reward: 310, rep: 40, theme: ["#100d1c", "#ff4fd8", "#46d9ff"], place: "tokyo", sign: "Tokyo Express", mood: "neon
tunnel", unlock: 44 },
  { id: "sahara", name: "Sahara Desert Rally", length: 6100, target: "Dodge 34 rivals", type: "dodges", goal: 34,
reward: 330, rep: 44, theme: ["#160f0a", "#ffb74a", "#f4fbf8"], place: "desert", sign: "Sahara Rally", mood: "sand
storm", unlock: 66 },
  { id: "rainforest", name: "Amazon Rainforest Rush", length: 5400, target: "Score 4600 points", type: "score", goal:
4600, reward: 320, rep: 42, theme: ["#07130d", "#36d98a", "#ffd166"], place: "rainforest", sign: "Amazon Route", mood:
 "jungle rain", unlock: 88 },
  { id: "eurotour", name: "Swiss Alps Grand Tour", length: 6400, target: "Score 5600 points", type: "score", goal:
5600, reward: 360, rep: 48, theme: ["#091119", "#dce8ef", "#46d9ff"], place: "europe", sign: "Swiss Alps", mood: "euro
 pass", unlock: 110 }
];

const vehicleDefs = [
  { id: "street", name: "Street Supercar", type: "car", desc: "Fast all-around modern racing feel.", speed: 1,
handling: 1, mass: 1, color: "#1bb7e8", price: 0, unlock: 0, class: "road" },
  { id: "rally", name: "Rally Coupe", type: "car", desc: "Widebody dirt-road racer with fast lane changes.", speed:
0.96, handling: 1.16, mass: 0.96, color: "#ff8c42", price: 1800, unlock: 28, class: "road", model: "Rally widebody" },
  { id: "drift", name: "Drift Street Coupe", type: "car", desc: "Slide-focused city build with extra drift control.",
speed: 0.98, handling: 1.24, mass: 0.92, color: "#ff4fd8", price: 2200, unlock: 38, class: "road", model: "Drift kit"
},
  { id: "f1", name: "F1 Open-Wheel", type: "f1", desc: "Sharp steering and high top speed.", speed: 1.18, handling:
1.18, mass: 0.72, color: "#ff3348", price: 2600, unlock: 45, class: "road" },
  { id: "grandprix", name: "Grand Prix Prototype", type: "prototype", desc: "Stable, fast, low race body.", speed:
1.12, handling: 1.08, mass: 0.82, color: "#f4fbf8", price: 3200, unlock: 58, class: "road" },
  { id: "truck", name: "Performance Truck", type: "truck", desc: "Heavier, stable, strong contact resistance.", speed:
 0.88, handling: 0.82, mass: 1.35, color: "#ffd166", price: 1200, unlock: 20, class: "road" },
  { id: "buggy", name: "Desert Buggy", type: "truck", desc: "Light off-road machine built for dunes and shortcut
```

```
branches.", speed: 0.92, handling: 1.04, mass: 0.88, color: "#ffb74a", price: 2400, unlock: 50, class: "rally", model:
 "Open desert buggy" },
  { id: "semi", name: "Semi Truck Racer", type: "semi", desc: "Huge highway pull, heavy drafting, upgrade into a
freight rocket.", speed: 0.72, handling: 0.58, mass: 2.1, color: "#dce8ef", price: 3400, unlock: 60, class: "freight"
},
  { id: "tractor", name: "Racing Tractor", type: "tractor", desc: "Farm rally machine with tough grip and upgradeable
agility.", speed: 0.62, handling: 0.7, mass: 1.65, color: "#36d98a", price: 1400, unlock: 22, class: "farm" },
  { id: "monster", name: "Monster Truck", type: "monster", desc: "Huge stance, slower but tough.", speed: 0.76,
handling: 0.7, mass: 1.75, color: "#bbf24a", price: 4200, unlock: 72, class: "monster" },
  { id: "tank", name: "Armored Tank", type: "tank", desc: "Slow, heavy, almost unstoppable.", speed: 0.56, handling:
0.52, mass: 2.35, color: "#6d7667", price: 5200, unlock: 92, class: "military" },
  { id: "heavytank", name: "Heavy Battle Tank", type: "tank", desc: "Bulkier armor, slower turn-in, stronger route-
clearing hits.", speed: 0.48, handling: 0.46, mass: 2.75, color: "#3f4a38", price: 7600, unlock: 138, class:
"military", model: "Heavy armor" },
  { id: "snowmobile", name: "Snowmobile", type: "snowmobile", desc: "Light and quick on snow routes.", speed: 0.94,
handling: 1.15, mass: 0.58, color: "#f4fbf8", price: 1600, unlock: 26, class: "snow" },
  { id: "boat", name: "Race Boat", type: "boat", desc: "Best fit for harbor water sprints.", speed: 1.05, handling:
0.86, mass: 0.92, color: "#46d9ff", price: 1900, unlock: 30, class: "water" },
  { id: "hydro", name: "Hydroplane Racer", type: "boat", desc: "Higher-speed water rocket for harbor and marina
cuts.", speed: 1.18, handling: 0.78, mass: 0.78, color: "#6fffe9", price: 3600, unlock: 64, class: "water", model:
"Hydroplane" },
  { id: "patrolboat", name: "Armored Patrol Boat", type: "boat", desc: "Heavy water build for chase scenarios and
rough wakes.", speed: 0.88, handling: 0.74, mass: 1.35, color: "#dce8ef", price: 4600, unlock: 82, class: "water",
model: "Patrol hull" },
  { id: "helicopter", name: "Pursuit Helicopter", type: "helicopter", desc: "Air-style handling with wide steering.",
speed: 0.92, handling: 0.98, mass: 0.9, color: "#dce8ef", price: 6000, unlock: 110, class: "air" },
  { id: "attackheli", name: "Attack Helicopter", type: "helicopter", desc: "Faster air mission build with stronger
pursuit handling.", speed: 1.02, handling: 0.9, mass: 1.05, color: "#6d7667", price: 7800, unlock: 145, class: "air",
model: "Attack rotor" },
  { id: "airplane", name: "Sport Airplane", type: "airplane", desc: "Fast runway and airfield races.", speed: 1.22,
handling: 0.78, mass: 0.74, color: "#ff5b6b", price: 7000, unlock: 128, class: "air" },
  { id: "jetplane", name: "Jet Stunt Plane", type: "airplane", desc: "High-speed sky racer for long airfield and
mountain routes.", speed: 1.34, handling: 0.68, mass: 0.82, color: "#f4fbf8", price: 8800, unlock: 158, class: "air",
model: "Jet stunt wing" }
];

const opponentNames = ["Vega", "Knox", "Ryder", "Nova", "Sable", "Mako", "Jett", "Blitz"];

const upgradeDefs = [
  { id: "engine", name: "Ion Engine", desc: "Higher top speed and faster score flow.", base: 320 },
  { id: "tires", name: "Grip Tires", desc: "Quicker steering and smoother dodges.", base: 280 },
  { id: "shield", name: "Pulse Shield", desc: "Protects focus during collisions.", base: 360 },
  { id: "magnet", name: "Coin Magnet", desc: "Pulls coins from wider lanes.", base: 300 },
  { id: "boost", name: "Clean Boost", desc: "Longer boost bursts with less focus drain.", base: 340 }
];

const paintPalette = ["#1bb7e8", "#ff3348", "#ff8c42", "#ffd166", "#bbf24a", "#f4fbf8", "#ff4fd8", "#36d98a",
"#6fffe9", "#dce8ef", "#6d7667", "#3f4a38"];

const vehicleRaceRules = {
  car: { places: ["coast", "city", "canyon", "alpine", "tokyo", "desert", "rainforest", "europe"], label: "road
circuits" },
  f1: { places: ["coast", "city", "alpine", "tokyo", "europe"], label: "road and grand prix circuits" },
  prototype: { places: ["coast", "city", "alpine", "tokyo", "europe"], label: "road and grand prix circuits" },
  truck: { places: ["coast", "city", "canyon", "alpine", "freight", "farm", "desert", "rainforest"], label: "road,
freight, and rally routes" },
  semi: { places: ["freight", "coast", "city", "desert"], label: "freight routes" },
  tractor: { places: ["farm", "rainforest"], label: "farm rally routes" },
  monster: { places: ["canyon", "freight", "farm", "desert", "rainforest"], label: "monster truck arenas" },
  tank: { places: ["airfield", "desert", "freight", "canyon"], label: "military routes" },
  snowmobile: { places: ["snow"], label: "snow routes" },
  boat: { places: ["harbor"], label: "water routes" },
  helicopter: { places: ["airfield", "desert", "rainforest", "europe"], label: "sky routes" },
  airplane: { places: ["airfield", "desert", "europe"], label: "sky routes" }
};

const multiplayerScenarios = [
  { id: "solo", name: "Solo Race", desc: "Standard race against the route pack.", allies: 0, extraOpponents: 0,
heatBoost: 0, reward: 1 },
  { id: "ghostduel", name: "Local Ghost Duel", desc: "Adds a Player 2 ghost and one extra rival for pass-and-play
style races.", allies: 1, extraOpponents: 1, heatBoost: 0.02, reward: 1.05 },
  { id: "crewrelay", name: "Crew Relay", desc: "Adds two local ghost teammates and a bigger rival field.", allies: 2,
extraOpponents: 2, heatBoost: 0.04, reward: 1.1 },
  { id: "pursuitcrew", name: "Hot Pursuit Crew", desc: "Crew race with rising police pressure over longer routes.",
allies: 1, extraOpponents: 2, hotPursuit: true, heatBoost: 0.22, reward: 1.18 }
];

const missionDefs = [
  { id: "first", text: "Finish your first race", reward: 90, test: (p) => p.stats.races >= 1 },
  { id: "collector", text: "Collect 75 total coins", reward: 130, test: (p) => p.stats.totalCoins >= 75 },
  { id: "steady", text: "Complete 3 races with focus above 50", reward: 190, test: (p) => p.stats.steadyRuns >= 3 },
  { id: "vault", text: "Earn 120 reputation", reward: 260, test: (p) => p.rep >= 120 }
];
```

```
const directorEvents = [
  { id: "coinrush", name: "AI Coin Rush", focus: "more coin gates", coinRate: 1.42, traffic: 0.92, reward: 1.12 },
  { id: "draftline", name: "AI Draft Line", focus: "speed and clean dodges", coinRate: 0.95, traffic: 1.1, reward:
1.18 },
  { id: "shieldlab", name: "AI Shield Lab", focus: "recovery after impacts", coinRate: 1.06, traffic: 0.86, reward:
1.08 },
  { id: "neonmix", name: "AI Neon Mix", focus: "balanced variety", coinRate: 1.14, traffic: 1, reward: 1.1 }
];

let profiles = loadProfiles();
let selectedAge = "rookie";
let activeProfile = null;
let selectedRace = races[0];
let selectedScenario = scenarioById(localStorage.getItem("velocityVaultScenarioMode") || "solo");
let view = "login";
let tab = "races";
let toastTimer = 0;
let deferredInstallPrompt = null;
let cameraMode = localStorage.getItem("velocityVaultCameraMode") || "chase";
let rendererMode = localStorage.getItem("velocityVaultRendererMode") || "webgl";
let touchDriveMode = localStorage.getItem("velocityVaultTouchDriveMode") === "toggle" ? "toggle" : "hold";
let touchControlSize = localStorage.getItem("velocityVaultTouchControlSize") === "full" ? "full" : "mini";
let webglRenderer = null;
let audioSystem = null;

const input = {
  left: false,
  right: false,
  gas: false,
  brake: false,
  boost: false,
  paused: false,
  touchSteer: 0,
  gamepadSteer: 0,
  gamepadGas: false,
  gamepadBrake: false,
  gamepadBoost: false,
  gamepadPauseHeld: false,
  gamepadResetHeld: false
};

const controllerState = {
  connected: false,
  name: "",
  index: null
};

const mobileTouchState = {
  active: new Map(),
  usingTouchEvents: false,
  stickSteer: 0,
  driveStickActive: false,
  latchedStickControl: "",
  modeToggleAt: 0,
  floatingStickId: null,
  floatingStickKey: "",
  floatingStickOriginX: 0,
  floatingStickOriginY: 0,
  floatingStickKnobX: 0,
  floatingStickKnobY: 0
};

const raceState = {
  active: false,
  distance: 0,
  speed: 0,
  focus: 100,
  score: 0,
  coins: 0,
  dodges: 0,
  overtakes: 0,
  passesAgainst: 0,
  civilianHits: 0,
  penaltyCoins: 0,
  penaltyRep: 0,
  combo: 1,
  lane: 0,
  x: 0,
  visualLane: 0,
  lateralVelocity: 0,
  steerAngle: 0,
  throttleLoad: 0,
  brakeHeat: 0,
  slip: 0,
```

```
  damage: 0,
  damageAlertTimer: 0,
  damageAlertLabel: "",
  hazardWarningTimer: 0,
  hazardWarningLabel: "",
  resetTimer: 0,
  resetReason: "",
  crashCooldown: 0,
  roadCurve: 0,
  roadTurn: 0,
  roadOffset: 0,
  spawnClock: 0,
  coinClock: 0,
  civilianClock: 0,
  oncomingClock: 0,
  cannonCooldown: 0,
  routeFeatureClock: 0,
  hideCooldown: 0,
  scenarioId: "solo",
  scenarioLabel: "Solo Race",
  teamScore: 0,
  rivals: [],
  police: [],
  opponents: [],
  coinsOnRoad: [],
  civilians: [],
  oncoming: [],
  routeFeatures: [],
  particles: [],
  elapsed: 0,
  heat: 0,
  heatClock: 0,
  chaseActive: false,
  cameraShake: 0,
  countdown: 0,
  goalIntroTimer: 0,
  finished: false,
  director: null
};

function loadProfiles() {
  try {
    const raw = localStorage.getItem(storeKey);
    return raw ? JSON.parse(raw) : {};
  } catch {
    return {};
  }
}

function saveProfiles() {
  localStorage.setItem(storeKey, JSON.stringify(profiles));
}

function selectedVehicle() {
  const id = activeProfile && activeProfile.selectedVehicle ? activeProfile.selectedVehicle : "street";
  return vehicleDefs.find((vehicle) => vehicle.id === id) || vehicleDefs[0];
}

function vehicleById(id) {
  return vehicleDefs.find((vehicle) => vehicle.id === id) || vehicleDefs[0];
}

function scenarioById(id) {
  return multiplayerScenarios.find((scenario) => scenario.id === id) || multiplayerScenarios[0];
}

function activeScenario() {
  selectedScenario = scenarioById(selectedScenario && selectedScenario.id);
  return selectedScenario;
}

function defaultUpgrades() {
  return { engine: 0, tires: 0, shield: 0, magnet: 0, boost: 0 };
}

function defaultBuild(vehicle = vehicleDefs[0]) {
  return {
    color: vehicle.color,
    trim: "#050807",
    upgrades: defaultUpgrades(),
    plate: ""
  };
}
```

```
function normalizeBuild(vehicle, build, legacyUpgrades = null) {
  const base = defaultBuild(vehicle);
  const source = build && typeof build === "object" ? build : {};
  const upgrades = Object.assign(
    base.upgrades,
    legacyUpgrades && typeof legacyUpgrades === "object" ? legacyUpgrades : {},
    source.upgrades || {}
  );
  Object.keys(upgrades).forEach((key) => {
    upgrades[key] = Math.max(0, Math.min(5, Number(upgrades[key]) || 0));
  });
  return {
    color: source.color || base.color,
    trim: source.trim || base.trim,
    upgrades,
    plate: sanitizePlate(source.plate || "")
  };
}

function sanitizePlate(value) {
  const source = String(value || "").toUpperCase().replace(/[^A-Z0-9_-]/g, "");
  return source.slice(0, 8);
}

function driverPlate(profile = activeProfile) {
  if (!profile) return "RACER";
  const build = vehicleBuild(profile, profile.selectedVehicle || "street");
  return sanitizePlate(build.plate || profile.name || "RACER");
}

function vehicleBuild(profile = activeProfile, vehicleId = null) {
  const vehicle = vehicleById(vehicleId || (profile && profile.selectedVehicle) || "street");
  if (!profile) return defaultBuild(vehicle);
  profile.vehicleBuilds = profile.vehicleBuilds && typeof profile.vehicleBuilds === "object" ? profile.vehicleBuilds :
 {};
  const legacy = vehicleOwned(profile, vehicle.id) ? profile.upgrades : null;
  profile.vehicleBuilds[vehicle.id] = normalizeBuild(vehicle, profile.vehicleBuilds[vehicle.id], legacy);
  return profile.vehicleBuilds[vehicle.id];
}

function activeVehicleUpgrades(profile = activeProfile) {
  return vehicleBuild(profile, profile && profile.selectedVehicle).upgrades;
}

function selectedVehicleColor(vehicle = selectedVehicle()) {
  return vehicleBuild(activeProfile, vehicle.id).color || vehicle.color;
}

function vehicleOwned(profile = activeProfile, vehicleId = "street") {
  if (!profile) return vehicleId === "street";
  const owned = Array.isArray(profile.ownedVehicles) ? profile.ownedVehicles : ["street"];
  return owned.includes(vehicleId);
}

function vehicleUnlocked(profile = activeProfile, vehicle = vehicleDefs[0]) {
  return !vehicle.unlock || (profile && profile.rep >= vehicle.unlock);
}

function raceCompatibleWithVehicle(race = selectedRace, vehicle = selectedVehicle()) {
  const rule = vehicleRaceRules[vehicle.type] || vehicleRaceRules.car;
  return rule.places.includes(race.place);
}

function compatibilityLabel(vehicle = selectedVehicle()) {
  const rule = vehicleRaceRules[vehicle.type] || vehicleRaceRules.car;
  return rule.label;
}

function firstCompatibleRace(vehicle = selectedVehicle(), profile = activeProfile) {
  return races.find((race, index) => {
    const requiredRep = race.unlock ?? index * 22;
    return (!profile || profile.rep >= requiredRep) && raceCompatibleWithVehicle(race, vehicle);
  }) || races[0];
}

function buyOrSelectVehicle(vehicle) {
  if (!activeProfile) return;
  if (!vehicleUnlocked(activeProfile, vehicle)) {
    showToast(`${vehicle.name} needs ${vehicle.unlock} reputation.`);
    return;
  }
  activeProfile.ownedVehicles = Array.isArray(activeProfile.ownedVehicles) ? activeProfile.ownedVehicles : ["street"];
  if (!vehicleOwned(activeProfile, vehicle.id)) {
    if (activeProfile.coins < vehicle.price) {
```

```
      showToast(`${vehicle.name} costs ${vehicle.price} coins.`);
      return;
    }
    activeProfile.coins -= vehicle.price;
    activeProfile.ownedVehicles.push(vehicle.id);
    activeProfile.vehicleBuilds = activeProfile.vehicleBuilds && typeof activeProfile.vehicleBuilds === "object" ?
activeProfile.vehicleBuilds : {};
    activeProfile.vehicleBuilds[vehicle.id] = normalizeBuild(vehicle, null, null);
    activeProfile.vehicleBuilds[vehicle.id].plate = sanitizePlate(activeProfile.name);
    showToast(`${vehicle.name} added to your garage.`);
  } else if (activeProfile.selectedVehicle === vehicle.id) {
    cycleVehiclePaint(vehicle);
    return;
  }
  activeProfile.selectedVehicle = vehicle.id;
  const build = vehicleBuild(activeProfile, vehicle.id);
  activeProfile.upgrades = Object.assign(defaultUpgrades(), build.upgrades);
  if (!raceCompatibleWithVehicle(selectedRace, vehicle)) selectedRace = firstCompatibleRace(vehicle, activeProfile);
  profiles[activeProfile.id] = activeProfile;
  saveProfiles();
  renderHub();
  updateHud();
  showToast(`${vehicle.name} selected for ${compatibilityLabel(vehicle)}.`);
}

function cycleVehiclePaint(vehicle = selectedVehicle()) {
  if (!activeProfile) return;
  const build = vehicleBuild(activeProfile, vehicle.id);
  const current = build.color || vehicle.color;
  const currentIndex = paintPalette.indexOf(current);
  build.color = paintPalette[(currentIndex + 1 + paintPalette.length) % paintPalette.length];
  profiles[activeProfile.id] = activeProfile;
  saveProfiles();
  renderHub();
  showToast(`${vehicle.name} paint changed.`);
}

function raceLength(race = selectedRace) {
  const baseLength = race && race.length ? race.length : races[0].length;
  return Math.round(baseLength * raceDistanceMultiplier);
}

const routeWorlds = {
  coast: { country: "USA", scene: "Pacific Coast", cue: "ocean cliffs", turn: 1.05, seed: 1.2 },
  city: { country: "USA", scene: "Chicago Loop", cue: "wet downtown", turn: 0.95, seed: 2.4 },
  canyon: { country: "USA", scene: "Sedona Canyon", cue: "red rock sweepers", turn: 1.24, seed: 3.6 },
  alpine: { country: "USA", scene: "Rocky Mountain Pass", cue: "storm pass", turn: 1.32, seed: 4.8 },
  harbor: { country: "USA", scene: "Miami Harbor", cue: "bridge run", turn: 0.92, seed: 5.7 },
  snow: { country: "USA", scene: "Aspen Snowfields", cue: "ice bends", turn: 1.08, seed: 6.9 },
  airfield: { country: "USA", scene: "Nevada Airfield", cue: "runway chicanes", turn: 0.72, seed: 7.4 },
  freight: { country: "USA", scene: "Texas Freightway", cue: "wide interstate", turn: 0.82, seed: 8.1 },
  farm: { country: "USA", scene: "Iowa Backroads", cue: "rolling farm roads", turn: 0.88, seed: 8.9 },
  tokyo: { country: "Japan", scene: "Tokyo Expressway", cue: "neon overpasses", turn: 1.18, seed: 9.6 },
  desert: { country: "Morocco", scene: "Sahara Rally", cue: "sand ridges", turn: 1.12, seed: 10.5 },
  rainforest: { country: "Brazil", scene: "Amazon Rainforest", cue: "jungle tunnels", turn: 1.02, seed: 11.4 },
  europe: { country: "Switzerland", scene: "Swiss Alps Tour", cue: "village switchbacks", turn: 1.36, seed: 12.2 }
};

const genAiSceneDesigns = {
  coast: { style: "coastal grand tour", surface: "#1b2424", accent: "#46d9ff", light: "#ffd166", props: ["surf",
"cliff", "camera", "crowd", "brake"] },
  city: { style: "downtown street circuit", surface: "#151b1d", accent: "#46d9ff", light: "#f4fbf8", props: ["tower",
"barrier", "crowd", "crosswalk", "brake"] },
  canyon: { style: "red-rock drift course", surface: "#2a1c14", accent: "#ff5b6b", light: "#ffd166", props: ["rock",
"dust", "camera", "barrier", "brake"] },
  alpine: { style: "mountain pass race", surface: "#1b2727", accent: "#bbf24a", light: "#dce8ef", props: ["pine",
"snowcap", "crowd", "guard", "brake"] },
  harbor: { style: "waterfront tunnel sprint", surface: "#14242a", accent: "#46d9ff", light: "#ffd166", props:
["crane", "boat", "barrier", "camera", "brake"] },
  snow: { style: "winter rally stage", surface: "#344141", accent: "#f4fbf8", light: "#46d9ff", props: ["snowbank",
"pine", "flag", "crowd", "brake"] },
  airfield: { style: "runway speed trial", surface: "#252723", accent: "#ffd166", light: "#ff5b6b", props: ["hangar",
"beacon", "plane", "barrier", "brake"] },
  freight: { style: "freightway endurance race", surface: "#22241d", accent: "#ffd166", light: "#46d9ff", props:
["trailer", "gantry", "crowd", "barrier", "brake"] },
  farm: { style: "rural rally festival", surface: "#3d3424", accent: "#bbf24a", light: "#ffd166", props: ["barn",
"field", "tractor", "crowd", "brake"] },
  tokyo: { style: "neon expressway battle", surface: "#151424", accent: "#ff4fd8", light: "#46d9ff", props: ["neon",
"tunnel", "tower", "crowd", "brake"] },
  desert: { style: "open desert rally", surface: "#4b3321", accent: "#ffb74a", light: "#f4fbf8", props: ["dune",
"dust", "camera", "barrier", "brake"] },
  rainforest: { style: "jungle wet rally", surface: "#17241e", accent: "#36d98a", light: "#ffd166", props: ["canopy",
"rain", "bridge", "crowd", "brake"] },
  europe: { style: "euro alpine tour", surface: "#263235", accent: "#dce8ef", light: "#46d9ff", props: ["village",
```

```
"snowcap", "guard", "crowd", "brake"] }
};

function routeWorldInfo(place = "city") {
  return routeWorlds[place] || routeWorlds.city;
}

function genAiSceneDesign(place = "city") {
  return genAiSceneDesigns[place] || genAiSceneDesigns.city;
}

function roadTurnAt(distance, race = selectedRace) {
  const info = routeWorldInfo(race && race.place ? race.place : "city");
  const d = Number(distance) || 0;
  const longSweep = Math.sin(d * 0.00034 + info.seed) * 0.82;
  const countryBend = Math.sin(d * 0.00072 + info.seed * 1.9) * 0.42;
  const shortSet = Math.sin(d * 0.00118 + info.seed * 0.58) * 0.18;
  const scenicSettle = Math.sin(d * 0.00009 + info.seed * 3.4) * 0.22;
  return Math.max(-1.32, Math.min(1.32, (longSweep + countryBend + shortSet + scenicSettle) * info.turn));
}

function routeTurnName(turn) {
  if (turn > 0.58) return "Right sweep";
  if (turn < -0.58) return "Left sweep";
  if (turn > 0.24) return "Right bend";
  if (turn < -0.24) return "Left bend";
  return "Fast straight";
}

function loadSavedRace() {
  try {
    const raw = localStorage.getItem(saveKey);
    return raw ? JSON.parse(raw) : null;
  } catch {
    return null;
  }
}

function clearSavedRace() {
  localStorage.removeItem(saveKey);
}

function makeProfile(name, pin, age) {
  return {
    id: `${age}:${name.toLowerCase()}`,
    name,
    pinHash: pin ? hashPin(pin) : "",
    age,
    coins: 1250,
    rep: 44,
    selectedVehicle: "street",
    upgrades: defaultUpgrades(),
    ownedVehicles: ["street"],
    vehicleBuilds: {
      street: Object.assign(defaultBuild(vehicleById("street")), { plate: sanitizePlate(name) })
    },
    completedMissions: [],
    stats: { races: 0, wins: 0, totalCoins: 0, steadyRuns: 0, bestScore: 0 },
    garageStarterVersion: starterGarageVersion
  };
}

function normalizeProfile(profile) {
  profile.coins = Number(profile.coins) || 0;
  profile.rep = Number(profile.rep) || 0;
  profile.selectedVehicle = profile.selectedVehicle || "street";
  profile.upgrades = Object.assign(defaultUpgrades(), profile.upgrades || {});
  profile.ownedVehicles = Array.isArray(profile.ownedVehicles) ? profile.ownedVehicles : ["street"];
  if (!profile.ownedVehicles.includes("street")) profile.ownedVehicles.unshift("street");
  profile.vehicleBuilds = profile.vehicleBuilds && typeof profile.vehicleBuilds === "object" ? profile.vehicleBuilds :
 {};
  vehicleDefs.forEach((vehicle) => {
    if (profile.ownedVehicles.includes(vehicle.id) || vehicle.id === profile.selectedVehicle) {
      profile.vehicleBuilds[vehicle.id] = normalizeBuild(vehicle, profile.vehicleBuilds[vehicle.id],
profile.upgrades);
      if (!profile.vehicleBuilds[vehicle.id].plate) profile.vehicleBuilds[vehicle.id].plate =
sanitizePlate(profile.name);
    }
  });
  if (!vehicleOwned(profile, profile.selectedVehicle)) profile.selectedVehicle = "street";
  profile.upgrades = Object.assign(defaultUpgrades(), vehicleBuild(profile, profile.selectedVehicle).upgrades);
  profile.completedMissions = Array.isArray(profile.completedMissions) ? profile.completedMissions : [];
  profile.stats = Object.assign({ races: 0, wins: 0, totalCoins: 0, steadyRuns: 0, bestScore: 0 }, profile.stats ||
{});
```

```
  if (profile.garageStarterVersion !== starterGarageVersion) {
    profile.coins = Math.max(profile.coins, 1250);
    profile.rep = Math.max(profile.rep, 44);
    profile.garageStarterVersion = starterGarageVersion;
  }
  return profile;
}

function hashPin(pin) {
  let hash = 2166136261;
  for (let i = 0; i < pin.length; i += 1) {
    hash ^= pin.charCodeAt(i);
    hash = Math.imul(hash, 16777619);
  }
  return (hash >>> 0).toString(36);
}

function sanitizeName(value) {
  return value.replace(/[^a-zA-Z0-9_-]/g, "").slice(0, 16) || "TurboAce";
}

function showView(next) {
  view = next;
  $$(".view").forEach((el) => el.classList.remove("active"));
  $(`#${next}View`).classList.add("active");
  document.body.classList.toggle("race-live", next === "race");
  if (next !== "race") clearTouchDriveInputs();
  setTouchDriveMode(touchDriveMode, true);
  syncViewportSize();
  fitCanvas();
}

function showToast(message) {
  const toast = $("#toast");
  toast.textContent = message;
  toast.classList.add("show");
  clearTimeout(toastTimer);
  toastTimer = setTimeout(() => toast.classList.remove("show"), 2300);
}

function startAudio() {
  if (audioSystem || !window.AudioContext && !window.webkitAudioContext) return;
  const AudioCtor = window.AudioContext || window.webkitAudioContext;
  const ctxAudio = new AudioCtor();
  const master = ctxAudio.createGain();
  master.gain.value = 0.045;
  master.connect(ctxAudio.destination);

  const engine = ctxAudio.createOscillator();
  const engineGain = ctxAudio.createGain();
  engine.type = "sawtooth";
  engine.frequency.value = 72;
  engineGain.gain.value = 0.18;
  engine.connect(engineGain);
  engineGain.connect(master);
  engine.start();

  const siren = ctxAudio.createOscillator();
  const sirenGain = ctxAudio.createGain();
  siren.type = "triangle";
  siren.frequency.value = 620;
  sirenGain.gain.value = 0;
  siren.connect(sirenGain);
  sirenGain.connect(master);
  siren.start();

  audioSystem = { ctx: ctxAudio, master, engine, engineGain, siren, sirenGain };
}

function updateAudio() {
  if (!audioSystem) return;
  const now = audioSystem.ctx.currentTime;
  const speed = raceState.active ? raceState.speed : 0;
  const boostInput = input.boost || input.gamepadBoost;
  audioSystem.engine.frequency.setTargetAtTime(64 + speed * 0.62 + (boostInput ? 70 : 0), now, 0.045);
  audioSystem.engineGain.gain.setTargetAtTime(raceState.active ? 0.12 + Math.min(0.18, speed / 1200) : 0.035, now,
0.08);
  const sirenLevel = raceState.active && raceState.chaseActive ? 0.14 : 0;
  const sirenSweep = 620 + Math.sin(raceState.elapsed * 7.2) * 210;
  audioSystem.siren.frequency.setTargetAtTime(sirenSweep, now, 0.03);
  audioSystem.sirenGain.gain.setTargetAtTime(sirenLevel, now, 0.06);
}

function playHitSound(kind = "impact") {
```

```
  if (!audioSystem) return;
  const now = audioSystem.ctx.currentTime;
  const osc = audioSystem.ctx.createOscillator();
  const gain = audioSystem.ctx.createGain();
  osc.type = kind === "coin" ? "sine" : "square";
  osc.frequency.value = kind === "coin" ? 880 : kind === "police" ? 150 : 210;
  gain.gain.setValueAtTime(kind === "coin" ? 0.07 : 0.11, now);
  gain.gain.exponentialRampToValueAtTime(0.001, now + (kind === "coin" ? 0.14 : 0.22));
  osc.connect(gain);
  gain.connect(audioSystem.master);
  osc.start(now);
  osc.stop(now + 0.24);
}

function profilePower(profile) {
  const build = vehicleBuild(profile, profile && profile.selectedVehicle);
  return Object.values(build.upgrades).reduce((sum, level) => sum + level, 0);
}

function directorDay() {
  return Math.floor(Date.now() / 86400000);
}

function getDirector(profile) {
  const day = directorDay();
  const seed = profile.name.length * 17 + profile.rep * 3 + profile.stats.races * 11 + day;
  const event = directorEvents[Math.abs(seed) % directorEvents.length];
  const upgrades = activeVehicleUpgrades(profile);
  let weakest = upgradeDefs[0];
  upgradeDefs.forEach((def) => {
    if (upgrades[def.id] < upgrades[weakest.id]) weakest = def;
  });
  const winRate = profile.stats.races ? profile.stats.wins / profile.stats.races : 0;
  const assist = winRate < 0.35 ? "extra reward support" : winRate > 0.7 ? "faster rival pacing" : "balanced variety";
  return {
    day,
    event,
    recommended: weakest,
    assist,
    reward: event.reward + (winRate < 0.35 ? 0.08 : 0),
    traffic: event.traffic + (winRate > 0.7 ? 0.08 : 0),
    coinRate: event.coinRate + (profile.stats.totalCoins < 40 ? 0.12 : 0)
  };
}

function renderDirector() {
  if (!activeProfile) return;
  const director = getDirector(activeProfile);
  const card = $("#directorCard");
  card.innerHTML = `
    <strong>${director.event.name}</strong>
    <span>Local agentic tuning is active for this driver. It studies only this profile's on-device race stats, then
rotates events and upgrade nudges to keep the game fresh.</span>
    <ul>
      <li>Today's focus: ${director.event.focus}.</li>
      <li>Recommended upgrade: ${director.recommended.name}.</li>
      <li>Adaptive assist: ${director.assist}.</li>
      <li>Bonus rewards: ${Math.round((director.reward - 1) * 100)}%.</li>
    </ul>
  `;
}

function upgradeCost(profile, def) {
  const level = activeVehicleUpgrades(profile)[def.id];
  return Math.round(def.base * Math.pow(1.92, level));
}

function setActiveProfile(profile) {
  activeProfile = profile;
  selectedRace = races[Math.min(profile.stats.races, races.length - 1)];
  if (!raceCompatibleWithVehicle(selectedRace, selectedVehicle())) selectedRace =
firstCompatibleRace(selectedVehicle(), profile);
  renderHub();
  updateHud();
  showView("garage");
  showToast(`Welcome, ${profile.name}. Garage unlocked.`);
}

function enterProfile(openRace = false) {
  const name = sanitizeName($("#driverName").value);
  const pin = $("#driverPin").value.replace(/\D/g, "").slice(0, 6);
  const id = `${selectedAge}:${name.toLowerCase()}`;
  const existing = profiles[id];
  if (existing && existing.pinHash && existing.pinHash !== hashPin(pin)) {
```

```
    showToast("PIN did not match this local profile.");
    return null;
  }
  const profile = normalizeProfile(existing || makeProfile(name, pin, selectedAge));
  profiles[profile.id] = profile;
  saveProfiles();
  setActiveProfile(profile);
  if (openRace) {
    selectedRace = races[Math.min(profile.stats.races, races.length - 1)];
    if (!raceCompatibleWithVehicle(selectedRace, selectedVehicle())) selectedRace =
firstCompatibleRace(selectedVehicle(), profile);
    launchRace();
  }
  return profile;
}

function renderHub() {
  if (!activeProfile) return;
  $("#driverSummary").textContent = `${ageBands[activeProfile.age].label} | ${activeProfile.coins} coins |
${activeProfile.rep} rep | ${selectedVehicle().name} | ${activeScenario().name} | Plate ${driverPlate(activeProfile)}
| Power ${profilePower(activeProfile)}`;
  renderRaces();
  renderScenarios();
  renderVehicles();
  renderUpgrades();
  renderMissions();
  renderDirector();
  renderSavedRaceButton();
}

function renderSavedRaceButton() {
  const btn = $("#resumeRace");
  const saved = loadSavedRace();
  const canResume = saved && activeProfile && saved.profileId === activeProfile.id;
  btn.disabled = !canResume;
  if (canResume) {
    const race = races.find((item) => item.id === saved.selectedRaceId);
    btn.textContent = `Resume ${race ? race.name : "Saved Race"}`;
  } else {
    btn.textContent = saved ? "Saved Race For Another Driver" : "No Saved Race";
  }
}

function renderRaces() {
  const list = $("#raceList");
  list.innerHTML = "";
  const director = getDirector(activeProfile);
  const vehicle = selectedVehicle();
  races.forEach((race, index) => {
    const requiredRep = race.unlock ?? index * 22;
    const locked = activeProfile.rep < requiredRep;
    const incompatible = !raceCompatibleWithVehicle(race, vehicle);
    const card = document.createElement("button");
    card.type = "button";
    card.className = `race-card ${selectedRace.id === race.id ? "selected" : ""}`;
    card.disabled = locked || incompatible;
    card.innerHTML = `
      <div class="card-top"><strong>${race.name}</strong><span class="badge">${locked ? `${requiredRep} rep` :
incompatible ? compatibilityLabel(vehicle) : `${Math.round(race.reward * director.reward)} coins`}</span></div>
      <div class="tiny">${race.target} | ${Math.round(raceLength(race) / 100)} sectors |
${routeWorldInfo(race.place).scene} | ${director.event.name}</div>
    `;
    card.addEventListener("click", () => {
      if (locked || incompatible) return;
      selectedRace = race;
      renderRaces();
    });
    list.appendChild(card);
  });
}

function renderScenarios() {
  const list = $("#scenarioList");
  if (!list) return;
  list.innerHTML = "";
  const current = activeScenario();
  multiplayerScenarios.forEach((scenario) => {
    const card = document.createElement("button");
    card.type = "button";
    card.className = `scenario-card ${current.id === scenario.id ? "selected" : ""}`;
    card.innerHTML = `
      <div class="card-top"><strong>${scenario.name}</strong><span class="badge">${scenario.hotPursuit ? "Pursuit" :
scenario.allies ? "Local MP" : "Solo"}</span></div>
      <div class="tiny">${scenario.desc}</div>
```

```
    `;
    card.addEventListener("click", () => {
      selectedScenario = scenario;
      localStorage.setItem("velocityVaultScenarioMode", scenario.id);
      renderScenarios();
      renderHub();
      showToast(`${scenario.name} selected.`);
    });
    list.appendChild(card);
  });
}

function renderVehicles() {
  const list = $("#vehicleList");
  if (!list) return;
  list.innerHTML = "";
  const current = selectedVehicle();
  vehicleDefs.forEach((vehicle) => {
    const owned = vehicleOwned(activeProfile, vehicle.id);
    const unlocked = vehicleUnlocked(activeProfile, vehicle);
    const build = vehicleBuild(activeProfile, vehicle.id);
    const status = current.id === vehicle.id ? "Active" : owned ? "Select" : !unlocked ? `${vehicle.unlock} rep` :
`${vehicle.price} coins`;
    const card = document.createElement("button");
    card.type = "button";
    card.className = `vehicle-card ${current.id === vehicle.id ? "selected" : ""} ${!unlocked ? "locked" : ""}`;
    card.innerHTML = `
      <div class="vehicle-preview asset-preview" data-type="${vehicle.type}" style="--vehicle-color:${build.color ||
vehicle.color}">
        <canvas class="vehicle-preview-canvas" width="180" height="116" aria-hidden="true"></canvas>
      </div>
      <div class="card-top"><strong>${vehicle.name}</strong><span class="badge">${status}</span></div>
      <div class="tiny">${vehicle.desc} | ${vehicle.model ? `Model ${vehicle.model} | ` :
""}${compatibilityLabel(vehicle)} | Plate ${build.plate || sanitizePlate(activeProfile.name)} | Tap active vehicle to
repaint</div>
    `;
    drawVehiclePreview(card.querySelector(".vehicle-preview-canvas"), Object.assign({}, vehicle, { color: build.color
|| vehicle.color }));
    card.addEventListener("click", () => buyOrSelectVehicle(vehicle));
    list.appendChild(card);
  });
}

function drawVehiclePreview(canvasEl, vehicle) {
  if (!canvasEl || !vehicle) return;
  const previewCtx = canvasEl.getContext("2d");
  const w = canvasEl.width;
  const h = canvasEl.height;
  previewCtx.clearRect(0, 0, w, h);
  const road = previewCtx.createLinearGradient(0, 0, 0, h);
  road.addColorStop(0, "rgba(21,31,30,0.96)");
  road.addColorStop(0.58, "rgba(10,16,15,0.98)");
  road.addColorStop(1, "rgba(5,8,7,0.98)");
  previewCtx.fillStyle = road;
  previewCtx.fillRect(0, 0, w, h);
  previewCtx.strokeStyle = "rgba(244,251,248,0.12)";
  previewCtx.lineWidth = 2;
  previewCtx.beginPath();
  previewCtx.moveTo(w * 0.18, 0);
  previewCtx.lineTo(w * 0.08, h);
  previewCtx.moveTo(w * 0.82, 0);
  previewCtx.lineTo(w * 0.92, h);
  previewCtx.stroke();
  previewCtx.strokeStyle = "rgba(255,209,102,0.32)";
  previewCtx.setLineDash([12, 10]);
  previewCtx.beginPath();
  previewCtx.moveTo(w * 0.5, h * 0.08);
  previewCtx.lineTo(w * 0.5, h * 0.96);
  previewCtx.stroke();
  previewCtx.setLineDash([]);

  const assets = window.VelocityPhoneAssets;
  const sprite = assets && assets.ready && typeof assets.getVehicleSprite === "function"
    ? assets.getVehicleSprite(vehicle.type, vehicle.color, { damage: 0 })
    : null;
  const baseWidth = ["semi", "truck", "monster", "tank", "tractor"].includes(vehicle.type) ? 100 : 88;
  const spriteW = vehicle.type === "semi" ? 122 : baseWidth;
  const spriteH = vehicle.type === "semi" ? 104 : ["airplane", "helicopter"].includes(vehicle.type) ? 98 : 102;
  previewCtx.save();
  previewCtx.translate(w * 0.5, h * 0.56);
  drawPreviewGroundContact(previewCtx, spriteW, spriteH, vehicle.type);
  if (sprite) {
    previewCtx.imageSmoothingEnabled = true;
    previewCtx.drawImage(sprite, -spriteW / 2, -spriteH * 0.54, spriteW, spriteH);
```

```
  } else {
    previewCtx.fillStyle = vehicle.color;
    roundRect(-spriteW * 0.28, -spriteH * 0.42, spriteW * 0.56, spriteH * 0.8, 8);
    previewCtx.fill();
  }
  previewCtx.restore();
}

function drawPreviewGroundContact(previewCtx, w, h, vehicleType = "car") {
  previewCtx.save();
  const air = ["airplane", "helicopter"].includes(vehicleType);
  const water = vehicleType === "boat";
  const contactY = h * (air ? 0.46 : water ? 0.5 : 0.56);
  const shadow = previewCtx.createRadialGradient(0, contactY, w * 0.08, 0, contactY + h * 0.06, w * 0.64);
  shadow.addColorStop(0, air ? "rgba(0,0,0,0.28)" : "rgba(0,0,0,0.62)");
  shadow.addColorStop(1, "rgba(0,0,0,0)");
  previewCtx.fillStyle = shadow;
  previewCtx.beginPath();
  previewCtx.ellipse(0, contactY + h * 0.08, w * 0.62, h * 0.13, 0, 0, Math.PI * 2);
  previewCtx.fill();
  if (!air && !water) {
    previewCtx.fillStyle = "rgba(3,5,5,0.78)";
    previewCtx.beginPath();
    previewCtx.ellipse(-w * 0.28, h * 0.38, w * 0.14, h * 0.055, 0, 0, Math.PI * 2);
    previewCtx.ellipse(w * 0.28, h * 0.38, w * 0.14, h * 0.055, 0, 0, Math.PI * 2);
    previewCtx.ellipse(-w * 0.28, h * 0.65, w * 0.16, h * 0.07, 0, 0, Math.PI * 2);
    previewCtx.ellipse(w * 0.28, h * 0.65, w * 0.16, h * 0.07, 0, 0, Math.PI * 2);
    previewCtx.fill();
  }
  previewCtx.restore();
}

function renderUpgrades() {
  const list = $("#upgradeList");
  list.innerHTML = "";
  const director = getDirector(activeProfile);
  const build = vehicleBuild(activeProfile, activeProfile.selectedVehicle);
  upgradeDefs.forEach((def) => {
    const level = build.upgrades[def.id];
    const cost = upgradeCost(activeProfile, def);
    const card = document.createElement("div");
    card.className = "upgrade-card";
    card.innerHTML = `
      <div class="card-top"><strong>${def.name}</strong><span class="badge">${director.recommended.id === def.id ? "AI
 Pick" : `Lv ${level}/5`}</span></div>
      <div class="tiny">${def.desc}</div>
      <div class="upgrade-action">
        <div class="bar"><i></i></div>
        <button type="button" ${level >= 5 || activeProfile.coins < cost ? "disabled" : ""}>${level >= 5 ? "MAX" :
`Buy ${cost}`}</button>
      </div>
    `;
    card.querySelector(".bar i").style.width = `${(level / 5) * 100}%`;
    card.querySelector("button").addEventListener("click", () => buyUpgrade(def));
    list.appendChild(card);
  });
}

function renderMissions() {
  const list = $("#missionList");
  list.innerHTML = "";
  missionDefs.forEach((mission) => {
    const done = activeProfile.completedMissions.includes(mission.id);
    const ready = !done && mission.test(activeProfile);
    const card = document.createElement("div");
    card.className = "mission-card";
    card.innerHTML = `
      <div class="card-top"><strong>${mission.text}</strong><span class="badge">${done ? "Claimed" :
`${mission.reward} coins`}</span></div>
      <div class="tiny">${ready ? "Ready to claim." : done ? "Nice work." : "Keep racing to unlock this
reward."}</div>
      ${ready ? "<button class=\"primary\" type=\"button\">Claim Reward</button>" : ""}
    `;
    const claim = card.querySelector("button");
    if (claim) claim.addEventListener("click", () => claimMission(mission));
    list.appendChild(card);
  });
}

function buyUpgrade(def) {
  const build = vehicleBuild(activeProfile, activeProfile.selectedVehicle);
  const level = build.upgrades[def.id];
  if (level >= 5) return;
  const cost = upgradeCost(activeProfile, def);
```

```
  if (activeProfile.coins < cost) return showToast("More coins needed.");
  activeProfile.coins -= cost;
  build.upgrades[def.id] += 1;
  activeProfile.upgrades = Object.assign(defaultUpgrades(), build.upgrades);
  profiles[activeProfile.id] = activeProfile;
  saveProfiles();
  renderHub();
  updateHud();
  showToast(`${selectedVehicle().name} ${def.name} upgraded.`);
}

function claimMission(mission) {
  activeProfile.completedMissions.push(mission.id);
  activeProfile.coins += mission.reward;
  activeProfile.rep += 8;
  profiles[activeProfile.id] = activeProfile;
  saveProfiles();
  renderHub();
  updateHud();
  showToast("Mission reward claimed.");
}

function launchRace() {
  startAudio();
  if (audioSystem && audioSystem.ctx.state === "suspended") audioSystem.ctx.resume();
  const age = ageBands[activeProfile.age];
  const director = getDirector(activeProfile);
  const vehicle = selectedVehicle();
  const scenario = activeScenario();
  if (!vehicleOwned(activeProfile, vehicle.id)) {
    showToast("Buy or select an owned vehicle first.");
    renderVehicles();
    return;
  }
  if (!raceCompatibleWithVehicle(selectedRace, vehicle)) {
    selectedRace = firstCompatibleRace(vehicle, activeProfile);
    renderRaces();
    showToast(`${vehicle.name} switched to a ${compatibilityLabel(vehicle)} route.`);
  }
  Object.assign(raceState, {
    active: true,
    distance: 0,
    speed: 0,
    focus: 100,
    score: 0,
    coins: 0,
    dodges: 0,
    overtakes: 0,
    passesAgainst: 0,
    civilianHits: 0,
    penaltyCoins: 0,
    penaltyRep: 0,
    combo: 1,
    lane: 0,
    x: 0,
    visualLane: 0,
    lateralVelocity: 0,
    steerAngle: 0,
    throttleLoad: 0,
    brakeHeat: 0,
    slip: 0,
    damage: 0,
    damageAlertTimer: 0,
    damageAlertLabel: "",
    hazardWarningTimer: 0,
    hazardWarningLabel: "",
    resetTimer: 0,
    resetReason: "",
    crashCooldown: 0,
    roadCurve: 0,
    roadTurn: 0,
    roadOffset: 0,
    spawnClock: 0,
    coinClock: 0,
    civilianClock: 5.5,
    oncomingClock: 8,
    cannonCooldown: 0,
    routeFeatureClock: 4.5,
    hideCooldown: 0,
    scenarioId: scenario.id,
    scenarioLabel: scenario.name,
    teamScore: 0,
    rivals: [],
    police: [],
```

```
    opponents: makeOpponents(vehicle, age, director),
    coinsOnRoad: [],
    civilians: [],
    oncoming: [],
    routeFeatures: [],
    particles: [],
    elapsed: 0,
    heat: 0,
    heatClock: 1.7,
    chaseActive: false,
    cameraShake: 0,
    countdown: 0,
    goalIntroTimer: 5.8,
    finished: false,
    director
  });
  $("#raceTitle").textContent = selectedRace.name;
  const route = routeWorldInfo(selectedRace.place);
  $("#raceBrief").textContent = `Goal: ${raceGoalText()} | ${route.country}: ${route.scene} |
${compatibilityLabel(vehicle)} | ${scenario.name}`;
  $("#modeChip").textContent = `${route.country} | ${route.scene}`;
  $("#missionChip").textContent = `${vehicle.name} | ${scenario.name} | ${selectedRace.target}`;
  showView("race");
  saveProfiles();
  showToast(`${vehicle.name} on grid. ${raceGoalText()}`);
}

function saveRace() {
  if (!activeProfile) return;
  if (!raceState.active) {
    saveProfiles();
    showToast("Garage progress saved on this device.");
    renderSavedRaceButton();
    return;
  }
  const snapshot = {
    version: 1,
    savedAt: Date.now(),
    profileId: activeProfile.id,
    selectedRaceId: selectedRace.id,
    cameraMode,
    inputPaused: input.paused,
    state: {
      active: true,
      distance: raceState.distance,
      speed: raceState.speed,
      focus: raceState.focus,
      score: raceState.score,
      coins: raceState.coins,
      dodges: raceState.dodges,
      overtakes: raceState.overtakes,
      passesAgainst: raceState.passesAgainst,
      civilianHits: raceState.civilianHits,
      penaltyCoins: raceState.penaltyCoins,
      penaltyRep: raceState.penaltyRep,
      combo: raceState.combo,
      lane: raceState.lane,
      x: raceState.x,
      visualLane: raceState.visualLane,
      lateralVelocity: raceState.lateralVelocity,
      steerAngle: raceState.steerAngle,
      throttleLoad: raceState.throttleLoad,
      brakeHeat: raceState.brakeHeat,
      slip: raceState.slip,
      damage: raceState.damage,
      resetTimer: raceState.resetTimer,
      resetReason: raceState.resetReason,
      crashCooldown: raceState.crashCooldown,
      roadCurve: raceState.roadCurve,
      roadTurn: raceState.roadTurn,
      roadOffset: raceState.roadOffset,
      spawnClock: raceState.spawnClock,
      coinClock: raceState.coinClock,
      civilianClock: raceState.civilianClock,
      oncomingClock: raceState.oncomingClock,
      cannonCooldown: raceState.cannonCooldown,
      routeFeatureClock: raceState.routeFeatureClock,
      hideCooldown: raceState.hideCooldown,
      scenarioId: raceState.scenarioId,
      scenarioLabel: raceState.scenarioLabel,
      teamScore: raceState.teamScore,
      rivals: raceState.rivals,
      police: raceState.police,
      opponents: raceState.opponents,
```

```
      coinsOnRoad: raceState.coinsOnRoad,
      civilians: raceState.civilians,
      oncoming: raceState.oncoming,
      routeFeatures: raceState.routeFeatures,
      elapsed: raceState.elapsed,
      heat: raceState.heat,
      heatClock: raceState.heatClock,
      chaseActive: raceState.chaseActive,
      cameraShake: 0,
      countdown: 0,
      goalIntroTimer: 0,
      finished: false
    }
  };
  localStorage.setItem(saveKey, JSON.stringify(snapshot));
  saveProfiles();
  renderSavedRaceButton();
  showToast("Race saved on this device.");
}

function resumeSavedRace() {
  const saved = loadSavedRace();
  if (!saved || !activeProfile || saved.profileId !== activeProfile.id) {
    showToast("No saved race for this driver.");
    renderSavedRaceButton();
    return;
  }
  const race = races.find((item) => item.id === saved.selectedRaceId) || races[0];
  selectedRace = race;
  selectedScenario = scenarioById(saved.state.scenarioId || selectedScenario.id);
  setCameraMode(saved.cameraMode || cameraMode, true);
  const director = getDirector(activeProfile);
  Object.assign(raceState, saved.state, {
    active: true,
    particles: [],
    opponents: saved.state.opponents || makeOpponents(selectedVehicle(), ageBands[activeProfile.age], director),
    civilians: saved.state.civilians || [],
    oncoming: saved.state.oncoming || [],
    routeFeatures: saved.state.routeFeatures || [],
    director,
    cameraShake: 0
  });
  raceState.visualLane = Number(saved.state.visualLane);
  if (!Number.isFinite(raceState.visualLane)) raceState.visualLane = raceState.lane || 0;
  raceState.lateralVelocity = Number(saved.state.lateralVelocity) || 0;
  raceState.steerAngle = Number(saved.state.steerAngle) || 0;
  raceState.throttleLoad = Number(saved.state.throttleLoad) || 0;
  raceState.brakeHeat = Number(saved.state.brakeHeat) || 0;
  raceState.slip = Number(saved.state.slip) || 0;
  raceState.damage = Math.max(0, Math.min(100, Number(saved.state.damage) || 0));
  raceState.resetTimer = Math.max(0, Number(saved.state.resetTimer) || 0);
  raceState.resetReason = saved.state.resetReason || "";
  raceState.crashCooldown = Math.max(0, Number(saved.state.crashCooldown) || 0);
  raceState.goalIntroTimer = Math.max(0, Number(saved.state.goalIntroTimer) || 0);
  raceState.roadCurve = Number(saved.state.roadCurve) || 0;
  raceState.roadTurn = Number(saved.state.roadTurn) || raceState.roadCurve || 0;
  raceState.overtakes = Number(saved.state.overtakes) || 0;
  raceState.passesAgainst = Number(saved.state.passesAgainst) || 0;
  raceState.civilianHits = Number(saved.state.civilianHits) || 0;
  raceState.penaltyCoins = Number(saved.state.penaltyCoins) || 0;
  raceState.penaltyRep = Number(saved.state.penaltyRep) || 0;
  raceState.cannonCooldown = Number(saved.state.cannonCooldown) || 0;
  raceState.routeFeatureClock = Number(saved.state.routeFeatureClock) || 0;
  raceState.hideCooldown = Number(saved.state.hideCooldown) || 0;
  raceState.scenarioId = selectedScenario.id;
  raceState.scenarioLabel = selectedScenario.name;
  raceState.teamScore = Number(saved.state.teamScore) || 0;
  raceState.opponents.forEach((opponent) => {
    const gap = Number(opponent.distance) - raceState.distance;
    opponent.wasAhead = gap > 4;
    opponent.passCooldown = Math.max(0, Number(opponent.passCooldown) || 0);
    opponent.surgePhase = Number(opponent.surgePhase) || Math.random() * Math.PI * 2;
  });
  input.paused = Boolean(saved.inputPaused);
  $("#pauseBtn").textContent = input.paused ? "Play" : "Pause";
  $("#raceTitle").textContent = selectedRace.name;
  const route = routeWorldInfo(selectedRace.place);
  $("#raceBrief").textContent = `Goal: ${raceGoalText()} | ${route.country}: ${route.scene} |
${compatibilityLabel(selectedVehicle())} | ${selectedScenario.name}`;
  $("#modeChip").textContent = `${route.country} | ${route.scene}`;
  $("#missionChip").textContent = raceState.chaseActive ? `Police heat ${Math.round(raceState.heat)}% | Escape clean`
: `${selectedRace.target} | ${director.event.name}`;
  startAudio();
  if (audioSystem && audioSystem.ctx.state === "suspended") audioSystem.ctx.resume();
```

```
  showView("race");
  updateRaceUi();
  showToast("Saved race resumed.");
}

function endRace(manual = false) {
  if (!raceState.active) return;
  if (manual) {
    saveRace();
    raceState.active = false;
    updateHud();
    showView("garage");
    renderHub();
    return;
  }
  raceState.active = false;
  const success = missionProgress() >= 1 && raceState.focus > 0;
  const finishPosition = playerPosition();
  const age = ageBands[activeProfile.age];
  const directorReward = raceState.director ? raceState.director.reward : 1;
  const scenarioReward = scenarioById(raceState.scenarioId).reward || 1;
  const podiumBonus = finishPosition === 1 ? 120 : finishPosition <= 3 ? 60 : 0;
  const grossReward = Math.round((selectedRace.reward + raceState.coins * 5 + (success ? 70 : 18) + podiumBonus +
(raceState.teamScore || 0) * 0.08) * age.rewards * directorReward * scenarioReward);
  const reward = Math.max(0, grossReward - Math.round(raceState.penaltyCoins || 0));
  const grossRep = success ? selectedRace.rep + (finishPosition === 1 ? 8 : 0) : Math.ceil(selectedRace.rep / 3);
  const rep = Math.max(0, grossRep - Math.round(raceState.penaltyRep || 0));
  activeProfile.coins += reward;
  activeProfile.rep += rep;
  activeProfile.upgrades = Object.assign(defaultUpgrades(), activeVehicleUpgrades(activeProfile));
  activeProfile.stats.races += 1;
  activeProfile.stats.wins += success && finishPosition === 1 ? 1 : 0;
  activeProfile.stats.totalCoins += raceState.coins;
  activeProfile.stats.bestScore = Math.max(activeProfile.stats.bestScore, Math.round(raceState.score));
  if (raceState.focus >= 50) activeProfile.stats.steadyRuns += 1;
  profiles[activeProfile.id] = activeProfile;
  saveProfiles();
  clearSavedRace();
  $("#finishTitle").textContent = success && finishPosition === 1 ? "Race Won" : success ? "Challenge Cleared" : "Race
 Complete";
  $("#finishStats").innerHTML = `
    <div><span>Position</span><strong>${finishPosition}/${raceRankings().length}</strong></div>
    <div><span>Coins earned</span><strong>${reward}</strong></div>
    <div><span>Reputation</span><strong>+${rep}</strong></div>
    <div><span>Penalties</span><strong>${raceState.civilianHits || 0}</strong></div>
    <div><span>Overtakes</span><strong>${raceState.overtakes}</strong></div>
    <div><span>Crew score</span><strong>${Math.round(raceState.teamScore || 0)}</strong></div>
    <div><span>Score</span><strong>${Math.round(raceState.score)}</strong></div>
    <div><span>Damage</span><strong>${Math.round(raceState.damage)}%</strong></div>
    <div><span>Focus left</span><strong>${Math.max(0, Math.round(raceState.focus))}</strong></div>
  `;
  updateHud();
  renderHub();
  showView("finish");
}

function missionProgress() {
  if (selectedRace.type === "coins") return raceState.coins / selectedRace.goal;
  if (selectedRace.type === "focus") return raceState.focus / selectedRace.goal;
  if (selectedRace.type === "dodges") return raceState.dodges / selectedRace.goal;
  return raceState.score / selectedRace.goal;
}

function raceGoalText(race = selectedRace, vehicle = selectedVehicle()) {
  const route = routeWorldInfo(race.place);
  const scenario = activeScenario();
  const clean = vehicle.type === "tank" ? "clear the route" : vehicle.type === "boat" ? "hold the water line" :
["helicopter", "airplane"].includes(vehicle.type) ? "fly the gates" : "race the pack";
  const pursuit = scenario.hotPursuit ? "Police pressure climbs the longer you stay visible. " : "";
  return `${race.target}; ${clean} through ${route.scene}. ${pursuit}Use hideouts and shortcut branches when they
appear.`;
}

function updateHud() {
  $("#profileChip").textContent = activeProfile ? `${activeProfile.name} | ${ageBands[activeProfile.age].label}` : "No
 Driver";
  $("#coinStat").textContent = activeProfile ? activeProfile.coins : 0;
  $("#repLabel").textContent = raceState.active ? "Heat" : "Rep";
  $("#repStat").textContent = raceState.active ? Math.round(raceState.heat) : activeProfile ? activeProfile.rep : 0;
  $("#speedStat").textContent = Math.max(0, Math.round(raceState.speed));
  $("#focusStat").textContent = Math.max(0, Math.round(raceState.focus));
  const damageStat = $("#damageStat");
  if (damageStat) damageStat.textContent = raceState.active ? `${Math.round(raceState.damage)}%` : "0%";
  if (raceState.active) {
```

```
    const pos = playerPosition();
    const leader = raceRankings()[0];
    const fieldSize = raceRankings().length;
    const turnName = routeTurnName(raceState.roadTurn || raceState.roadCurve || 0);
    $("#missionChip").textContent = raceState.chaseActive
      ? `P${pos}/${fieldSize} | ${turnName} | Heat ${Math.round(raceState.heat)}%`
      : `P${pos}/${fieldSize} | ${turnName} | Leader ${leader.name}`;
    if ((raceState.teamScore || 0) > 0) $("#missionChip").textContent = `P${pos}/${fieldSize} | Crew
${Math.round(raceState.teamScore)} | ${turnName}`;
    if (raceState.speed < 8) $("#missionChip").textContent = "Hold Gas to accelerate | Brake to stop/reverse";
    if ((raceState.hazardWarningTimer || 0) > 0) $("#missionChip").textContent = raceState.hazardWarningLabel ||
"Traffic ahead";
    if ((raceState.damageAlertTimer || 0) > 0) $("#missionChip").textContent = raceState.damageAlertLabel || "Damage
taken";
    if (raceState.resetTimer > 0) $("#missionChip").textContent = `Vehicle reset in ${Math.ceil(raceState.resetTimer)}
 | ${raceState.resetReason}`;
  }
}

function setCameraMode(mode, quiet = false) {
  cameraMode = mode;
  localStorage.setItem("velocityVaultCameraMode", mode);
  $$(".camera-btn").forEach((btn) => btn.classList.toggle("active", btn.dataset.camera === mode));
  if (!quiet) {
    const labels = { chase: "Third-person chase camera", hood: "Low hood camera", cockpit: "Windshield driver view" };
    showToast(labels[mode]);
  }
}

function applyPhoneGraphicsDefaults() {
  const key = "velocityVaultPhoneGraphicsDefaultsV56";
  if (!phoneGraphicsActive() || localStorage.getItem(key)) return;
  rendererMode = "canvas";
  if (cameraMode === "chase") cameraMode = "hood";
  touchDriveMode = "hold";
  touchControlSize = "mini";
  localStorage.setItem("velocityVaultRendererMode", rendererMode);
  localStorage.setItem("velocityVaultCameraMode", cameraMode);
  localStorage.setItem("velocityVaultTouchDriveMode", touchDriveMode);
  localStorage.setItem("velocityVaultTouchControlSize", touchControlSize);
  localStorage.setItem(key, "applied");
}

function initWebGLRenderer() {
  if (!glCanvas || webglRenderer) return;
  try {
    webglRenderer = window.VelocityWebGLPipeline ? window.VelocityWebGLPipeline(glCanvas) : null;
  } catch {
    webglRenderer = null;
    rendererMode = "canvas";
    localStorage.setItem("velocityVaultRendererMode", rendererMode);
  }
}

function useWebGLRenderer() {
  if (phoneGraphicsActive()) return false;
  return rendererMode === "webgl" && webglRenderer && webglRenderer.ready;
}

function phoneGraphicsActive() {
  const assets = window.VelocityPhoneAssets;
  if (assets && typeof assets.isPhoneViewport === "function" && assets.isPhoneViewport()) return true;
  return window.matchMedia && window.matchMedia("(pointer: coarse)").matches;
}

function phoneCleanRoadActive() {
  return phoneGraphicsActive();
}

function setRendererMode(mode, quiet = false) {
  if (mode === "webgl" && phoneGraphicsActive()) mode = "canvas";
  rendererMode = mode;
  if (mode === "webgl") initWebGLRenderer();
  if (mode === "webgl" && !webglRenderer) rendererMode = "canvas";
  localStorage.setItem("velocityVaultRendererMode", rendererMode);
  $$(".renderer-btn").forEach((btn) => btn.classList.toggle("active", btn.dataset.renderer === rendererMode));
  $(".stage").classList.toggle("webgl", useWebGLRenderer());
  if (!quiet) showToast(useWebGLRenderer() ? "WebGL 3D renderer enabled." : "Classic 2D renderer enabled.");
}

function clearTouchDriveInputs() {
  input.left = false;
  input.right = false;
  input.touchSteer = 0;
```

```
  input.gas = false;
  input.brake = false;
  input.boost = false;
  mobileTouchState.active.clear();
  mobileTouchState.stickSteer = 0;
  mobileTouchState.driveStickActive = false;
  mobileTouchState.latchedStickControl = "";
  mobileTouchState.floatingStickId = null;
  mobileTouchState.floatingStickKey = "";
  mobileTouchState.floatingStickOriginX = 0;
  mobileTouchState.floatingStickOriginY = 0;
  mobileTouchState.floatingStickKnobX = 0;
  mobileTouchState.floatingStickKnobY = 0;
  setFloatingStickVisual(false);
}

function useFloatingStickControls() {
  return document.body.classList.contains("race-live") && touchControlSize === "mini";
}

function floatingStickRadius() {
  const stick = $(".drive-stick");
  if (!stick) return 58;
  const rect = stick.getBoundingClientRect();
  return Math.max(42, Math.min(72, Math.min(rect.width || 116, rect.height || 116) * 0.44));
}

function setFloatingStickVisual(active, originX = mobileTouchState.floatingStickOriginX, originY =
mobileTouchState.floatingStickOriginY, knobX = 0, knobY = 0) {
  const stick = $(".drive-stick");
  if (!stick) return;
  stick.classList.toggle("floating-stick-active", Boolean(active));
  stick.style.setProperty("--stick-origin-x", `${Math.round(originX)}px`);
  stick.style.setProperty("--stick-origin-y", `${Math.round(originY)}px`);
  stick.style.setProperty("--stick-knob-x", `${Math.round(knobX)}px`);
  stick.style.setProperty("--stick-knob-y", `${Math.round(knobY)}px`);
}

function floatingStickControlAt(clientX, clientY) {
  const radius = floatingStickRadius();
  const dx = clientX - mobileTouchState.floatingStickOriginX;
  const dy = clientY - mobileTouchState.floatingStickOriginY;
  const distance = Math.hypot(dx, dy);
  const limit = distance > radius ? radius / distance : 1;
  const knobX = dx * limit;
  const knobY = dy * limit;
  mobileTouchState.floatingStickKnobX = knobX;
  mobileTouchState.floatingStickKnobY = knobY;
  setFloatingStickVisual(true, mobileTouchState.floatingStickOriginX, mobileTouchState.floatingStickOriginY, knobX,
knobY);
  const steer = Math.max(-1, Math.min(1, dx / radius));
  const controls = ["stick"];
  if (dy > radius * 0.72) {
    controls.push("brake");
  } else {
    controls.push("gas");
  }
  if (steer < -0.07) controls.push("left");
  if (steer > 0.07) controls.push("right");
  if (Math.abs(steer) > 0.035) controls.push(`steer=${steer.toFixed(2)}`);
  return controls.join(":");
}

function beginFloatingStick(id, clientX, clientY) {
  if (!useFloatingStickControls()) return false;
  if (mobileTouchState.floatingStickId !== null && mobileTouchState.floatingStickId !== id) return false;
  const key = `floating-stick-${id}`;
  mobileTouchState.floatingStickId = id;
  mobileTouchState.floatingStickKey = key;
  mobileTouchState.floatingStickOriginX = clientX;
  mobileTouchState.floatingStickOriginY = clientY;
  mobileTouchState.latchedStickControl = "";
  mobileTouchState.active.set(key, floatingStickControlAt(clientX, clientY));
  applyMobileTouchSnapshot();
  return true;
}

function moveFloatingStick(id, clientX, clientY) {
  if (mobileTouchState.floatingStickId !== id) return false;
  mobileTouchState.active.set(mobileTouchState.floatingStickKey, floatingStickControlAt(clientX, clientY));
  applyMobileTouchSnapshot();
  return true;
}
```

```
function endFloatingStick(id) {
  if (mobileTouchState.floatingStickId !== id) return false;
  mobileTouchState.active.delete(mobileTouchState.floatingStickKey);
  mobileTouchState.floatingStickId = null;
  mobileTouchState.floatingStickKey = "";
  mobileTouchState.floatingStickKnobX = 0;
  mobileTouchState.floatingStickKnobY = 0;
  setFloatingStickVisual(false);
  applyMobileTouchSnapshot();
  return true;
}

function setTouchDriveMode(mode, quiet = false) {
  touchDriveMode = mode === "toggle" ? "toggle" : "hold";
  localStorage.setItem("velocityVaultTouchDriveMode", touchDriveMode);
  document.body.classList.toggle("touch-toggle", touchDriveMode === "toggle");
  const label = $("#touchModeLabel");
  const modeButton = $(".control-mode");
  if (label) label.textContent = touchDriveMode === "toggle" ? "Toggle" : "Hold";
  if (modeButton) modeButton.setAttribute("aria-pressed", touchDriveMode === "toggle" ? "true" : "false");
  if (!quiet) {
    clearTouchDriveInputs();
    updateRaceUi();
    showToast(touchDriveMode === "toggle" ? "Touch controls set to toggle. Tap Gas or Brake to hold it on." : "Touch
controls set to hold. Press and hold to drive.");
  }
}

function setTouchControlSize(size, quiet = false) {
  touchControlSize = size === "full" ? "full" : "mini";
  localStorage.setItem("velocityVaultTouchControlSize", touchControlSize);
  document.body.classList.toggle("control-mini", touchControlSize === "mini");
  const label = $("#touchSizeLabel");
  const sizeButton = $(".control-size");
  if (label) label.textContent = touchControlSize === "mini" ? "Full" : "Mini";
  if (sizeButton) {
    sizeButton.setAttribute("aria-pressed", touchControlSize === "mini" ? "true" : "false");
    sizeButton.setAttribute("aria-label", touchControlSize === "mini" ? "Switch to full controls" : "Switch to mini
controls");
  }
  if (!quiet) {
    clearTouchDriveInputs();
    updateRaceUi();
    showToast(touchControlSize === "mini" ? "Floating one-thumb joystick on. Drag anywhere on the driving screen." :
"Full controls on. Tap Mini for more screen space.");
  }
}

function setDriveInput(control, pressed) {
  if (control === "left") {
    input.left = pressed;
    if (pressed) input.right = false;
  } else if (control === "right") {
    input.right = pressed;
    if (pressed) input.left = false;
  } else if (control === "neutral") {
    input.left = false;
    input.right = false;
  } else if (control === "gas") {
    input.gas = pressed;
    if (pressed) input.brake = false;
  } else if (control === "brake") {
    input.brake = pressed;
    if (pressed) input.gas = false;
  } else if (control === "boost") {
    input.boost = pressed;
  }
}

function toggleDriveInput(control) {
  if (control === "left") {
    setDriveInput("left", !input.left);
  } else if (control === "right") {
    setDriveInput("right", !input.right);
  } else if (control === "neutral") {
    setDriveInput("neutral", true);
  } else if (control === "gas") {
    setDriveInput("gas", !input.gas);
  } else if (control === "brake") {
    setDriveInput("brake", !input.brake);
  } else if (control === "boost") {
    setDriveInput("boost", !input.boost);
  }
}
```

```
function isDriveStickControl(control) {
  return typeof control === "string" && control.startsWith("stick:");
}

function driveStickControlAt(stick, clientX, clientY) {
  const rect = stick.getBoundingClientRect();
  if (!rect.width || !rect.height) return "";
  const clampedX = Math.max(rect.left, Math.min(rect.right, clientX));
  const clampedY = Math.max(rect.top, Math.min(rect.bottom, clientY));
  const nx = (clampedX - rect.left) / rect.width - 0.5;
  const ny = (clampedY - rect.top) / rect.height - 0.5;
  const dead = 0.08;
  const controls = ["stick"];
  if (touchControlSize === "mini") {
    if (ny > 0.34) controls.push("brake");
    else controls.push("gas");
  } else {
    if (ny < -dead) controls.push("gas");
    if (ny > dead) controls.push("brake");
  }
  if (nx < -dead) controls.push("left");
  if (nx > dead) controls.push("right");
  if (controls.length === 1) controls.push("neutral");
  const steer = Math.max(-1, Math.min(1, nx / 0.42));
  if (Math.abs(steer) > dead) controls.push(`steer=${steer.toFixed(2)}`);
  return controls.join(":");
}

function addTouchControls(control, controls) {
  if (!control) return;
  if (!isDriveStickControl(control)) {
    controls.add(control);
    return;
  }
  control.split(":").forEach((part) => {
    if (part && part !== "stick") controls.add(part);
  });
}

function driveStickSteerValue(control) {
  if (!isDriveStickControl(control)) return 0;
  const analog = control.split(":").find((part) => part.startsWith("steer="));
  if (analog) {
    const value = Number(analog.slice(6));
    if (Number.isFinite(value)) return Math.max(-1, Math.min(1, value));
  }
  const parts = control.split(":");
  const left = parts.includes("left");
  const right = parts.includes("right");
  return right && !left ? 1 : left && !right ? -1 : 0;
}

function mobileControlAt(clientX, clientY) {
  if (document.body.classList.contains("control-mini")) {
    const miniStick = $(".drive-stick");
    if (miniStick) {
      const rect = miniStick.getBoundingClientRect();
      const margin = Math.max(24, Math.min(rect.width, rect.height) * 0.22);
      const insideStick =
        clientX >= rect.left - margin &&
        clientX <= rect.right + margin &&
        clientY >= rect.top - margin &&
        clientY <= rect.bottom + margin;
      if (insideStick) return driveStickControlAt(miniStick, clientX, clientY);
    }
  }
  const el = document.elementFromPoint(clientX, clientY);
  const stick = el && el.closest ? el.closest(".drive-stick") : null;
  if (stick && document.body.classList.contains("control-mini")) return driveStickControlAt(stick, clientX, clientY);
  const button = el && el.closest ? el.closest(".mobile-control") : null;
  return button && button.dataset ? button.dataset.control : "";
}

function applyMobileTouchSnapshot() {
  const hadDriveStick = mobileTouchState.driveStickActive;
  const controls = new Set();
  let hasDriveStick = false;
  let stickSteer = 0;
  if (touchDriveMode === "toggle" && mobileTouchState.latchedStickControl) {
    hasDriveStick = true;
    stickSteer = driveStickSteerValue(mobileTouchState.latchedStickControl);
    addTouchControls(mobileTouchState.latchedStickControl, controls);
  }
```

```
  mobileTouchState.active.forEach((control) => {
    if (isDriveStickControl(control)) {
      hasDriveStick = true;
      const value = driveStickSteerValue(control);
      if (Math.abs(value) > Math.abs(stickSteer)) stickSteer = value;
    }
    addTouchControls(control, controls);
  });
  if (touchDriveMode === "toggle" && !hasDriveStick && !hadDriveStick) return;
  mobileTouchState.driveStickActive = hasDriveStick;
  mobileTouchState.stickSteer = hasDriveStick ? stickSteer : 0;
  const neutral = controls.has("neutral");
  const left = controls.has("left");
  const right = controls.has("right");
  const gas = controls.has("gas");
  const brake = controls.has("brake");
  input.left = neutral ? false : left && !right;
  input.right = neutral ? false : right && !left;
  input.touchSteer = neutral ? 0 : mobileTouchState.stickSteer;
  input.gas = gas && !brake;
  input.brake = brake && !gas;
  input.boost = controls.has("boost");
  if (touchDriveMode === "toggle" && neutral) mobileTouchState.latchedStickControl = "";
  updateRaceUi();
}

function handleMobileTouchStart(event) {
  mobileTouchState.usingTouchEvents = true;
  event.preventDefault();
  startAudio();
  if (audioSystem && audioSystem.ctx.state === "suspended") audioSystem.ctx.resume();
  Array.from(event.changedTouches).forEach((touch) => {
    const el = document.elementFromPoint(touch.clientX, touch.clientY);
    if (useFloatingStickControls() && !(el && el.closest && el.closest(".mobile-control:not(.stick-up):not(.stick-
left):not(.stick-center):not(.stick-right):not(.stick-down)"))) {
      beginFloatingStick(touch.identifier, touch.clientX, touch.clientY);
      return;
    }
    if (useFloatingStickControls() && el && el.closest && el.closest(".drive-stick")) {
      beginFloatingStick(touch.identifier, touch.clientX, touch.clientY);
      return;
    }
    const control = mobileControlAt(touch.clientX, touch.clientY);
    if (!control) return;
    if (control === "size") {
      setTouchControlSize(touchControlSize === "mini" ? "full" : "mini");
      return;
    }
    if (control === "mode") {
      const now = performance.now();
      if (now - mobileTouchState.modeToggleAt > 260) {
        mobileTouchState.modeToggleAt = now;
        setTouchDriveMode(touchDriveMode === "toggle" ? "hold" : "toggle");
      }
      return;
    }
    if (touchDriveMode === "toggle") {
      if (isDriveStickControl(control)) {
        mobileTouchState.latchedStickControl = control.includes("neutral") ? "" : control;
        mobileTouchState.active.set(touch.identifier, control);
      } else {
        toggleDriveInput(control);
        updateRaceUi();
      }
      return;
    }
    mobileTouchState.active.set(touch.identifier, control);
  });
  applyMobileTouchSnapshot();
}

function handleMobileTouchMove(event) {
  event.preventDefault();
  Array.from(event.changedTouches).forEach((touch) => {
    if (moveFloatingStick(touch.identifier, touch.clientX, touch.clientY)) return;
    const previous = mobileTouchState.active.get(touch.identifier);
    const control = mobileControlAt(touch.clientX, touch.clientY);
    if (touchDriveMode === "toggle") {
      if (isDriveStickControl(control)) {
        mobileTouchState.latchedStickControl = control.includes("neutral") ? "" : control;
        mobileTouchState.active.set(touch.identifier, control);
      } else if (isDriveStickControl(previous)) {
        mobileTouchState.active.delete(touch.identifier);
      }
```

```
      return;
    }
    if (control && control !== "mode" && control !== "size") {
      mobileTouchState.active.set(touch.identifier, control);
    } else {
      mobileTouchState.active.delete(touch.identifier);
    }
  });
  applyMobileTouchSnapshot();
}

function handleMobileTouchEnd(event) {
  event.preventDefault();
  Array.from(event.changedTouches).forEach((touch) => {
    if (endFloatingStick(touch.identifier)) return;
    mobileTouchState.active.delete(touch.identifier);
  });
  applyMobileTouchSnapshot();
}

function bindDriveStickPointerControl() {
  const stick = $(".drive-stick");
  if (!stick) return;
  const pointerKey = "drive-stick-pointer";
  const applyStick = (event) => {
    if (event.pointerType === "touch" && mobileTouchState.usingTouchEvents) return;
    if (useFloatingStickControls()) return;
    if (touchControlSize !== "mini") return;
    event.preventDefault();
    const control = driveStickControlAt(stick, event.clientX, event.clientY);
    if (touchDriveMode === "toggle") mobileTouchState.latchedStickControl = control.includes("neutral") ? "" :
control;
    mobileTouchState.active.set(pointerKey, control);
    applyMobileTouchSnapshot();
  };
  const stopStick = (event) => {
    if (event && event.pointerType === "touch" && mobileTouchState.usingTouchEvents) return;
    if (event) event.preventDefault();
    mobileTouchState.active.delete(pointerKey);
    applyMobileTouchSnapshot();
    if (stick.releasePointerCapture && event && event.pointerId !== undefined) {
      try {
        stick.releasePointerCapture(event.pointerId);
      } catch {
        // Capture may already be released by the browser.
      }
    }
  };
  stick.addEventListener("pointerdown", (event) => {
    if (event.pointerType === "touch" && mobileTouchState.usingTouchEvents) return;
    startAudio();
    if (audioSystem && audioSystem.ctx.state === "suspended") audioSystem.ctx.resume();
    if (stick.setPointerCapture && event.pointerId !== undefined) {
      try {
        stick.setPointerCapture(event.pointerId);
      } catch {
        // Some browsers reject capture after synthetic pointer events.
      }
    }
    applyStick(event);
  });
  stick.addEventListener("pointermove", applyStick);
  stick.addEventListener("pointerup", stopStick);
  stick.addEventListener("pointercancel", stopStick);
  stick.addEventListener("lostpointercapture", stopStick);
}

function floatingStickPointerAllowed(event) {
  if (!useFloatingStickControls()) return false;
  if (event.pointerType === "touch" || event.pointerType === "pen") return true;
  const compactViewport = window.matchMedia && window.matchMedia("(max-width: 980px)").matches;
  return phoneGraphicsActive() || compactViewport;
}

function bindFloatingPhoneJoystick() {
  const stick = $(".drive-stick");
  const layer = $(".mobile-drive-controls");
  const targets = [canvas, stick, layer].filter(Boolean);
  const start = (event) => {
    if (!floatingStickPointerAllowed(event)) return;
    const target = event.target;
    if (target && target.closest && target.closest(".mobile-control:not(.stick-up):not(.stick-left):not(.stick-
center):not(.stick-right):not(.stick-down)")) return;
    event.preventDefault();
```

```
    startAudio();
    if (audioSystem && audioSystem.ctx.state === "suspended") audioSystem.ctx.resume();
    const id = `pointer-${event.pointerId}`;
    if (!beginFloatingStick(id, event.clientX, event.clientY)) return;
    if (event.currentTarget.setPointerCapture && event.pointerId !== undefined) {
      try {
        event.currentTarget.setPointerCapture(event.pointerId);
      } catch {
        // Synthetic pointer events may not allow capture.
      }
    }
  };
  const move = (event) => {
    const id = `pointer-${event.pointerId}`;
    if (!moveFloatingStick(id, event.clientX, event.clientY)) return;
    event.preventDefault();
  };
  const stop = (event) => {
    const id = `pointer-${event.pointerId}`;
    if (!endFloatingStick(id)) return;
    event.preventDefault();
    if (event.currentTarget.releasePointerCapture && event.pointerId !== undefined) {
      try {
        event.currentTarget.releasePointerCapture(event.pointerId);
      } catch {
        // Capture may already be gone.
      }
    }
  };
  targets.forEach((target) => {
    target.addEventListener("pointerdown", start);
    target.addEventListener("pointermove", move);
    target.addEventListener("pointerup", stop);
    target.addEventListener("pointercancel", stop);
    target.addEventListener("lostpointercapture", stop);
  });
}

function registerOfflineApp() {
  if (!("serviceWorker" in navigator)) return;
  if (!location.protocol.startsWith("http")) return;
  navigator.serviceWorker.register("./sw.js").then(() => {
    showToast("Offline app mode ready.");
  }).catch(() => {
    showToast("Offline app mode needs HTTPS hosting.");
  });
}

function installApp() {
  const isStandalone = window.matchMedia("(display-mode: standalone)").matches || navigator.standalone;
  if (isStandalone) {
    showToast("Velocity Vault is already running as an app.");
    return;
  }
  if (deferredInstallPrompt) {
    deferredInstallPrompt.prompt();
    deferredInstallPrompt.userChoice.finally(() => {
      deferredInstallPrompt = null;
    });
    return;
  }
  if (!location.protocol.startsWith("http")) {
    showToast("Install needs the secure GitHub Pages link. Then use Share > Add to Home Screen on iPhone, or Install
on Android.");
    return;
  }
  showToast("iPhone: Safari Share > Add to Home Screen. Android/desktop: browser menu > Install / Add to Phone.");
}

function makeOpponents(playerVehicle, age, director) {
  const routeTypes = raceTrafficTypes(selectedRace, playerVehicle);
  const compatiblePool = vehicleDefs.filter((vehicle) => vehicle.id !== playerVehicle.id &&
(raceCompatibleWithVehicle(selectedRace, vehicle) || routeTypes.includes(vehicle.type)));
  const pool = compatiblePool.length ? compatiblePool : vehicleDefs.filter((vehicle) => vehicle.id !==
playerVehicle.id);
  const scenario = activeScenario();
  const grid = [
    { gap: 1080, lane: -1.95, bias: 0.58, speed: 28 },
    { gap: 5200, lane: 0.86, bias: 0.66, speed: 34 },
    { gap: -1800, lane: 1.34, bias: 0.74, speed: 32 },
    { gap: 9800, lane: 1.95, bias: 0.74, speed: 42 },
    { gap: -4500, lane: -1.28, bias: 0.84, speed: 38 },
    { gap: 14400, lane: -0.76, bias: 0.78, speed: 44 },
    { gap: -7200, lane: 1.95, bias: 0.86, speed: 40 },
```

```
    { gap: 18800, lane: 1.18, bias: 0.82, speed: 48 }
  ];
  const allyGrid = [
    { gap: -720, lane: 1.72, bias: 0.72, speed: 31 },
    { gap: -2380, lane: -1.72, bias: 0.82, speed: 36 }
  ];
  const roster = [
    ...opponentNames.slice(0, 5 + (scenario.extraOpponents || 0)).map((name) => ({ name, ally: false })),
    ...Array.from({ length: scenario.allies || 0 }, (_, index) => ({ name: `P${index + 2} Ghost`, ally: true }))
  ];
  let rivalIndex = 0;
  let allyIndex = 0;
  return roster.map((entry, index) => {
    const { name, ally } = entry;
    const vehicle = ally ? playerVehicle : pool[(rivalIndex * 2 + selectedRace.id.length) % pool.length];
    const slot = ally ? allyGrid[allyIndex % allyGrid.length] : grid[rivalIndex % grid.length];
    if (ally) allyIndex += 1;
    else rivalIndex += 1;
    const startGap = slot.gap + (Math.random() - 0.5) * 18;
    return {
      name,
      vehicleId: vehicle.id,
      lane: slot.lane,
      homeLane: slot.lane,
      distance: startGap,
      speed: slot.speed + index * 2,
      targetBias: slot.bias + Math.random() * 0.035 + (ally ? 0.035 : 0),
      focus: 100,
      damage: 0,
      wrecked: false,
      color: ally ? selectedVehicleColor(playerVehicle) : vehicle.color,
      multiplayerRole: ally ? "ally" : "rival",
      wobble: Math.random() * Math.PI * 2,
      surgePhase: Math.random() * Math.PI * 2,
      packPhase: Math.random() * Math.PI * 2,
      spreadGap: slot.gap,
      bumpCooldown: 0,
      passCooldown: 0,
      wasAhead: startGap > 0,
      laneVelocity: 0,
      spin: 0,
      finished: false,
      ageSpeed: age.speed,
      aiTune: director.traffic
    };
  });
}

function updateOpponents(dt, maxSpeed) {
  const length = raceLength();
  const playerVehicle = selectedVehicle();
  const fieldSpeed = maxSpeed / Math.max(0.68, playerVehicle.speed);
  raceState.opponents.forEach((opponent, index) => {
    if (opponent.finished) return;
    opponent.damage = Math.max(0, Math.min(100, Number(opponent.damage) || 0));
    opponent.laneVelocity = Number(opponent.laneVelocity) || 0;
    opponent.spin = Number(opponent.spin) || 0;
    opponent.passCooldown = Math.max(0, (Number(opponent.passCooldown) || 0) - dt);
    opponent.surgePhase = Number(opponent.surgePhase) || index * 1.3;
    opponent.packPhase = Number(opponent.packPhase) || index * 0.9;
    const vehicle = vehicleById(opponent.vehicleId);
    const routePush = routeVehicleBoost(selectedRace.place, vehicle.type, true);
    const gapToPlayer = opponent.distance - raceState.distance;
    let racePressure = gapToPlayer > 0
      ? Math.max(-0.07, -gapToPlayer / 2400)
      : Math.min(0.2, Math.abs(gapToPlayer) / 980);
    if (gapToPlayer < -20 && raceState.elapsed < 26) racePressure = Math.min(racePressure, 0.045);
    const duelSurge = Math.abs(gapToPlayer) < 260 ? Math.sin(raceState.elapsed * (1.05 + index * 0.08) +
opponent.surgePhase) * 0.085 : 0;
    let spreadPressure = 0;
    raceState.opponents.forEach((other) => {
      if (other === opponent || other.finished) return;
      const otherGap = other.distance - opponent.distance;
      const minGap = phoneGraphicsActive() ? 420 : 320;
      if (Math.abs(otherGap) < minGap) spreadPressure += otherGap >= 0 ? -0.075 : 0.075;
    });
    const damageDrag = Math.max(0.28, 1 - ((opponent.damage || 0) / 100) * 0.58);
    const vehiclePace = fieldSpeed * (0.78 + vehicle.speed * 0.24);
    const packPace = 0.92 + opponent.aiTune * 0.045 + racePressure + duelSurge + Math.max(-0.08, Math.min(0.08,
spreadPressure));
    const target = opponent.wrecked
      ? Math.max(18, maxSpeed * 0.16 * damageDrag)
      : vehiclePace * opponent.targetBias * routePush * packPace * damageDrag;
    opponent.speed += (target - opponent.speed) * Math.min(1, dt * (opponent.wrecked ? 0.45 : 0.85 + vehicle.handling
```

```
* 0.35));
    opponent.distance = Math.min(length + 80, opponent.distance + Math.max(0, opponent.speed) * dt);
    opponent.wobble += dt * (0.85 + index * 0.08);
    const homeLane = Number.isFinite(Number(opponent.homeLane)) ? Number(opponent.homeLane) : ((index % 5) - 2);
    const flowLane = homeLane * 0.82 + Math.sin(opponent.wobble + opponent.packPhase) * 0.42 +
Math.sin(opponent.wobble * 0.47 + index) * 0.18;
    const closePassWindow = gapToPlayer > -180 && gapToPlayer < 260;
    const sideFromPlayer = Math.sign(opponent.lane - raceState.lane) || (index % 2 ? -1 : 1);
    const passSide = closePassWindow ? sideFromPlayer : (index % 2 ? -1 : 1);
    const clearance = closePassWindow ? 1.86 : 1.08;
    const duelLane = Math.max(-2.08, Math.min(2.08, raceState.lane + passSide * clearance + Math.sin(opponent.wobble *
 1.6) * 0.14));
    const laneTarget = closePassWindow && !opponent.wrecked ? duelLane : flowLane;
    if (!opponent.wrecked) {
      opponent.lane += (laneTarget - opponent.lane) * dt * (closePassWindow ? 1.35 : 0.42) * vehicle.handling *
Math.max(0.42, damageDrag);
    }
    opponent.lane += (opponent.laneVelocity || 0) * dt;
    opponent.laneVelocity = (opponent.laneVelocity || 0) * Math.max(0, 1 - dt * (opponent.wrecked ? 0.75 : 1.35));
    opponent.lane = Math.max(-2.15, Math.min(2.15, opponent.lane));
    opponent.spin += (opponent.wrecked ? 1.8 : 0.2) * dt * Math.sign(opponent.laneVelocity || 1);
    opponent.bumpCooldown = Math.max(0, (opponent.bumpCooldown || 0) - dt);
    const phoneMode = phoneGraphicsActive();
    const laneOverlap = Math.abs(opponent.lane - raceState.lane);
    const wheelToWheel = Math.abs(opponent.distance - raceState.distance) < (phoneMode ? 42 : 54) && laneOverlap <
(phoneMode ? 0.72 : 0.78);
    if (wheelToWheel && opponent.bumpCooldown <= 0 && raceState.resetTimer <= 0) {
      opponent.bumpCooldown = phoneMode ? 1.45 : 1.15;
      const side = Math.sign(opponent.lane - raceState.lane) || (index % 2 ? -1 : 1);
      const shove = 0.82 - Math.min(0.82, laneOverlap);
      opponent.speed *= phoneMode ? 0.9 : 0.84;
      opponent.laneVelocity += side * (phoneMode ? 2.25 : 1.75) * (1 + shove);
      opponent.focus = Math.max(0, opponent.focus - (phoneMode ? 10 : 16));
      raceState.speed *= phoneMode ? 0.93 : 0.87;
      raceState.lateralVelocity -= side * (phoneMode ? 0.78 : 1.12) * (1 + shove * 0.65);
      raceState.lane = Math.max(-2.18, Math.min(2.18, raceState.lane - side * (phoneMode ? 0.055 : 0.07) * (1 +
shove)));
      raceState.cameraShake = Math.max(raceState.cameraShake, phoneMode ? 4 : 7);
      applyVehicleDamage((phoneMode ? 3.2 : 8) / Math.max(0.72, selectedVehicle().mass), `${opponent.name} side
contact`);
      applyOpponentDamage(opponent, (phoneMode ? 12 : 17) / Math.max(0.74, vehicle.mass), `${opponent.name} damaged`,
canvas.width / 2 + raceState.lane * laneWidth(), canvas.height * 0.76, opponent.color || "#ffd166");
      burst(canvas.width / 2 + raceState.lane * laneWidth(), canvas.height * 0.76, opponent.color || "#ffd166");
      playHitSound("impact");
    }
    if (opponent.multiplayerRole === "ally" && Math.abs(gapToPlayer) < 560 && !opponent.wrecked) {
      raceState.teamScore = (raceState.teamScore || 0) + dt * (10 + Math.max(0, raceState.speed) * 0.035);
    }
    const gapAfter = opponent.distance - raceState.distance;
    const previouslyAhead = opponent.wasAhead !== false;
    if (previouslyAhead && gapAfter < -2 && raceState.elapsed > 1.4) {
      raceState.overtakes = (raceState.overtakes || 0) + 1;
      raceState.score += 240 * raceState.combo;
      raceState.teamScore = (raceState.teamScore || 0) + (opponent.multiplayerRole === "ally" ? 80 : 120);
      raceState.combo = Math.min(5, raceState.combo + 0.22);
      opponent.passCooldown = 1.8;
      showToast(`Overtake: ${opponent.name}`);
      burst(canvas.width / 2 + raceState.lane * laneWidth(), canvas.height * 0.72, opponent.color || "#ffd166");
      playHitSound("coin");
    } else if (!previouslyAhead && gapAfter > 6 && opponent.passCooldown <= 0 && raceState.elapsed > 3) {
      raceState.passesAgainst = (raceState.passesAgainst || 0) + 1;
      raceState.combo = Math.max(1, raceState.combo * 0.94);
      opponent.passCooldown = 2.1;
      showToast(`${opponent.name} passed you. Draft back.`);
    }
    if (gapAfter > 3) opponent.wasAhead = true;
    if (gapAfter < -2) opponent.wasAhead = false;
    if (opponent.distance >= length) opponent.finished = true;
  });
  enforceOpponentSpacing(length, dt);
}

function phonePassLaneSlots(playerLane = raceState.lane) {
  const slots = [-1.95, -0.72, 0.72, 1.95];
  return slots.sort((a, b) => Math.abs(b - playerLane) - Math.abs(a - playerLane));
}

function enforceOpponentSpacing(length, dt) {
  const active = raceState.opponents
    .filter((opponent) => opponent && !opponent.finished)
    .sort((a, b) => a.distance - b.distance);
  const phoneMode = phoneGraphicsActive();
  const minGap = phoneMode ? 1500 : 1250;
  const laneSlots = phonePassLaneSlots();
```

```
  active.forEach((opponent, index) => {
    if (index > 0) {
      const previous = active[index - 1];
      const gap = opponent.distance - previous.distance;
      if (gap < minGap) {
        opponent.distance = Math.min(length + 80, previous.distance + minGap);
        opponent.speed = Math.max(opponent.speed, previous.speed * 0.98);
      }
    }
    const playerGap = opponent.distance - raceState.distance;
    if (playerGap > -420 && playerGap < 2200 && !opponent.wrecked) {
      const nearPlayerLane = Math.abs(opponent.lane - raceState.lane) < 0.9 && playerGap > -110 && playerGap < 520;
      const laneConflict = active.some((other) => {
        if (other === opponent || other.finished) return false;
        const otherGap = other.distance - opponent.distance;
        return Math.abs(otherGap) < (phoneMode ? 420 : 360) && Math.abs(other.lane - opponent.lane) < 0.72;
      });
      const openSlot = laneSlots.find((slot) => Math.abs(slot - raceState.lane) > 1.05 && active.every((other) => {
        if (other === opponent) return true;
        const otherGap = other.distance - raceState.distance;
        return Math.abs(otherGap - playerGap) > (phoneMode ? 420 : 360) || Math.abs(other.lane - slot) > 0.72;
      }));
      const currentHome = Number.isFinite(Number(opponent.homeLane)) ? Number(opponent.homeLane) : opponent.lane;
      const laneTarget = nearPlayerLane || laneConflict
        ? (openSlot ?? (opponent.lane < raceState.lane ? -1.95 : 1.95))
        : currentHome;
      opponent.homeLane = laneTarget;
      const laneRate = nearPlayerLane ? 2.4 : laneConflict ? 1.25 : 0.24;
      opponent.lane += (laneTarget - opponent.lane) * Math.min(1, dt * laneRate);
      if (nearPlayerLane) opponent.speed *= Math.max(0.96, 1 - dt * 0.16);
      opponent.lane = Math.max(-2.15, Math.min(2.15, opponent.lane));
    }
  });
}

function visiblePhoneTrafficCount() {
  if (!phoneGraphicsActive()) return 0;
  let count = 0;
  const inRange = (object) => {
    const gap = ensureRoadDistance(object) - raceState.distance;
    return gap > -180 && gap < 1150;
  };
  raceState.rivals.forEach((rival) => {
    if (!rival.wrecked && inRange(rival)) count += 1;
  });
  raceState.police.forEach((unit) => {
    if (!unit.wrecked && inRange(unit)) count += 1;
  });
  raceState.opponents.forEach((opponent) => {
    const gap = opponent.distance - raceState.distance;
    if (!opponent.wrecked && gap > -180 && gap < 1150) count += 1;
  });
  (raceState.oncoming || []).forEach((unit) => {
    if (!unit.wrecked && inRange(unit)) count += 1;
  });
  return count;
}

function choosePhoneSpawnLane() {
  const slots = phonePassLaneSlots();
  const busy = (lane) => {
    const closeToPlayer = Math.abs(lane - raceState.lane) < 1.05;
    if (closeToPlayer) return true;
    return raceState.opponents.some((opponent) => {
      const gap = opponent.distance - raceState.distance;
      return gap > -160 && gap < 1200 && Math.abs(opponent.lane - lane) < 0.76;
    }) || raceState.rivals.some((rival) => {
      const gap = ensureRoadDistance(rival) - raceState.distance;
      return gap > -160 && gap < 1200 && Math.abs(rival.lane - lane) < 0.76;
    }) || raceState.police.some((unit) => {
      const gap = ensureRoadDistance(unit) - raceState.distance;
      return gap > -160 && gap < 1200 && Math.abs(unit.lane - lane) < 0.76;
    }) || (raceState.oncoming || []).some((unit) => {
      const gap = ensureRoadDistance(unit) - raceState.distance;
      return gap > -160 && gap < 1200 && Math.abs(unit.lane - lane) < 0.76;
    });
  };
  return slots.find((lane) => !busy(lane)) ?? slots.find((lane) => Math.abs(lane - raceState.lane) > 1.05) ?? -1.95;
}

function playerVehicleContact(object, kind = "traffic") {
  const objectLane = Number(object && object.lane) || 0;
  const laneGap = Math.abs(objectLane - raceState.lane);
  const gap = ensureRoadDistance(object) - raceState.distance;
```

```
  if (phoneGraphicsActive()) {
    const laneLimit = kind === "civilian" ? 0.24 : kind === "police" ? 0.32 : 0.3;
    const aheadLimit = kind === "civilian" ? 18 : kind === "police" ? 28 : 24;
    const behindLimit = kind === "civilian" ? 10 : kind === "police" ? 16 : 14;
    return {
      laneHit: laneGap < laneLimit,
      yHit: gap > -behindLimit && gap < aheadLimit,
      gap,
      laneGap
    };
  }
  const screenY = roadObjectY(object);
  const laneLimit = kind === "civilian" ? 0.3 : kind === "police" ? 0.42 : 0.4;
  const aheadLimit = kind === "civilian" ? 18 : kind === "police" ? 46 : 40;
  const behindLimit = kind === "civilian" ? 8 : kind === "police" ? 24 : 22;
  const screenAligned = screenY > canvas.height * 0.64 && screenY < canvas.height * 1.02;
  return {
    laneHit: laneGap < laneLimit,
    yHit: gap > -behindLimit && gap < aheadLimit && screenAligned,
    gap,
    screenY,
    laneGap
  };
}

function visiblePlayerVehicleContact(object, kind = "traffic") {
  const contact = playerVehicleContact(object, kind);
  const gap = Number.isFinite(contact.gap) ? contact.gap : ensureRoadDistance(object) - raceState.distance;
  const screenY = Number.isFinite(contact.screenY) ? contact.screenY : roadObjectY(object);
  const phoneMode = phoneGraphicsActive();
  const visibleTop = phoneMode ? canvas.height * 0.5 : canvas.height * 0.58;
  const visibleBottom = phoneMode ? canvas.height * 1.02 : canvas.height * 0.98;
  const aheadLimit = phoneMode
    ? (kind === "civilian" ? 18 : kind === "police" ? 26 : 22)
    : (kind === "civilian" ? 18 : kind === "police" ? 48 : 42);
  const rearForgiveness = phoneMode ? -8 : kind === "civilian" ? -8 : -26;
  const visible = screenY >= visibleTop && screenY <= visibleBottom;
  const avoidableOverlap = gap >= rearForgiveness && gap <= aheadLimit;
  const sideScrapeLane = kind === "civilian" ? 0.32 : kind === "police" ? (phoneMode ? 0.7 : 0.78) : (phoneMode ? 0.66
 : 0.74);
  const sideScrapeGap = gap >= (phoneMode ? -22 : -32) && gap <= (kind === "police" ? (phoneMode ? 46 : 60) :
(phoneMode ? 42 : 52));
  const sideScrape = kind !== "civilian" && contact.laneGap < sideScrapeLane && visible && sideScrapeGap;
  return {
    ...contact,
    gap,
    screenY,
    visible,
    hit: Boolean((contact.laneHit && contact.yHit && visible && avoidableOverlap) || sideScrape)
  };
}

function updateHazardWarning(object, kind = "traffic") {
  if (!raceState.active || raceState.resetTimer > 0) return;
  const gap = ensureRoadDistance(object) - raceState.distance;
  const laneGap = Math.abs((Number(object && object.lane) || 0) - raceState.lane);
  const screenY = roadObjectY(object);
  const phoneMode = phoneGraphicsActive();
  const warningRange = phoneMode ? 260 : 190;
  if (gap > 38 && gap < warningRange && laneGap < 0.96 && screenY > canvas.height * 0.42 && screenY < canvas.height *
0.88) {
    raceState.hazardWarningTimer = Math.max(raceState.hazardWarningTimer || 0, 0.35);
    raceState.hazardWarningLabel = kind === "police" ? "POLICE AHEAD - CHANGE LANES" : "CAR AHEAD - CHANGE LANES";
  }
}

function routeVehicleBoost(place, vehicleType, rival = false) {
  if (place === "snow" && vehicleType === "snowmobile") return rival ? 1.12 : 1.1;
  if (place === "harbor" && vehicleType === "boat") return rival ? 1.14 : 1.12;
  if (place === "airfield" && (vehicleType === "airplane" || vehicleType === "helicopter")) return rival ? 1.12 : 1.1;
  if (place === "freight" && (vehicleType === "semi" || vehicleType === "truck")) return rival ? 1.16 : 1.14;
  if (place === "farm" && vehicleType === "tractor") return rival ? 1.18 : 1.16;
  if (place === "tokyo" && (vehicleType === "car" || vehicleType === "f1" || vehicleType === "prototype")) return
rival ? 1.1 : 1.08;
  if (place === "desert" && (vehicleType === "monster" || vehicleType === "truck")) return rival ? 1.12 : 1.1;
  if (place === "rainforest" && (vehicleType === "truck" || vehicleType === "monster" || vehicleType === "tractor"))
return rival ? 1.1 : 1.08;
  if (place === "europe" && (vehicleType === "f1" || vehicleType === "prototype" || vehicleType === "car")) return
rival ? 1.1 : 1.08;
  return 1;
}

function raceTrafficTypes(race = selectedRace, vehicle = selectedVehicle()) {
  if (vehicle.type === "boat" || race.place === "harbor") return ["boat", "boat", "boat", "boat"];
```

```
  if (vehicle.type === "airplane" || vehicle.type === "helicopter") return ["airplane", "helicopter", "airplane",
"helicopter"];
  if (vehicle.type === "tank") return ["tank", "truck", "semi", "monster"];
  if (vehicle.type === "monster") return ["monster", "truck", "monster", "semi"];
  if (vehicle.type === "tractor") return ["tractor", "tractor", "truck"];
  if (vehicle.type === "semi") return ["semi", "semi", "truck"];
  if (vehicle.type === "snowmobile" || race.place === "snow") return ["snowmobile", "snowmobile", "truck"];
  if (race.place === "freight") return ["semi", "semi", "truck", "monster"];
  if (race.place === "farm") return ["tractor", "tractor", "truck", "monster"];
  if (race.place === "tokyo") return ["car", "f1", "prototype", "semi"];
  if (race.place === "desert") return ["monster", "truck", "semi", "car"];
  if (race.place === "rainforest") return ["truck", "monster", "tractor", "car"];
  return ["car", "f1", "prototype", "truck", "monster"];
}

function raceAllowsPolice(race = selectedRace, vehicle = selectedVehicle()) {
  if (activeScenario().hotPursuit) return true;
  if (race.place === "airfield" && vehicle.type === "tank") return true;
  return Boolean(race.hotPursuit || /hot pursuit/i.test(`${race.name} ${race.mood}`));
}

function pursuitIntensity() {
  const scenario = activeScenario();
  const routeProgress = raceLength() ? Math.max(0, Math.min(1, raceState.distance / raceLength())) : 0;
  const longRacePressure = Math.max(0, Math.min(1, raceState.elapsed / 160));
  const heatPressure = Math.max(0, Math.min(1, raceState.heat / 100));
  const scenarioPressure = scenario.hotPursuit ? 0.34 : scenario.heatBoost || 0;
  return Math.max(0, Math.min(1, heatPressure * 0.52 + longRacePressure * 0.24 + routeProgress * 0.18 +
scenarioPressure));
}

function raceHasOncomingTraffic(race = selectedRace, vehicle = selectedVehicle()) {
  if (["boat", "airplane", "helicopter", "tank"].includes(vehicle.type)) return false;
  return ["city", "tokyo", "coast", "desert", "europe"].includes(race.place);
}

function raceAllowsCivilians(race = selectedRace, vehicle = selectedVehicle()) {
  if (["airplane", "helicopter", "boat"].includes(vehicle.type)) return false;
  return !["airfield", "snow"].includes(race.place);
}

function raceIsNight(race = selectedRace) {
  return race.place === "city" || race.place === "tokyo" || /night|neon|tunnel/i.test(race.name + " " + race.mood);
}

function raceRankings() {
  const racers = [{ name: "You", distance: raceState.distance, player: true }];
  raceState.opponents.forEach((opponent) => racers.push({
    name: opponent.name,
    distance: opponent.distance,
    player: false
  }));
  racers.sort((a, b) => b.distance - a.distance);
  return racers;
}

function playerPosition() {
  return raceRankings().findIndex((racer) => racer.player) + 1;
}

function spawnRival() {
  const age = ageBands[activeProfile.age];
  const director = raceState.director || getDirector(activeProfile);
  const laneOptions = [-2, -1, 0, 1, 2].filter((candidate) => Math.abs(candidate - raceState.lane) > 0.72);
  const lane = phoneGraphicsActive() ? choosePhoneSpawnLane() : (laneOptions[Math.floor(Math.random() *
laneOptions.length)] ?? (Math.floor(Math.random() * 5) - 2));
  const routeTypes = raceTrafficTypes(selectedRace, selectedVehicle());
  const type = routeTypes[Math.floor(Math.random() * routeTypes.length)];
  const def = vehicleDefs.find((vehicle) => vehicle.type === type) || vehicleDefs[0];
  raceState.rivals.push({
    lane,
    distance: roadSpawnDistance(0.34, 0.43),
    w: type === "semi" ? 82 : type === "tractor" ? 64 : type === "monster" || type === "truck" ? 68 : 54,
    h: type === "airplane" || type === "helicopter" ? 104 : 92,
    speed: (38 + Math.random() * 58) * age.traffic * director.traffic,
    color: Math.random() > 0.45 ? def.color : (Math.random() > 0.5 ? "#ff5b6b" : "#ffd166"),
    type,
    passed: false,
    contactCooldown: 0,
    damage: 0,
    wrecked: false,
    laneVelocity: 0,
    spin: 0
  });
```

```
}

function spawnPoliceUnit() {
  const lane = phoneGraphicsActive() ? choosePhoneSpawnLane() : Math.floor(Math.random() * 5) - 2;
  const intensity = pursuitIntensity();
  const heavy = intensity > 0.68 && Math.random() < 0.38;
  raceState.police.push({
    lane,
    distance: roadSpawnDistance(0.34, 0.42),
    w: heavy ? 74 : 62,
    h: heavy ? 118 : 112,
    speed: (heavy ? 48 : 58) + Math.random() * 62 + raceState.heat * (heavy ? 0.2 : 0.26),
    type: heavy ? "truck" : "car",
    label: heavy ? "POLICE SUV" : "POLICE",
    passed: false,
    contactCooldown: 0,
    damage: 0,
    wrecked: false,
    laneVelocity: 0,
    spin: 0
  });
}

function spawnCivilian() {
  const side = Math.random() > 0.5 ? 1 : -1;
  const crossing = Math.random() < 0.26;
  raceState.civilians.push({
    lane: crossing ? side * (1.9 + Math.random() * 0.22) : side * (2.34 + Math.random() * 0.28),
    distance: roadSpawnDistance(0.24, 0.34),
    side,
    crossing,
    speed: crossing ? 0.34 + Math.random() * 0.26 : 0,
    step: Math.random() * Math.PI * 2,
    hit: false,
    warned: false
  });
}

function spawnOncomingTraffic() {
  const laneChoices = [-1.55, -0.48, 0.48, 1.55].filter((lane) => Math.abs(lane - raceState.lane) > 0.48);
  const routeTypes = raceTrafficTypes(selectedRace, selectedVehicle()).filter((type) => !["boat", "airplane",
"helicopter"].includes(type));
  const type = routeTypes[Math.floor(Math.random() * routeTypes.length)] || "car";
  const def = vehicleDefs.find((vehicle) => vehicle.type === type) || vehicleDefs[0];
  raceState.oncoming.push({
    lane: laneChoices[Math.floor(Math.random() * laneChoices.length)] || -1.55,
    distance: roadSpawnDistance(0.18, 0.3) + 360,
    speed: 54 + Math.random() * 44,
    w: type === "semi" ? 82 : type === "monster" || type === "truck" ? 68 : 56,
    h: type === "semi" ? 112 : 94,
    color: def.color,
    type,
    damage: 0,
    wrecked: false,
    contactCooldown: 0,
    laneVelocity: 0,
    spin: Math.PI
  });
}

function routeFeatureCatalog(race = selectedRace, vehicle = selectedVehicle()) {
  if (vehicle.type === "boat") {
    return [
      { type: "hide", label: "Marina Cover", detail: "duck behind docks", icon: "dock" },
      { type: "shortcut", label: "Canal Cut", detail: "side-water sprint", icon: "water" }
    ];
  }
  if (vehicle.type === "airplane" || vehicle.type === "helicopter") {
    return [
      { type: "hide", label: "Cloud Cover", detail: "drop heat in cloud banks", icon: "cloud" },
      { type: "shortcut", label: "Sky Gate", detail: "faster air corridor", icon: "gate" }
    ];
  }
  const byPlace = {
    city: [
      { type: "hide", label: "Parking Garage", detail: "building hideout", icon: "building" },
      { type: "hide", label: "Pole Cutoff", detail: "telephone pole shadow", icon: "pole" },
      { type: "shortcut", label: "Alley Branch", detail: "extra city road", icon: "road" }
    ],
    tokyo: [
      { type: "hide", label: "Neon Garage", detail: "building hideout", icon: "building" },
      { type: "shortcut", label: "Tunnel Branch", detail: "expressway slip road", icon: "road" }
    ],
    canyon: [
```

```
      { type: "hide", label: "Cave Pull-Off", detail: "cave hideout", icon: "cave" },
      { type: "shortcut", label: "Rock Cut", detail: "side canyon track", icon: "mountain" }
    ],
    alpine: [
      { type: "hide", label: "Mountain Tunnel", detail: "mountain cover", icon: "mountain" },
      { type: "shortcut", label: "Service Road", detail: "extra mountain road", icon: "road" }
    ],
    desert: [
      { type: "hide", label: "Dune Cave", detail: "desert cave cover", icon: "cave" },
      { type: "shortcut", label: "Wadi Track", detail: "extra desert track", icon: "road" }
    ],
    rainforest: [
      { type: "hide", label: "Canopy Cover", detail: "jungle hideout", icon: "building" },
      { type: "shortcut", label: "Bridge Bypass", detail: "extra jungle track", icon: "road" }
    ],
    freight: [
      { type: "hide", label: "Truck Stop", detail: "building hideout", icon: "building" },
      { type: "shortcut", label: "Service Lane", detail: "extra freight road", icon: "road" }
    ],
    farm: [
      { type: "hide", label: "Barn Cover", detail: "building hideout", icon: "building" },
      { type: "shortcut", label: "Field Track", detail: "extra dirt track", icon: "road" }
    ],
    airfield: [
      { type: "hide", label: "Hangar Cover", detail: "building hideout", icon: "building" },
      { type: "shortcut", label: "Taxiway Cut", detail: "extra runway lane", icon: "road" }
    ],
    snow: [
      { type: "hide", label: "Pine Shelter", detail: "snow cover", icon: "mountain" },
      { type: "shortcut", label: "Ice Cut", detail: "extra snow track", icon: "road" }
    ],
    coast: [
      { type: "hide", label: "Cliff Pull-Off", detail: "coastal cover", icon: "mountain" },
      { type: "shortcut", label: "Beach Road", detail: "extra coast road", icon: "road" }
    ],
    europe: [
      { type: "hide", label: "Village Arch", detail: "building hideout", icon: "building" },
      { type: "shortcut", label: "Switchback Cut", detail: "extra alpine road", icon: "road" }
    ]
  };
  return byPlace[race.place] || byPlace.city;
}

function spawnRouteFeature() {
  const catalog = routeFeatureCatalog();
  const wantsHide = (raceState.chaseActive || raceState.heat > 42 || activeScenario().hotPursuit) && Math.random() <
0.62;
  const candidates = catalog.filter((feature) => feature.type === (wantsHide ? "hide" : "shortcut"));
  const feature = (candidates.length ? candidates : catalog)[Math.floor(Math.random() * (candidates.length ?
candidates.length : catalog.length))];
  const side = Math.random() > 0.5 ? 1 : -1;
  raceState.routeFeatures.push({
    id: `${feature.type}:${Date.now()}:${Math.random()}`,
    type: feature.type,
    label: feature.label,
    detail: feature.detail,
    icon: feature.icon,
    lane: side * (feature.type === "hide" ? 2.1 : 1.82),
    distance: roadSpawnDistance(0.16, 0.28) + 260,
    side,
    pulse: Math.random() * Math.PI * 2,
    used: false,
    warned: false
  });
}

function updateRouteFeatures(dt) {
  raceState.routeFeatures = Array.isArray(raceState.routeFeatures) ? raceState.routeFeatures : [];
  raceState.routeFeatures.forEach((feature) => {
    feature.pulse = (feature.pulse || 0) + dt * 4;
    const gap = ensureRoadDistance(feature) - raceState.distance;
    const laneGap = Math.abs((feature.lane || 0) - raceState.lane);
    if (!feature.warned && gap > 36 && gap < 260 && laneGap < 1.25) {
      feature.warned = true;
      raceState.hazardWarningTimer = Math.max(raceState.hazardWarningTimer || 0, 0.75);
      raceState.hazardWarningLabel = feature.type === "hide" ? `${feature.label.toUpperCase()} - SLOW AND MOVE OVER` :
 `${feature.label.toUpperCase()} - CUT IN`;
    }
    if (feature.used || gap < -18 || gap > 44 || laneGap > 0.46) return;
    if (feature.type === "hide") {
      if (raceState.speed > 118) {
        raceState.hazardWarningTimer = Math.max(raceState.hazardWarningTimer || 0, 0.6);
        raceState.hazardWarningLabel = "TOO FAST TO HIDE - BRAKE";
        return;
```

```
      }
      feature.used = true;
      raceState.hideCooldown = 2.4;
      raceState.heat = Math.max(0, raceState.heat - (activeScenario().hotPursuit ? 34 : 26));
      raceState.chaseActive = raceState.heat > 20 && raceState.police.length > 0;
      raceState.focus = Math.min(100, raceState.focus + 5);
      raceState.score += 420;
      raceState.teamScore = (raceState.teamScore || 0) + 80;
      showToast(`${feature.label}: heat dropped.`);
      burst(canvas.width / 2 + raceState.lane * laneWidth(), canvas.height * 0.72, "#46d9ff");
    } else {
      feature.used = true;
      const gain = Math.min(460, Math.max(180, raceLength() * 0.018));
      raceState.distance = Math.max(raceState.distance, Math.min(raceLength() - 120, raceState.distance + gain));
      raceState.speed = Math.min(raceState.speed + 24, raceState.speed * 1.08 + 12);
      raceState.score += 520 * raceState.combo;
      raceState.combo = Math.min(5, raceState.combo + 0.28);
      raceState.teamScore = (raceState.teamScore || 0) + 90;
      showToast(`${feature.label}: shortcut gained.`);
      burst(canvas.width / 2 + raceState.lane * laneWidth(), canvas.height * 0.72, "#bbf24a");
    }
  });
  raceState.routeFeatures = raceState.routeFeatures.filter((feature) => roadObjectY(feature) < canvas.height + 220 &&
feature.distance > raceState.distance - 360 && !feature.used);
}

function routeFeatureSize(feature) {
  if (feature.type === "hide") return feature.icon === "pole" ? { w: 42, h: 108 } : { w: 92, h: 86 };
  return { w: 104, h: 42 };
}

function fireTankCannon() {
  raceState.cannonCooldown = 1.25;
  const targets = [
    ...raceState.rivals.map((target) => ({ target, group: "traffic", gap: ensureRoadDistance(target) -
raceState.distance })),
    ...raceState.police.map((target) => ({ target, group: "police", gap: ensureRoadDistance(target) -
raceState.distance })),
    ...raceState.oncoming.map((target) => ({ target, group: "traffic", gap: ensureRoadDistance(target) -
raceState.distance })),
    ...raceState.opponents.map((target) => ({ target, group: "opponent", gap: target.distance - raceState.distance }))
  ].filter((item) => item.target && !item.target.wrecked && item.gap > 18 && item.gap < 520 &&
Math.abs((item.target.lane || 0) - raceState.lane) < 0.86)
    .sort((a, b) => a.gap - b.gap);
  const hit = targets[0];
  const x = canvas.width / 2 + raceState.lane * laneWidth();
  const y = canvas.height * 0.68;
  burst(x, y, "#ffd166");
  playHitSound("police");
  raceState.focus = Math.max(0, raceState.focus - 2.5);
  if (!hit) {
    showToast("Tank cannon fired. No target in lane.");
    return;
  }
  if (hit.group === "opponent") {
    applyOpponentDamage(hit.target, 58, `${hit.target.name} disabled`, x, y, hit.target.color || "#ffd166");
  } else {
    applyTrafficDamage(hit.target, 72, hit.group === "police" ? "Interceptor disabled" : "Vehicle cleared", x, y,
hit.target.color || "#ffd166");
  }
  raceState.score += 260;
  showToast("Route cleared.");
}

function spawnCoin() {
  const lane = Math.floor(Math.random() * 5) - 2;
  raceState.coinsOnRoad.push({ lane, distance: roadSpawnDistance(0.35, 0.45), r: 13, pulse: Math.random() * 10 });
}

function tick(dt) {
  if (!raceState.active || input.paused) return;
  const upgrades = activeVehicleUpgrades(activeProfile);
  const age = ageBands[activeProfile.age];
  const director = raceState.director || getDirector(activeProfile);
  const vehicle = selectedVehicle();
  const scenario = activeScenario();
  const gasInput = input.gas || input.gamepadGas;
  const brakeInput = input.brake || input.gamepadBrake;
  const boostInput = input.boost || input.gamepadBoost;
  raceState.crashCooldown = Math.max(0, raceState.crashCooldown - dt);
  raceState.damageAlertTimer = Math.max(0, (raceState.damageAlertTimer || 0) - dt);
  raceState.hazardWarningTimer = Math.max(0, (raceState.hazardWarningTimer || 0) - dt);
  raceState.goalIntroTimer = Math.max(0, (raceState.goalIntroTimer || 0) - dt);
  if (raceState.resetTimer > 0) {
```

```
    raceState.resetTimer = Math.max(0, raceState.resetTimer - dt);
    raceState.speed *= Math.max(0, 1 - dt * 5.6);
    raceState.lateralVelocity *= Math.max(0, 1 - dt * 6.8);
    raceState.steerAngle *= Math.max(0, 1 - dt * 6.4);
    raceState.slip = Math.max(raceState.slip * 0.9, 0.55);
    raceState.cameraShake = Math.max(raceState.cameraShake, 2 + raceState.resetTimer * 1.8);
    emitDrivingEffects(dt, false, true, false, 0);
    if (raceState.resetTimer <= 0) completeVehicleReset();
    updateRaceUi();
    updateAudio();
    return;
  }
  const surfaceBoost = routeVehicleBoost(selectedRace.place, vehicle.type);
  const damageRatio = Math.min(0.86, (raceState.damage || 0) / 120);
  const performanceHealth = Math.max(0.42, 1 - damageRatio * 0.44);
  const handlingHealth = Math.max(0.48, 1 - damageRatio * 0.34);
  const maxSpeed = (245 + upgrades.engine * 26) * age.speed * vehicle.speed * surfaceBoost * performanceHealth;
  const keySteer = Number(input.right) - Number(input.left);
  const touchSteer = Number(input.touchSteer) || 0;
  const manualSteer = Math.abs(touchSteer) > Math.abs(keySteer) ? touchSteer : keySteer;
  const steerInput = Math.abs(input.gamepadSteer) > Math.abs(manualSteer) ? input.gamepadSteer : manualSteer;
  const boostPower = boostInput && gasInput && raceState.focus > 2 ? (60 + upgrades.boost * 17) * performanceHealth :
0;
  const speedLimit = maxSpeed + boostPower;
  const forwardSpeed = Math.max(0, raceState.speed);
  const reverseSpeed = Math.min(0, raceState.speed);
  const throttleAccel = (42 + upgrades.engine * 4.8) * age.speed * vehicle.speed * surfaceBoost * performanceHealth;
  const boostAccel = boostInput && gasInput && raceState.focus > 2 ? (54 + upgrades.boost * 9) * performanceHealth :
0;
  const brakeForce = ((88 + upgrades.tires * 5) / Math.max(0.72, vehicle.mass)) * Math.max(0.64, 1 - damageRatio *
0.24);
  const reverseAccel = 34 / Math.max(0.8, vehicle.mass);
  const rollingDrag = raceState.speed > 0 ? (10 + forwardSpeed * 0.035 + forwardSpeed * forwardSpeed * 0.00045) : (14
+ Math.abs(reverseSpeed) * 0.12);

  if (gasInput) {
    raceState.speed += (throttleAccel + boostAccel) * dt;
  }
  if (brakeInput) {
    raceState.speed -= (raceState.speed > 4 ? brakeForce : reverseAccel) * dt;
  }
  if (!gasInput && !brakeInput && Math.abs(raceState.speed) > 0.05) {
    const drag = Math.min(Math.abs(raceState.speed), rollingDrag * dt);
    raceState.speed += raceState.speed > 0 ? -drag : drag;
  }
  raceState.speed = Math.max(-38, Math.min(speedLimit, raceState.speed));
  raceState.throttleLoad += ((gasInput ? 1 : 0) - raceState.throttleLoad) * Math.min(1, dt * 4.4);
  raceState.brakeHeat += ((brakeInput && Math.abs(raceState.speed) > 12 ? 1 : 0) - raceState.brakeHeat) * Math.min(1,
dt * 5.2);
  if (boostInput && gasInput && raceState.focus > 2) raceState.focus -= dt * Math.max(4, 13 - upgrades.boost * 1.4);

  const speedGrip = Math.max(0.54, Math.min(1.18, Math.abs(raceState.speed) / 112));
  const steerTarget = steerInput * (0.92 + upgrades.tires * 0.045) * handlingHealth * (raceState.speed < -1 ? -0.62 :
1);
  raceState.steerAngle += (steerTarget - raceState.steerAngle) * Math.min(1, dt * (5.4 + vehicle.handling * 2.2) *
handlingHealth);
  const lateralAccel = raceState.steerAngle * speedGrip * (4.2 + vehicle.handling * 1.35 + upgrades.tires * 0.28) *
handlingHealth;
  raceState.lateralVelocity += lateralAccel * dt;
  const grip = (2.15 + vehicle.handling * 1.25 + upgrades.tires * 0.32 + (brakeInput ? 0.75 : 0)) * handlingHealth;
  raceState.lateralVelocity *= Math.max(0, 1 - dt * grip);
  if (Math.abs(steerInput) > 0.05 && Math.abs(raceState.speed) < 46) {
    raceState.lateralVelocity += steerInput * dt * (2.1 + vehicle.handling * 0.48) * handlingHealth;
  }
  raceState.lane += raceState.lateralVelocity * dt;
  raceState.x = raceState.lane;
  const targetVisualLane = raceState.lane;
  raceState.visualLane = Number.isFinite(raceState.visualLane) ? raceState.visualLane : targetVisualLane;
  raceState.visualLane += (targetVisualLane - raceState.visualLane) * Math.min(1, dt * 8.5);
  const offRoad = Math.max(0, Math.abs(raceState.lane) - 2.04);
  if (offRoad > 0) {
    raceState.lane = Math.max(-2.28, Math.min(2.28, raceState.lane));
    raceState.lateralVelocity -= Math.sign(raceState.lane) * offRoad * 6.5 * dt;
    raceState.speed *= Math.max(0.985, 1 - dt * (0.22 + offRoad * 0.24));
    raceState.focus -= dt * offRoad * 1.8;
    if (Math.abs(raceState.speed) > 90) {
      raceState.hazardWarningTimer = Math.max(raceState.hazardWarningTimer || 0, 0.45);
      raceState.hazardWarningLabel = "ROAD EDGE - STEER BACK";
    }
  }
  raceState.slip = Math.min(1, Math.abs(raceState.lateralVelocity) * 0.42 + Math.abs(raceState.steerAngle) * speedGrip
 * 0.18 + raceState.brakeHeat * 0.22 + offRoad * 0.5);
  raceState.distance = Math.max(0, raceState.distance + Math.max(0, raceState.speed) * dt);
  raceState.roadOffset += raceState.speed * dt;
```

```
  raceState.elapsed += dt;
  const turnTarget = roadTurnAt(raceState.distance);
  raceState.roadTurn += (turnTarget - (raceState.roadTurn || 0)) * Math.min(1, dt * 0.72);
  raceState.roadCurve += ((raceState.roadTurn || 0) - (raceState.roadCurve || 0)) * Math.min(1, dt * 0.65);
  raceState.score += dt * Math.max(0, raceState.speed) * raceState.combo * 0.32;
  raceState.heat = Math.min(100, raceState.heat + dt * (gasInput ? 0.8 + Math.max(0, raceState.speed) / 185 +
scenario.heatBoost : 0.1) + (boostInput && gasInput ? dt * 2.4 : 0));
  raceState.heatClock -= dt;
  raceState.cameraShake = Math.max(0, raceState.cameraShake - dt * 18);
  emitDrivingEffects(dt, gasInput, brakeInput, boostInput, steerInput);
  updateOpponents(dt, maxSpeed);
  raceState.spawnClock -= dt;
  raceState.coinClock -= dt;
  raceState.civilianClock = Math.max(0, (raceState.civilianClock || 0) - dt);
  raceState.oncomingClock = Math.max(0, (raceState.oncomingClock || 0) - dt);
  raceState.cannonCooldown = Math.max(0, (raceState.cannonCooldown || 0) - dt);
  raceState.routeFeatureClock = Math.max(0, (raceState.routeFeatureClock || 0) - dt);
  raceState.hideCooldown = Math.max(0, (raceState.hideCooldown || 0) - dt);
  const phoneMode = phoneGraphicsActive();
  const trafficRoom = !phoneMode || visiblePhoneTrafficCount() < 3;
  if (raceState.spawnClock <= 0 && raceState.speed > 45 && raceState.elapsed > 5 && trafficRoom) {
    spawnRival();
    raceState.spawnClock = phoneMode
      ? Math.max(3.4, 4.8 - age.traffic * director.traffic * 0.18)
      : Math.max(1.15, 2.1 - age.traffic * director.traffic * 0.16 - raceState.elapsed * 0.002);
  } else if (phoneMode && raceState.spawnClock <= 0) {
    raceState.spawnClock = 1.4;
  }
  if (raceState.coinClock <= 0 && raceState.speed > 30) {
    spawnCoin();
    raceState.coinClock = Math.max(0.78, (1.15 - upgrades.magnet * 0.045) / Math.max(0.8, director.coinRate * 0.82));
  }
  if (raceAllowsCivilians(selectedRace, vehicle) && raceState.civilianClock <= 0 && raceState.speed > 34) {
    spawnCivilian();
    raceState.civilianClock = 8.5 + Math.random() * 10 + (phoneMode ? 3 : 0);
  }
  if (raceHasOncomingTraffic(selectedRace, vehicle) && raceState.oncomingClock <= 0 && raceState.speed > 55 &&
(!phoneMode || visiblePhoneTrafficCount() < 3)) {
    spawnOncomingTraffic();
    raceState.oncomingClock = 7.5 + Math.random() * 8;
  }
  if (vehicle.type === "tank" && boostInput && raceState.cannonCooldown <= 0) fireTankCannon();
  if (raceState.routeFeatureClock <= 0 && raceState.speed > 38 && raceState.elapsed > 7) {
    spawnRouteFeature();
    raceState.routeFeatureClock = 8 + Math.random() * 8 + (phoneMode ? 3 : 0);
  }
  updateRouteFeatures(dt);
  const policeEligible = raceAllowsPolice(selectedRace, vehicle) || raceState.heat > 82;
  const intensity = pursuitIntensity();
  const policeLimit = phoneMode ? (intensity > 0.75 ? 3 : 2) : (intensity > 0.72 ? 5 : intensity > 0.48 ? 3 : 2);
  if (policeEligible && raceState.heat > 38 && raceState.heatClock <= 0 && raceState.speed > 45 &&
raceState.police.length < policeLimit && (!phoneMode || visiblePhoneTrafficCount() < 4)) {
    raceState.chaseActive = true;
    const spawnCount = !phoneMode && intensity > 0.68 && raceState.police.length < policeLimit - 1 ? 2 : 1;
    for (let i = 0; i < spawnCount; i += 1) spawnPoliceUnit();
    raceState.heatClock = Math.max(1.9, 9.2 - intensity * 6.1 - upgrades.engine * 0.08);
    showToast(intensity > 0.72 ? "Hot pursuit escalating. More units inbound." : "Pursuit unit entering the route.");
  }
  moveObjects(dt);
  if (raceState.focus <= 0) {
    raceState.focus = 18;
    raceState.damage = Math.max(raceState.damage || 0, 82);
    triggerVehicleReset("Focus recovered");
  }
  if (raceState.distance >= raceLength() && raceState.elapsed >= minimumRaceSeconds) endRace(false);
  updateRaceUi();
  updateAudio();
}

function moveObjects(dt) {
  const carLane = raceState.lane;
  const vehicle = selectedVehicle();
  raceState.rivals.forEach((rival) => {
    ensureRoadDistance(rival, -120);
    rival.damage = Math.max(0, Math.min(100, Number(rival.damage) || 0));
    rival.laneVelocity = Number(rival.laneVelocity) || 0;
    rival.spin = Number(rival.spin) || 0;
    rival.contactCooldown = Math.max(0, (rival.contactCooldown || 0) - dt);
    const damageDrag = Math.max(0.24, 1 - ((rival.damage || 0) / 100) * 0.66);
    const beforeY = roadObjectY(rival);
    if (rival.wrecked) {
      rival.speed *= Math.max(0, 1 - dt * 1.65);
      rival.spin += dt * (2.2 + Math.abs(rival.laneVelocity || 0)) * Math.sign(rival.laneVelocity || 1);
    } else {
```

```
      rival.speed *= Math.max(0, 1 - dt * (1 - damageDrag) * 0.18);
      const avoid = beforeY > canvas.height * 0.34 && beforeY < canvas.height * 0.7 && Math.abs(rival.lane - carLane)
< 0.85
        ? Math.sign(rival.lane - carLane || 1) * 0.55
        : 0;
      rival.laneVelocity += avoid * dt;
    }
    rival.lane += (rival.laneVelocity || 0) * dt;
    rival.laneVelocity = (rival.laneVelocity || 0) * Math.max(0, 1 - dt * 1.2);
    rival.lane = Math.max(-2.24, Math.min(2.24, rival.lane));
    rival.distance += Math.max(8, rival.speed * damageDrag) * dt;
    const screenY = roadObjectY(rival);
    if (!rival.passed && !rival.wrecked && screenY > canvas.height * 0.72) {
      rival.passed = true;
      raceState.dodges += 1;
      raceState.combo = Math.min(5, raceState.combo + 0.12);
    }
    updateHazardWarning(rival, "traffic");
    const contact = visiblePlayerVehicleContact(rival, "traffic");
    if (contact.hit && rival.contactCooldown <= 0 && raceState.resetTimer <= 0) {
      const phoneMode = phoneGraphicsActive();
      rival.contactCooldown = phoneMode ? 1.35 : 0.55;
      const shield = activeVehicleUpgrades(activeProfile).shield;
      const impact = Math.max(phoneMode ? 3.2 : 8, ((28 - shield * 2.7) * (phoneMode ? 0.28 : 1)) / Math.max(0.72,
vehicle.mass));
      const side = Math.sign(rival.lane - carLane) || (Math.random() > 0.5 ? 1 : -1);
      applyVehicleDamage(impact, "Visible traffic contact");
      applyTrafficDamage(rival, impact * 1.65, "Traffic disabled", canvas.width / 2 + carLane * laneWidth(),
canvas.height * 0.76, rival.color || "#ff5b6b");
      raceState.combo = 1;
      raceState.cameraShake = Math.max(raceState.cameraShake, phoneMode ? 4 : 7);
      raceState.speed *= Math.max(phoneMode ? 0.68 : 0.42, (phoneMode ? 0.88 : 0.78) - impact * 0.007);
      raceState.lateralVelocity -= side * (phoneMode ? 0.62 : 1.05);
      rival.laneVelocity += side * (phoneMode ? 1.75 : 1.35);
      rival.speed *= Math.max(phoneMode ? 0.42 : 0.24, (phoneMode ? 0.72 : 0.62) - impact * 0.006);
      burst(canvas.width / 2 + carLane * laneWidth(), canvas.height * 0.76, "#ff5b6b");
      playHitSound("impact");
    }
  });
  raceState.police.forEach((unit) => {
    ensureRoadDistance(unit, -170);
    unit.damage = Math.max(0, Math.min(100, Number(unit.damage) || 0));
    unit.laneVelocity = Number(unit.laneVelocity) || 0;
    unit.spin = Number(unit.spin) || 0;
    unit.contactCooldown = Math.max(0, (unit.contactCooldown || 0) - dt);
    const damageDrag = Math.max(0.22, 1 - ((unit.damage || 0) / 100) * 0.62);
    if (unit.wrecked) {
      unit.speed *= Math.max(0, 1 - dt * 1.45);
      unit.spin += dt * (2.4 + Math.abs(unit.laneVelocity || 0)) * Math.sign(unit.laneVelocity || 1);
    } else {
      const pursuitPull = Math.sign(carLane - unit.lane) * dt * (0.18 + raceState.heat / 260) * damageDrag;
      unit.lane += pursuitPull;
    }
    unit.lane += (unit.laneVelocity || 0) * dt;
    unit.laneVelocity = (unit.laneVelocity || 0) * Math.max(0, 1 - dt * 1.05);
    unit.lane = Math.max(-2.2, Math.min(2.2, unit.lane));
    unit.distance += Math.max(8, unit.speed * damageDrag) * dt;
    const screenY = roadObjectY(unit);
    if (!unit.passed && !unit.wrecked && screenY > canvas.height * 0.72) {
      unit.passed = true;
      raceState.dodges += 1;
      raceState.combo = Math.min(5, raceState.combo + 0.2);
      raceState.score += 180;
      raceState.heat = Math.max(10, raceState.heat - 4);
    }
    updateHazardWarning(unit, "police");
    const contact = visiblePlayerVehicleContact(unit, "police");
    if (contact.hit && unit.contactCooldown <= 0 && raceState.resetTimer <= 0) {
      const phoneMode = phoneGraphicsActive();
      unit.contactCooldown = phoneMode ? 1.45 : 0.65;
      const shield = activeVehicleUpgrades(activeProfile).shield;
      const impact = Math.max(phoneMode ? 4.5 : 12, ((38 - shield * 2.5) * (phoneMode ? 0.26 : 1)) / Math.max(0.72,
vehicle.mass));
      const side = Math.sign(unit.lane - carLane) || (Math.random() > 0.5 ? 1 : -1);
      applyVehicleDamage(impact, "Visible police contact");
      applyTrafficDamage(unit, impact * 1.25, "Interceptor damaged", canvas.width / 2 + carLane * laneWidth(),
canvas.height * 0.78, "#46d9ff");
      raceState.combo = 1;
      raceState.cameraShake = Math.max(raceState.cameraShake, phoneMode ? 6 : 12);
      raceState.heat = Math.min(100, raceState.heat + (phoneMode ? 7 : 12));
      raceState.speed *= Math.max(phoneMode ? 0.62 : 0.36, (phoneMode ? 0.84 : 0.72) - impact * 0.006);
      raceState.lateralVelocity -= side * (phoneMode ? 0.72 : 1.3);
      unit.laneVelocity += side * (phoneMode ? 1.8 : 1.5);
      unit.speed *= Math.max(phoneMode ? 0.38 : 0.22, (phoneMode ? 0.7 : 0.58) - impact * 0.004);
```

```
      burst(canvas.width / 2 + carLane * laneWidth(), canvas.height * 0.78, "#46d9ff");
      burst(canvas.width / 2 + carLane * laneWidth(), canvas.height * 0.78, "#ff3348");
      playHitSound("police");
    }
  });
  raceState.oncoming = Array.isArray(raceState.oncoming) ? raceState.oncoming : [];
  raceState.oncoming.forEach((unit) => {
    ensureRoadDistance(unit, -180);
    unit.damage = Math.max(0, Math.min(100, Number(unit.damage) || 0));
    unit.contactCooldown = Math.max(0, (unit.contactCooldown || 0) - dt);
    unit.laneVelocity = Number(unit.laneVelocity) || 0;
    unit.spin = Number(unit.spin) || Math.PI;
    const damageDrag = Math.max(0.26, 1 - ((unit.damage || 0) / 100) * 0.64);
    if (unit.wrecked) {
      unit.speed *= Math.max(0, 1 - dt * 1.5);
      unit.spin += dt * 2.1;
    } else {
      unit.distance -= Math.max(10, unit.speed * damageDrag) * dt;
      unit.lane += (unit.laneVelocity || 0) * dt;
      unit.laneVelocity *= Math.max(0, 1 - dt * 1.2);
    }
    updateHazardWarning(unit, "traffic");
    const contact = visiblePlayerVehicleContact(unit, "traffic");
    if (contact.hit && unit.contactCooldown <= 0 && raceState.resetTimer <= 0) {
      const phoneMode = phoneGraphicsActive();
      unit.contactCooldown = phoneMode ? 1.4 : 0.6;
      const shield = activeVehicleUpgrades(activeProfile).shield;
      const impact = Math.max(phoneMode ? 4 : 11, ((34 - shield * 2.4) * (phoneMode ? 0.3 : 1)) / Math.max(0.72,
vehicle.mass));
      const side = Math.sign(unit.lane - carLane) || (Math.random() > 0.5 ? 1 : -1);
      applyVehicleDamage(impact, "Oncoming traffic contact");
      applyTrafficDamage(unit, impact * 1.45, "Oncoming vehicle disabled", canvas.width / 2 + carLane * laneWidth(),
canvas.height * 0.76, unit.color || "#ffd166");
      raceState.combo = 1;
      raceState.cameraShake = Math.max(raceState.cameraShake, phoneMode ? 5 : 10);
      raceState.speed *= Math.max(phoneMode ? 0.6 : 0.32, (phoneMode ? 0.82 : 0.68) - impact * 0.006);
      raceState.lateralVelocity -= side * (phoneMode ? 0.7 : 1.25);
      unit.laneVelocity += side * 1.45;
      burst(canvas.width / 2 + carLane * laneWidth(), canvas.height * 0.76, unit.color || "#ffd166");
      playHitSound("impact");
    }
  });
  raceState.civilians = Array.isArray(raceState.civilians) ? raceState.civilians : [];
  raceState.civilians.forEach((person) => {
    ensureRoadDistance(person, -120);
    person.step = (person.step || 0) + dt * 6;
    if (person.crossing && !person.hit) {
      person.lane -= person.side * (person.speed || 0.42) * dt;
      if (Math.abs(person.lane) < 0.55 && !person.warned) {
        person.warned = true;
        raceState.hazardWarningTimer = Math.max(raceState.hazardWarningTimer || 0, 0.8);
        raceState.hazardWarningLabel = "CIVILIAN CROSSING - AVOID";
      }
    }
    const contact = visiblePlayerVehicleContact(person, "civilian");
    if (contact.hit && !person.hit && raceState.resetTimer <= 0) {
      person.hit = true;
      raceState.civilianHits = (raceState.civilianHits || 0) + 1;
      raceState.penaltyCoins = (raceState.penaltyCoins || 0) + 160;
      raceState.penaltyRep = (raceState.penaltyRep || 0) + 5;
      raceState.coins = Math.max(0, raceState.coins - 2);
      raceState.score = Math.max(0, raceState.score - 520);
      raceState.focus = Math.max(0, raceState.focus - 12);
      raceState.combo = 1;
      raceState.cameraShake = Math.max(raceState.cameraShake, 6);
      burst(canvas.width / 2 + raceState.lane * laneWidth(), canvas.height * 0.78, "#f4fbf8");
      showToast("Civilian hit. Reputation and coins penalized.");
      playHitSound("impact");
    }
  });
  const magnet = activeVehicleUpgrades(activeProfile).magnet;
  raceState.coinsOnRoad.forEach((coin) => {
    ensureRoadDistance(coin, -60);
    const screenY = roadObjectY(coin);
    coin.pulse += dt * 8;
    const catchRange = 0.25 + magnet * 0.065;
    const laneHit = Math.abs(coin.lane - carLane) < catchRange;
    const yHit = screenY > canvas.height * 0.62 && screenY < canvas.height * 0.9;
    if (laneHit && yHit && !coin.hit) {
      coin.hit = true;
      raceState.coins += 1;
      raceState.score += 120 * raceState.combo;
      raceState.combo = Math.min(5, raceState.combo + 0.18);
      burst(canvas.width / 2 + carLane * laneWidth(), canvas.height * 0.76, "#ffd166");
```

```
      playHitSound("coin");
    }
  });
  raceState.rivals = raceState.rivals.filter((r) => roadObjectY(r) < canvas.height + 220 && r.distance >
raceState.distance - 360);
  raceState.police = raceState.police.filter((p) => roadObjectY(p) < canvas.height + 240 && p.distance >
raceState.distance - 390);
  raceState.oncoming = raceState.oncoming.filter((p) => roadObjectY(p) < canvas.height + 260 && p.distance >
raceState.distance - 420);
  raceState.civilians = raceState.civilians.filter((p) => roadObjectY(p) < canvas.height + 160 && p.distance >
raceState.distance - 280 && !p.hit);
  if (!raceState.police.length && raceState.heat < 16) raceState.chaseActive = false;
  raceState.coinsOnRoad = raceState.coinsOnRoad.filter((c) => roadObjectY(c) < canvas.height + 80 && !c.hit);
  raceState.particles.forEach((p) => {
    p.x += p.vx * dt;
    p.y += p.vy * dt;
    p.vx *= Math.max(0, 1 - dt * 1.8);
    p.vy *= Math.max(0, 1 - dt * 1.2);
    p.life -= dt;
  });
  raceState.particles = raceState.particles.filter((p) => p.life > 0);
}

function applyOpponentDamage(opponent, amount, reason, x, y, color) {
  if (!opponent || amount <= 0) return;
  opponent.damage = Math.max(0, Math.min(100, (opponent.damage || 0) + amount));
  opponent.bumpCooldown = Math.max(opponent.bumpCooldown || 0, 0.7);
  opponent.focus = Math.max(0, (opponent.focus || 100) - amount * 0.85);
  if (opponent.damage >= 100 && !opponent.wrecked) {
    opponent.wrecked = true;
    opponent.speed *= 0.22;
    opponent.laneVelocity += (Math.random() > 0.5 ? 1 : -1) * 1.4;
    opponent.spin += 0.8;
    showToast(`${reason}. Rival limping.`);
  }
  burst(x, y, color || "#ffd166");
  if (opponent.damage > 48) emitVehicleSmoke(x, y, opponent.damage, color || "#888888");
}

function applyTrafficDamage(target, amount, reason, x, y, color) {
  if (!target || amount <= 0) return;
  target.damage = Math.max(0, Math.min(100, (target.damage || 0) + amount));
  if (target.damage >= 100 && !target.wrecked) {
    target.wrecked = true;
    target.speed *= 0.18;
    target.spin += 1.2;
    target.laneVelocity += (Math.random() > 0.5 ? 1 : -1) * 1.2;
    showToast(reason);
  }
  if (target.damage > 35) emitVehicleSmoke(x, y, target.damage, color || "#777777");
}

function emitVehicleSmoke(x, y, damage, color = "#777777") {
  const count = Math.min(10, 3 + Math.floor(damage / 18));
  for (let i = 0; i < count; i += 1) {
    const life = 0.45 + Math.random() * 0.5;
    raceState.particles.push({
      x: x + (Math.random() - 0.5) * 32,
      y: y + (Math.random() - 0.5) * 22,
      vx: (Math.random() - 0.5) * 80,
      vy: -40 - Math.random() * 80,
      life,
      maxLife: life,
      radius: 10 + Math.random() * 18,
      color: damage > 72 ? "rgba(22,22,20,0.48)" : "rgba(120,126,120,0.34)"
    });
  }
}

function applyVehicleDamage(amount, reason = "Impact", quiet = false) {
  if (!raceState.active || raceState.resetTimer > 0 || amount <= 0) return;
  const shield = activeProfile ? activeVehicleUpgrades(activeProfile).shield : 0;
  const vehicle = selectedVehicle();
  const massResist = Math.max(0.55, Math.min(1.15, 1.18 / Math.max(0.72, vehicle.mass)));
  const shieldResist = Math.max(0.58, 1 - shield * 0.055);
  const finalAmount = amount * massResist * shieldResist;
  raceState.damage = Math.max(0, Math.min(100, (raceState.damage || 0) + finalAmount));
  raceState.focus = Math.max(0, raceState.focus - Math.max(3, finalAmount * 0.64));
  raceState.crashCooldown = Math.max(raceState.crashCooldown, quiet ? 0.18 : 0.75);
  raceState.damageAlertTimer = Math.max(raceState.damageAlertTimer || 0, quiet ? 0.65 : 1.45);
  raceState.damageAlertLabel = `${reason}: +${Math.max(1, Math.round(finalAmount))}%`;
  if (!quiet) {
    const label = raceState.damage >= 100 ? "Critical damage" : `${reason}: ${Math.round(raceState.damage)}% damage`;
    showToast(label);
```

```
  }
  if (raceState.damage >= 100) triggerVehicleReset(reason);
}

function triggerVehicleReset(reason = "Critical damage") {
  if (!raceState.active || raceState.resetTimer > 0) return;
  raceState.resetTimer = 3.2;
  raceState.resetReason = reason;
  raceState.speed = Math.min(18, Math.max(-4, raceState.speed * 0.18));
  raceState.lateralVelocity = 0;
  raceState.steerAngle = 0;
  raceState.combo = 1;
  raceState.cameraShake = Math.max(raceState.cameraShake, 18);
  raceState.focus = Math.max(8, raceState.focus - 7);
  clearTouchDriveInputs();
  burst(canvas.width / 2 + raceState.lane * laneWidth(), canvas.height * 0.78, "#ff5b6b");
  burst(canvas.width / 2 + raceState.lane * laneWidth(), canvas.height * 0.78, "#ffd166");
  playHitSound("impact");
  showToast(`${reason}. Resetting vehicle.`);
}

function completeVehicleReset() {
  raceState.resetTimer = 0;
  raceState.resetReason = "";
  raceState.damage = Math.min(42, raceState.damage * 0.42);
  raceState.lane = 0;
  raceState.x = 0;
  raceState.visualLane = 0;
  raceState.speed = 0;
  raceState.lateralVelocity = 0;
  raceState.steerAngle = 0;
  raceState.slip = 0;
  raceState.crashCooldown = 1.2;
  showToast("Vehicle reset. Damage stabilized.");
}

function manualResetVehicle() {
  if (!raceState.active) return;
  if (raceState.resetTimer > 0) return;
  raceState.damage = Math.min(100, raceState.damage + 6);
  triggerVehicleReset("Manual reset");
}

function burst(x, y, color) {
  for (let i = 0; i < 16; i += 1) {
    const life = 0.35 + Math.random() * 0.35;
    raceState.particles.push({
      x,
      y,
      vx: (Math.random() - 0.5) * 240,
      vy: (Math.random() - 0.8) * 220,
      life,
      maxLife: life,
      radius: 4 + Math.random() * 4,
      color
    });
  }
}

function emitDrivingEffects(dt, gasInput, brakeInput, boostInput, steerInput) {
  if (!raceState.active || canvas.width <= 0 || canvas.height <= 0) return;
  const speed = Math.abs(raceState.speed);
  const x = canvas.width / 2 + (raceState.lane - cameraLaneOffset(1)) * laneWidth();
  const baseY = cameraMode === "cockpit" ? canvas.height * 0.78 : cameraMode === "hood" ? canvas.height * 0.93 :
canvas.height * 0.87;
  const slipChance = Math.min(0.75, raceState.slip * 0.34 + (brakeInput && speed > 24 ? 0.12 : 0) +
(Math.abs(steerInput) > 0 && speed > 90 ? 0.06 : 0));
  if (Math.random() < slipChance * dt * 18) {
    const side = Math.random() > 0.5 ? -1 : 1;
    const life = 0.35 + Math.random() * 0.28;
    raceState.particles.push({
      x: x + side * laneWidth() * 0.24 + (Math.random() - 0.5) * 18,
      y: baseY + Math.random() * 24,
      vx: -raceState.lateralVelocity * 34 + (Math.random() - 0.5) * 42,
      vy: -32 - Math.random() * 34,
      life,
      maxLife: life,
      radius: 10 + Math.random() * 18,
      color: brakeInput ? "rgba(190,205,205,0.42)" : "rgba(120,134,128,0.32)"
    });
  }
  const driftPower = Math.max(0, Math.min(1, ((raceState.slip || 0) - 0.16) * 1.95 +
Math.abs(raceState.lateralVelocity || 0) * 0.07));
  if (driftPower > 0.08 && speed > 36 && Math.random() < dt * (18 + driftPower * 36)) {
```

```
    const side = Math.sign(raceState.lateralVelocity || raceState.steerAngle || (Math.random() - 0.5)) || 1;
    const life = 0.42 + Math.random() * 0.38;
    const dusty = selectedRace.place === "desert" || selectedRace.place === "canyon" || selectedRace.place === "farm";
    raceState.particles.push({
      x: x - side * laneWidth() * (0.18 + Math.random() * 0.18),
      y: baseY + 8 + Math.random() * 18,
      vx: -side * (34 + Math.random() * 72) - raceState.lateralVelocity * 18,
      vy: -18 - Math.random() * 44,
      life,
      maxLife: life,
      radius: 16 + Math.random() * 34,
      color: dusty ? "rgba(255,183,74,0.28)" : "rgba(205,218,214,0.34)"
    });
  }
  if (boostInput && gasInput && raceState.focus > 2 && Math.random() < dt * 22) {
    const life = 0.25 + Math.random() * 0.18;
    raceState.particles.push({
      x: x + (Math.random() - 0.5) * laneWidth() * 0.5,
      y: baseY + 18 + Math.random() * 22,
      vx: (Math.random() - 0.5) * 36,
      vy: 80 + Math.random() * 90,
      life,
      maxLife: life,
      radius: 12 + Math.random() * 16,
      color: "rgba(255,209,102,0.58)"
    });
  }
  if (gasInput && speed < 20 && Math.random() < dt * 8) {
    const life = 0.28 + Math.random() * 0.22;
    raceState.particles.push({
      x: x + (Math.random() - 0.5) * laneWidth() * 0.42,
      y: baseY + 12,
      vx: (Math.random() - 0.5) * 24,
      vy: 34 + Math.random() * 36,
      life,
      maxLife: life,
      radius: 8 + Math.random() * 12,
      color: "rgba(244,251,248,0.26)"
    });
  }
  const damage = raceState.damage || 0;
  if (damage > 24 && Math.random() < dt * (4 + damage * 0.12)) {
    const life = 0.48 + Math.random() * 0.5;
    raceState.particles.push({
      x: x + (Math.random() - 0.5) * laneWidth() * 0.38,
      y: baseY + 8 + Math.random() * 18,
      vx: (Math.random() - 0.5) * 36 - raceState.lateralVelocity * 18,
      vy: 38 + Math.random() * 80,
      life,
      maxLife: life,
      radius: 12 + Math.random() * 24,
      color: damage > 70 ? "rgba(25,25,24,0.42)" : "rgba(135,138,132,0.28)"
    });
  }
}

function updateRaceUi() {
  $("#distanceBar").style.width = `${Math.min(100, (raceState.distance / raceLength()) * 100)}%`;
  $("#missionBar").style.width = `${Math.min(100, missionProgress() * 100)}%`;
  const damageBar = $("#damageBar");
  if (damageBar) damageBar.style.width = `${Math.min(100, raceState.damage || 0)}%`;
  const resetCar = $("#resetCar");
  if (resetCar) resetCar.textContent = raceState.resetTimer > 0 ? `Reset ${Math.ceil(raceState.resetTimer)}` : "Reset
Car";
  $("#gasBtn").classList.toggle("pressed", input.gas || input.gamepadGas);
  $("#brakeBtn").classList.toggle("pressed", input.brake || input.gamepadBrake);
  $("#boostBtn").classList.toggle("pressed", input.boost || input.gamepadBoost);
  $$(".mobile-control").forEach((button) => {
    const control = button.dataset.control;
    const steer = Number(input.touchSteer) || 0;
    const pressed = control === "left" ? input.left || steer < -0.08
      : control === "right" ? input.right || steer > 0.08
        : control === "neutral" ? !input.left && !input.right && Math.abs(steer) <= 0.08
          : control === "gas" ? input.gas || input.gamepadGas
            : control === "brake" ? input.brake || input.gamepadBrake
              : control === "boost" ? input.boost || input.gamepadBoost
                : control === "mode" ? touchDriveMode === "toggle"
                  : control === "size" ? touchControlSize === "mini"
                    : false;
    button.classList.toggle("pressed", pressed);
  });
  const touchLabel = $("#touchModeLabel");
  if (touchLabel) touchLabel.textContent = touchDriveMode === "toggle" ? "Toggle" : "Hold";
  const sizeLabel = $("#touchSizeLabel");
```

```
  if (sizeLabel) sizeLabel.textContent = touchControlSize === "mini" ? "Full" : "Mini";
  updateHud();
}

function laneWidth() {
  if (cameraMode === "cockpit") return Math.min(canvas.width * 0.145, 118);
  if (cameraMode === "hood") return Math.min(canvas.width * 0.13, 104);
  return Math.min(canvas.width * 0.115, 92);
}

function cameraLaneOffset(depth = 1) {
  if (!raceState.active) return 0;
  const lane = Number.isFinite(raceState.visualLane) ? raceState.visualLane : raceState.lane || 0;
  const follow = Math.max(0.18, Math.min(1.08, depth));
  return lane * follow;
}

let last = performance.now();
function loop(now = performance.now()) {
  const dt = Math.min(0.04, (now - last) / 1000);
  last = now;
  pollGamepad();
  tick(dt);
  drawFrame();
}

function drawFrame() {
  const w = canvas.width;
  const h = canvas.height;
  const shake = raceState.cameraShake;
  const theme = selectedRace ? selectedRace.theme : races[0].theme;
  if (useWebGLRenderer()) {
    $(".stage").classList.add("webgl");
    ctx.clearRect(0, 0, w, h);
    webglRenderer.resize(w, h);
    webglRenderer.render({
      width: w,
      height: h,
      raceState,
      selectedRace: selectedRace || races[0],
      selectedVehicle: Object.assign({}, selectedVehicle(), { color: selectedVehicleColor(selectedVehicle()) }),
      vehicleDefs,
      cameraMode
    });
    ctx.save();
    if (shake > 0) ctx.translate((Math.random() - 0.5) * shake, (Math.random() - 0.5) * shake * 0.55);
    drawWebGLRouteAtmosphere(w, h, theme);
    drawPhoneConsoleChasePass(w, h, theme);
    drawRealisticDrivingPass(w, h, theme);
    drawWebGLFlatPavementPass(w, h, theme);
    drawGenAiRacingScenePass(w, h, theme);
    drawRouteStageLandmarks(w, h, theme);
    drawRouteLightingPass(w, h, theme);
    if (raceState.active) {
      drawObjects();
      drawCar(w, h);
    } else {
      drawDemoPursuitTraffic(w, h);
    }
    drawDriftStylePass(w, h, theme);
    drawCameraOverlay(w, h, theme);
    drawDamageOverlay(w, h, theme);
    if (raceState.active) drawRaceStandings(w, h);
    drawParticles();
    drawPhoneFilmicPost(w, h, theme);
    drawCinematicGrade(w, h, theme);
    drawPhoneConsolePostPass(w, h, theme);
    if (!raceState.active) drawAttract(w, h);
    drawRaceGoalIntro(w, h, theme);
    if (input.paused && raceState.active) drawPause(w, h);
    ctx.restore();
    requestAnimationFrame(loop);
    return;
  }
  $(".stage").classList.remove("webgl");
  if (phoneGraphicsActive()) {
    drawPhoneCanvasFrame(w, h, theme, shake);
    requestAnimationFrame(loop);
    return;
  }
  ctx.save();
  if (shake > 0) ctx.translate((Math.random() - 0.5) * shake, (Math.random() - 0.5) * shake * 0.55);
  const sky = ctx.createLinearGradient(0, 0, 0, h);
  sky.addColorStop(0, theme[0]);
```

```
  sky.addColorStop(0.6, "#08100f");
  sky.addColorStop(1, "#050807");
  ctx.fillStyle = sky;
  ctx.fillRect(0, 0, w, h);
  drawScenery(w, h, theme);
  drawRoad(w, h, theme);
  drawPhoneConsoleChasePass(w, h, theme);
  drawRealisticDrivingPass(w, h, theme);
  drawPhoneAssetTexturePass(w, h, theme);
  drawRoadWeightPass(w, h, theme);
  drawRoadMotionPass(w, h, theme);
  drawPhoneUltraGraphicsPass(w, h, theme);
  drawGenAiRacingScenePass(w, h, theme);
  drawRouteStageLandmarks(w, h, theme);
  drawRouteLightingPass(w, h, theme);
  if (!raceState.active) drawDemoPursuitTraffic(w, h);
  drawObjects();
  drawCar(w, h);
  drawDriftStylePass(w, h, theme);
  drawCameraOverlay(w, h, theme);
  drawDamageOverlay(w, h, theme);
  if (raceState.active) drawRaceStandings(w, h);
  if (raceState.active) drawPhoneLaneRadar(w, h, theme);
  drawParticles();
  drawPhoneFilmicPost(w, h, theme);
  drawCinematicGrade(w, h, theme);
  drawPhoneConsolePostPass(w, h, theme);
  if (!raceState.active) drawAttract(w, h);
  drawRaceGoalIntro(w, h, theme);
  if (input.paused && raceState.active) drawPause(w, h);
  ctx.restore();
  requestAnimationFrame(loop);
}

function drawPhoneCanvasFrame(w, h, theme, shake) {
  ctx.save();
  if (shake > 0) ctx.translate((Math.random() - 0.5) * shake, (Math.random() - 0.5) * shake * 0.55);
  const sky = ctx.createLinearGradient(0, 0, 0, h);
  sky.addColorStop(0, theme[0]);
  sky.addColorStop(0.62, "#08100f");
  sky.addColorStop(1, "#050807");
  ctx.fillStyle = sky;
  ctx.fillRect(0, 0, w, h);
  drawPhoneConsoleChasePass(w, h, theme);
  drawRealisticDrivingPass(w, h, theme);
  drawPhoneAssetTexturePass(w, h, theme);
  drawPhoneRoadContrastPass(w, h, theme);
  drawGenAiRacingScenePass(w, h, theme);
  drawRouteStageLandmarks(w, h, theme);
  drawRouteLightingPass(w, h, theme);
  if (!raceState.active) drawDemoPursuitTraffic(w, h);
  drawObjects();
  drawCar(w, h);
  drawDriftStylePass(w, h, theme);
  drawCameraOverlay(w, h, theme);
  drawDamageOverlay(w, h, theme);
  if (raceState.active) drawRaceStandings(w, h);
  if (raceState.active) drawPhoneLaneRadar(w, h, theme);
  drawParticles();
  drawPhoneFilmicPost(w, h, theme);
  drawCinematicGrade(w, h, theme);
  drawPhoneConsolePostPass(w, h, theme);
  if (!raceState.active) drawAttract(w, h);
  drawRaceGoalIntro(w, h, theme);
  if (input.paused && raceState.active) drawPause(w, h);
  ctx.restore();
}

function drawWebGLRouteAtmosphere(w, h, theme) {
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  const phoneMode = phoneGraphicsActive();
  const cleanRoad = phoneCleanRoadActive();
  const skyLimit = phoneMode ? h * 0.34 : h * 0.62;
  ctx.save();
  const sky = ctx.createLinearGradient(0, 0, 0, skyLimit);
  sky.addColorStop(0, `${theme[0]}99`);
  sky.addColorStop(0.6, "rgba(5,8,7,0.08)");
  sky.addColorStop(1, "rgba(5,8,7,0)");
  ctx.fillStyle = sky;
  ctx.fillRect(0, 0, w, skyLimit);

  const glow = ctx.createRadialGradient(w * 0.5, h * 0.42, w * 0.06, w * 0.5, h * 0.55, w * 0.62);
  glow.addColorStop(0, phoneMode ? "rgba(244,251,248,0.025)" : "rgba(244,251,248,0.08)");
  glow.addColorStop(1, "rgba(0,0,0,0)");
```

```
  ctx.fillStyle = glow;
  ctx.fillRect(0, 0, w, phoneMode ? h * 0.42 : h);

  if (place === "tokyo" || place === "city") {
    ctx.globalAlpha = phoneMode ? (place === "tokyo" ? 0.12 : 0.06) : (place === "tokyo" ? 0.18 : 0.08);
    for (let i = 0; i < 14; i += 1) {
      const x = (i * 83 + raceState.roadOffset * 0.07) % (w + 140) - 70;
      const y = phoneMode ? h * (0.19 + (i % 4) * 0.032) : h * (0.25 + (i % 5) * 0.058);
      const dashW = 24 + (i % 4) * 18;
      const neon = ctx.createLinearGradient(x, y, x + dashW, y);
      neon.addColorStop(0, "rgba(255,255,255,0)");
      neon.addColorStop(0.5, place === "tokyo" ? "rgba(255,79,216,0.44)" : "rgba(244,251,248,0.18)");
      neon.addColorStop(1, "rgba(255,255,255,0)");
      ctx.strokeStyle = neon;
      ctx.lineWidth = 1.4;
      ctx.beginPath();
      ctx.moveTo(x, y);
      ctx.lineTo(x + dashW, y + 1);
      ctx.stroke();
    }
    ctx.globalAlpha = 1;
  }

  if (place === "desert" || place === "canyon" || place === "freight") {
    const dust = ctx.createLinearGradient(0, h * 0.38, 0, h);
    dust.addColorStop(0, "rgba(255,183,74,0)");
    dust.addColorStop(0.72, "rgba(255,183,74,0.12)");
    dust.addColorStop(1, "rgba(255,183,74,0.18)");
    ctx.fillStyle = dust;
    ctx.fillRect(0, h * 0.34, w, h * 0.66);
  }

  if (!cleanRoad && (place === "rainforest" || place === "snow")) {
    ctx.strokeStyle = place === "snow" ? "rgba(244,251,248,0.28)" : "rgba(185,255,220,0.16)";
    ctx.lineWidth = place === "snow" ? 2 : 1;
    for (let i = 0; i < 58; i += 1) {
      const x = (i * 53 + raceState.elapsed * (place === "snow" ? 24 : 70)) % w;
      const y = (i * 37 + raceState.elapsed * (place === "snow" ? 18 : 110)) % h;
      ctx.beginPath();
      ctx.moveTo(x, y);
      ctx.lineTo(x - (place === "snow" ? 4 : 12), y + (place === "snow" ? 6 : 28));
      ctx.stroke();
    }
  }

  ctx.restore();
}

function drawPhoneConsoleChasePass(w, h, theme) {
  if (!phoneGraphicsActive()) return;
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  const speed = Math.max(0, raceState.speed || 0);
  const cleanRoad = phoneCleanRoadActive();
  const horizon = cameraMode === "cockpit" ? h * 0.27 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const roadTop = cameraMode === "cockpit" ? w * 0.11 : cameraMode === "hood" ? w * 0.14 : w * 0.13;
  const roadBottom = cameraMode === "cockpit" ? w * 1.02 : cameraMode === "hood" ? w * 0.98 : w * 1.08;
  const wet = ["city", "tokyo", "coast", "harbor", "rainforest"].includes(place);
  const warm = ["desert", "canyon", "farm", "freight"].includes(place);
  const webglActive = useWebGLRenderer();
  const turn = raceState.roadTurn || raceState.roadCurve || 0;
  const roadCenter = (t) => {
    const clamped = Math.max(0, Math.min(1.08, t));
    const farPull = (1 - clamped) * (1 - clamped);
    const roadWave = Math.sin(clamped * 4.2 + (raceState.roadOffset || 0) * 0.004) * Math.abs(turn) * w * 0.018;
    const laneShift = cameraLaneOffset(0.25 + clamped * 0.85) * laneWidth() * (0.18 + clamped * 0.95);
    return w * 0.5 + turn * farPull * w * 0.26 + roadWave - laneShift;
  };

  ctx.save();
  const skyBottom = webglActive ? horizon + h * 0.1 : horizon + h * 0.24;
  const skyWash = ctx.createLinearGradient(0, 0, 0, skyBottom);
  skyWash.addColorStop(0, `${theme[0]}d9`);
  skyWash.addColorStop(0.54, warm ? "rgba(178,118,56,0.18)" : "rgba(70,217,255,0.11)");
  skyWash.addColorStop(1, "rgba(5,8,7,0)");
  ctx.fillStyle = skyWash;
  ctx.fillRect(0, 0, w, skyBottom);

  drawPhoneSceneryBackdrop(w, h, horizon, place, theme);
  drawPhoneConsoleLandmarks(w, h, horizon, place, theme);
  if (webglActive) {
    ctx.restore();
    return;
  }
```

```
ctx.beginPath();
ctx.moveTo(roadCenter(0) - roadTop, horizon);
ctx.lineTo(roadCenter(0) + roadTop, horizon);
ctx.lineTo(roadCenter(1) + roadBottom * 0.62, h + 8);
ctx.lineTo(roadCenter(1) - roadBottom * 0.62, h + 8);
ctx.closePath();
ctx.clip();

const asphalt = ctx.createLinearGradient(0, horizon, 0, h);
asphalt.addColorStop(0, wet ? "#2c3a3a" : "#292f2c");
asphalt.addColorStop(0.36, wet ? "#182223" : "#171c1a");
asphalt.addColorStop(1, "#030505");
ctx.fillStyle = asphalt;
ctx.fillRect(0, horizon, w, h - horizon);

if (!cleanRoad) {
  const edgeAlpha = 0.92;
  for (let side = -1; side <= 1; side += 2) {
    const shoulder = ctx.createLinearGradient(0, horizon, 0, h);
    shoulder.addColorStop(0, "rgba(244,251,248,0.36)");
    shoulder.addColorStop(0.42, "rgba(244,251,248,0.2)");
    shoulder.addColorStop(1, "rgba(244,251,248,0.1)");
    ctx.globalAlpha = edgeAlpha;
    ctx.strokeStyle = shoulder;
    ctx.lineWidth = Math.max(3, w * 0.005);
    ctx.beginPath();
    ctx.moveTo(roadCenter(0) + side * roadTop * 0.96, horizon + 1);
    ctx.lineTo(roadCenter(1) + side * roadBottom * 0.62, h + 8);
    ctx.stroke();
    ctx.strokeStyle = side < 0 ? "rgba(255,91,107,0.5)" : "rgba(70,217,255,0.46)";
    ctx.lineWidth = Math.max(5, w * 0.008);
    ctx.globalAlpha = 0.46;
    ctx.beginPath();
    ctx.moveTo(roadCenter(0) + side * roadTop * 0.76, horizon + 8);
    ctx.lineTo(roadCenter(1) + side * roadBottom * 0.48, h + 8);
    ctx.stroke();
  }
}
ctx.globalAlpha = 1;

if (wet && !cleanRoad) {
  ctx.globalCompositeOperation = "screen";
  for (let i = 0; i < 12; i += 1) {
    const y = horizon + (((i * 62 + raceState.roadOffset * (0.72 + speed / 380)) % (h - horizon + 90)) - 45);
    if (y < horizon || y > h) continue;
    const t = (y - horizon) / Math.max(1, h - horizon);
    const ribbonW = w * (0.08 + t * 0.44);
    const x = roadCenter(t) + Math.sin(i * 1.9 + raceState.elapsed * 0.7) * w * 0.06;
    const shine = ctx.createLinearGradient(x - ribbonW, y, x + ribbonW, y);
    shine.addColorStop(0, "rgba(255,255,255,0)");
    shine.addColorStop(0.5, `rgba(210,245,255,${0.08 + t * 0.18})`);
    shine.addColorStop(1, "rgba(255,255,255,0)");
    ctx.strokeStyle = shine;
    ctx.lineWidth = 1 + t * 6;
    ctx.beginPath();
    ctx.moveTo(x - ribbonW, y);
    ctx.quadraticCurveTo(x, y + 3 + t * 12, x + ribbonW, y + 2);
    ctx.stroke();
  }
  ctx.globalCompositeOperation = "source-over";
}

const roadX = (lane, t) => roadCenter(t) + lane * laneWidth() * (0.26 + t * 1.24);
const paint = (lane, y, length, width, alpha) => {
  const t1 = Math.max(0, Math.min(1.08, (y - horizon) / Math.max(1, h - horizon)));
  const t2 = Math.max(0, Math.min(1.12, (y + length - horizon) / Math.max(1, h - horizon)));
  if (t2 <= 0 || t1 >= 1.12) return;
  const x1 = roadX(lane, t1);
  const x2 = roadX(lane, t2);
  ctx.globalAlpha = alpha;
  ctx.fillStyle = "rgba(230,238,234,0.82)";
  ctx.beginPath();
  ctx.moveTo(x1 - width * (0.25 + t1), y);
  ctx.lineTo(x1 + width * (0.25 + t1), y);
  ctx.lineTo(x2 + width * (0.28 + t2 * 1.25), y + length);
  ctx.lineTo(x2 - width * (0.28 + t2 * 1.25), y + length);
  ctx.closePath();
  ctx.fill();
};

const motion = raceState.roadOffset * (0.96 + Math.min(1.8, speed / 130));
const paintCount = cleanRoad ? 8 : 17;
for (let i = 0; i < paintCount; i += 1) {
  const y = horizon + (((i * 78 + motion) % (h - horizon + 120)) - 52);
```

```
    const t = Math.max(0, Math.min(1, (y - horizon) / Math.max(1, h - horizon)));
    paint(-0.5, y, 14 + t * 42, 1.5 + t * 5.2, 0.24 + t * 0.34);
    paint(0.5, y + 12, 14 + t * 38, 1.5 + t * 5.2, 0.2 + t * 0.3);
  }
  ctx.globalAlpha = 1;

  if (!cleanRoad) {
    for (let side = -1; side <= 1; side += 2) {
      for (let i = 0; i < 12; i += 1) {
      const y = horizon + (((i * 66 + motion * 0.74) % (h - horizon + 100)) - 28);
      if (y < horizon || y > h) continue;
      const t = Math.max(0, Math.min(1, (y - horizon) / Math.max(1, h - horizon)));
      const x = roadX(side * (1.72 + t * 0.24), t);
      const dashW = 10 + t * 46;
      const dashH = 2 + t * 6;
      ctx.globalAlpha = 0.1 + t * 0.34;
      ctx.fillStyle = side < 0 ? "rgba(255,91,107,0.32)" : "rgba(70,217,255,0.28)";
      ctx.save();
      ctx.translate(x, y);
      ctx.rotate(side * turn * -0.08);
      roundRect(-dashW / 2, -dashH / 2, dashW, dashH, 2);
      ctx.fill();
      ctx.restore();
      }
    }
  }
  ctx.globalAlpha = 1;

  if (!cleanRoad && Math.abs(turn) > 0.22) {
    const side = turn > 0 ? 1 : -1;
    ctx.save();
    ctx.globalCompositeOperation = "screen";
    for (let i = 0; i < 8; i += 1) {
      const y = horizon + (((i * 74 + motion * 0.82) % (h - horizon + 120)) - 36);
      const t = Math.max(0.04, Math.min(1, (y - horizon) / Math.max(1, h - horizon)));
      const x = roadX(side * 2.72, t);
      const boardW = 16 + t * 42;
      const boardH = 5 + t * 11;
      ctx.globalAlpha = 0.18 + t * 0.42;
      ctx.fillStyle = side > 0 ? `${theme[2]}aa` : `${theme[1]}aa`;
      ctx.save();
      ctx.translate(x, y);
      ctx.rotate(side * -0.34);
      roundRect(-boardW / 2, -boardH / 2, boardW, boardH, 2);
      ctx.fill();
      ctx.fillStyle = "rgba(244,251,248,0.76)";
      for (let dot = -1; dot <= 1; dot += 1) {
        ctx.beginPath();
        ctx.arc(dot * boardW * 0.18, 0, Math.max(1.4, boardH * 0.18), 0, Math.PI * 2);
        ctx.fill();
      }
      ctx.restore();
    }
    ctx.restore();
    ctx.globalAlpha = 1;
  }

  if (!cleanRoad && speed > 90) {
    ctx.globalCompositeOperation = "screen";
    ctx.globalAlpha = Math.min(0.2, (speed - 80) / 700);
    for (let side = -1; side <= 1; side += 2) {
      for (let i = 0; i < 7; i += 1) {
        const y = horizon + h * (0.12 + i * 0.11);
        ctx.strokeStyle = i % 2 ? "rgba(255,255,255,0.22)" : `${theme[1]}66`;
        ctx.lineWidth = 1 + i * 0.35;
        ctx.beginPath();
        ctx.moveTo(side < 0 ? 0 : w, y);
        ctx.lineTo(side < 0 ? w * 0.14 : w * 0.86, y + h * 0.015);
        ctx.stroke();
      }
    }
  }
  ctx.restore();
}

function drawPhoneRoadContrastPass(w, h, theme) {
  if (!phoneGraphicsActive()) return;
  if (phoneCleanRoadActive()) return;
  const horizon = cameraMode === "cockpit" ? h * 0.27 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const roadTop = cameraMode === "cockpit" ? w * 0.11 : cameraMode === "hood" ? w * 0.14 : w * 0.13;
  const roadBottom = cameraMode === "cockpit" ? w * 1.02 : cameraMode === "hood" ? w * 0.98 : w * 1.08;
  const turn = raceState.roadTurn || raceState.roadCurve || 0;
  const roadCenter = (t) => {
    const clamped = Math.max(0, Math.min(1.08, t));
```

```
    const farPull = (1 - clamped) * (1 - clamped);
    return w * 0.5 + turn * farPull * w * 0.26;
  };
  ctx.save();
  ctx.beginPath();
  ctx.moveTo(roadCenter(0) - roadTop, horizon);
  ctx.lineTo(roadCenter(0) + roadTop, horizon);
  ctx.lineTo(roadCenter(1) + roadBottom * 0.62, h + 8);
  ctx.lineTo(roadCenter(1) - roadBottom * 0.62, h + 8);
  ctx.closePath();
  ctx.clip();
  const shade = ctx.createLinearGradient(0, horizon, 0, h);
  shade.addColorStop(0, "rgba(0,0,0,0.16)");
  shade.addColorStop(0.48, "rgba(0,0,0,0.28)");
  shade.addColorStop(1, "rgba(0,0,0,0.48)");
  ctx.fillStyle = shade;
  ctx.fillRect(0, horizon, w, h - horizon);
  const motion = raceState.roadOffset * (1.08 + Math.min(1.8, Math.max(0, raceState.speed || 0) / 130));
  const laneX = (lane, t) => roadCenter(t) + lane * laneWidth() * (0.34 + t * 1.08);
  for (let lane = -1.5; lane <= 1.5; lane += 1) {
    for (let i = 0; i < 9; i += 1) {
      const y = horizon + (((i * 118 + motion) % (h - horizon + 160)) - 60);
      if (y < horizon || y > h + 50) continue;
      const t = Math.max(0, Math.min(1, (y - horizon) / Math.max(1, h - horizon)));
      const x = laneX(lane, t);
      ctx.globalAlpha = 0.38 + t * 0.28;
      ctx.fillStyle = "rgba(244,251,248,0.88)";
      roundRect(x - (2 + t * 4), y, 4 + t * 8, 16 + t * 36, 2 + t * 3);
      ctx.fill();
    }
  }
  ctx.globalAlpha = 1;
  for (let side = -1; side <= 1; side += 2) {
    ctx.strokeStyle = side < 0 ? "rgba(255,91,107,0.72)" : "rgba(70,217,255,0.68)";
    ctx.lineWidth = Math.max(4, w * 0.006);
    ctx.beginPath();
    ctx.moveTo(roadCenter(0) + side * roadTop * 0.82, horizon + 4);
    ctx.lineTo(roadCenter(1) + side * roadBottom * 0.5, h + 8);
    ctx.stroke();
  }
  ctx.restore();
}

function drawPhoneSceneryBackdrop(w, h, horizon, place, theme) {
  if (!phoneGraphicsActive()) return;
  const offset = (raceState.roadOffset || 0) * 0.018;
  ctx.save();
  ctx.globalAlpha = 0.9;
  const farY = horizon - h * 0.02;
  if (place === "city" || place === "tokyo") {
    for (let i = 0; i < 22; i += 1) {
      const x = ((i * 94 - offset * (1.2 + (i % 3) * 0.18)) % (w + 180)) - 90;
      const bw = 34 + (i % 5) * 13;
      const bh = h * (0.2 + (i % 6) * 0.045);
      const glow = place === "tokyo" ? "rgba(255,79,216,0.26)" : "rgba(70,217,255,0.2)";
      const tower = ctx.createLinearGradient(x, farY - bh, x + bw, farY);
      tower.addColorStop(0, glow);
      tower.addColorStop(0.45, "rgba(16,25,28,0.9)");
      tower.addColorStop(1, "rgba(5,8,7,0.82)");
      ctx.fillStyle = tower;
      roundRect(x, farY - bh, bw, bh, 3);
      ctx.fill();
      ctx.fillStyle = i % 2 ? `${theme[1]}88` : `${theme[2]}77`;
      for (let yy = 14; yy < bh - 10; yy += 24) {
        ctx.globalAlpha = 0.42;
        ctx.fillRect(x + 8 + (yy % 3) * 7, farY - bh + yy, 5, 12);
        ctx.fillRect(x + bw - 15, farY - bh + yy + 5, 5, 10);
      }
      ctx.globalAlpha = 0.9;
    }
  } else if (place === "coast" || place === "harbor") {
    const water = ctx.createLinearGradient(0, horizon - h * 0.01, 0, horizon + h * 0.2);
    water.addColorStop(0, "rgba(70,217,255,0.16)");
    water.addColorStop(1, "rgba(4,21,24,0.42)");
    ctx.fillStyle = water;
    ctx.fillRect(0, horizon - h * 0.02, w, h * 0.24);
    for (let i = 0; i < 12; i += 1) {
      const x = ((i * 128 - offset * 1.9) % (w + 180)) - 90;
      ctx.fillStyle = "rgba(8,38,43,0.86)";
      ctx.beginPath();
      ctx.ellipse(x, horizon + h * 0.06, 54 + (i % 4) * 12, 12 + (i % 3) * 4, 0, 0, Math.PI * 2);
      ctx.fill();
      ctx.fillStyle = "rgba(244,251,248,0.28)";
      roundRect(x - 26, horizon + h * 0.025, 52, 6, 2);
```

```
      ctx.fill();
    }
  } else if (place === "alpine" || place === "snow" || place === "europe") {
    for (let i = 0; i < 9; i += 1) {
      const x = ((i * 180 - offset) % (w + 260)) - 130;
      const ridgeW = 180 + (i % 3) * 45;
      const ridgeH = h * (0.13 + (i % 4) * 0.025);
      ctx.fillStyle = place === "snow" ? "rgba(210,228,230,0.42)" : "rgba(102,130,126,0.38)";
      ctx.beginPath();
      ctx.ellipse(x, horizon + h * 0.025, ridgeW * 0.5, ridgeH, -0.08, 0, Math.PI * 2);
      ctx.fill();
      ctx.fillStyle = "rgba(244,251,248,0.36)";
      roundRect(x - ridgeW * 0.22, horizon - ridgeH * 0.28, ridgeW * 0.44, 7, 4);
      ctx.fill();
    }
  } else {
    for (let i = 0; i < 26; i += 1) {
      const x = ((i * 82 - offset * 2.2) % (w + 160)) - 80;
      const treeH = h * (0.1 + (i % 5) * 0.018);
      ctx.fillStyle = place === "desert" || place === "canyon" ? "rgba(132,72,38,0.64)" : "rgba(24,78,52,0.66)";
      ctx.beginPath();
      ctx.ellipse(x, horizon + h * 0.04, 34 + (i % 4) * 8, treeH * 0.42, 0, 0, Math.PI * 2);
      ctx.fill();
      ctx.fillStyle = place === "farm" ? "rgba(187,242,74,0.24)" : "rgba(54,217,138,0.2)";
      ctx.fillRect(x - 2, horizon - treeH * 0.42, 4, treeH * 0.72);
    }
  }
  const roadside = ctx.createLinearGradient(0, horizon + h * 0.06, 0, h);
  roadside.addColorStop(0, "rgba(7,20,18,0)");
  roadside.addColorStop(1, "rgba(7,20,18,0.45)");
  ctx.fillStyle = roadside;
  ctx.fillRect(0, horizon + h * 0.05, w, h * 0.42);
  ctx.restore();
}

function drawPhoneConsoleLandmarks(w, h, horizon, place, theme) {
  ctx.save();
  ctx.globalAlpha = 0.52;
  const baseY = horizon + h * 0.05;
  const offset = (raceState.roadOffset * 0.025) % 180;
  const route = routeWorldInfo(place);
  for (let i = 0; i < 18; i += 1) {
    const side = i % 2 ? -1 : 1;
    const x = side < 0 ? (i * 74 - offset) % (w * 0.48) : w - ((i * 78 + offset) % (w * 0.48));
    if (place === "city" || place === "tokyo") {
      const bw = 20 + (i % 4) * 9;
      const bh = h * (0.14 + (i % 5) * 0.035);
      ctx.fillStyle = place === "tokyo" ? "rgba(18,14,36,0.82)" : "rgba(14,20,23,0.86)";
      roundRect(x, baseY - bh, bw, bh, 2);
      ctx.fill();
      ctx.fillStyle = i % 2 ? `${theme[1]}66` : `${theme[2]}55`;
      ctx.fillRect(x + bw * 0.22, baseY - bh + 8, bw * 0.12, bh * 0.62);
      ctx.fillRect(x + bw * 0.62, baseY - bh + 18, bw * 0.12, bh * 0.48);
    } else if (place === "coast" || place === "harbor") {
      ctx.fillStyle = "rgba(8,38,43,0.72)";
      ctx.beginPath();
      ctx.ellipse(x, baseY + 8, 48 + (i % 3) * 12, 12 + (i % 4) * 3, 0, 0, Math.PI * 2);
      ctx.fill();
      ctx.fillStyle = "rgba(244,251,248,0.16)";
      roundRect(x - 24, baseY - 5, 48, 5, 2);
      ctx.fill();
      ctx.fillStyle = `${theme[1]}33`;
      roundRect(x - 8, baseY - 24 - (i % 3) * 4, 16, 20 + (i % 3) * 4, 2);
      ctx.fill();
    } else if (place === "desert" || place === "canyon") {
      ctx.fillStyle = "rgba(132,72,38,0.58)";
      ctx.beginPath();
      ctx.moveTo(x - 68, baseY + 14);
      ctx.quadraticCurveTo(x, baseY - 26 - (i % 3) * 12, x + 74, baseY + 16);
      ctx.closePath();
      ctx.fill();
    } else if (place === "snow" || place === "alpine" || place === "europe") {
      ctx.fillStyle = "rgba(150,174,174,0.26)";
      ctx.beginPath();
      ctx.ellipse(x, baseY + 8, 68 + (i % 4) * 12, 20 + (i % 3) * 5, 0, 0, Math.PI * 2);
      ctx.fill();
      ctx.fillStyle = "rgba(244,251,248,0.42)";
      roundRect(x - 26, baseY - 3 - (i % 2) * 4, 52, 5, 3);
      ctx.fill();
    } else {
      ctx.fillStyle = place === "farm" ? "rgba(60,92,42,0.58)" : "rgba(24,78,52,0.55)";
      ctx.beginPath();
      ctx.ellipse(x, baseY + 8, 44 + (i % 4) * 12, 20 + (i % 3) * 6, 0, 0, Math.PI * 2);
      ctx.fill();
```

```
      ctx.fillStyle = "rgba(10,28,18,0.64)";
      ctx.fillRect(x - 2, baseY - 40, 4, 50);
    }
  }
  const signSide = Math.sin((raceState.roadOffset || 0) * 0.002 + route.seed) > 0 ? 1 : -1;
  const signX = signSide > 0 ? w * 0.82 : w * 0.18;
  const signY = horizon + h * 0.08;
  ctx.globalAlpha = 0.74;
  ctx.fillStyle = "rgba(4,9,9,0.68)";
  roundRect(signX - w * 0.095, signY - h * 0.035, w * 0.19, h * 0.07, 4);
  ctx.fill();
  ctx.strokeStyle = `${theme[1]}88`;
  ctx.lineWidth = 1.4;
  ctx.stroke();
  ctx.fillStyle = "rgba(244,251,248,0.9)";
  ctx.font = `700 ${Math.max(8, Math.min(13, w * 0.014))}px Inter, system-ui, sans-serif`;
  ctx.textAlign = "center";
  ctx.fillText(route.country.toUpperCase(), signX, signY - h * 0.005);
  ctx.font = `600 ${Math.max(7, Math.min(11, w * 0.012))}px Inter, system-ui, sans-serif`;
  ctx.fillStyle = `${theme[2]}dd`;
  ctx.fillText(route.scene.toUpperCase(), signX, signY + h * 0.018);
  ctx.restore();
}

function drawPhoneConsolePostPass(w, h, theme) {
  if (!phoneGraphicsActive()) return;
  const speed = Math.max(0, raceState.speed || 0);
  const turn = raceState.roadTurn || raceState.roadCurve || 0;
  ctx.save();
  ctx.globalCompositeOperation = "screen";
  const headlight = ctx.createRadialGradient(w * 0.5, h * 0.76, w * 0.04, w * 0.5, h * 0.76, w * 0.55);
  headlight.addColorStop(0, "rgba(244,251,248,0.12)");
  headlight.addColorStop(0.55, `${theme[1]}12`);
  headlight.addColorStop(1, "rgba(0,0,0,0)");
  ctx.fillStyle = headlight;
  ctx.fillRect(0, h * 0.35, w, h * 0.65);
  ctx.restore();

  if (Math.abs(turn) > 0.08) {
    ctx.save();
    const leanSide = turn > 0 ? 1 : -1;
    const pull = Math.min(0.26, Math.abs(turn) * 0.11 + speed / 1800);
    const turnGlow = ctx.createLinearGradient(leanSide > 0 ? w : 0, 0, leanSide > 0 ? w * 0.45 : w * 0.55, 0);
    turnGlow.addColorStop(0, `${theme[1]}33`);
    turnGlow.addColorStop(0.42, "rgba(244,251,248,0.04)");
    turnGlow.addColorStop(1, "rgba(0,0,0,0)");
    ctx.globalCompositeOperation = "screen";
    ctx.globalAlpha = pull;
    ctx.fillStyle = turnGlow;
    ctx.fillRect(0, 0, w, h);
    ctx.restore();
  }

  if (speed > 110) {
    ctx.save();
    ctx.globalAlpha = Math.min(0.24, (speed - 100) / 520);
    const blur = ctx.createLinearGradient(0, 0, w, 0);
    blur.addColorStop(0, "rgba(244,251,248,0.18)");
    blur.addColorStop(0.18, "rgba(244,251,248,0)");
    blur.addColorStop(0.82, "rgba(244,251,248,0)");
    blur.addColorStop(1, "rgba(244,251,248,0.18)");
    ctx.fillStyle = blur;
    ctx.fillRect(0, 0, w, h);
    ctx.restore();
  }

  ctx.save();
  const cinematic = ctx.createLinearGradient(0, 0, 0, h);
  cinematic.addColorStop(0, "rgba(0,0,0,0.24)");
  cinematic.addColorStop(0.5, "rgba(0,0,0,0)");
  cinematic.addColorStop(1, "rgba(0,0,0,0.36)");
  ctx.fillStyle = cinematic;
  ctx.fillRect(0, 0, w, h);
  ctx.restore();
}

function hashText(value = "") {
  let hash = 2166136261;
  for (let i = 0; i < value.length; i += 1) {
    hash ^= value.charCodeAt(i);
    hash = Math.imul(hash, 16777619);
  }
  return hash >>> 0;
}
```

```
function seededUnit(seed, salt = 0) {
  const n = Math.sin(seed * 12.9898 + salt * 78.233) * 43758.5453;
  return n - Math.floor(n);
}

function perspectiveRoadCenter(w, t) {
  const turn = raceState.roadTurn || raceState.roadCurve || 0;
  const clamped = Math.max(0, Math.min(1.08, t));
  const farPull = (1 - clamped) * (1 - clamped);
  const laneShift = cameraLaneOffset(0.24 + clamped * 0.86) * laneWidth() * (0.18 + clamped * 0.95);
  return w * 0.5 + turn * farPull * w * 0.27 + Math.sin(clamped * 4.1 + (raceState.roadOffset || 0) * 0.003) *
Math.abs(turn) * w * 0.015 - laneShift;
}

function drawGenAiRacingScenePass(w, h, theme) {
  if (!raceState.active && view !== "race") return;
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  const design = genAiSceneDesign(place);
  const route = routeWorldInfo(place);
  const phoneMode = phoneGraphicsActive();
  const horizon = cameraMode === "cockpit" ? h * 0.27 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const roadTop = cameraMode === "cockpit" ? w * 0.11 : cameraMode === "hood" ? w * 0.14 : w * 0.13;
  const roadBottom = cameraMode === "cockpit" ? w * 1.02 : cameraMode === "hood" ? w * 0.98 : w * 1.08;
  const seed = hashText(`${selectedRace ? selectedRace.id : "race"}:${design.style}`);
  drawGenAiHeroBackdrop(w, h, horizon, design, theme, place, route, seed);
  drawGenAiSurfaceDetails(w, h, theme, horizon, roadTop, roadBottom, design, seed);
  ctx.save();
  const count = phoneMode ? 20 : 30;
  const offset = (raceState.roadOffset || 0) * (phoneMode ? 1.08 : 0.84);
  for (let i = 0; i < count; i += 1) {
    const side = i % 2 ? -1 : 1;
    const loopSpan = h - horizon + 240;
    const raw = ((i * 76 + offset * (0.52 + seededUnit(seed, i) * 0.34)) % loopSpan) - 92;
    const y = horizon + raw;
    if (y < horizon - 18 || y > h + 70) continue;
    const t = Math.max(0.02, Math.min(1.05, (y - horizon) / Math.max(1, h - horizon)));
    const center = perspectiveRoadCenter(w, t);
    const roadHalf = roadTop * 0.82 + (roadBottom * 0.58 - roadTop * 0.82) * t;
    const x = center + side * (roadHalf + w * (0.05 + seededUnit(seed, i + 4) * 0.11) * (0.42 + t));
    const scale = 0.36 + t * 1.06;
    const prop = design.props[i % design.props.length];
    drawGenAiTrackProp(prop, x, y, scale, side, design, theme, route, seed + i * 13);
  }
  drawGenAiCrowdAndCameras(w, h, horizon, roadTop, roadBottom, design);
  ctx.restore();
}

function routeStageInfo(place = "city", progress = 0) {
  const stageIndex = Math.max(0, Math.min(3, Math.floor(Math.max(0, Math.min(0.999, progress)) * 4)));
  const stages = {
    coast: [["Ocean Cliff", "cliff"], ["Beach Town", "palm"], ["Coastal Bridge", "bridge"], ["Finish Pier",
"checkpoint"]],
    city: [["Lakefront", "tower"], ["Downtown", "crowd"], ["Underpass", "tunnel"], ["Loop Finish", "checkpoint"]],
    canyon: [["Red Rocks", "cliff"], ["Cave Road", "tunnel"], ["Mesa Sweep", "dune"], ["Canyon Finish",
"checkpoint"]],
    alpine: [["Pine Pass", "pine"], ["Mountain Tunnel", "tunnel"], ["Snow Peaks", "mountain"], ["Summit Finish",
"checkpoint"]],
    harbor: [["Marina", "pier"], ["Container Docks", "crane"], ["Bridge Cut", "bridge"], ["Harbor Finish",
"checkpoint"]],
    snow: [["Aspen Forest", "pine"], ["Ice Village", "village"], ["Snow Tunnel", "tunnel"], ["Winter Finish",
"checkpoint"]],
    airfield: [["Runway", "beacon"], ["Hangars", "hangar"], ["Taxiway", "plane"], ["Airfield Finish", "checkpoint"]],
    freight: [["Truck Stop", "trailer"], ["Overpass", "bridge"], ["Depot", "hangar"], ["Freight Finish",
"checkpoint"]],
    farm: [["Cornfields", "field"], ["Barn Bend", "barn"], ["Dirt Cut", "tractor"], ["Farm Finish", "checkpoint"]],
    tokyo: [["Neon Entry", "neon"], ["Expressway", "tower"], ["Tunnel Run", "tunnel"], ["Tokyo Finish",
"checkpoint"]],
    desert: [["Dune Sea", "dune"], ["Oasis Road", "palm"], ["Rock Gate", "cliff"], ["Rally Finish", "checkpoint"]],
    rainforest: [["Jungle Entry", "canopy"], ["Bridge Bypass", "bridge"], ["Canopy Tunnel", "canopy"], ["Rain Finish",
 "checkpoint"]],
    europe: [["Village Road", "village"], ["Switchback", "mountain"], ["Stone Tunnel", "tunnel"], ["Alps Finish",
"checkpoint"]]
  };
  const selected = (stages[place] || stages.city)[stageIndex];
  return { index: stageIndex, label: selected[0], prop: selected[1] };
}

function drawRouteStageLandmarks(w, h, theme) {
  if (!raceState.active && view !== "race") return;
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  const length = raceLength();
  const progress = length ? Math.max(0, Math.min(1, raceState.distance / length)) : 0;
  const stage = routeStageInfo(place, progress);
```

```
  const design = genAiSceneDesign(place);
  const horizon = cameraMode === "cockpit" ? h * 0.27 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const phoneMode = phoneGraphicsActive();
  const seed = hashText(`${place}:${stage.label}:${stage.index}`);
  ctx.save();
  ctx.globalAlpha = phoneMode ? 0.72 : 0.82;
  drawRouteHorizonSetPiece(w, h, horizon, stage, design, theme, seed);
  const loopSpan = h - horizon + 260;
  const count = phoneMode ? 13 : 18;
  for (let i = 0; i < count; i += 1) {
    const raw = ((i * 132 + (raceState.roadOffset || 0) * (0.72 + seededUnit(seed, i) * 0.22) + stage.index * 43) %
loopSpan) - 84;
    const y = horizon + raw;
    if (y < horizon - 12 || y > h + 80) continue;
    const t = Math.max(0.04, Math.min(1.1, (y - horizon) / Math.max(1, h - horizon)));
    const side = i % 2 === 0 ? -1 : 1;
    const center = perspectiveRoadCenter(w, t);
    const sideOffset = w * (0.18 + t * 0.48 + seededUnit(seed, i + 10) * 0.07);
    const x = center + side * sideOffset;
    const scale = 0.38 + t * (phoneMode ? 0.92 : 1.22);
    const prop = i % 5 === 0 ? stage.prop : design.props[(i + stage.index) % design.props.length];
    drawRouteStageProp(prop, x, y, scale, side, design, theme, seed + i * 17);
  }
  drawRouteStageSign(w, h, horizon, stage, design, theme);
  drawFinishGate(w, h, theme);
  ctx.restore();
}

function drawRouteHorizonSetPiece(w, h, horizon, stage, design, theme, seed) {
  const x = w * (0.5 + Math.sin((raceState.distance || 0) * 0.00008 + seed) * 0.05);
  const y = horizon + h * 0.035;
  ctx.save();
  ctx.globalAlpha *= 0.62;
  if (stage.prop === "tunnel") {
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    roundRect(x - w * 0.16, y - h * 0.06, w * 0.32, h * 0.14, 12);
    ctx.fill();
    ctx.strokeStyle = `${design.accent}77`;
    ctx.lineWidth = Math.max(2, w * 0.004);
    ctx.stroke();
    ctx.fillStyle = "rgba(0,0,0,0.55)";
    ctx.beginPath();
    ctx.ellipse(x, y + h * 0.045, w * 0.09, h * 0.06, 0, 0, Math.PI * 2);
    ctx.fill();
  } else if (stage.prop === "bridge") {
    ctx.strokeStyle = `${design.light}88`;
    ctx.lineWidth = Math.max(2, w * 0.004);
    ctx.beginPath();
    ctx.moveTo(x - w * 0.27, y + h * 0.04);
    ctx.quadraticCurveTo(x, y - h * 0.11, x + w * 0.27, y + h * 0.04);
    ctx.stroke();
    for (let i = -3; i <= 3; i += 1) {
      ctx.beginPath();
      ctx.moveTo(x + i * w * 0.07, y - h * 0.045);
      ctx.lineTo(x + i * w * 0.055, y + h * 0.055);
      ctx.stroke();
    }
  } else if (stage.prop === "checkpoint") {
    ctx.fillStyle = "rgba(5,8,7,0.72)";
    roundRect(x - w * 0.13, y - h * 0.055, w * 0.26, h * 0.05, 6);
    ctx.fill();
    ctx.strokeStyle = theme[1] || "#46d9ff";
    ctx.lineWidth = 2;
    ctx.stroke();
  } else {
    ctx.fillStyle = `${design.accent}22`;
    ctx.beginPath();
    ctx.ellipse(x, y, w * 0.22, h * 0.045, 0, 0, Math.PI * 2);
    ctx.fill();
  }
  ctx.restore();
}

function drawRouteStageProp(prop, x, y, scale, side, design, theme, seed) {
  ctx.save();
  ctx.translate(x, y);
  ctx.scale(scale, scale);
  ctx.globalAlpha *= Math.max(0.42, Math.min(0.96, 0.42 + scale * 0.22));
  const accent = design.accent || theme[1] || "#46d9ff";
  const light = design.light || theme[2] || "#ffd166";
  if (prop === "tower" || prop === "neon" || prop === "village") {
    const bw = prop === "village" ? 52 : 34;
    const bh = prop === "village" ? 48 : 118;
    ctx.fillStyle = prop === "neon" ? "rgba(18,10,32,0.9)" : "rgba(10,18,20,0.88)";
```

```
    roundRect(-bw / 2, -bh, bw, bh, 4);
    ctx.fill();
    ctx.fillStyle = prop === "neon" ? `${accent}cc` : `${light}99`;
    for (let i = 0; i < 5; i += 1) ctx.fillRect(-bw * 0.3, -bh + 12 + i * 18, bw * 0.6, 4);
    if (prop === "village") {
      ctx.fillStyle = `${light}88`;
      ctx.beginPath();
      ctx.moveTo(-bw * 0.6, -bh);
      ctx.lineTo(0, -bh - 28);
      ctx.lineTo(bw * 0.6, -bh);
      ctx.closePath();
      ctx.fill();
    }
  } else if (prop === "palm" || prop === "pine" || prop === "canopy") {
    ctx.strokeStyle = prop === "palm" ? "rgba(140,100,55,0.9)" : "rgba(30,78,52,0.92)";
    ctx.lineWidth = 5;
    ctx.beginPath();
    ctx.moveTo(0, 12);
    ctx.lineTo(side * 6, -70);
    ctx.stroke();
    ctx.fillStyle = prop === "pine" ? "rgba(34,93,59,0.86)" : "rgba(54,217,138,0.62)";
    for (let i = 0; i < 3; i += 1) {
      ctx.beginPath();
      ctx.ellipse(side * (4 + i * 3), -70 + i * 12, 36 - i * 7, 12, side * 0.4, 0, Math.PI * 2);
      ctx.fill();
    }
  } else if (prop === "bridge" || prop === "crane" || prop === "beacon") {
    ctx.strokeStyle = `${light}99`;
    ctx.lineWidth = 4;
    ctx.beginPath();
    ctx.moveTo(0, 16);
    ctx.lineTo(0, -86);
    ctx.moveTo(-42 * side, -62);
    ctx.lineTo(52 * side, -76);
    ctx.stroke();
    ctx.fillStyle = `${accent}aa`;
    ctx.beginPath();
    ctx.arc(0, -90, 7, 0, Math.PI * 2);
    ctx.fill();
  } else if (prop === "hangar" || prop === "barn" || prop === "trailer" || prop === "field" || prop === "tractor") {
    ctx.fillStyle = prop === "barn" ? "rgba(148,42,34,0.84)" : "rgba(22,29,29,0.84)";
    roundRect(-54, -42, 108, 54, 5);
    ctx.fill();
    ctx.fillStyle = prop === "barn" ? "rgba(255,209,102,0.7)" : `${light}77`;
    ctx.beginPath();
    ctx.moveTo(-62, -42);
    ctx.lineTo(0, -76);
    ctx.lineTo(62, -42);
    ctx.closePath();
    ctx.fill();
    if (prop === "trailer" || prop === "tractor") {
      ctx.fillStyle = "rgba(5,8,7,0.9)";
      ctx.beginPath();
      ctx.arc(-34, 14, 8, 0, Math.PI * 2);
      ctx.arc(34, 14, 8, 0, Math.PI * 2);
      ctx.fill();
    }
  } else {
    ctx.fillStyle = prop === "dune" ? "rgba(255,183,74,0.58)" : prop === "mountain" ? "rgba(130,160,160,0.5)" :
"rgba(132,72,38,0.58)";
    ctx.beginPath();
    ctx.moveTo(-58, 16);
    ctx.quadraticCurveTo(-8, -68 - seededUnit(seed, 1) * 28, 64, 18);
    ctx.closePath();
    ctx.fill();
    ctx.fillStyle = "rgba(244,251,248,0.18)";
    ctx.fillRect(-18, -18, 42, 5);
  }
  ctx.restore();
}

function drawRouteStageSign(w, h, horizon, stage, design, theme) {
  const x = w * 0.2;
  const y = horizon + h * 0.08;
  ctx.save();
  ctx.globalAlpha *= 0.88;
  ctx.fillStyle = "rgba(5,8,7,0.78)";
  roundRect(x - w * 0.095, y - 18, w * 0.19, 38, 6);
  ctx.fill();
  ctx.strokeStyle = design.accent || theme[1] || "#46d9ff";
  ctx.lineWidth = 2;
  ctx.stroke();
  ctx.fillStyle = "#f4fbf8";
  ctx.font = `900 ${Math.max(9, Math.min(15, w * 0.013))}px system-ui`;
```

```
  ctx.textAlign = "center";
  ctx.textBaseline = "middle";
  ctx.fillText(stage.label.toUpperCase(), x, y + 1);
  ctx.restore();
}

function drawFinishGate(w, h, theme) {
  if (!raceState.active) return;
  const length = raceLength();
  const gap = length - raceState.distance;
  if (gap > 2600 || gap < -420) return;
  const horizon = cameraMode === "cockpit" ? h * 0.27 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const p = roadObjectPos(0, length);
  const y = Math.max(horizon + h * 0.05, Math.min(h * 0.94, p.y));
  const scale = Math.max(0.34, Math.min(1.24, p.scale || 1));
  const half = laneWidth() * (2.15 + scale * 0.65);
  const x = p.x;
  const lineH = Math.max(8, 20 * scale);
  ctx.save();
  ctx.globalAlpha = Math.max(0.48, Math.min(0.96, 1 - Math.max(0, gap - 1800) / 1600));
  for (let i = 0; i < 12; i += 1) {
    ctx.fillStyle = i % 2 ? "#050807" : "#f4fbf8";
    const tileW = (half * 2) / 12;
    ctx.fillRect(x - half + i * tileW, y + lineH * 0.25, tileW + 1, lineH);
  }
  ctx.fillStyle = "rgba(5,8,7,0.82)";
  roundRect(x - half * 0.92, y - 86 * scale, 12 * scale, 108 * scale, 4 * scale);
  ctx.fill();
  roundRect(x + half * 0.92 - 12 * scale, y - 86 * scale, 12 * scale, 108 * scale, 4 * scale);
  ctx.fill();
  ctx.fillStyle = "rgba(5,8,7,0.88)";
  roundRect(x - half * 0.72, y - 104 * scale, half * 1.44, 34 * scale, 5 * scale);
  ctx.fill();
  ctx.strokeStyle = theme[1] || "#46d9ff";
  ctx.lineWidth = Math.max(1.5, 2.5 * scale);
  ctx.stroke();
  ctx.fillStyle = "#f4fbf8";
  ctx.font = `900 ${Math.max(9, 17 * scale)}px system-ui`;
  ctx.textAlign = "center";
  ctx.textBaseline = "middle";
  ctx.fillText("FINISH", x, y - 87 * scale);
  ctx.restore();
}

function drawGenAiHeroBackdrop(w, h, horizon, design, theme, place, route, seed) {
  ctx.save();
  const phoneMode = phoneGraphicsActive();
  const alpha = phoneMode ? 0.58 : 0.72;
  ctx.globalAlpha = alpha;
  const farBase = horizon + h * 0.025;
  if (place === "city" || place === "tokyo") {
    for (let i = 0; i < 18; i += 1) {
      const bw = w * (0.025 + seededUnit(seed, i) * 0.035);
      const bh = h * (0.16 + seededUnit(seed, i + 11) * 0.22);
      const x = ((i * w * 0.078 - (raceState.roadOffset || 0) * 0.018) % (w + 140)) - 70;
      const tower = ctx.createLinearGradient(x, farBase - bh, x + bw, farBase);
      tower.addColorStop(0, place === "tokyo" ? "rgba(255,79,216,0.18)" : "rgba(70,217,255,0.12)");
      tower.addColorStop(0.32, "rgba(15,25,28,0.72)");
      tower.addColorStop(1, "rgba(5,8,7,0.92)");
      ctx.fillStyle = tower;
      roundRect(x, farBase - bh, bw, bh, 3);
      ctx.fill();
      ctx.fillStyle = i % 2 ? `${design.accent}66` : `${design.light}55`;
      for (let yy = 12; yy < bh - 8; yy += 22) ctx.fillRect(x + bw * 0.24, farBase - bh + yy, bw * 0.52, 3);
    }
  } else if (place === "coast" || place === "harbor") {
    const water = ctx.createLinearGradient(0, horizon - h * 0.01, 0, horizon + h * 0.19);
    water.addColorStop(0, "rgba(70,217,255,0.22)");
    water.addColorStop(1, "rgba(4,22,26,0.54)");
    ctx.fillStyle = water;
    ctx.fillRect(0, horizon - h * 0.01, w, h * 0.22);
    for (let i = 0; i < 7; i += 1) {
      const x = ((i * w * 0.18 - (raceState.roadOffset || 0) * 0.026) % (w + 160)) - 80;
      ctx.fillStyle = place === "harbor" ? "rgba(10,42,46,0.78)" : "rgba(35,54,45,0.72)";
      ctx.beginPath();
      ctx.ellipse(x, horizon + h * 0.085, w * 0.08, h * 0.028, 0, 0, Math.PI * 2);
      ctx.fill();
      if (place === "harbor") {
        ctx.strokeStyle = `${design.light}88`;
        ctx.lineWidth = 3;
        ctx.beginPath();
        ctx.moveTo(x, horizon + h * 0.07);
        ctx.lineTo(x + w * 0.035, horizon - h * 0.09);
        ctx.lineTo(x + w * 0.09, horizon - h * 0.085);
```

```
        ctx.stroke();
      }
    }
  } else if (place === "canyon" || place === "desert") {
    for (let i = 0; i < 8; i += 1) {
      const x = ((i * w * 0.16 - (raceState.roadOffset || 0) * 0.014) % (w + 180)) - 90;
      const ridgeH = h * (0.09 + seededUnit(seed, i + 3) * 0.12);
      ctx.fillStyle = place === "desert" ? "rgba(255,183,74,0.24)" : "rgba(255,91,107,0.22)";
      ctx.beginPath();
      ctx.moveTo(x - w * 0.08, farBase + h * 0.03);
      ctx.quadraticCurveTo(x, farBase - ridgeH, x + w * 0.12, farBase + h * 0.035);
      ctx.closePath();
      ctx.fill();
    }
  } else if (place === "alpine" || place === "snow" || place === "europe") {
    for (let i = 0; i < 6; i += 1) {
      const x = i * w * 0.2 - ((raceState.roadOffset || 0) * 0.01 % (w * 0.2));
      const peak = h * (0.18 + seededUnit(seed, i) * 0.13);
      ctx.fillStyle = "rgba(130,160,160,0.36)";
      ctx.beginPath();
      ctx.moveTo(x - w * 0.16, farBase + h * 0.05);
      ctx.lineTo(x, farBase - peak);
      ctx.lineTo(x + w * 0.18, farBase + h * 0.05);
      ctx.closePath();
      ctx.fill();
      ctx.fillStyle = "rgba(244,251,248,0.42)";
      ctx.beginPath();
      ctx.moveTo(x - w * 0.035, farBase - peak * 0.58);
      ctx.lineTo(x, farBase - peak);
      ctx.lineTo(x + w * 0.045, farBase - peak * 0.56);
      ctx.closePath();
      ctx.fill();
    }
  } else if (place === "farm" || place === "freight") {
    for (let i = 0; i < 8; i += 1) {
      const x = ((i * w * 0.15 - (raceState.roadOffset || 0) * 0.02) % (w + 140)) - 70;
      ctx.fillStyle = place === "farm" ? "rgba(95,70,42,0.7)" : "rgba(30,34,30,0.78)";
      roundRect(x, farBase - h * 0.08, w * 0.075, h * 0.08, 4);
      ctx.fill();
      ctx.fillStyle = place === "farm" ? "rgba(160,40,31,0.72)" : `${design.light}66`;
      ctx.beginPath();
      ctx.moveTo(x - w * 0.01, farBase - h * 0.08);
      ctx.lineTo(x + w * 0.037, farBase - h * 0.13);
      ctx.lineTo(x + w * 0.085, farBase - h * 0.08);
      ctx.closePath();
      ctx.fill();
    }
  } else if (place === "rainforest") {
    for (let i = 0; i < 16; i += 1) {
      const x = ((i * w * 0.08 - (raceState.roadOffset || 0) * 0.025) % (w + 120)) - 60;
      const treeH = h * (0.13 + seededUnit(seed, i) * 0.12);
      ctx.strokeStyle = "rgba(24,78,52,0.72)";
      ctx.lineWidth = 4;
      ctx.beginPath();
      ctx.moveTo(x, farBase + h * 0.07);
      ctx.lineTo(x, farBase - treeH);
      ctx.stroke();
      ctx.fillStyle = "rgba(54,217,138,0.32)";
      ctx.beginPath();
      ctx.ellipse(x, farBase - treeH, w * 0.045, h * 0.035, -0.3, 0, Math.PI * 2);
      ctx.fill();
    }
  }
  ctx.globalAlpha = alpha * 0.82;
  ctx.fillStyle = "rgba(5,8,7,0.74)";
  roundRect(w * 0.39, horizon - h * 0.055, w * 0.22, h * 0.045, 6);
  ctx.fill();
  ctx.strokeStyle = `${design.accent}77`;
  ctx.lineWidth = 1.4;
  ctx.stroke();
  ctx.fillStyle = "#f4fbf8";
  ctx.font = `900 ${Math.max(8, Math.min(13, w * 0.013))}px system-ui`;
  ctx.textAlign = "center";
  ctx.fillText(route.scene.toUpperCase(), w * 0.5, horizon - h * 0.026);
  ctx.restore();
}

function drawGenAiSurfaceDetails(w, h, theme, horizon, roadTop, roadBottom, design, seed) {
  ctx.save();
  const phoneMode = phoneGraphicsActive();
  if (phoneCleanRoadActive()) {
    ctx.restore();
    return;
  }
```

```
  const laneX = (lane, t) => perspectiveRoadCenter(w, t) + lane * laneWidth() * (0.34 + t * 1.05);
  ctx.beginPath();
  ctx.moveTo(perspectiveRoadCenter(w, 0) - roadTop, horizon);
  ctx.lineTo(perspectiveRoadCenter(w, 0) + roadTop, horizon);
  ctx.lineTo(perspectiveRoadCenter(w, 1) + roadBottom * 0.63, h + 10);
  ctx.lineTo(perspectiveRoadCenter(w, 1) - roadBottom * 0.63, h + 10);
  ctx.closePath();
  ctx.clip();
  const tint = ctx.createLinearGradient(0, horizon, 0, h);
  tint.addColorStop(0, `${design.surface}11`);
  tint.addColorStop(0.5, `${design.surface}42`);
  tint.addColorStop(1, "rgba(0,0,0,0.42)");
  ctx.fillStyle = tint;
  ctx.fillRect(0, horizon, w, h - horizon);
  const speed = Math.max(0, raceState.speed || 0);
  const motion = (raceState.roadOffset || 0) * (1.2 + Math.min(1.4, speed / 180));
  const grimeCount = phoneMode ? 0 : 42;
  for (let i = 0; i < grimeCount; i += 1) {
    const y = horizon + (((i * 47 + motion * (0.54 + seededUnit(seed, i) * 0.28)) % (h - horizon + 150)) - 42);
    if (y < horizon || y > h + 20) continue;
    const t = Math.max(0, Math.min(1, (y - horizon) / Math.max(1, h - horizon)));
    const x = laneX(-2 + seededUnit(seed, i + 20) * 4, t);
    const len = phoneMode ? 5 + t * 20 : 10 + t * 70;
    ctx.globalAlpha = phoneMode ? 0.035 + t * 0.045 : 0.08 + t * 0.18;
    ctx.strokeStyle = phoneMode ? "rgba(244,251,248,0.22)" : (seededUnit(seed, i + 30) > 0.54 ?
"rgba(244,251,248,0.34)" : "rgba(0,0,0,0.78)");
    ctx.lineWidth = phoneMode ? 0.7 + t * 1.1 : 1 + t * 3.5;
    ctx.beginPath();
    ctx.moveTo(x, y);
    ctx.lineTo(x + Math.sin(i) * len * 0.18, y + len * 0.16);
    ctx.stroke();
  }
  const racingLine = ctx.createLinearGradient(0, horizon, 0, h);
  racingLine.addColorStop(0, "rgba(255,255,255,0)");
  racingLine.addColorStop(0.42, `${design.accent}28`);
  racingLine.addColorStop(1, "rgba(255,255,255,0)");
  ctx.strokeStyle = racingLine;
  ctx.lineWidth = Math.max(4, w * 0.006);
  ctx.globalAlpha = phoneMode ? 0.18 : 0.42;
  ctx.beginPath();
  for (let step = 0; step <= 18; step += 1) {
    const t = step / 18;
    const y = horizon + (h - horizon) * t;
    const lane = Math.sin((raceState.distance || 0) * 0.0009 + t * 3.5) * 0.72;
    const x = laneX(lane, t);
    if (step === 0) ctx.moveTo(x, y);
    else ctx.lineTo(x, y);
  }
  ctx.stroke();
  ctx.globalAlpha = 1;
  ctx.restore();
}

function drawGenAiTrackProp(prop, x, y, scale, side, design, theme, route, seed) {
  ctx.save();
  ctx.translate(x, y);
  ctx.scale(scale, scale);
  ctx.globalAlpha = Math.max(0.25, Math.min(0.9, 0.34 + scale * 0.28));
  const accent = design.accent;
  const light = design.light;
  if (prop === "tower" || prop === "neon" || prop === "village") {
    const bw = 24 + seededUnit(seed, 1) * 26;
    const bh = 70 + seededUnit(seed, 2) * 90;
    ctx.fillStyle = prop === "neon" ? "rgba(14,10,30,0.82)" : "rgba(12,18,20,0.8)";
    roundRect(-bw / 2, -bh, bw, bh, 3);
    ctx.fill();
    ctx.fillStyle = prop === "neon" ? `${accent}aa` : `${light}77`;
    for (let i = 0; i < 5; i += 1) ctx.fillRect(-bw * 0.32, -bh + 10 + i * 15, bw * 0.64, 4);
  } else if (prop === "cliff" || prop === "rock" || prop === "dune" || prop === "snowcap") {
    ctx.fillStyle = prop === "snowcap" ? "rgba(220,232,239,0.48)" : prop === "dune" ? "rgba(255,183,74,0.38)" :
"rgba(132,72,38,0.52)";
    ctx.beginPath();
    ctx.moveTo(-54, 12);
    ctx.quadraticCurveTo(-10, -42 - seededUnit(seed, 3) * 26, 58, 14);
    ctx.closePath();
    ctx.fill();
    ctx.fillStyle = "rgba(244,251,248,0.25)";
    ctx.fillRect(-18, -8, 38, 5);
  } else if (prop === "crowd" || prop === "camera") {
    ctx.fillStyle = "rgba(5,8,7,0.74)";
    roundRect(-45, -22, 90, 22, 4);
    ctx.fill();
    for (let i = 0; i < 9; i += 1) {
      ctx.fillStyle = i % 3 === 0 ? accent : i % 3 === 1 ? light : "#f4fbf8";
```

```
      ctx.beginPath();
      ctx.arc(-34 + i * 8, -26 - (i % 2) * 3, 3.2, 0, Math.PI * 2);
      ctx.fill();
    }
    if (prop === "camera") {
      ctx.strokeStyle = "rgba(244,251,248,0.42)";
      ctx.lineWidth = 2;
      ctx.beginPath();
      ctx.moveTo(30, -20);
      ctx.lineTo(42, -42);
      ctx.stroke();
      ctx.fillStyle = "rgba(5,8,7,0.9)";
      roundRect(35, -48, 20, 12, 3);
      ctx.fill();
    }
  } else if (prop === "barrier" || prop === "guard" || prop === "brake") {
    const text = prop === "brake" ? "100" : route.country.toUpperCase();
    ctx.fillStyle = "rgba(5,8,7,0.76)";
    roundRect(-42, -30, 84, 24, 4);
    ctx.fill();
    ctx.strokeStyle = accent;
    ctx.lineWidth = 2;
    ctx.stroke();
    ctx.fillStyle = "#f4fbf8";
    ctx.font = "900 12px system-ui";
    ctx.textAlign = "center";
    ctx.fillText(text, 0, -14);
  } else if (prop === "crane" || prop === "gantry" || prop === "hangar" || prop === "tunnel" || prop === "bridge") {
    ctx.strokeStyle = `${light}88`;
    ctx.lineWidth = 4;
    ctx.beginPath();
    ctx.moveTo(-55, -8);
    ctx.lineTo(-55, -72);
    ctx.lineTo(52, -72);
    ctx.lineTo(52, -8);
    ctx.stroke();
    ctx.fillStyle = "rgba(5,8,7,0.62)";
    roundRect(-35, -58, 70, 18, 3);
    ctx.fill();
  } else {
    ctx.fillStyle = `${accent}66`;
    ctx.beginPath();
    ctx.ellipse(0, -14, 34, 16, 0, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "rgba(244,251,248,0.28)";
    ctx.beginPath();
    ctx.moveTo(0, -8);
    ctx.lineTo(side * 6, 32);
    ctx.stroke();
  }
  ctx.restore();
}

function drawGenAiCrowdAndCameras(w, h, horizon, roadTop, roadBottom, design) {
  const phoneMode = phoneGraphicsActive();
  const bands = phoneMode ? 2 : 3;
  ctx.save();
  for (let side = -1; side <= 1; side += 2) {
    for (let band = 0; band < bands; band += 1) {
      const t = 0.18 + band * 0.18;
      const y = horizon + (h - horizon) * t + Math.sin(raceState.elapsed + band) * 2;
      const center = perspectiveRoadCenter(w, t);
      const roadHalf = roadTop * 0.92 + (roadBottom * 0.58 - roadTop * 0.92) * t;
      const x = center + side * (roadHalf + w * (0.05 + band * 0.025));
      const standW = w * (0.12 + band * 0.03);
      const standH = h * (0.025 + band * 0.012);
      ctx.globalAlpha = 0.18 + band * 0.09;
      ctx.fillStyle = "rgba(5,8,7,0.78)";
      roundRect(x - standW / 2, y - standH, standW, standH, 4);
      ctx.fill();
      for (let i = 0; i < 16; i += 1) {
        ctx.fillStyle = i % 4 === 0 ? design.accent : i % 4 === 1 ? design.light : "rgba(244,251,248,0.76)";
        ctx.fillRect(x - standW * 0.42 + i * standW * 0.052, y - standH - 3 - (i % 2) * 2, 2 + band, 5 + band);
      }
    }
  }
  ctx.restore();
}

function drawDriftStylePass(w, h, theme) {
  if (!raceState.active || raceState.resetTimer > 0) return;
  if (phoneCleanRoadActive()) return;
  const speed = Math.abs(raceState.speed || 0);
  const drift = Math.max(0, Math.min(1, ((raceState.slip || 0) - 0.14) * 1.95 + Math.abs(raceState.lateralVelocity ||
```

```
0) * 0.07));
  if (drift <= 0.03 || speed < 30) return;
  const vehicle = selectedVehicle();
  const carX = w / 2 + raceState.lane * laneWidth();
  const baseY = cameraMode === "cockpit" ? h * 0.78 : cameraMode === "hood" ? h * 0.91 : h * 0.87;
  const steer = raceState.steerAngle || 0;
  const slide = raceState.lateralVelocity || 0;
  ctx.save();
  ctx.globalAlpha = Math.min(0.86, 0.26 + drift * 0.58);
  ctx.strokeStyle = "rgba(1,2,2,0.72)";
  ctx.lineWidth = Math.max(2, w * 0.005 + drift * 4);
  ctx.lineCap = "round";
  for (let side = -1; side <= 1; side += 2) {
    const tireX = carX + side * laneWidth() * 0.2;
    ctx.beginPath();
    ctx.moveTo(tireX, baseY + h * 0.02);
    ctx.bezierCurveTo(tireX - slide * 18 - steer * 30, baseY + h * 0.08, tireX - slide * 34 - steer * 55, baseY + h *
0.2, tireX - slide * 50 - steer * 78, h + 22);
    ctx.stroke();
  }
  ctx.globalCompositeOperation = "screen";
  const smoke = ctx.createRadialGradient(carX, baseY + h * 0.02, 4, carX - slide * 35, baseY + h * 0.04, w * (0.12 +
drift * 0.14));
  const smokeColor = vehicle.type === "snowmobile" ? "rgba(244,251,248," : selectedRace.place === "desert" ||
selectedRace.place === "canyon" ? "rgba(255,183,74," : "rgba(190,205,205,";
  smoke.addColorStop(0, `${smokeColor}${0.12 + drift * 0.18})`);
  smoke.addColorStop(0.5, `${smokeColor}${0.06 + drift * 0.1})`);
  smoke.addColorStop(1, `${smokeColor}0)`);
  ctx.fillStyle = smoke;
  ctx.fillRect(0, h * 0.48, w, h * 0.52);
  if (drift > 0.35 && cameraMode !== "cockpit") {
    ctx.fillStyle = theme[1];
    ctx.font = `900 ${Math.max(13, Math.min(20, w * 0.018))}px system-ui`;
    ctx.textAlign = "center";
    ctx.globalAlpha = Math.min(0.62, drift);
    ctx.fillText("DRIFT", carX + Math.sign(slide || steer || 1) * laneWidth() * 0.55, baseY - h * 0.18);
  }
  ctx.restore();
}

function drawWebGLFlatPavementPass(w, h, theme) {
  if (phoneGraphicsActive()) {
    const floorY = cameraMode === "cockpit" ? h * 0.52 : cameraMode === "hood" ? h * 0.56 : h * 0.58;
    const speed = Math.max(0, Math.abs(raceState.speed || 0));
    const offset = raceState.roadOffset || 0;
    ctx.save();
    const shade = ctx.createLinearGradient(0, floorY, 0, h);
    shade.addColorStop(0, "rgba(0,0,0,0)");
    shade.addColorStop(0.35, "rgba(5,7,7,0.42)");
    shade.addColorStop(1, "rgba(0,0,0,0.72)");
    ctx.fillStyle = shade;
    ctx.fillRect(0, floorY, w, h - floorY);
    if (phoneCleanRoadActive()) {
      ctx.restore();
      return;
    }
    for (let i = 0; i < 16; i += 1) {
      const y = floorY + (((i * 34 + offset * (0.34 + speed / 760)) % (h - floorY + 60)) - 20);
      if (y < floorY || y > h) continue;
      const t = (y - floorY) / Math.max(1, h - floorY);
      ctx.globalAlpha = 0.08 + t * 0.16;
      ctx.strokeStyle = i % 2 ? "rgba(244,251,248,0.12)" : "rgba(2,3,3,0.48)";
      ctx.lineWidth = 1 + t * 3;
      ctx.beginPath();
      ctx.moveTo(w * 0.18, y);
      ctx.lineTo(w * 0.82, y + 1);
      ctx.stroke();
    }
    ctx.restore();
    return;
  }
  const floorY = cameraMode === "cockpit" ? h * 0.36 : cameraMode === "hood" ? h * 0.39 : h * 0.42;
  const turn = raceState.roadTurn || raceState.roadCurve || 0;
  const speed = Math.max(0, Math.abs(raceState.speed || 0));
  const offset = raceState.roadOffset || 0;
  ctx.save();

  const roadTop = w * (cameraMode === "cockpit" ? 0.16 : cameraMode === "hood" ? 0.19 : 0.18);
  const roadBottom = w * (cameraMode === "cockpit" ? 0.96 : cameraMode === "hood" ? 1.04 : 1.1);
  const roadCenter = (t) => w * 0.5 + turn * (1 - t) * (1 - t) * w * 0.12;
  ctx.beginPath();
  ctx.moveTo(roadCenter(0) - roadTop, floorY);
  ctx.lineTo(roadCenter(0) + roadTop, floorY);
  ctx.lineTo(roadCenter(1) + roadBottom * 0.55, h + 10);
```

```
  ctx.lineTo(roadCenter(1) - roadBottom * 0.55, h + 10);
  ctx.closePath();
  const asphalt = ctx.createLinearGradient(0, floorY, 0, h);
  asphalt.addColorStop(0, "rgba(19,23,23,0.99)");
  asphalt.addColorStop(0.42, "rgba(11,14,14,1)");
  asphalt.addColorStop(1, "rgba(2,3,3,1)");
  ctx.fillStyle = asphalt;
  ctx.fill();
  ctx.clip();

  for (let i = 0; i < 20; i += 1) {
    const y = floorY + (((i * 32 + offset * (0.42 + speed / 650)) % (h - floorY + 72)) - 24);
    if (y < floorY || y > h) continue;
    const t = (y - floorY) / Math.max(1, h - floorY);
    ctx.globalAlpha = 0.08 + t * 0.2;
    ctx.strokeStyle = i % 2 ? "rgba(244,251,248,0.14)" : "rgba(6,8,8,0.42)";
    ctx.lineWidth = 1 + t * 3.5;
    ctx.beginPath();
    const center = roadCenter(t);
    const half = w * (0.24 + t * 0.42);
    ctx.moveTo(center - half, y);
    ctx.lineTo(center + half, y + 1);
    ctx.stroke();
  }

  for (let side = -1; side <= 1; side += 2) {
    for (let i = 0; i < 8; i += 1) {
      const y = floorY + (((i * 58 + offset * 0.42) % (h - floorY + 80)) - 16);
      if (y < floorY || y > h) continue;
      const t = (y - floorY) / Math.max(1, h - floorY);
      const center = roadCenter(t);
      const roadHalf = w * (0.25 + t * 0.4);
      ctx.globalAlpha = 0.1 + t * 0.18;
      ctx.fillStyle = side < 0 ? "rgba(255,91,107,0.22)" : "rgba(70,217,255,0.2)";
      roundRect(center + side * roadHalf - side * (14 + t * 24), y, 14 + t * 32, 2 + t * 4, 2);
      ctx.fill();
    }
  }
  ctx.restore();
}

function drawScenery(w, h, theme) {
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  ctx.save();
  if (place === "coast") drawCoastalScenery(w, h, theme);
  if (place === "city") drawMetroScenery(w, h, theme);
  if (place === "canyon") drawCanyonScenery(w, h, theme);
  if (place === "alpine") drawAlpineScenery(w, h, theme);
  if (place === "harbor") drawHarborScenery(w, h, theme);
  if (place === "snow") drawSnowScenery(w, h, theme);
  if (place === "airfield") drawAirfieldScenery(w, h, theme);
  if (place === "freight") drawFreightScenery(w, h, theme);
  if (place === "farm") drawFarmScenery(w, h, theme);
  if (place === "tokyo") drawTokyoScenery(w, h, theme);
  if (place === "desert") drawWorldDesertScenery(w, h, theme);
  if (place === "rainforest") drawRainforestScenery(w, h, theme);
  if (place === "europe") drawEuropeanScenery(w, h, theme);
  drawAtmosphericDepth(w, h, theme);
  drawRoadSigns(w, h, theme);
  ctx.restore();
}

function drawRouteLightingPass(w, h, theme) {
  const night = raceIsNight(selectedRace);
  const turn = raceState.roadTurn || raceState.roadCurve || 0;
  ctx.save();
  if (!night) {
    const sunX = w * (0.18 + (Math.sin((raceState.distance || 0) * 0.00004) + 1) * 0.08);
    const sunY = h * 0.16;
    const glow = ctx.createRadialGradient(sunX, sunY, 4, sunX, sunY, w * 0.34);
    glow.addColorStop(0, "rgba(255,209,102,0.26)");
    glow.addColorStop(1, "rgba(255,209,102,0)");
    ctx.fillStyle = glow;
    ctx.fillRect(0, 0, w, h * 0.5);
    ctx.restore();
    return;
  }
  const skyShade = ctx.createLinearGradient(0, 0, 0, h);
  skyShade.addColorStop(0, "rgba(0,0,0,0.34)");
  skyShade.addColorStop(0.48, "rgba(0,0,0,0.08)");
  skyShade.addColorStop(1, "rgba(0,0,0,0.26)");
  ctx.fillStyle = skyShade;
  ctx.fillRect(0, 0, w, h);
  ctx.globalCompositeOperation = "screen";
```

```
  const horizon = phoneGraphicsActive() ? h * 0.34 : h * 0.44;
  for (let i = 0; i < 11; i += 1) {
    const depth = (i + 1) / 12;
    const y = horizon + Math.pow(depth, 1.7) * h * 0.52;
    const center = perspectiveRoadCenter(w, depth) + turn * (1 - depth) * w * 0.05;
    const half = w * (0.18 + depth * 0.43);
    for (let side = -1; side <= 1; side += 2) {
      const x = center + side * half;
      const lamp = ctx.createRadialGradient(x, y, 2, x, y, w * (0.025 + depth * 0.04));
      lamp.addColorStop(0, "rgba(255,229,148,0.34)");
      lamp.addColorStop(1, "rgba(255,229,148,0)");
      ctx.fillStyle = lamp;
      ctx.fillRect(x - w * 0.09, y - h * 0.08, w * 0.18, h * 0.16);
      ctx.strokeStyle = "rgba(244,251,248,0.14)";
      ctx.lineWidth = Math.max(1, 2 * depth);
      ctx.beginPath();
      ctx.moveTo(x, y);
      ctx.lineTo(x, y + h * (0.04 + depth * 0.04));
      ctx.stroke();
    }
  }
  ctx.restore();
}

function drawAtmosphericDepth(w, h, theme) {
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  const haze = ctx.createLinearGradient(0, h * 0.16, 0, h * 0.55);
  haze.addColorStop(0, "rgba(244,251,248,0.06)");
  haze.addColorStop(0.65, "rgba(70,217,255,0.05)");
  haze.addColorStop(1, "rgba(5,8,7,0)");
  ctx.fillStyle = haze;
  ctx.fillRect(0, h * 0.12, w, h * 0.44);

  if (place === "coast") {
    for (let i = 0; i < 10; i += 1) {
      const x = ((i * 131 + raceState.roadOffset * 0.03) % (w + 120)) - 60;
      ctx.strokeStyle = "rgba(244,251,248,0.22)";
      ctx.lineWidth = 3;
      ctx.beginPath();
      ctx.moveTo(x, h * 0.31);
      ctx.lineTo(x - 20, h * 0.42);
      ctx.stroke();
      ctx.fillStyle = "rgba(187,242,74,0.18)";
      ctx.beginPath();
      ctx.ellipse(x - 24, h * 0.28, 22, 7, -0.35, 0, Math.PI * 2);
      ctx.ellipse(x + 3, h * 0.27, 20, 6, 0.35, 0, Math.PI * 2);
      ctx.fill();
    }
  }

  if (place === "city") {
    ctx.strokeStyle = "rgba(255,255,255,0.16)";
    ctx.lineWidth = 4;
    for (let i = 0; i < 5; i += 1) {
      const x = w * (0.12 + i * 0.19);
      ctx.beginPath();
      ctx.moveTo(x, h * 0.34);
      ctx.lineTo(x + Math.sin(i) * 36, h * 0.2);
      ctx.stroke();
    }
  }

  if (place === "alpine") {
    ctx.fillStyle = "rgba(244,251,248,0.22)";
    for (let i = 0; i < 8; i += 1) {
      const x = ((i * 151 + raceState.roadOffset * 0.02) % (w + 160)) - 80;
      ctx.beginPath();
      ctx.moveTo(x, h * 0.31);
      ctx.lineTo(x + 38, h * 0.18);
      ctx.lineTo(x + 76, h * 0.31);
      ctx.closePath();
      ctx.fill();
    }
  }

  if (place === "harbor") {
    ctx.strokeStyle = "rgba(244,251,248,0.24)";
    ctx.lineWidth = 3;
    for (let i = 0; i < 9; i += 1) {
      const x = ((i * 137 + raceState.roadOffset * 0.05) % (w + 150)) - 75;
      ctx.beginPath();
      ctx.moveTo(x, h * 0.24);
      ctx.lineTo(x + 22, h * 0.36);
      ctx.stroke();
```

```
    }
  }

  if (place === "snow") {
    ctx.fillStyle = "rgba(244,251,248,0.42)";
    for (let i = 0; i < 70; i += 1) {
      const x = (i * 59 + raceState.roadOffset * 0.06) % w;
      const y = h * 0.12 + ((i * 37 + raceState.elapsed * 28) % (h * 0.42));
      ctx.fillRect(x, y, 2, 2);
    }
  }

  if (place === "airfield") {
    ctx.strokeStyle = "rgba(255,209,102,0.22)";
    ctx.lineWidth = 2;
    for (let i = 0; i < 13; i += 1) {
      const x = ((i * 91 + raceState.roadOffset * 0.08) % (w + 120)) - 60;
      ctx.beginPath();
      ctx.moveTo(x, h * 0.36);
      ctx.lineTo(x + 34, h * 0.36);
      ctx.stroke();
    }
  }

  if (place === "tokyo") {
    ctx.strokeStyle = "rgba(255,79,216,0.22)";
    ctx.lineWidth = 3;
    for (let i = 0; i < 12; i += 1) {
      const x = ((i * 91 + raceState.roadOffset * 0.11) % (w + 120)) - 60;
      ctx.beginPath();
      ctx.moveTo(x, h * 0.19);
      ctx.lineTo(x + 18, h * 0.43);
      ctx.stroke();
    }
  }

  if (place === "farm" || place === "freight") {
    ctx.strokeStyle = "rgba(255,209,102,0.16)";
    ctx.lineWidth = 2;
    for (let i = 0; i < 16; i += 1) {
      const y = h * 0.38 + i * 12;
      ctx.beginPath();
      ctx.moveTo(0, y);
      ctx.lineTo(w, y + Math.sin(i) * 10);
      ctx.stroke();
    }
  }

  if (place === "rainforest") {
    ctx.fillStyle = "rgba(54,217,138,0.16)";
    for (let i = 0; i < 70; i += 1) {
      const x = ((i * 47 + raceState.roadOffset * 0.05) % (w + 80)) - 40;
      const y = h * 0.18 + (i % 18) * 16;
      ctx.beginPath();
      ctx.ellipse(x, y, 16 + (i % 5) * 3, 6, Math.sin(i), 0, Math.PI * 2);
      ctx.fill();
    }
  }
}

function drawMetroScenery(w, h, theme) {
  const lake = ctx.createLinearGradient(0, h * 0.33, 0, h * 0.56);
  lake.addColorStop(0, "rgba(38,84,96,0.38)");
  lake.addColorStop(1, "rgba(5,8,7,0.16)");
  ctx.fillStyle = lake;
  ctx.fillRect(0, h * 0.34, w, h * 0.18);
  ctx.strokeStyle = "rgba(244,251,248,0.12)";
  for (let i = 0; i < 7; i += 1) {
    const y = h * 0.39 + i * 18;
    ctx.beginPath();
    ctx.moveTo(0, y);
    ctx.bezierCurveTo(w * 0.3, y - 8, w * 0.62, y + 10, w, y - 3);
    ctx.stroke();
  }
  for (let i = 0; i < 42; i += 1) {
    const x = ((i * 93 + raceState.roadOffset * 0.08) % (w + 180)) - 90;
    const bh = 92 + ((i * 47) % 230);
    const tower = ctx.createLinearGradient(x, h * 0.34 - bh, x + 70, h * 0.34);
    tower.addColorStop(0, i % 3 === 0 ? "rgba(70,217,255,0.26)" : "rgba(255,255,255,0.12)");
    tower.addColorStop(1, "rgba(5,8,7,0.4)");
    ctx.fillStyle = tower;
    ctx.fillRect(x, h * 0.34 - bh, 42 + (i % 5) * 11, bh);
    ctx.fillStyle = theme[i % 2 + 1];
    ctx.globalAlpha = 0.58;
```

```
    for (let y = 18; y < bh - 12; y += 22) ctx.fillRect(x + 9 + (y % 3) * 8, h * 0.34 - bh + y, 7, 8);
    ctx.globalAlpha = 1;
  }
  ctx.strokeStyle = "rgba(255,255,255,0.14)";
  ctx.lineWidth = 2;
  ctx.beginPath();
  ctx.moveTo(w * 0.07, h * 0.34);
  ctx.lineTo(w * 0.93, h * 0.34);
  ctx.stroke();
}

function drawCoastalScenery(w, h, theme) {
  const sun = ctx.createRadialGradient(w * 0.16, h * 0.16, 8, w * 0.16, h * 0.16, 130);
  sun.addColorStop(0, "rgba(255,209,102,0.58)");
  sun.addColorStop(1, "rgba(255,209,102,0)");
  ctx.fillStyle = sun;
  ctx.fillRect(0, 0, w, h * 0.4);
  const water = ctx.createLinearGradient(0, h * 0.32, 0, h * 0.58);
  water.addColorStop(0, "rgba(70,217,255,0.2)");
  water.addColorStop(1, "rgba(5,8,7,0.1)");
  ctx.fillStyle = water;
  ctx.fillRect(0, h * 0.32, w, h * 0.22);
  ctx.strokeStyle = "rgba(244,251,248,0.18)";
  ctx.lineWidth = 2;
  for (let i = 0; i < 8; i += 1) {
    const y = h * 0.37 + i * 18;
    ctx.beginPath();
    ctx.moveTo(0, y);
    ctx.bezierCurveTo(w * 0.25, y + 14, w * 0.5, y - 12, w, y + 8);
    ctx.stroke();
  }
  ctx.strokeStyle = theme[1];
  ctx.lineWidth = 5;
  ctx.beginPath();
  ctx.moveTo(w * 0.08, h * 0.32);
  ctx.quadraticCurveTo(w * 0.5, h * 0.12, w * 0.92, h * 0.32);
  ctx.stroke();
  for (let i = 0; i < 9; i += 1) {
    const x = w * 0.12 + i * w * 0.095;
    ctx.strokeStyle = "rgba(244,251,248,0.3)";
    ctx.lineWidth = 3;
    ctx.beginPath();
    ctx.moveTo(x, h * 0.18);
    ctx.lineTo(x, h * 0.34);
    ctx.stroke();
  }
}

function drawCanyonScenery(w, h, theme) {
  const layers = [
    { y: 0.35, color: "rgba(255,91,107,0.28)", scale: 1 },
    { y: 0.42, color: "rgba(255,209,102,0.22)", scale: 0.74 },
    { y: 0.49, color: "rgba(111,54,30,0.45)", scale: 0.55 }
  ];
  layers.forEach((layer, layerIndex) => {
    ctx.fillStyle = layer.color;
    ctx.beginPath();
    ctx.moveTo(0, h * layer.y);
    for (let i = 0; i <= 10; i += 1) {
      const x = i * w * 0.1;
      const y = h * layer.y - (((i * 37 + layerIndex * 19) % 90) + 20) * layer.scale;
      ctx.lineTo(x, y);
    }
    ctx.lineTo(w, h * 0.56);
    ctx.lineTo(0, h * 0.56);
    ctx.closePath();
    ctx.fill();
  });
  ctx.fillStyle = "rgba(255,209,102,0.32)";
  ctx.beginPath();
  ctx.arc(w * 0.82, h * 0.16, 42, 0, Math.PI * 2);
  ctx.fill();
}

function drawAlpineScenery(w, h, theme) {
  const fog = ctx.createLinearGradient(0, h * 0.16, 0, h * 0.56);
  fog.addColorStop(0, "rgba(185,216,224,0.16)");
  fog.addColorStop(1, "rgba(185,216,224,0)");
  ctx.fillStyle = fog;
  ctx.fillRect(0, h * 0.14, w, h * 0.44);
  ctx.fillStyle = "rgba(244,251,248,0.16)";
  ctx.beginPath();
  ctx.moveTo(0, h * 0.38);
  ctx.lineTo(w * 0.18, h * 0.16);
```

```
  ctx.lineTo(w * 0.34, h * 0.38);
  ctx.lineTo(w * 0.54, h * 0.12);
  ctx.lineTo(w * 0.78, h * 0.38);
  ctx.lineTo(w, h * 0.2);
  ctx.lineTo(w, h * 0.52);
  ctx.lineTo(0, h * 0.52);
  ctx.closePath();
  ctx.fill();
  ctx.fillStyle = "rgba(187,242,74,0.1)";
  for (let i = 0; i < 38; i += 1) {
    const x = ((i * 71 + raceState.roadOffset * 0.04) % (w + 80)) - 40;
    const y = h * 0.36 + (i % 5) * 20;
    ctx.beginPath();
    ctx.moveTo(x, y - 38);
    ctx.lineTo(x - 16, y);
    ctx.lineTo(x + 16, y);
    ctx.closePath();
    ctx.fill();
  }
}

function drawHarborScenery(w, h, theme) {
  const water = ctx.createLinearGradient(0, h * 0.28, 0, h * 0.62);
  water.addColorStop(0, "rgba(70,217,255,0.28)");
  water.addColorStop(1, "rgba(4,24,32,0.48)");
  ctx.fillStyle = water;
  ctx.fillRect(0, h * 0.3, w, h * 0.28);
  ctx.strokeStyle = "rgba(244,251,248,0.18)";
  for (let i = 0; i < 10; i += 1) {
    const y = h * 0.34 + i * 18;
    ctx.beginPath();
    ctx.moveTo(0, y);
    ctx.bezierCurveTo(w * 0.28, y + 12, w * 0.56, y - 10, w, y + 6);
    ctx.stroke();
  }
  for (let i = 0; i < 16; i += 1) {
    const x = ((i * 113 + raceState.roadOffset * 0.05) % (w + 160)) - 80;
    const y = h * (0.3 + (i % 3) * 0.055);
    ctx.fillStyle = i % 2 ? "rgba(255,209,102,0.28)" : "rgba(70,217,255,0.24)";
    ctx.beginPath();
    ctx.moveTo(x, y);
    ctx.lineTo(x + 36, y + 8);
    ctx.lineTo(x + 24, y + 18);
    ctx.lineTo(x - 18, y + 14);
    ctx.closePath();
    ctx.fill();
  }
  ctx.fillStyle = "rgba(5,8,7,0.42)";
  for (let i = 0; i < 12; i += 1) {
    const x = ((i * 97 + raceState.roadOffset * 0.03) % (w + 100)) - 50;
    ctx.fillRect(x, h * 0.22, 28 + (i % 4) * 18, h * 0.12);
  }
}

function drawSnowScenery(w, h, theme) {
  const skyGlow = ctx.createLinearGradient(0, h * 0.1, 0, h * 0.54);
  skyGlow.addColorStop(0, "rgba(244,251,248,0.2)");
  skyGlow.addColorStop(1, "rgba(118,168,188,0)");
  ctx.fillStyle = skyGlow;
  ctx.fillRect(0, h * 0.1, w, h * 0.48);
  ctx.fillStyle = "rgba(244,251,248,0.28)";
  ctx.beginPath();
  ctx.moveTo(0, h * 0.42);
  ctx.lineTo(w * 0.14, h * 0.18);
  ctx.lineTo(w * 0.3, h * 0.42);
  ctx.lineTo(w * 0.48, h * 0.14);
  ctx.lineTo(w * 0.72, h * 0.42);
  ctx.lineTo(w, h * 0.2);
  ctx.lineTo(w, h * 0.54);
  ctx.lineTo(0, h * 0.54);
  ctx.closePath();
  ctx.fill();
  ctx.fillStyle = "rgba(10,35,33,0.48)";
  for (let i = 0; i < 44; i += 1) {
    const x = ((i * 67 + raceState.roadOffset * 0.05) % (w + 90)) - 45;
    const y = h * 0.36 + (i % 4) * 24;
    ctx.beginPath();
    ctx.moveTo(x, y - 32);
    ctx.lineTo(x - 13, y);
    ctx.lineTo(x + 13, y);
    ctx.closePath();
    ctx.fill();
  }
}
```

```
function drawAirfieldScenery(w, h, theme) {
  const desert = ctx.createLinearGradient(0, h * 0.32, 0, h * 0.58);
  desert.addColorStop(0, "rgba(255,209,102,0.18)");
  desert.addColorStop(1, "rgba(77,55,35,0.34)");
  ctx.fillStyle = desert;
  ctx.fillRect(0, h * 0.32, w, h * 0.26);
  ctx.fillStyle = "rgba(5,8,7,0.5)";
  for (let i = 0; i < 9; i += 1) {
    const x = ((i * 151 + raceState.roadOffset * 0.04) % (w + 180)) - 90;
    const y = h * 0.31 + (i % 2) * 24;
    ctx.fillRect(x, y, 104, 42);
    ctx.beginPath();
    ctx.moveTo(x + 10, y);
    ctx.lineTo(x + 52, y - 28);
    ctx.lineTo(x + 94, y);
    ctx.closePath();
    ctx.fill();
  }
  ctx.strokeStyle = "rgba(244,251,248,0.32)";
  ctx.lineWidth = 3;
  for (let i = 0; i < 5; i += 1) {
    const x = ((i * 197 + raceState.roadOffset * 0.08) % (w + 220)) - 110;
    ctx.beginPath();
    ctx.moveTo(x, h * 0.25);
    ctx.lineTo(x + 80, h * 0.29);
    ctx.lineTo(x + 36, h * 0.32);
    ctx.stroke();
  }
}

function drawFreightScenery(w, h, theme) {
  const plains = ctx.createLinearGradient(0, h * 0.34, 0, h * 0.58);
  plains.addColorStop(0, "rgba(255,209,102,0.22)");
  plains.addColorStop(1, "rgba(57,66,31,0.28)");
  ctx.fillStyle = plains;
  ctx.fillRect(0, h * 0.34, w, h * 0.24);
  ctx.strokeStyle = "rgba(244,251,248,0.18)";
  ctx.lineWidth = 3;
  for (let i = 0; i < 10; i += 1) {
    const x = ((i * 146 + raceState.roadOffset * 0.08) % (w + 180)) - 90;
    ctx.beginPath();
    ctx.moveTo(x, h * 0.3);
    ctx.lineTo(x + 32, h * 0.46);
    ctx.stroke();
  }
  for (let i = 0; i < 8; i += 1) {
    const x = ((i * 181 + raceState.roadOffset * 0.045) % (w + 220)) - 110;
    ctx.fillStyle = "rgba(5,8,7,0.48)";
    roundRect(x, h * 0.28, 120, 34, 4);
    ctx.fill();
    ctx.fillStyle = theme[1];
    ctx.globalAlpha = 0.45;
    ctx.fillRect(x + 14, h * 0.3, 28, 6);
    ctx.fillRect(x + 58, h * 0.3, 42, 6);
    ctx.globalAlpha = 1;
  }
}

function drawFarmScenery(w, h, theme) {
  const fields = ctx.createLinearGradient(0, h * 0.32, 0, h * 0.6);
  fields.addColorStop(0, "rgba(187,242,74,0.2)");
  fields.addColorStop(1, "rgba(56,88,31,0.42)");
  ctx.fillStyle = fields;
  ctx.fillRect(0, h * 0.32, w, h * 0.28);
  ctx.strokeStyle = "rgba(255,209,102,0.22)";
  ctx.lineWidth = 2;
  for (let i = 0; i < 20; i += 1) {
    const y = h * 0.36 + i * 13;
    ctx.beginPath();
    ctx.moveTo(0, y);
    ctx.bezierCurveTo(w * 0.25, y + 8, w * 0.62, y - 7, w, y + 4);
    ctx.stroke();
  }
  for (let i = 0; i < 9; i += 1) {
    const x = ((i * 173 + raceState.roadOffset * 0.045) % (w + 200)) - 100;
    const y = h * 0.32 + (i % 3) * 26;
    ctx.fillStyle = i % 2 ? "rgba(160,40,31,0.5)" : "rgba(244,251,248,0.38)";
    ctx.fillRect(x, y, 58, 38);
    ctx.fillStyle = "rgba(117,86,47,0.72)";
    ctx.beginPath();
    ctx.moveTo(x - 5, y);
    ctx.lineTo(x + 29, y - 24);
    ctx.lineTo(x + 63, y);
```

```
    ctx.closePath();
    ctx.fill();
  }
}

function drawTokyoScenery(w, h, theme) {
  const glow = ctx.createLinearGradient(0, h * 0.08, 0, h * 0.56);
  glow.addColorStop(0, "rgba(255,79,216,0.12)");
  glow.addColorStop(0.56, "rgba(70,217,255,0.1)");
  glow.addColorStop(1, "rgba(5,8,7,0)");
  ctx.fillStyle = glow;
  ctx.fillRect(0, h * 0.08, w, h * 0.5);
  for (let i = 0; i < 44; i += 1) {
    const x = ((i * 83 + raceState.roadOffset * 0.12) % (w + 180)) - 90;
    const bh = 100 + ((i * 41) % 260);
    ctx.fillStyle = i % 2 ? "rgba(7,12,24,0.74)" : "rgba(16,10,28,0.78)";
    ctx.fillRect(x, h * 0.36 - bh, 36 + (i % 4) * 16, bh);
    ctx.fillStyle = i % 3 ? theme[1] : theme[2];
    ctx.globalAlpha = 0.6;
    for (let y = 18; y < bh - 12; y += 24) ctx.fillRect(x + 8, h * 0.36 - bh + y, 10 + (i % 3) * 7, 7);
    ctx.globalAlpha = 1;
  }
  ctx.strokeStyle = "rgba(244,251,248,0.22)";
  ctx.lineWidth = 5;
  ctx.beginPath();
  ctx.moveTo(0, h * 0.37);
  ctx.quadraticCurveTo(w * 0.5, h * 0.28, w, h * 0.37);
  ctx.stroke();
}

function drawWorldDesertScenery(w, h, theme) {
  const sand = ctx.createLinearGradient(0, h * 0.3, 0, h * 0.6);
  sand.addColorStop(0, "rgba(255,183,74,0.24)");
  sand.addColorStop(1, "rgba(92,55,27,0.42)");
  ctx.fillStyle = sand;
  ctx.fillRect(0, h * 0.3, w, h * 0.3);
  for (let layer = 0; layer < 3; layer += 1) {
    ctx.fillStyle = `rgba(255,183,74,${0.12 + layer * 0.08})`;
    ctx.beginPath();
    ctx.moveTo(0, h * (0.42 + layer * 0.04));
    for (let i = 0; i <= 8; i += 1) {
      const x = i * w * 0.125;
      const y = h * (0.42 + layer * 0.04) - Math.sin(i * 0.8 + layer + raceState.roadOffset * 0.001) * 34;
      ctx.lineTo(x, y);
    }
    ctx.lineTo(w, h * 0.62);
    ctx.lineTo(0, h * 0.62);
    ctx.closePath();
    ctx.fill();
  }
  ctx.fillStyle = "rgba(244,251,248,0.18)";
  ctx.beginPath();
  ctx.arc(w * 0.78, h * 0.16, 48, 0, Math.PI * 2);
  ctx.fill();
}

function drawRainforestScenery(w, h, theme) {
  const canopy = ctx.createLinearGradient(0, h * 0.2, 0, h * 0.6);
  canopy.addColorStop(0, "rgba(54,217,138,0.16)");
  canopy.addColorStop(1, "rgba(10,54,35,0.5)");
  ctx.fillStyle = canopy;
  ctx.fillRect(0, h * 0.24, w, h * 0.36);
  ctx.strokeStyle = "rgba(54,217,138,0.22)";
  ctx.lineWidth = 5;
  for (let i = 0; i < 36; i += 1) {
    const x = ((i * 71 + raceState.roadOffset * 0.04) % (w + 100)) - 50;
    const y = h * 0.34 + (i % 5) * 24;
    ctx.beginPath();
    ctx.moveTo(x, y + 50);
    ctx.lineTo(x + Math.sin(i) * 16, y - 45);
    ctx.stroke();
    ctx.fillStyle = i % 2 ? "rgba(54,217,138,0.32)" : "rgba(187,242,74,0.18)";
    ctx.beginPath();
    ctx.ellipse(x - 12, y - 34, 30, 10, -0.4, 0, Math.PI * 2);
    ctx.ellipse(x + 16, y - 28, 28, 9, 0.45, 0, Math.PI * 2);
    ctx.fill();
  }
  ctx.strokeStyle = "rgba(244,251,248,0.14)";
  ctx.lineWidth = 2;
  for (let i = 0; i < 18; i += 1) {
    const x = (i * 53 + raceState.elapsed * 40) % w;
    ctx.beginPath();
    ctx.moveTo(x, h * 0.16);
    ctx.lineTo(x - 18, h * 0.58);
```

```
    ctx.stroke();
  }
}

function drawEuropeanScenery(w, h, theme) {
  const valley = ctx.createLinearGradient(0, h * 0.3, 0, h * 0.58);
  valley.addColorStop(0, "rgba(220,232,239,0.18)");
  valley.addColorStop(1, "rgba(24,58,52,0.32)");
  ctx.fillStyle = valley;
  ctx.fillRect(0, h * 0.3, w, h * 0.28);
  ctx.fillStyle = "rgba(220,232,239,0.36)";
  ctx.beginPath();
  ctx.moveTo(0, h * 0.4);
  ctx.lineTo(w * 0.12, h * 0.13);
  ctx.lineTo(w * 0.28, h * 0.4);
  ctx.lineTo(w * 0.48, h * 0.1);
  ctx.lineTo(w * 0.72, h * 0.4);
  ctx.lineTo(w * 0.9, h * 0.18);
  ctx.lineTo(w, h * 0.4);
  ctx.lineTo(w, h * 0.56);
  ctx.lineTo(0, h * 0.56);
  ctx.closePath();
  ctx.fill();
  for (let i = 0; i < 12; i += 1) {
    const x = ((i * 137 + raceState.roadOffset * 0.035) % (w + 160)) - 80;
    const y = h * 0.33 + (i % 3) * 28;
    ctx.fillStyle = "rgba(244,251,248,0.44)";
    ctx.fillRect(x, y, 44, 30);
    ctx.fillStyle = "rgba(160,40,31,0.58)";
    ctx.beginPath();
    ctx.moveTo(x - 4, y);
    ctx.lineTo(x + 22, y - 18);
    ctx.lineTo(x + 48, y);
    ctx.closePath();
    ctx.fill();
  }
}

function drawRoadSigns(w, h, theme) {
  const signText = selectedRace && selectedRace.sign ? selectedRace.sign : "Race Route";
  const x = w * 0.75;
  const y = h * 0.25 + Math.sin(raceState.roadOffset * 0.004) * 8;
  ctx.strokeStyle = "rgba(244,251,248,0.28)";
  ctx.lineWidth = 4;
  ctx.beginPath();
  ctx.moveTo(x, y + 28);
  ctx.lineTo(x, y + 94);
  ctx.stroke();
  ctx.fillStyle = "rgba(5,8,7,0.76)";
  roundRect(x - 82, y - 18, 164, 46, 6);
  ctx.fill();
  ctx.strokeStyle = theme[1];
  ctx.lineWidth = 2;
  ctx.stroke();
  ctx.fillStyle = "#f4fbf8";
  ctx.font = "900 14px system-ui";
  ctx.textAlign = "center";
  ctx.fillText(signText, x, y + 10);
}

function drawRoad(w, h, theme) {
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  const horizon = cameraMode === "cockpit" ? h * 0.28 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const roadTop = cameraMode === "cockpit" ? w * 0.12 : cameraMode === "hood" ? w * 0.15 : w * 0.14;
  const roadBottom = cameraMode === "cockpit" ? w * 1.05 : cameraMode === "hood" ? w * 0.95 : w * 1.02;
  const roadCenter = (t) => w * 0.5 - cameraLaneOffset(0.24 + t * 0.86) * laneWidth() * (0.18 + t * 0.95);
  const topCenter = roadCenter(0);
  const bottomCenter = roadCenter(1);
  const glare = ctx.createRadialGradient(bottomCenter, h * 0.62, w * 0.08, bottomCenter, h * 0.74, w * 0.75);
  glare.addColorStop(0, "rgba(244,251,248,0.08)");
  glare.addColorStop(1, "rgba(244,251,248,0)");
  ctx.fillStyle = glare;
  ctx.fillRect(0, horizon, w, h - horizon);
  ctx.fillStyle = "rgba(0,0,0,0.34)";
  ctx.beginPath();
  ctx.moveTo(topCenter - roadTop, horizon);
  ctx.lineTo(topCenter + roadTop, horizon);
  ctx.lineTo(bottomCenter + roadBottom, h);
  ctx.lineTo(bottomCenter - roadBottom, h);
  ctx.closePath();
  ctx.fill();
  ctx.fillStyle = "#222826";
  const asphalt = ctx.createLinearGradient(0, horizon, 0, h);
  if (place === "snow") {
```

```
  asphalt.addColorStop(0, "#596967");
  asphalt.addColorStop(0.45, "#394845");
  asphalt.addColorStop(1, "#1a2422");
} else if (place === "harbor") {
  asphalt.addColorStop(0, "#28434a");
  asphalt.addColorStop(0.45, "#182d33");
  asphalt.addColorStop(1, "#0b1619");
} else if (place === "airfield") {
  asphalt.addColorStop(0, "#3d3f3b");
  asphalt.addColorStop(0.45, "#282a27");
  asphalt.addColorStop(1, "#151613");
} else if (place === "freight") {
  asphalt.addColorStop(0, "#393c34");
  asphalt.addColorStop(0.45, "#252820");
  asphalt.addColorStop(1, "#131610");
} else if (place === "farm") {
  asphalt.addColorStop(0, "#5d5542");
  asphalt.addColorStop(0.45, "#3d3526");
  asphalt.addColorStop(1, "#211b13");
} else if (place === "tokyo") {
  asphalt.addColorStop(0, "#252335");
  asphalt.addColorStop(0.45, "#171722");
  asphalt.addColorStop(1, "#0b0a12");
} else if (place === "desert") {
  asphalt.addColorStop(0, "#6b5234");
  asphalt.addColorStop(0.45, "#473622");
  asphalt.addColorStop(1, "#21170e");
} else if (place === "rainforest") {
  asphalt.addColorStop(0, "#314037");
  asphalt.addColorStop(0.45, "#1e2b24");
  asphalt.addColorStop(1, "#101812");
} else if (place === "europe") {
  asphalt.addColorStop(0, "#3d4648");
  asphalt.addColorStop(0.45, "#283234");
  asphalt.addColorStop(1, "#141c1d");
} else {
  asphalt.addColorStop(0, "#2e3330");
  asphalt.addColorStop(0.45, "#1f2422");
  asphalt.addColorStop(1, "#111615");
}
ctx.fillStyle = asphalt;
ctx.beginPath();
ctx.moveTo(topCenter - roadTop * 0.86, horizon);
ctx.lineTo(topCenter + roadTop * 0.86, horizon);
ctx.lineTo(bottomCenter + roadBottom * 0.7, h);
ctx.lineTo(bottomCenter - roadBottom * 0.7, h);
ctx.closePath();
ctx.fill();
ctx.save();
ctx.globalAlpha = 0.17;
for (let i = 0; i < 130; i += 1) {
  const y = ((i * 23 + raceState.roadOffset * 0.9) % (h + 40)) - 20;
  if (y < horizon) continue;
  const t = (y - horizon) / (h - horizon);
  const x = roadCenter(t) + (Math.sin(i * 13.7) * roadBottom * 0.55 * t);
  ctx.fillStyle = i % 3 ? "#f4fbf8" : "rgba(180,192,188,0.72)";
  ctx.fillRect(x, y, 1 + t * 3, 1 + t * 5);
}
ctx.restore();
ctx.globalAlpha = 0.16;
ctx.strokeStyle = "#f4fbf8";
ctx.lineWidth = 1;
for (let i = 0; i < 26; i += 1) {
  const y = ((i * 42 + raceState.roadOffset * 0.6) % (h + 80)) - 40;
  const t = Math.max(0, (y - horizon) / (h - horizon));
  const center = roadCenter(t);
  ctx.beginPath();
  ctx.moveTo(center - roadBottom * 0.65 * t, y);
  ctx.lineTo(center + roadBottom * 0.65 * t, y + 4);
  ctx.stroke();
}
ctx.globalAlpha = 1;
const baseRoadX = (lane, t) => roadCenter(t) + lane * laneWidth() * (0.3 + t * 1.05);
const basePaintSegment = (lane, y, length, width, style) => {
  const y1 = Math.max(horizon, y);
  const y2 = Math.min(h + 80, y + length);
  const t1 = Math.max(0, Math.min(1.08, (y1 - horizon) / (h - horizon)));
  const t2 = Math.max(0, Math.min(1.12, (y2 - horizon) / (h - horizon)));
  const x1 = baseRoadX(lane, t1);
  const x2 = baseRoadX(lane, t2);
  const w1 = width * (0.24 + t1 * 0.58);
  const w2 = width * (0.24 + t2 * 0.98);
  ctx.fillStyle = style;
  ctx.beginPath();
```

```
    ctx.moveTo(x1 - w1 / 2, y1);
    ctx.lineTo(x1 + w1 / 2, y1);
    ctx.lineTo(x2 + w2 / 2, y2);
    ctx.lineTo(x2 - w2 / 2, y2);
    ctx.closePath();
    ctx.fill();
  };
  for (let lane = -1.5; lane <= 1.5; lane += 1) {
    for (let i = 0; i < 14; i += 1) {
      const y = ((i * 116 + raceState.roadOffset * 1.42) % (h + 190)) - 95;
      if (y < horizon) continue;
      const t = Math.max(0, (y - horizon) / (h - horizon));
      const dashH = 20 + t * 72;
      const x = baseRoadX(lane, t);
      const dash = ctx.createLinearGradient(x, y, x, y + dashH);
      dash.addColorStop(0, "rgba(244,251,248,0.08)");
      dash.addColorStop(0.35, "rgba(244,251,248,0.64)");
      dash.addColorStop(1, "rgba(244,251,248,0.12)");
      basePaintSegment(lane, y, dashH, 7 + t * 8, dash);
    }
  }
  ctx.strokeStyle = "rgba(244,251,248,0.72)";
  ctx.lineWidth = 2;
  ctx.globalAlpha = 0.42;
  for (let lane = -2.5; lane <= 2.5; lane += 1) {
    ctx.beginPath();
    ctx.moveTo(topCenter + lane * laneWidth() * 0.28, horizon);
    ctx.lineTo(bottomCenter + lane * laneWidth() * 1.28, h);
    ctx.stroke();
  }
  ctx.strokeStyle = "rgba(244,251,248,0.58)";
  ctx.lineWidth = 4;
  ctx.globalAlpha = 0.7;
  ctx.beginPath();
  ctx.moveTo(topCenter - roadTop * 0.9, horizon);
  ctx.lineTo(bottomCenter - roadBottom * 0.7, h);
  ctx.moveTo(topCenter + roadTop * 0.9, horizon);
  ctx.lineTo(bottomCenter + roadBottom * 0.7, h);
  ctx.stroke();
  ctx.globalAlpha = 1;
  drawRoadsideDetails(w, h, theme, horizon, roadBottom);
  drawGuardrails(w, h, theme, horizon, roadTop, roadBottom);
  drawHeadlightBeams(w, h, theme, horizon);
}

function drawGuardrails(w, h, theme, horizon, roadTop, roadBottom) {
  ctx.save();
  const topCenter = perspectiveRoadCenter(w, 0.04);
  const bottomCenter = perspectiveRoadCenter(w, 1);
  for (let side = -1; side <= 1; side += 2) {
    ctx.strokeStyle = "rgba(205,218,214,0.38)";
    ctx.lineWidth = 5;
    ctx.beginPath();
    ctx.moveTo(topCenter + side * roadTop * 1.08, horizon + 12);
    ctx.lineTo(bottomCenter + side * roadBottom * 0.82, h);
    ctx.stroke();
    ctx.strokeStyle = "rgba(5,8,7,0.78)";
    ctx.lineWidth = 2;
    ctx.beginPath();
    ctx.moveTo(topCenter + side * roadTop * 1.04, horizon + 26);
    ctx.lineTo(bottomCenter + side * roadBottom * 0.78, h);
    ctx.stroke();
    for (let i = 0; i < 18; i += 1) {
      const y = ((i * 74 + raceState.roadOffset * 1.2) % (h + 120)) - 60;
      if (y < horizon) continue;
      const t = (y - horizon) / (h - horizon);
      const x = perspectiveRoadCenter(w, t) + side * (roadTop * 1.08 + (roadBottom * 0.82 - roadTop * 1.08) * t);
      ctx.fillStyle = i % 2 ? "rgba(255,255,255,0.34)" : "rgba(180,192,188,0.34)";
      roundRect(x - side * 4, y, side * 10, 22 + t * 34, 3);
      ctx.fill();
    }
  }
  ctx.restore();
}

function drawRoadsideDetails(w, h, theme, horizon, roadBottom) {
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  for (let i = 0; i < 22; i += 1) {
    const y = ((i * 92 + raceState.roadOffset * 1.08) % (h + 160)) - 80;
    if (y < horizon) continue;
    const t = (y - horizon) / (h - horizon);
    const side = i % 2 === 0 ? -1 : 1;
    const x = perspectiveRoadCenter(w, t) + side * roadBottom * (0.28 + t * 0.64);
    const size = 10 + t * 28;
```

```
  if (place === "canyon") {
    const rock = ctx.createLinearGradient(x - size, y - size, x + size, y + size);
    rock.addColorStop(0, "rgba(255,150,83,0.46)");
    rock.addColorStop(1, "rgba(82,42,27,0.56)");
    ctx.fillStyle = rock;
    ctx.beginPath();
    ctx.moveTo(x, y - size);
    ctx.lineTo(x - size * 0.7, y + size);
    ctx.lineTo(x + size * 0.8, y + size * 0.7);
    ctx.closePath();
    ctx.fill();
  } else if (place === "alpine") {
    ctx.fillStyle = "rgba(23,45,35,0.72)";
    ctx.beginPath();
    ctx.moveTo(x, y - size * 1.2);
    ctx.lineTo(x - size * 0.7, y + size);
    ctx.lineTo(x + size * 0.7, y + size);
    ctx.closePath();
    ctx.fill();
    ctx.strokeStyle = "rgba(244,251,248,0.2)";
    ctx.beginPath();
    ctx.moveTo(x, y - size * 0.9);
    ctx.lineTo(x, y + size * 0.8);
    ctx.stroke();
  } else if (place === "harbor") {
    ctx.fillStyle = "rgba(117,86,47,0.72)";
    ctx.fillRect(x - size * 0.2, y - size * 0.35, size * 0.4, size * 1.6);
    ctx.strokeStyle = "rgba(244,251,248,0.2)";
    ctx.lineWidth = Math.max(2, size * 0.08);
    ctx.beginPath();
    ctx.moveTo(x - size * 0.7, y + size * 0.12);
    ctx.lineTo(x + size * 0.7, y + size * 0.12);
    ctx.stroke();
  } else if (place === "snow") {
    ctx.fillStyle = "rgba(244,251,248,0.76)";
    ctx.beginPath();
    ctx.ellipse(x, y + size * 0.72, size * 0.8, size * 0.22, 0, 0, Math.PI * 2);
    ctx.fill();
    ctx.fillStyle = "rgba(10,35,33,0.72)";
    ctx.beginPath();
    ctx.moveTo(x, y - size);
    ctx.lineTo(x - size * 0.62, y + size * 0.6);
    ctx.lineTo(x + size * 0.62, y + size * 0.6);
    ctx.closePath();
    ctx.fill();
  } else if (place === "airfield") {
    ctx.fillStyle = i % 3 === 0 ? "rgba(255,209,102,0.9)" : "rgba(244,251,248,0.5)";
    ctx.beginPath();
    ctx.arc(x, y, size * 0.2, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "rgba(244,251,248,0.22)";
    ctx.lineWidth = 2;
    ctx.beginPath();
    ctx.moveTo(x, y);
    ctx.lineTo(x, y + size * 1.4);
    ctx.stroke();
  } else if (place === "freight") {
    ctx.fillStyle = "rgba(244,251,248,0.24)";
    roundRect(x - size * 0.7, y - size * 0.45, size * 1.4, size * 0.54, 4);
    ctx.fill();
    ctx.fillStyle = "rgba(5,8,7,0.76)";
    ctx.fillRect(x - size * 0.52, y - size * 0.3, size * 0.32, size * 0.12);
    ctx.fillRect(x + size * 0.16, y - size * 0.3, size * 0.28, size * 0.12);
  } else if (place === "farm") {
    ctx.fillStyle = i % 3 === 0 ? "rgba(187,242,74,0.58)" : "rgba(255,209,102,0.46)";
    ctx.beginPath();
    ctx.moveTo(x, y - size * 0.85);
    ctx.lineTo(x - size * 0.36, y + size * 0.2);
    ctx.lineTo(x + size * 0.36, y + size * 0.2);
    ctx.closePath();
    ctx.fill();
    ctx.strokeStyle = "rgba(117,86,47,0.62)";
    ctx.lineWidth = Math.max(2, size * 0.08);
    ctx.beginPath();
    ctx.moveTo(x, y + size * 0.2);
    ctx.lineTo(x, y + size);
    ctx.stroke();
  } else if (place === "tokyo") {
    ctx.fillStyle = i % 2 ? "rgba(255,79,216,0.66)" : "rgba(70,217,255,0.58)";
    roundRect(x - size * 0.26, y - size * 1.2, size * 0.52, size * 1.5, 4);
    ctx.fill();
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    ctx.fillRect(x - size * 0.13, y - size * 1.05, size * 0.26, size * 0.96);
  } else if (place === "desert") {
```

```
      ctx.fillStyle = "rgba(255,183,74,0.52)";
      ctx.beginPath();
      ctx.moveTo(x, y - size * 0.15);
      ctx.lineTo(x - size, y + size * 0.68);
      ctx.lineTo(x + size, y + size * 0.68);
      ctx.closePath();
      ctx.fill();
    } else if (place === "rainforest") {
      ctx.strokeStyle = "rgba(38,78,46,0.82)";
      ctx.lineWidth = Math.max(3, size * 0.12);
      ctx.beginPath();
      ctx.moveTo(x, y - size * 0.4);
      ctx.lineTo(x, y + size * 1.3);
      ctx.stroke();
      ctx.fillStyle = "rgba(54,217,138,0.58)";
      ctx.beginPath();
      ctx.ellipse(x - size * 0.32, y - size * 0.46, size * 0.74, size * 0.24, -0.5, 0, Math.PI * 2);
      ctx.ellipse(x + size * 0.32, y - size * 0.36, size * 0.7, size * 0.22, 0.5, 0, Math.PI * 2);
      ctx.fill();
    } else if (place === "europe") {
      ctx.fillStyle = "rgba(244,251,248,0.68)";
      ctx.beginPath();
      ctx.moveTo(x, y - size);
      ctx.lineTo(x - size * 0.7, y + size * 0.7);
      ctx.lineTo(x + size * 0.7, y + size * 0.7);
      ctx.closePath();
      ctx.fill();
    } else {
      ctx.strokeStyle = "rgba(244,251,248,0.28)";
      ctx.lineWidth = Math.max(2, size * 0.08);
      ctx.beginPath();
      ctx.moveTo(x, y - size);
      ctx.lineTo(x, y + size * 1.5);
      ctx.stroke();
      ctx.fillStyle = theme[1];
      ctx.globalAlpha = 0.65;
      ctx.beginPath();
      ctx.arc(x, y - size, size * 0.22, 0, Math.PI * 2);
      ctx.fill();
      ctx.globalAlpha = 1;
      if (place === "city" && i % 5 === 0) {
        const signW = size * 2.1;
        const signX = side < 0 ? x - signW : x;
        ctx.fillStyle = "rgba(5,8,7,0.82)";
        roundRect(signX, y - size * 1.2, signW, size * 0.78, 4);
        ctx.fill();
        ctx.strokeStyle = theme[1];
        ctx.lineWidth = 1.5;
        ctx.stroke();
        ctx.fillStyle = "#f4fbf8";
        ctx.font = `900 ${Math.max(8, size * 0.22)}px system-ui`;
        ctx.textAlign = "center";
        ctx.fillText("DOWNTOWN", signX + signW / 2, y - size * 0.73);
      }
    }
  }
}

function drawHeadlightBeams(w, h, theme, horizon) {
  const boostInput = input.boost || input.gamepadBoost;
  const alpha = cameraMode === "cockpit" ? 0.18 : 0.11;
  const beam = ctx.createLinearGradient(0, horizon, 0, h);
  beam.addColorStop(0, "rgba(255,255,255,0)");
  beam.addColorStop(0.62, `rgba(255,248,214,${alpha})`);
  beam.addColorStop(1, `rgba(70,217,255,${boostInput ? 0.22 : 0.08})`);
  ctx.fillStyle = beam;
  ctx.beginPath();
  ctx.moveTo(w * 0.43, h * 0.7);
  ctx.lineTo(w * 0.17, h);
  ctx.lineTo(w * 0.83, h);
  ctx.lineTo(w * 0.57, h * 0.7);
  ctx.closePath();
  ctx.fill();
  if (!boostInput) return;
  ctx.strokeStyle = theme[1];
  ctx.lineWidth = 3;
  ctx.globalAlpha = 0.45;
  for (let i = 0; i < 8; i += 1) {
    const x = w * (0.18 + i * 0.09);
    ctx.beginPath();
    ctx.moveTo(w * 0.5, h * 0.52);
    ctx.lineTo(x, h);
    ctx.stroke();
  }
```

```
  ctx.globalAlpha = 1;
}

function objectPos(lane, y) {
  const h = canvas.height;
  const t = Math.max(0, y / h);
  const turn = raceState.roadTurn || raceState.roadCurve || 0;
  const phoneCurve = phoneGraphicsActive() ? 0.29 : 0.19;
  const curveShift = turn * (1 - Math.min(1, t)) * (1 - Math.min(1, t) * 0.24) * canvas.width * phoneCurve;
  const laneSpread = laneWidth() * (0.42 + t * 0.72);
  return {
    x: canvas.width / 2 + curveShift + (lane - cameraLaneOffset(t)) * laneSpread,
    y,
    scale: 0.52 + t * 0.55
  };
}

function phoneRoadObjectPos(lane, distance) {
  const w = canvas.width;
  const h = canvas.height;
  const gap = Number(distance) - raceState.distance;
  const horizon = cameraMode === "cockpit" ? h * 0.27 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const nearY = cameraMode === "cockpit" ? h * 0.83 : cameraMode === "hood" ? h * 0.9 : h * 0.87;
  const farGap = 1680;
  let depth;
  if (gap >= 0) {
    const raw = 1 - Math.max(0, Math.min(1, gap / farGap));
    depth = 0.12 + Math.pow(raw, 0.62) * 0.86;
  } else {
    depth = 1 + Math.min(0.32, Math.abs(gap) / 520);
  }
  const clampedDepth = Math.max(0.09, Math.min(1.22, depth));
  const y = gap >= 0
    ? horizon + (nearY - horizon) * clampedDepth
    : nearY + Math.min(h * 0.16, Math.abs(gap) * 0.16);
  const turn = raceState.roadTurn || raceState.roadCurve || 0;
  const farPull = (1 - Math.min(1, clampedDepth)) * (1 - Math.min(1, clampedDepth));
  const curveShift = turn * farPull * w * 0.28;
  const laneSpread = laneWidth() * (0.54 + Math.min(1.08, clampedDepth) * 1.02);
  return {
    x: w / 2 + curveShift + (lane - cameraLaneOffset(clampedDepth)) * laneSpread,
    y,
    scale: Math.max(0.34, Math.min(1.26, 0.32 + clampedDepth * 0.84)),
    depth: clampedDepth
  };
}

function roadSpawnDistance(minRatio = 0.34, maxRatio = 0.44) {
  const h = Math.max(1, canvas.height || window.innerHeight || 720);
  const ratio = minRatio + Math.random() * Math.max(0.01, maxRatio - minRatio);
  return raceState.distance + (h * 0.68 - h * ratio) / 0.11;
}

function screenYFromRoadDistance(distance) {
  return canvas.height * 0.68 - (distance - raceState.distance) * 0.11;
}

function roadDistanceFromScreenY(y) {
  const h = Math.max(1, canvas.height || window.innerHeight || 720);
  const clampedY = Math.max(h * 0.34, Math.min(h * 0.98, Number(y) || h * 0.34));
  return raceState.distance + (h * 0.68 - clampedY) / 0.11;
}

function ensureRoadDistance(object, fallbackY = -100) {
  if (!object) return raceState.distance;
  const distance = Number(object.distance);
  if (Number.isFinite(distance)) return distance;
  object.distance = roadDistanceFromScreenY(Number.isFinite(Number(object.y)) ? Number(object.y) : fallbackY);
  return object.distance;
}

function roadObjectY(object) {
  const y = screenYFromRoadDistance(ensureRoadDistance(object));
  object.y = y;
  return y;
}

function roadObjectPos(lane, distance) {
  if (phoneGraphicsActive()) return phoneRoadObjectPos(lane, distance);
  if (useWebGLRenderer()) return visibleRoadObjectPos(lane, distance);
  return objectPos(lane, screenYFromRoadDistance(distance));
}

function roadContactSink(scale = 1, vehicleType = "car") {
```

```
  if (vehicleType === "airplane" || vehicleType === "helicopter" || vehicleType === "boat") return 0;
  const h = canvas.height || 720;
  const heavy = ["semi", "truck", "monster", "tank", "tractor"].includes(vehicleType);
  return Math.max(12, Math.min(h * 0.12, h * (heavy ? 0.058 : 0.046) + scale * (heavy ? 24 : 20)));
}

function visibleRoadObjectPos(lane, distance) {
  const w = canvas.width;
  const h = canvas.height;
  if (webglRenderer && typeof webglRenderer.projectRoadPoint === "function") {
    const projected = webglRenderer.projectRoadPoint(lane, distance);
    if (projected && Number.isFinite(projected.x) && Number.isFinite(projected.y)) {
      const clampedY = Math.max(h * 0.34, Math.min(h * 1.06, projected.y));
      const t = Math.max(0, Math.min(1, (clampedY - h * 0.34) / Math.max(1, h * 0.7)));
      const turn = raceState.roadTurn || raceState.roadCurve || 0;
      const curveShift = turn * (1 - t) * (1 - t * 0.22) * w * 0.17;
      const laneSpread = laneWidth() * (0.32 + t * 1.08);
      return {
        x: w / 2 + curveShift + (lane - cameraLaneOffset(t)) * laneSpread,
        y: clampedY,
        scale: Math.max(0.34, Math.min(1.24, projected.scale || (0.36 + t * 0.78))),
        depth: t
      };
    }
  }
  const rawGap = Number(distance) - raceState.distance;
  const gap = Math.max(-140, rawGap);
  const horizon = cameraMode === "cockpit" ? h * 0.44 : cameraMode === "hood" ? h * 0.48 : h * 0.5;
  const nearY = cameraMode === "cockpit" ? h * 0.84 : cameraMode === "hood" ? h * 0.9 : h * 0.88;
  const depth = gap >= 0
    ? Math.max(0.18, Math.min(1, 1 / (1 + gap / 88)))
    : Math.min(1.28, 1 + Math.abs(gap) / 360);
  const y = gap >= 0
    ? horizon + (nearY - horizon) * depth
    : nearY + Math.min(h * 0.2, Math.abs(gap) * 0.18);
  const turn = raceState.roadTurn || raceState.roadCurve || 0;
  const phoneCurve = phoneGraphicsActive() ? 0.27 : 0.17;
  const curveShift = turn * (1 - depth) * (1 - depth * 0.22) * w * phoneCurve;
  const laneSpread = laneWidth() * (0.32 + depth * 1.08);
  return {
    x: w / 2 + curveShift + (lane - cameraLaneOffset(depth)) * laneSpread,
    y,
    scale: Math.min(1.34, 0.36 + depth * 0.78),
    depth
  };
}

function isVehicleScreenYVisible(y) {
  return y >= canvas.height * 0.44 && y <= canvas.height * 1.08;
}

function drawObjects() {
  const draws = [];
  const phoneMode = phoneGraphicsActive();
  const phoneDrawnVehicles = [];
  const canDrawPhoneVehicle = (gap, lane, priority = false) => {
    if (!phoneMode) return true;
    if (gap > 1180) return false;
    if (gap < -130) return false;
    if (!priority && gap < 95 && Math.abs(lane - raceState.lane) < 0.95) return false;
    const conflict = phoneDrawnVehicles.some((item) => Math.abs(item.gap - gap) < 250 && Math.abs(item.lane - lane) <
1.05);
    if (conflict && !priority) return false;
    phoneDrawnVehicles.push({ gap, lane });
    return true;
  };
  raceState.opponents.forEach((opponent) => {
    const gap = opponent.distance - raceState.distance;
    if (gap < (phoneMode ? -180 : -55)) return;
    if (phoneMode && gap > 1250) return;
    if (!phoneMode && gap > 3600) return;
    if (!canDrawPhoneVehicle(gap, opponent.lane, true)) return;
    const p = roadObjectPos(opponent.lane, opponent.distance);
    if (!isVehicleScreenYVisible(p.y)) return;
    const vehicle = vehicleById(opponent.vehicleId);
    const groundY = p.y + roadContactSink(p.scale, vehicle.type);
    const label = opponent.name;
    draws.push({
      y: groundY,
      draw: () => {
        if (phoneMode) drawPhoneOpponentGapCue(p.x, groundY, p.scale, gap, opponent.lane, opponent.damage || 0,
opponent.wrecked);
        drawTrafficRearCar(p.x, groundY, 70 * p.scale, 112 * p.scale, opponent.color, false, vehicle.type, label,
opponent.damage || 0, opponent.wrecked, opponent.spin || 0);
```

```
      }
    });
  });
  raceState.coinsOnRoad.forEach((coin) => {
    const p = roadObjectPos(coin.lane, ensureRoadDistance(coin));
    if (!isVehicleScreenYVisible(p.y)) return;
    draws.push({
      y: p.y,
      draw: () => drawRouteMarker(p.x, p.y, (coin.r * 2.8) * p.scale, coin.pulse)
    });
  });
  raceState.rivals.forEach((rival) => {
    const distance = ensureRoadDistance(rival);
    const gap = distance - raceState.distance;
    const priority = gap > -30 && gap < 260 && Math.abs(rival.lane - raceState.lane) < 1.08;
    if (!canDrawPhoneVehicle(gap, rival.lane, priority)) return;
    const p = roadObjectPos(rival.lane, distance);
    if (!isVehicleScreenYVisible(p.y)) return;
    const groundY = p.y + roadContactSink(p.scale, rival.type || "car");
    draws.push({
      y: groundY,
      draw: () => drawTrafficRearCar(p.x, groundY, rival.w * p.scale * 1.1, rival.h * p.scale * 0.82, rival.color,
false, rival.type || "car", "", rival.damage || 0, rival.wrecked, rival.spin || 0)
    });
  });
  raceState.police.forEach((unit) => {
    const distance = ensureRoadDistance(unit);
    const gap = distance - raceState.distance;
    const priority = gap > -30 && gap < 280 && Math.abs(unit.lane - raceState.lane) < 1.12;
    if (!canDrawPhoneVehicle(gap, unit.lane, priority)) return;
    const p = roadObjectPos(unit.lane, distance);
    if (!isVehicleScreenYVisible(p.y)) return;
    const unitType = unit.type || "car";
    const groundY = p.y + roadContactSink(p.scale, unitType);
    draws.push({
      y: groundY,
      draw: () => drawTrafficRearCar(p.x, groundY, unit.w * p.scale * 1.16, unit.h * p.scale * 0.84, "#f4fbf8", true,
unitType, unit.label || "", unit.damage || 0, unit.wrecked, unit.spin || 0)
    });
  });
  (raceState.oncoming || []).forEach((unit) => {
    const distance = ensureRoadDistance(unit);
    const gap = distance - raceState.distance;
    const priority = gap > -30 && gap < 320 && Math.abs(unit.lane - raceState.lane) < 1.12;
    if (!canDrawPhoneVehicle(gap, unit.lane, priority)) return;
    const p = roadObjectPos(unit.lane, distance);
    if (!isVehicleScreenYVisible(p.y)) return;
    const groundY = p.y + roadContactSink(p.scale, unit.type || "car");
    draws.push({
      y: groundY,
      draw: () => drawTrafficRearCar(p.x, groundY, unit.w * p.scale * 1.08, unit.h * p.scale * 0.82, unit.color ||
"#ffd166", false, unit.type || "car", "ONCOMING", unit.damage || 0, unit.wrecked, unit.spin || Math.PI)
    });
  });
  (raceState.civilians || []).forEach((person) => {
    const distance = ensureRoadDistance(person);
    const p = roadObjectPos(person.lane, distance);
    if (!isVehicleScreenYVisible(p.y)) return;
    draws.push({
      y: p.y + 8,
      draw: () => drawCivilianMarker(p.x, p.y, p.scale, person)
    });
  });
  (raceState.routeFeatures || []).forEach((feature) => {
    const distance = ensureRoadDistance(feature);
    const gap = distance - raceState.distance;
    if (gap < -120 || gap > 1800) return;
    const p = roadObjectPos(feature.lane, distance);
    if (!isVehicleScreenYVisible(p.y)) return;
    draws.push({
      y: p.y + 16,
      draw: () => drawRouteFeatureMarker(p.x, p.y, p.scale, feature)
    });
  });
  draws.sort((a, b) => a.y - b.y).forEach((item) => item.draw());
}

function drawRouteFeatureMarker(x, y, scale, feature) {
  const s = Math.max(0.36, Math.min(1.22, scale || 1));
  const size = routeFeatureSize(feature);
  const w = size.w * s;
  const h = size.h * s;
  const accent = feature.type === "hide" ? "#46d9ff" : "#bbf24a";
  ctx.save();
```

```
  ctx.translate(x, y + 6 * s);
  ctx.globalAlpha = 0.92;
  const glow = ctx.createRadialGradient(0, 8 * s, 4, 0, 8 * s, w * 0.86);
  glow.addColorStop(0, feature.type === "hide" ? "rgba(70,217,255,0.22)" : "rgba(187,242,74,0.22)");
  glow.addColorStop(1, "rgba(0,0,0,0)");
  ctx.fillStyle = glow;
  ctx.fillRect(-w, -h * 0.6, w * 2, h * 1.4);
  ctx.fillStyle = "rgba(0,0,0,0.4)";
  ctx.beginPath();
  ctx.ellipse(0, h * 0.42, w * 0.62, h * 0.12, 0, 0, Math.PI * 2);
  ctx.fill();
  if (feature.type === "shortcut") {
    ctx.fillStyle = "rgba(12,18,14,0.86)";
    ctx.beginPath();
    ctx.moveTo(-w * 0.58, h * 0.34);
    ctx.lineTo(w * 0.58, h * 0.18);
    ctx.lineTo(w * 0.48, h * 0.42);
    ctx.lineTo(-w * 0.62, h * 0.56);
    ctx.closePath();
    ctx.fill();
    ctx.strokeStyle = accent;
    ctx.lineWidth = Math.max(1.5, 3 * s);
    ctx.stroke();
    ctx.fillStyle = accent;
    roundRect(-w * 0.42, -h * 0.26, w * 0.84, h * 0.26, 5 * s);
    ctx.fill();
  } else if (feature.icon === "pole") {
    ctx.strokeStyle = "#d8c08b";
    ctx.lineWidth = Math.max(2, 5 * s);
    ctx.beginPath();
    ctx.moveTo(0, -h * 0.62);
    ctx.lineTo(0, h * 0.42);
    ctx.stroke();
    ctx.strokeStyle = "rgba(244,251,248,0.58)";
    ctx.lineWidth = Math.max(1, 2 * s);
    ctx.beginPath();
    ctx.moveTo(-w * 0.46, -h * 0.45);
    ctx.lineTo(w * 0.46, -h * 0.45);
    ctx.moveTo(-w * 0.5, -h * 0.38);
    ctx.lineTo(w * 0.5, -h * 0.38);
    ctx.stroke();
  } else {
    ctx.fillStyle = feature.icon === "cave" ? "rgba(13,10,8,0.94)" : feature.icon === "mountain" ?
"rgba(36,45,42,0.94)" : "rgba(13,21,21,0.94)";
    if (feature.icon === "cave" || feature.icon === "mountain") {
      ctx.beginPath();
      ctx.moveTo(-w * 0.6, h * 0.36);
      ctx.lineTo(-w * 0.18, -h * 0.48);
      ctx.lineTo(w * 0.18, -h * 0.36);
      ctx.lineTo(w * 0.6, h * 0.36);
      ctx.closePath();
      ctx.fill();
      ctx.fillStyle = "rgba(0,0,0,0.72)";
      ctx.beginPath();
      ctx.ellipse(0, h * 0.18, w * 0.24, h * 0.2, 0, 0, Math.PI * 2);
      ctx.fill();
    } else {
      roundRect(-w * 0.48, -h * 0.52, w * 0.96, h * 0.92, 5 * s);
      ctx.fill();
      ctx.fillStyle = "rgba(244,251,248,0.16)";
      for (let i = 0; i < 3; i += 1) {
        roundRect(-w * 0.34, -h * 0.34 + i * h * 0.22, w * 0.68, h * 0.06, 2 * s);
        ctx.fill();
      }
    }
    ctx.strokeStyle = accent;
    ctx.lineWidth = Math.max(1.4, 2.4 * s);
    ctx.stroke();
  }
  ctx.fillStyle = "rgba(5,8,7,0.82)";
  roundRect(-w * 0.54, -h * 0.74, w * 1.08, h * 0.22, 5 * s);
  ctx.fill();
  ctx.strokeStyle = accent;
  ctx.lineWidth = Math.max(1, 1.5 * s);
  ctx.stroke();
  ctx.fillStyle = "#f4fbf8";
  ctx.font = `900 ${Math.max(7, Math.min(12, 9 * s))}px system-ui`;
  ctx.textAlign = "center";
  ctx.textBaseline = "middle";
  ctx.fillText(feature.type === "hide" ? "HIDE" : "CUT", 0, -h * 0.63);
  ctx.restore();
}

function drawCivilianMarker(x, y, scale, person) {
```

```
  const s = Math.max(0.42, Math.min(1.18, scale || 1));
  const step = Math.sin(person.step || 0) * 3 * s;
  ctx.save();
  ctx.translate(x, y + 8 * s);
  ctx.globalAlpha = person.crossing ? 0.92 : 0.72;
  ctx.fillStyle = "rgba(0,0,0,0.42)";
  ctx.beginPath();
  ctx.ellipse(0, 13 * s, 12 * s, 4 * s, 0, 0, Math.PI * 2);
  ctx.fill();
  ctx.strokeStyle = person.crossing ? "#ffd166" : "rgba(244,251,248,0.72)";
  ctx.lineWidth = Math.max(1.4, 2.4 * s);
  ctx.lineCap = "round";
  ctx.beginPath();
  ctx.moveTo(0, -5 * s);
  ctx.lineTo(0, 8 * s);
  ctx.moveTo(-5 * s, 0);
  ctx.lineTo(5 * s, 2 * s);
  ctx.moveTo(0, 8 * s);
  ctx.lineTo(-5 * s, 15 * s + step);
  ctx.moveTo(0, 8 * s);
  ctx.lineTo(6 * s, 15 * s - step);
  ctx.stroke();
  ctx.fillStyle = "rgba(244,251,248,0.9)";
  ctx.beginPath();
  ctx.arc(0, -10 * s, 4.2 * s, 0, Math.PI * 2);
  ctx.fill();
  if (person.crossing) {
    ctx.fillStyle = "rgba(255,209,102,0.16)";
    ctx.beginPath();
    ctx.arc(0, 1 * s, 24 * s, 0, Math.PI * 2);
    ctx.fill();
  }
  ctx.restore();
}

function drawPhoneOpponentGapCue(x, y, scale, gap, lane, damage = 0, wrecked = false) {
  if (!phoneGraphicsActive()) return;
  const ahead = gap >= 0;
  const label = ahead ? `${Math.round(Math.max(0, gap))}m` : "PASS";
  const laneName = lane < -0.7 ? "L" : lane > 0.7 ? "R" : "MID";
  const badgeW = Math.max(40, Math.min(62, 38 + scale * 24));
  const badgeH = Math.max(13, Math.min(18, 12 + scale * 6));
  const badgeY = y - Math.max(34, 52 * scale);
  ctx.save();
  ctx.globalAlpha = wrecked ? 0.54 : 0.78;
  ctx.fillStyle = damage > 60 ? "rgba(255,91,107,0.78)" : "rgba(5,8,7,0.76)";
  roundRect(x - badgeW / 2, badgeY - badgeH / 2, badgeW, badgeH, 5);
  ctx.fill();
  ctx.strokeStyle = ahead ? "rgba(70,217,255,0.88)" : "rgba(187,242,74,0.88)";
  ctx.lineWidth = 1.4;
  ctx.stroke();
  ctx.fillStyle = "#f4fbf8";
  ctx.font = `900 ${Math.max(8, Math.min(11, 8 + scale * 4))}px Inter, system-ui, sans-serif`;
  ctx.textAlign = "center";
  ctx.textBaseline = "middle";
  ctx.fillText(label, x, badgeY - badgeH * 0.08);
  ctx.fillStyle = "rgba(244,251,248,0.72)";
  ctx.font = `800 ${Math.max(6, Math.min(8, 6 + scale * 2))}px Inter, system-ui, sans-serif`;
  ctx.fillText(laneName, x, badgeY + badgeH * 0.34);
  ctx.restore();
}

function drawRouteMarker(x, y, size, pulse) {
  ctx.save();
  ctx.translate(x, y);
  const glow = 0.18 + Math.sin(pulse) * 0.05;
  const phoneMode = phoneGraphicsActive();
  ctx.globalAlpha = 0.45;
  ctx.fillStyle = phoneMode ? `rgba(70,217,255,${glow})` : `rgba(255,209,102,${glow})`;
  ctx.beginPath();
  ctx.ellipse(0, size * 0.25, size * 1.1, size * 0.28, 0, 0, Math.PI * 2);
  ctx.fill();
  ctx.globalAlpha = 1;
  ctx.fillStyle = phoneMode ? "rgba(70,217,255,0.82)" : "rgba(255,209,102,0.82)";
  roundRect(-size * 0.46, -size * 0.1, size * 0.92, size * 0.22, size * 0.11);
  ctx.fill();
  ctx.fillStyle = "rgba(244,251,248,0.72)";
  ctx.beginPath();
  ctx.arc(0, size * 0.01, Math.max(2, size * 0.13), 0, Math.PI * 2);
  ctx.fill();
  ctx.strokeStyle = phoneMode ? "rgba(70,217,255,0.45)" : "rgba(255,209,102,0.45)";
  ctx.lineWidth = Math.max(1, size * 0.045);
  ctx.beginPath();
  ctx.ellipse(0, size * 0.02, size * 0.68, size * 0.26, 0, 0, Math.PI * 2);
```

```
  ctx.stroke();
  ctx.restore();
}

function drawRaceStandings(w, h) {
  const pos = playerPosition();
  const vehicle = selectedVehicle();
  ctx.save();
  const panelW = Math.min(178, w * 0.34);
  const panelH = 66;
  const x = w - panelW - 14;
  const y = cameraMode === "chase" ? h * 0.13 : 14;
  ctx.fillStyle = "rgba(5,8,7,0.68)";
  roundRect(x, y, panelW, panelH, 7);
  ctx.fill();
  ctx.strokeStyle = "rgba(244,251,248,0.16)";
  ctx.stroke();
  ctx.fillStyle = pos === 1 ? "#ffd166" : "#f4fbf8";
  ctx.font = "900 18px system-ui";
  ctx.textAlign = "left";
  const fieldSize = raceRankings().length;
  ctx.fillText(`P${pos}/${fieldSize}`, x + 12, y + 23);
  ctx.fillStyle = "rgba(244,251,248,0.72)";
  ctx.font = "800 11px system-ui";
  const leader = raceRankings()[0];
  ctx.fillText(`${leader.name} leads`, x + 12, y + 39);
  ctx.fillStyle = "rgba(187,242,74,0.9)";
  ctx.fillText((raceState.teamScore || 0) > 0 ? `Crew ${Math.round(raceState.teamScore)}` : `${raceState.overtakes ||
0} overtakes`, x + 12, y + 54);
  ctx.fillStyle = vehicle.color;
  roundRect(x + panelW - 48, y + 15, 32, 18, 5);
  ctx.fill();
  ctx.restore();
}

function drawTrafficLabel(w, h, label) {
  if (!label) return;
  ctx.fillStyle = "rgba(5,8,7,0.72)";
  roundRect(-w * 0.36, -h * 0.76, w * 0.72, h * 0.16, 4);
  ctx.fill();
  ctx.fillStyle = "#f4fbf8";
  ctx.font = `${Math.max(7, w * 0.12)}px system-ui`;
  ctx.textAlign = "center";
  ctx.fillText(label, 0, -h * 0.64);
}

function drawVehicleGroundContact(w, h, vehicleType = "car", speedFactor = 0.5) {
  const speed = Math.max(0.18, Math.min(1.15, Math.abs(speedFactor)));
  const air = ["airplane", "helicopter"].includes(vehicleType);
  const water = vehicleType === "boat";
  const snow = vehicleType === "snowmobile";
  ctx.save();
  const contactY = h * (air ? 0.58 : water ? 0.76 : 0.92);
  const contact = ctx.createRadialGradient(0, contactY - h * 0.05, w * 0.08, 0, contactY, w * 0.82);
  contact.addColorStop(0, air ? "rgba(0,0,0,0.34)" : "rgba(0,0,0,0.82)");
  contact.addColorStop(1, "rgba(0,0,0,0)");
  ctx.fillStyle = contact;
  ctx.beginPath();
  ctx.ellipse(0, contactY, w * 0.78, h * (air ? 0.1 : 0.2), 0, 0, Math.PI * 2);
  ctx.fill();

  if (!air && !water) {
    ctx.fillStyle = snow ? "rgba(244,251,248,0.54)" : "rgba(0,0,0,0.96)";
    ctx.beginPath();
    ctx.ellipse(-w * 0.34, h * 0.58, w * 0.14, h * 0.052, 0, 0, Math.PI * 2);
    ctx.ellipse(w * 0.34, h * 0.58, w * 0.14, h * 0.052, 0, 0, Math.PI * 2);
    ctx.ellipse(-w * 0.35, h * 0.91, w * 0.19, h * 0.082, 0, 0, Math.PI * 2);
    ctx.ellipse(w * 0.35, h * 0.91, w * 0.19, h * 0.082, 0, 0, Math.PI * 2);
    ctx.fill();
  }

  ctx.globalAlpha = (air ? 0.14 : 0.34) + speed * 0.18;
  ctx.strokeStyle = water ? "rgba(70,217,255,0.62)" : snow ? "rgba(244,251,248,0.7)" : "rgba(3,5,5,0.82)";
  ctx.lineWidth = Math.max(1.2, w * 0.018);
  ctx.lineCap = "round";
  ctx.beginPath();
  ctx.moveTo(-w * 0.32, contactY - h * 0.05);
  ctx.lineTo(-w * (0.4 + speed * 0.24), contactY + h * 0.24);
  ctx.moveTo(w * 0.32, contactY - h * 0.05);
  ctx.lineTo(w * (0.4 + speed * 0.24), contactY + h * 0.24);
  ctx.stroke();
  ctx.restore();
}
```

```
function spriteContactRatio(vehicleType = "car") {
  if (vehicleType === "airplane" || vehicleType === "helicopter") return 0.68;
  if (vehicleType === "boat") return 0.78;
  if (vehicleType === "snowmobile") return 0.94;
  return 0.955;
}

function spriteContactLift(h, vehicleType = "car") {
  return h * spriteContactRatio(vehicleType);
}

function drawRoadContactShadow(w, h, vehicleType = "car", intensity = 1) {
  const air = ["airplane", "helicopter"].includes(vehicleType);
  const water = vehicleType === "boat";
  const snow = vehicleType === "snowmobile";
  ctx.save();
  ctx.globalAlpha = Math.max(0.28, Math.min(1, intensity)) * (air ? 0.36 : 0.96);
  const shadow = ctx.createRadialGradient(0, -h * 0.025, w * 0.08, 0, 0, w * 0.78);
  shadow.addColorStop(0, water ? "rgba(10,70,82,0.74)" : "rgba(0,0,0,0.92)");
  shadow.addColorStop(1, "rgba(0,0,0,0)");
  ctx.fillStyle = shadow;
  ctx.beginPath();
  ctx.ellipse(0, 0, w * (air ? 0.45 : 0.76), h * (air ? 0.065 : 0.13), 0, 0, Math.PI * 2);
  ctx.fill();

  if (!air && !water) {
    ctx.globalAlpha = Math.max(0.52, Math.min(1, intensity));
    ctx.fillStyle = snow ? "rgba(244,251,248,0.78)" : "rgba(0,0,0,0.98)";
    const tireW = w * 0.19;
    const tireH = h * 0.075;
    ctx.beginPath();
    ctx.ellipse(-w * 0.36, -h * 0.01, tireW, tireH, 0, 0, Math.PI * 2);
    ctx.ellipse(w * 0.36, -h * 0.01, tireW, tireH, 0, 0, Math.PI * 2);
    ctx.fill();
  }
  ctx.restore();
}

function drawVehicleRoadLock(w, h, vehicleType = "car", speedFactor = 0.5) {
  const air = ["airplane", "helicopter"].includes(vehicleType);
  const water = vehicleType === "boat";
  const snow = vehicleType === "snowmobile";
  const speed = Math.max(0.18, Math.min(1.15, Math.abs(speedFactor)));
  ctx.save();
  ctx.globalAlpha = air ? 0.3 : 0.94;
  ctx.fillStyle = water ? "rgba(70,217,255,0.56)" : snow ? "rgba(244,251,248,0.7)" : "rgba(0,0,0,0.98)";
  if (air || water) {
    ctx.beginPath();
    ctx.ellipse(0, h * 0.78, w * 0.42, h * 0.06, 0, 0, Math.PI * 2);
    ctx.fill();
  } else {
    roundRect(-w * 0.52, h * 0.55, w * 0.18, h * 0.5, Math.max(4, w * 0.04));
    roundRect(w * 0.34, h * 0.55, w * 0.18, h * 0.5, Math.max(4, w * 0.04));
    ctx.fill();
    ctx.fillStyle = snow ? "rgba(244,251,248,0.78)" : "rgba(0,0,0,1)";
    ctx.beginPath();
    ctx.ellipse(-w * 0.38, h * 0.74, w * 0.12, h * 0.046, 0, 0, Math.PI * 2);
    ctx.ellipse(w * 0.38, h * 0.74, w * 0.12, h * 0.046, 0, 0, Math.PI * 2);
    ctx.ellipse(-w * 0.36, h * 0.98, w * 0.18, h * 0.072, 0, 0, Math.PI * 2);
    ctx.ellipse(w * 0.36, h * 0.98, w * 0.18, h * 0.072, 0, 0, Math.PI * 2);
    ctx.fill();
  }

  ctx.globalAlpha = air ? 0.18 : 0.54;
  ctx.strokeStyle = water ? "rgba(70,217,255,0.65)" : snow ? "rgba(244,251,248,0.72)" : "rgba(1,2,2,0.72)";
  ctx.lineWidth = Math.max(1.4, w * 0.02);
  ctx.lineCap = "round";
  ctx.beginPath();
  const trailY = air || water ? h * 0.78 : h * 1.0;
  ctx.moveTo(-w * 0.28, trailY);
  ctx.lineTo(-w * (0.32 + speed * 0.2), trailY + h * 0.2);
  ctx.moveTo(w * 0.28, trailY);
  ctx.lineTo(w * (0.32 + speed * 0.2), trailY + h * 0.2);
  ctx.stroke();
  ctx.restore();
}

function drawGroundPinnedVehicleContact(w, h, vehicleType = "car", speedFactor = 0.5) {
  const air = ["airplane", "helicopter"].includes(vehicleType);
  const water = vehicleType === "boat";
  const snow = vehicleType === "snowmobile";
  const speed = Math.max(0.18, Math.min(1.15, Math.abs(speedFactor)));
  ctx.save();
  ctx.globalAlpha = air ? 0.34 : 0.96;
```

```
  const shadow = ctx.createRadialGradient(0, -h * 0.04, w * 0.08, 0, 0, w * 0.82);
  shadow.addColorStop(0, water ? "rgba(10,70,82,0.76)" : "rgba(0,0,0,0.9)");
  shadow.addColorStop(1, "rgba(0,0,0,0)");
  ctx.fillStyle = shadow;
  ctx.beginPath();
  ctx.ellipse(0, 0, w * (air ? 0.48 : 0.82), h * (air ? 0.07 : 0.13), 0, 0, Math.PI * 2);
  ctx.fill();

  if (!air && !water) {
    ctx.globalAlpha = 1;
    ctx.fillStyle = snow ? "rgba(244,251,248,0.82)" : "rgba(0,0,0,0.98)";
    ctx.beginPath();
    ctx.ellipse(-w * 0.36, -h * 0.006, w * 0.17, h * 0.052, 0, 0, Math.PI * 2);
    ctx.ellipse(w * 0.36, -h * 0.006, w * 0.17, h * 0.052, 0, 0, Math.PI * 2);
    ctx.fill();
    ctx.globalAlpha = 0.34 + speed * 0.16;
    ctx.strokeStyle = snow ? "rgba(244,251,248,0.72)" : "rgba(1,2,2,0.74)";
    ctx.lineWidth = Math.max(1.3, w * 0.018);
    ctx.lineCap = "round";
    ctx.beginPath();
    ctx.moveTo(-w * 0.3, 0);
    ctx.lineTo(-w * (0.36 + speed * 0.24), h * 0.18);
    ctx.moveTo(w * 0.3, 0);
    ctx.lineTo(w * (0.36 + speed * 0.24), h * 0.18);
    ctx.stroke();
  }
  ctx.restore();
}

function drawVisibleTireRoadLock(w, h, vehicleType = "car") {
  const air = ["airplane", "helicopter"].includes(vehicleType);
  const water = vehicleType === "boat";
  if (air || water) return;
  const wide = ["semi", "truck", "monster", "tank", "tractor"].includes(vehicleType);
  const snow = vehicleType === "snowmobile";
  const tireSpread = wide ? 0.42 : 0.36;
  const tireW = w * (wide ? 0.18 : 0.15);
  const tireH = h * (wide ? 0.055 : 0.045);
  ctx.save();
  ctx.globalAlpha = snow ? 0.68 : 0.95;
  ctx.fillStyle = snow ? "rgba(244,251,248,0.72)" : "rgba(0,0,0,0.98)";
  roundRect(-w * (wide ? 0.64 : 0.58), -h * 0.035, w * (wide ? 1.28 : 1.16), h * 0.072, Math.max(2, w * 0.025));
  ctx.fill();
  ctx.globalAlpha = 0.42;
  ctx.strokeStyle = snow ? "rgba(244,251,248,0.82)" : "rgba(244,251,248,0.16)";
  ctx.lineWidth = Math.max(1, w * 0.008);
  ctx.beginPath();
  ctx.moveTo(-w * 0.48, -h * 0.002);
  ctx.lineTo(w * 0.48, -h * 0.002);
  ctx.stroke();
  ctx.globalAlpha = snow ? 0.5 : 0.82;
  ctx.strokeStyle = snow ? "rgba(244,251,248,0.78)" : "rgba(0,0,0,0.78)";
  ctx.lineWidth = Math.max(3, w * 0.034);
  ctx.beginPath();
  ctx.moveTo(-w * tireSpread, tireH * 0.24);
  ctx.lineTo(-w * tireSpread, h * 0.26);
  ctx.moveTo(w * tireSpread, tireH * 0.24);
  ctx.lineTo(w * tireSpread, h * 0.26);
  ctx.stroke();
  ctx.globalAlpha = 1;
  ctx.fillStyle = snow ? "rgba(244,251,248,0.86)" : "rgba(0,0,0,0.98)";
  ctx.beginPath();
  ctx.ellipse(-w * tireSpread, -h * 0.006, tireW * 1.14, tireH * 1.28, 0, 0, Math.PI * 2);
  ctx.ellipse(w * tireSpread, -h * 0.006, tireW * 1.14, tireH * 1.28, 0, 0, Math.PI * 2);
  ctx.fill();
  ctx.globalAlpha = 0.58;
  ctx.strokeStyle = snow ? "rgba(244,251,248,0.72)" : "rgba(1,2,2,0.76)";
  ctx.lineWidth = Math.max(1.4, w * 0.02);
  ctx.lineCap = "round";
  ctx.beginPath();
  ctx.moveTo(-w * tireSpread, tireH * 0.35);
  ctx.lineTo(-w * (tireSpread + 0.18), h * 0.16);
  ctx.moveTo(w * tireSpread, tireH * 0.35);
  ctx.lineTo(w * (tireSpread + 0.18), h * 0.16);
  ctx.stroke();
  ctx.restore();
}

function drawPhoneAssetVehicleSprite(w, h, color, vehicleType = "car", police = false, damage = 0, contactAnchored =
false) {
  const assets = window.VelocityPhoneAssets;
  if (!assets || !assets.ready || typeof assets.getVehicleSprite !== "function") return false;
  const type = police ? "car" : vehicleType;
  const sprite = assets.getVehicleSprite(type, color, { police, damage });
```

```
  if (!sprite) return false;
  const wide = ["semi", "truck", "monster", "tank", "tractor"].includes(type);
  const floating = ["boat", "airplane", "helicopter"].includes(type);
  const snow = type === "snowmobile";
  const spriteW = w * (contactAnchored ? (type === "semi" ? 1.86 : wide ? 1.78 : snow ? 1.68 : 1.72) : (type ===
"semi" ? 1.72 : wide ? 1.58 : 1.5));
  const spriteH = h * (contactAnchored ? (type === "semi" ? 1.18 : wide ? 1.12 : snow ? 1.0 : 1.06) : (type === "semi"
 ? 1.66 : floating || snow ? 1.48 : 1.56));
  const contactRatio = spriteContactRatio(type);
  const speedFactor = Math.abs(raceState.speed || 0) / 220;
  ctx.save();
  ctx.imageSmoothingEnabled = true;
  if (contactAnchored) {
    drawGroundPinnedVehicleContact(w, h, type, speedFactor);
    ctx.drawImage(sprite, -spriteW / 2, -spriteH * contactRatio, spriteW, spriteH);
    drawGroundPinnedVehicleContact(w * 0.72, h * 0.72, type, speedFactor);
    drawVisibleTireRoadLock(w, h, type);
  } else {
    drawVehicleGroundContact(w, h, type, speedFactor);
    ctx.drawImage(sprite, -spriteW / 2, -spriteH * 0.3, spriteW, spriteH);
    drawVehicleRoadLock(w, h, type, speedFactor);
  }
  ctx.restore();
  return true;
}

function drawPhoneAssetTexturePass(w, h, theme) {
  const assets = window.VelocityPhoneAssets;
  if (!assets || !assets.ready || typeof assets.getRoadTexture !== "function") return;
  if (phoneCleanRoadActive()) return;
  const texture = assets.getRoadTexture(selectedRace ? selectedRace.place : "city", theme);
  const pattern = ctx.createPattern(texture, "repeat");
  if (!pattern) return;
  const phoneMode = assets.isPhoneViewport && assets.isPhoneViewport();
  const clipPhoneRoad = () => {
    const horizon = cameraMode === "cockpit" ? h * 0.27 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
    const roadTop = cameraMode === "cockpit" ? w * 0.11 : cameraMode === "hood" ? w * 0.14 : w * 0.13;
    const roadBottom = cameraMode === "cockpit" ? w * 1.02 : cameraMode === "hood" ? w * 0.98 : w * 1.08;
    const turn = raceState.roadTurn || raceState.roadCurve || 0;
    const roadCenter = (t) => {
      const clamped = Math.max(0, Math.min(1.08, t));
      const farPull = (1 - clamped) * (1 - clamped);
      return w * 0.5 + turn * farPull * w * 0.26;
    };
    ctx.beginPath();
    ctx.moveTo(roadCenter(0) - roadTop, horizon);
    ctx.lineTo(roadCenter(0) + roadTop, horizon);
    ctx.lineTo(roadCenter(1) + roadBottom * 0.62, h + 8);
    ctx.lineTo(roadCenter(1) - roadBottom * 0.62, h + 8);
    ctx.closePath();
    ctx.clip();
  };
  ctx.save();
  if (phoneMode) clipPhoneRoad();
  ctx.globalAlpha = phoneMode ? 0.08 : 0.13;
  const tile = texture.width || 256;
  ctx.translate(-((raceState.roadOffset * 0.28) % tile), -((raceState.roadOffset * 0.08) % tile));
  ctx.fillStyle = pattern;
  ctx.fillRect(-256, h * 0.3, w + 512, h * 0.78 + 256);
  ctx.restore();

  ctx.save();
  if (phoneMode) clipPhoneRoad();
  const bloom = ctx.createRadialGradient(w * 0.5, h * 0.72, w * 0.05, w * 0.5, h * 0.78, w * 0.58);
  bloom.addColorStop(0, "rgba(244,251,248,0.07)");
  bloom.addColorStop(0.52, "rgba(244,251,248,0.035)");
  bloom.addColorStop(1, "rgba(0,0,0,0)");
  ctx.fillStyle = bloom;
  ctx.fillRect(0, h * 0.34, w, h * 0.66);
  ctx.restore();
}

function drawRoadWeightPass(w, h, theme) {
  const horizon = cameraMode === "cockpit" ? h * 0.28 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const roadTop = cameraMode === "cockpit" ? w * 0.12 : cameraMode === "hood" ? w * 0.15 : w * 0.14;
  const roadBottom = cameraMode === "cockpit" ? w * 1.05 : cameraMode === "hood" ? w * 0.95 : w * 1.02;
  const roadCenter = (t) => w * 0.5 - cameraLaneOffset(0.24 + t * 0.86) * laneWidth() * (0.18 + t * 0.95);
  const topCenter = roadCenter(0);
  const bottomCenter = roadCenter(1);
  ctx.save();
  ctx.beginPath();
  ctx.moveTo(topCenter - roadTop * 0.88, horizon);
  ctx.lineTo(topCenter + roadTop * 0.88, horizon);
  ctx.lineTo(bottomCenter + roadBottom * 0.7, h);
```

```
  ctx.lineTo(bottomCenter - roadBottom * 0.7, h);
  ctx.closePath();
  ctx.clip();

  const asphaltWeight = ctx.createLinearGradient(0, horizon, 0, h);
  asphaltWeight.addColorStop(0, "rgba(0,0,0,0)");
  asphaltWeight.addColorStop(0.55, "rgba(0,0,0,0.2)");
  asphaltWeight.addColorStop(1, "rgba(0,0,0,0.64)");
  ctx.fillStyle = asphaltWeight;
  ctx.fillRect(0, horizon, w, h - horizon);

  ctx.globalAlpha = 0.32;
  for (let lane = -1.5; lane <= 1.5; lane += 1) {
    for (let i = 0; i < 7; i += 1) {
      const y = horizon + (((i * 92 + raceState.roadOffset * 0.72) % (h - horizon + 110)) - 30);
      if (y < horizon || y > h) continue;
      const t = Math.max(0, Math.min(1, (y - horizon) / Math.max(1, h - horizon)));
      const x = roadCenter(t) + lane * laneWidth() * (0.28 + t * 1.1);
      ctx.fillStyle = "rgba(0,0,0,0.56)";
      roundRect(x - (4 + t * 7), y, 8 + t * 14, 2 + t * 5, 2);
      ctx.fill();
    }
  }
  ctx.restore();
}

function drawRoadMotionPass(w, h, theme) {
  const speed = Math.max(0, Math.abs(raceState.speed || 0));
  if (speed < 3 && raceState.active) return;
  const horizon = cameraMode === "cockpit" ? h * 0.28 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const roadTop = cameraMode === "cockpit" ? w * 0.12 : cameraMode === "hood" ? w * 0.15 : w * 0.14;
  const roadBottom = cameraMode === "cockpit" ? w * 1.05 : cameraMode === "hood" ? w * 0.95 : w * 1.02;
  const motion = raceState.roadOffset * (1.35 + Math.min(1.9, speed / 130));
  const loop = (value, span) => ((value % span) + span) % span;
  const roadCenter = (t) => w * 0.5 - cameraLaneOffset(0.24 + t * 0.86) * laneWidth() * (0.18 + t * 0.95);
  const roadX = (lane, t) => roadCenter(t) + lane * laneWidth() * (0.28 + t * 1.18);
  const topCenter = roadCenter(0);
  const bottomCenter = roadCenter(1);
  const paintSegment = (lane, y, length, width, style, alpha = 1) => {
    const y1 = Math.max(horizon, y);
    const y2 = Math.min(h + 80, y + length);
    if (y2 <= horizon) return;
    const t1 = Math.max(0, Math.min(1.08, (y1 - horizon) / (h - horizon)));
    const t2 = Math.max(0, Math.min(1.12, (y2 - horizon) / (h - horizon)));
    const x1 = roadX(lane, t1);
    const x2 = roadX(lane, t2);
    const w1 = width * (0.22 + t1 * 0.72);
    const w2 = width * (0.22 + t2 * 1.15);
    ctx.globalAlpha = alpha;
    ctx.fillStyle = style;
    ctx.beginPath();
    ctx.moveTo(x1 - w1 / 2, y1);
    ctx.lineTo(x1 + w1 / 2, y1);
    ctx.lineTo(x2 + w2 / 2, y2);
    ctx.lineTo(x2 - w2 / 2, y2);
    ctx.closePath();
    ctx.fill();
  };

  ctx.save();
  ctx.beginPath();
  ctx.moveTo(topCenter - roadTop * 0.88, horizon);
  ctx.lineTo(topCenter + roadTop * 0.88, horizon);
  ctx.lineTo(bottomCenter + roadBottom * 0.7, h);
  ctx.lineTo(bottomCenter - roadBottom * 0.7, h);
  ctx.closePath();
  ctx.clip();

  const bandAlpha = Math.min(0.42, 0.12 + speed / 520);
  for (let i = 0; i < 30; i += 1) {
    const span = h - horizon + 180;
    const y = horizon - 90 + loop(i * 58 + motion * 1.18, span);
    if (y < horizon || y > h + 30) continue;
    const t = Math.max(0, Math.min(1, (y - horizon) / (h - horizon)));
    const roadHalf = roadTop * 0.62 + (roadBottom * 0.63 - roadTop * 0.62) * t;
    const center = roadCenter(t);
    ctx.globalAlpha = bandAlpha * (0.3 + t * 0.9);
    ctx.strokeStyle = i % 2 ? "rgba(0,0,0,0.78)" : "rgba(244,251,248,0.18)";
    ctx.lineWidth = 1 + t * 7;
    ctx.beginPath();
    ctx.moveTo(center - roadHalf, y);
    ctx.lineTo(center + roadHalf, y + 3 + t * 14);
    ctx.stroke();
  }
```

```
  const dashAlpha = Math.min(0.9, 0.4 + speed / 310);
  for (let lane = -1.5; lane <= 1.5; lane += 1) {
    for (let i = 0; i < 11; i += 1) {
      const span = h - horizon + 260;
      const y = horizon - 130 + loop(i * 128 + motion * 1.72, span);
      if (y < horizon || y > h + 70) continue;
      const t = Math.max(0, Math.min(1, (y - horizon) / (h - horizon)));
      const dashH = 14 + t * 42;
      const x = roadX(lane, t);
      const dash = ctx.createLinearGradient(x, y, x, y + dashH);
      dash.addColorStop(0, "rgba(244,251,248,0)");
      dash.addColorStop(0.28, "rgba(244,251,248,0.78)");
      dash.addColorStop(1, "rgba(244,251,248,0.08)");
      paintSegment(lane, y, dashH, 8 + t * 7, dash, dashAlpha * (0.32 + t * 0.5));
    }
  }

  ctx.globalAlpha = Math.min(0.5, speed / 360);
  ctx.strokeStyle = "rgba(244,251,248,0.5)";
  ctx.lineWidth = 2;
  for (let i = 0; i < 26; i += 1) {
    const span = h - horizon + 220;
    const y = horizon - 90 + loop(i * 46 + motion * 2.1, span);
    if (y < h * 0.5) continue;
    const t = Math.max(0, Math.min(1, (y - horizon) / (h - horizon)));
    const side = i % 2 ? -1 : 1;
    const x = roadCenter(t) + side * (roadTop * 1.08 + (roadBottom * 0.82 - roadTop * 1.08) * t);
    ctx.beginPath();
    ctx.moveTo(x, y);
    ctx.lineTo(x + side * w * (0.04 + t * 0.1), y + h * (0.04 + t * 0.12));
    ctx.stroke();
  }

  ctx.restore();
  ctx.globalAlpha = 1;
}

function drawPhoneUltraGraphicsPass(w, h, theme) {
  if (!phoneGraphicsActive()) return;
  if (phoneCleanRoadActive()) return;
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  const horizon = cameraMode === "cockpit" ? h * 0.28 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const roadTop = cameraMode === "cockpit" ? w * 0.12 : cameraMode === "hood" ? w * 0.15 : w * 0.14;
  const roadBottom = cameraMode === "cockpit" ? w * 1.05 : cameraMode === "hood" ? w * 0.95 : w * 1.02;
  const speed = Math.max(0, raceState.speed || 0);

  ctx.save();
  ctx.beginPath();
  ctx.moveTo(w / 2 - roadTop * 0.88, horizon);
  ctx.lineTo(w / 2 + roadTop * 0.88, horizon);
  ctx.lineTo(w / 2 + roadBottom * 0.7, h);
  ctx.lineTo(w / 2 - roadBottom * 0.7, h);
  ctx.closePath();
  ctx.clip();

  const rubber = ctx.createLinearGradient(0, horizon, 0, h);
  rubber.addColorStop(0, "rgba(0,0,0,0)");
  rubber.addColorStop(0.42, "rgba(0,0,0,0.18)");
  rubber.addColorStop(1, "rgba(0,0,0,0.46)");
  ctx.fillStyle = rubber;
  ctx.fillRect(0, horizon, w, h - horizon);

  ctx.globalAlpha = 0.38;
  ctx.strokeStyle = "rgba(1,2,2,0.78)";
  ctx.lineWidth = Math.max(2, w * 0.004);
  for (let lane = -1; lane <= 1; lane += 2) {
    const tireBase = laneWidth() * (0.18 + lane * 0.03);
    ctx.beginPath();
    ctx.moveTo(w * 0.5 + lane * roadTop * 0.22, horizon + h * 0.04);
    ctx.bezierCurveTo(
      w * 0.5 + lane * tireBase,
      h * 0.55,
      w * 0.5 + lane * tireBase * 1.85 - (raceState.steerAngle || 0) * 18,
      h * 0.78,
      w * 0.5 + lane * tireBase * 2.4 - (raceState.lateralVelocity || 0) * 22,
      h
    );
    ctx.stroke();
  }

  ctx.globalAlpha = place === "tokyo" || place === "city" || place === "harbor" || place === "rainforest" ? 0.36 :
0.18;
  for (let i = 0; i < 18; i += 1) {
```

```
    const y = ((i * 67 + raceState.roadOffset * 1.1) % (h + 130)) - 40;
    if (y < horizon) continue;
    const t = Math.max(0, (y - horizon) / (h - horizon));
    const spread = w * (0.1 + t * 0.44);
    const x = w * 0.5 + Math.sin(i * 2.11) * w * 0.08 * t;
    const reflection = ctx.createLinearGradient(x - spread, y, x + spread, y);
    reflection.addColorStop(0, "rgba(255,255,255,0)");
    reflection.addColorStop(0.46, "rgba(244,251,248,0.22)");
    reflection.addColorStop(0.52, "rgba(255,255,255,0.22)");
    reflection.addColorStop(1, "rgba(255,255,255,0)");
    ctx.strokeStyle = reflection;
    ctx.lineWidth = 1 + t * 7;
    ctx.beginPath();
    ctx.moveTo(x - spread, y);
    ctx.quadraticCurveTo(x, y + 5 + t * 18, x + spread, y + 2);
    ctx.stroke();
  }

  ctx.restore();

  ctx.save();
  const foreground = ctx.createLinearGradient(0, h * 0.64, 0, h);
  foreground.addColorStop(0, "rgba(0,0,0,0)");
  foreground.addColorStop(0.52, speed > 70 ? "rgba(0,0,0,0.22)" : "rgba(0,0,0,0.14)");
  foreground.addColorStop(1, "rgba(0,0,0,0.48)");
  ctx.fillStyle = foreground;
  ctx.fillRect(0, h * 0.58, w, h * 0.42);

  if (speed > 45) {
    ctx.globalAlpha = Math.min(0.34, speed / 620);
    ctx.strokeStyle = "rgba(244,251,248,0.42)";
    ctx.lineWidth = 3;
    for (let i = 0; i < 18; i += 1) {
      const side = i % 2 ? -1 : 1;
      const x = side < 0 ? (i * 59 + raceState.roadOffset * 0.8) % (w * 0.28) : w - ((i * 59 + raceState.roadOffset *
0.8) % (w * 0.28));
      const y = h * (0.46 + (i % 9) * 0.06);
      ctx.beginPath();
      ctx.moveTo(x, y);
      ctx.lineTo(x + side * w * 0.12, y + h * 0.18);
      ctx.stroke();
    }
  }
  ctx.restore();
}

function drawPhoneFilmicPost(w, h, theme) {
  if (!phoneGraphicsActive()) return;
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  ctx.save();
  ctx.globalCompositeOperation = "screen";
  const bloom = ctx.createRadialGradient(w * 0.5, h * 0.54, w * 0.05, w * 0.5, h * 0.58, w * 0.7);
  bloom.addColorStop(0, "rgba(255,255,255,0.06)");
  bloom.addColorStop(0.42, "rgba(244,251,248,0.06)");
  bloom.addColorStop(1, "rgba(0,0,0,0)");
  ctx.fillStyle = bloom;
  ctx.fillRect(0, 0, w, h);
  ctx.restore();

  ctx.save();
  ctx.globalAlpha = 0.12;
  ctx.fillStyle = place === "desert" || place === "canyon" ? "rgba(255,183,74,0.22)"
    : place === "snow" || place === "europe" ? "rgba(220,232,239,0.22)"
      : "rgba(70,217,255,0.16)";
  ctx.fillRect(0, 0, w, h);
  ctx.globalAlpha = 0.08;
  ctx.fillStyle = "rgba(255,255,255,0.72)";
  for (let i = 0; i < 90; i += 1) {
    const x = (i * 71 + Math.floor(raceState.elapsed * 13) * 17) % w;
    const y = (i * 43 + Math.floor(raceState.elapsed * 11) * 23) % h;
    ctx.fillRect(x, y, 1, 1);
  }
  ctx.restore();
}

function drawPhoneLaneRadar(w, h, theme) {
  if (!phoneGraphicsActive() || !raceState.active) return;
  const panelW = Math.min(260, w * 0.31);
  const panelH = 34;
  const x = w * 0.5 - panelW / 2;
  const y = 12;
  const startX = x + panelW * 0.16;
  const span = panelW * 0.72;
  const midY = y + panelH * 0.54;
```

```
  ctx.save();
  ctx.globalAlpha = 0.86;
  ctx.fillStyle = "rgba(5,8,7,0.58)";
  roundRect(x, y, panelW, panelH, 7);
  ctx.fill();
  ctx.strokeStyle = "rgba(244,251,248,0.16)";
  ctx.lineWidth = 1;
  ctx.stroke();
  for (let lane = -2; lane <= 2; lane += 1) {
    const rowY = midY + lane * panelH * 0.085;
    ctx.globalAlpha = lane === 0 ? 0.45 : 0.24;
    ctx.strokeStyle = lane === 0 ? "rgba(244,251,248,0.58)" : "rgba(244,251,248,0.32)";
    ctx.beginPath();
    ctx.moveTo(startX, rowY);
    ctx.lineTo(startX + span, rowY);
    ctx.stroke();
  }
  ctx.globalAlpha = 1;
  ctx.fillStyle = theme[1] || "#46d9ff";
  ctx.beginPath();
  ctx.arc(startX, midY, 4.6, 0, Math.PI * 2);
  ctx.fill();
  raceState.opponents.forEach((opponent) => {
    const gap = opponent.distance - raceState.distance;
    if (gap < -220 || gap > 1680 || opponent.finished) return;
    const px = startX + Math.max(0, Math.min(1, (gap + 220) / 1900)) * span;
    const py = midY + Math.max(-2.2, Math.min(2.2, opponent.lane)) * panelH * 0.085;
    ctx.globalAlpha = opponent.wrecked ? 0.45 : 0.9;
    ctx.fillStyle = opponent.wrecked ? "#ff5b6b" : (opponent.color || theme[2] || "#ffd166");
    ctx.beginPath();
    ctx.arc(px, py, 3.8, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "rgba(5,8,7,0.7)";
    ctx.lineWidth = 1.2;
    ctx.stroke();
  });
  (raceState.oncoming || []).forEach((unit) => {
    const gap = ensureRoadDistance(unit) - raceState.distance;
    if (gap < -220 || gap > 1680 || unit.wrecked) return;
    const px = startX + Math.max(0, Math.min(1, (gap + 220) / 1900)) * span;
    const py = midY + Math.max(-2.2, Math.min(2.2, unit.lane)) * panelH * 0.085;
    ctx.globalAlpha = 0.9;
    ctx.fillStyle = "#ff5b6b";
    ctx.beginPath();
    ctx.arc(px, py, 3.6, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "rgba(5,8,7,0.7)";
    ctx.lineWidth = 1.2;
    ctx.stroke();
  });
  ctx.restore();
}

function drawTrafficRearCar(x, y, width, height, color, police = false, vehicleType = "car", label = "", damage = 0,
wrecked = false, spin = 0) {
  ctx.save();
  ctx.translate(x, y);
  const t = Math.max(0.3, Math.min(1.25, y / canvas.height));
  const w = width * (0.76 + t * 0.28);
  const h = height * (0.72 + t * 0.12);

  drawRoadContactShadow(w, h, vehicleType, 1);

  if (wrecked || damage > 45) ctx.rotate(Math.max(-0.22, Math.min(0.22, (spin || 0) * 0.18)));

  if (drawPhoneAssetVehicleSprite(w, h, color, vehicleType, police, damage, true)) {
    ctx.save();
    ctx.translate(0, -spriteContactLift(h, vehicleType));
    drawOpponentRaceIdentity(w, h, label, police, vehicleType, damage, wrecked);
    drawTrafficLabel(w, h, label);
    drawTrafficDamageBadge(w, h, damage, wrecked);
    ctx.restore();
    ctx.restore();
    return;
  }

  ctx.translate(0, -h * 0.46);

  if (!police && vehicleType !== "car") {
    drawSpecialVehicleSilhouette(w, h, color, vehicleType, false);
    drawOpponentRaceIdentity(w, h, label, false, vehicleType, damage, wrecked);
    drawVehicleDamageMarks(w, h, damage);
    drawTrafficDamageBadge(w, h, damage, wrecked);
    drawTrafficLabel(w, h, label);
```

```
    ctx.restore();
    return;
  }

  ctx.fillStyle = "#050807";
  roundRect(-w * 0.56, -h * 0.03, w * 0.16, h * 0.36, 5);
  ctx.fill();
  roundRect(w * 0.4, -h * 0.03, w * 0.16, h * 0.36, 5);
  ctx.fill();
  drawWheelRim(-w * 0.49, h * 0.08, w * 0.075, h * 0.15);
  drawWheelRim(w * 0.49, h * 0.08, w * 0.075, h * 0.15);
  drawWheelRim(-w * 0.48, h * 0.3, w * 0.065, h * 0.105, "rgba(255,209,102,0.24)");
  drawWheelRim(w * 0.48, h * 0.3, w * 0.065, h * 0.105, "rgba(255,209,102,0.24)");

  const body = ctx.createLinearGradient(-w * 0.48, -h * 0.48, w * 0.48, h * 0.44);
  body.addColorStop(0, police ? "#ffffff" : shade(color, 38));
  body.addColorStop(0.48, police ? "#dce2e1" : color);
  body.addColorStop(1, police ? "#9da9a7" : shade(color, -44));
  ctx.fillStyle = body;
  ctx.beginPath();
  ctx.moveTo(-w * 0.34, -h * 0.5);
  ctx.lineTo(w * 0.34, -h * 0.5);
  ctx.lineTo(w * 0.52, -h * 0.12);
  ctx.lineTo(w * 0.42, h * 0.42);
  ctx.lineTo(-w * 0.42, h * 0.42);
  ctx.lineTo(-w * 0.52, -h * 0.12);
  ctx.closePath();
  ctx.fill();
  ctx.strokeStyle = "rgba(255,255,255,0.22)";
  ctx.lineWidth = Math.max(1, w * 0.016);
  ctx.stroke();
  drawVehicleTrimDetails(w, h, color, police);
  drawOpponentRaceIdentity(w, h, label, police, vehicleType, damage, wrecked);

  ctx.fillStyle = "rgba(5,8,7,0.86)";
  roundRect(-w * 0.28, -h * 0.35, w * 0.56, h * 0.2, 7);
  ctx.fill();
  ctx.fillStyle = "rgba(222,249,255,0.22)";
  roundRect(-w * 0.22, -h * 0.31, w * 0.24, h * 0.035, 3);
  ctx.fill();
  ctx.fillStyle = "rgba(5,8,7,0.92)";
  roundRect(-w * 0.34, h * 0.02, w * 0.68, h * 0.21, 8);
  ctx.fill();
  ctx.strokeStyle = "rgba(244,251,248,0.18)";
  ctx.lineWidth = Math.max(1, w * 0.012);
  ctx.beginPath();
  ctx.moveTo(-w * 0.26, h * 0.06);
  ctx.lineTo(w * 0.26, h * 0.06);
  ctx.moveTo(0, h * 0.02);
  ctx.lineTo(0, h * 0.23);
  ctx.stroke();

  if (police) {
    ctx.fillStyle = "#111817";
    ctx.fillRect(-w * 0.38, -h * 0.08, w * 0.76, h * 0.09);
    ctx.fillStyle = "#f4fbf8";
    ctx.font = `${Math.max(7, w * 0.13)}px system-ui`;
    ctx.textAlign = "center";
    ctx.fillText("POLICE", 0, h * 0.17);
    const flash = Math.sin(raceState.elapsed * 18) > 0;
    ctx.fillStyle = flash ? "#ff3348" : "#46d9ff";
    roundRect(-w * 0.18, -h * 0.22, w * 0.36, h * 0.06, 3);
    ctx.fill();
  }

  drawTrafficLabel(w, h, label);

  ctx.fillStyle = "#ff3348";
  roundRect(-w * 0.35, h * 0.3, w * 0.18, h * 0.06, 3);
  ctx.fill();
  roundRect(w * 0.17, h * 0.3, w * 0.18, h * 0.06, 3);
  ctx.fill();
  ctx.fillStyle = "rgba(244,251,248,0.58)";
  roundRect(-w * 0.08, h * 0.32, w * 0.16, h * 0.035, 3);
  ctx.fill();
  ctx.fillStyle = "rgba(5,8,7,0.9)";
  roundRect(-w * 0.17, h * 0.43, w * 0.12, h * 0.035, 3);
  ctx.fill();
  roundRect(w * 0.05, h * 0.43, w * 0.12, h * 0.035, 3);
  ctx.fill();
  drawVehicleDamageMarks(w, h, damage);
  drawTrafficDamageBadge(w, h, damage, wrecked);
  ctx.restore();
}
```

```
function drawOpponentRaceIdentity(w, h, label = "", police = false, vehicleType = "car", damage = 0, wrecked = false)
{
  if (!label && !police) return;
  const seed = hashText(`${label || "traffic"}:${vehicleType}`);
  const number = String((seed % 89) + 10);
  const stripe = seededUnit(seed, 2) > 0.5 ? 1 : -1;
  const accent = police ? "#46d9ff" : (seededUnit(seed, 3) > 0.5 ? "#ffd166" : "#46d9ff");
  ctx.save();
  if (phoneCleanRoadActive()) {
    ctx.globalAlpha = wrecked ? 0.44 : 0.76;
    ctx.fillStyle = police ? "rgba(5,8,7,0.82)" : "rgba(5,8,7,0.7)";
    roundRect(-w * 0.17, h * 0.07, w * 0.34, h * 0.12, 4);
    ctx.fill();
    ctx.strokeStyle = police ? "#ff3348" : accent;
    ctx.lineWidth = Math.max(1, w * 0.011);
    ctx.stroke();
    ctx.fillStyle = "#f4fbf8";
    ctx.font = `900 ${Math.max(8, w * 0.14)}px system-ui`;
    ctx.textAlign = "center";
    ctx.fillText(police ? "HEAT" : number, 0, h * 0.17);
    if (damage > 55) {
      ctx.globalCompositeOperation = "screen";
      ctx.globalAlpha = Math.min(0.48, damage / 170);
      ctx.fillStyle = "#ff5b6b";
      ctx.beginPath();
      ctx.ellipse(0, h * 0.3, w * 0.16, h * 0.036, 0, 0, Math.PI * 2);
      ctx.fill();
    }
    ctx.restore();
    return;
  }
  ctx.globalAlpha = wrecked ? 0.48 : 0.82;
  ctx.strokeStyle = accent;
  ctx.lineWidth = Math.max(2, w * 0.028);
  ctx.beginPath();
  ctx.moveTo(-w * 0.32 * stripe, -h * 0.42);
  ctx.lineTo(w * 0.34 * stripe, h * 0.36);
  ctx.stroke();
  ctx.globalAlpha = wrecked ? 0.36 : 0.68;
  ctx.strokeStyle = "rgba(244,251,248,0.58)";
  ctx.lineWidth = Math.max(1, w * 0.012);
  ctx.beginPath();
  ctx.moveTo(-w * 0.2 * stripe, -h * 0.46);
  ctx.lineTo(w * 0.44 * stripe, h * 0.24);
  ctx.stroke();
  ctx.globalAlpha = 0.86;
  ctx.fillStyle = police ? "rgba(5,8,7,0.78)" : "rgba(5,8,7,0.66)";
  roundRect(-w * 0.16, h * 0.08, w * 0.32, h * 0.13, 4);
  ctx.fill();
  ctx.strokeStyle = police ? "#ff3348" : accent;
  ctx.lineWidth = Math.max(1, w * 0.01);
  ctx.stroke();
  ctx.fillStyle = "#f4fbf8";
  ctx.font = `900 ${Math.max(8, w * 0.15)}px system-ui`;
  ctx.textAlign = "center";
  ctx.fillText(police ? "HEAT" : number, 0, h * 0.18);
  if (label) {
    ctx.globalAlpha = 0.7;
    ctx.fillStyle = "rgba(222,249,255,0.32)";
    ctx.beginPath();
    ctx.arc(-w * 0.07, -h * 0.23, w * 0.045, 0, Math.PI * 2);
    ctx.arc(w * 0.07, -h * 0.23, w * 0.045, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "rgba(5,8,7,0.72)";
    ctx.lineWidth = Math.max(1, w * 0.012);
    ctx.beginPath();
    ctx.moveTo(-w * 0.1, -h * 0.18);
    ctx.lineTo(w * 0.1, -h * 0.18);
    ctx.stroke();
  }
  if (damage > 55) {
    ctx.globalCompositeOperation = "screen";
    ctx.globalAlpha = Math.min(0.55, damage / 150);
    ctx.fillStyle = "#ff5b6b";
    ctx.beginPath();
    ctx.ellipse(stripe * w * 0.24, h * 0.3, w * 0.12, h * 0.035, 0, 0, Math.PI * 2);
    ctx.fill();
  }
  ctx.restore();
}

function drawTrafficDamageBadge(w, h, damage, wrecked) {
  if (damage < 12 && !wrecked) return;
```

```
  ctx.save();
  const barWidth = w * 0.58;
  const pct = Math.min(1, damage / 100);
  ctx.fillStyle = "rgba(5,8,7,0.74)";
  roundRect(-barWidth / 2, -h * 0.88, barWidth, h * 0.055, 3);
  ctx.fill();
  ctx.fillStyle = wrecked ? "#ff5b6b" : damage > 58 ? "#ffd166" : "#bbf24a";
  roundRect(-barWidth / 2, -h * 0.88, barWidth * pct, h * 0.055, 3);
  ctx.fill();
  if (wrecked) {
    ctx.fillStyle = "rgba(255,91,107,0.72)";
    ctx.beginPath();
    ctx.moveTo(-w * 0.16, -h * 0.72);
    ctx.lineTo(0, -h * 0.56);
    ctx.lineTo(w * 0.16, -h * 0.72);
    ctx.closePath();
    ctx.fill();
  }
  ctx.restore();
}

function drawWheelRim(x, y, rx, ry, accent = "rgba(70,217,255,0.26)") {
  ctx.save();
  ctx.fillStyle = "rgba(4,6,6,0.96)";
  ctx.beginPath();
  ctx.ellipse(x, y, rx, ry, 0, 0, Math.PI * 2);
  ctx.fill();
  ctx.strokeStyle = "rgba(244,251,248,0.28)";
  ctx.lineWidth = Math.max(1, rx * 0.18);
  ctx.stroke();
  ctx.strokeStyle = accent;
  ctx.lineWidth = Math.max(1, rx * 0.12);
  ctx.beginPath();
  ctx.moveTo(x - rx * 0.58, y);
  ctx.lineTo(x + rx * 0.58, y);
  ctx.moveTo(x, y - ry * 0.55);
  ctx.lineTo(x, y + ry * 0.55);
  ctx.moveTo(x - rx * 0.38, y - ry * 0.38);
  ctx.lineTo(x + rx * 0.38, y + ry * 0.38);
  ctx.moveTo(x + rx * 0.38, y - ry * 0.38);
  ctx.lineTo(x - rx * 0.38, y + ry * 0.38);
  ctx.stroke();
  ctx.restore();
}

function drawVehicleTrimDetails(w, h, color, police = false) {
  ctx.save();
  const trim = police ? "#111817" : shade(color, -46);
  ctx.fillStyle = trim;
  roundRect(-w * 0.65, -h * 0.24, w * 0.14, h * 0.06, 3);
  ctx.fill();
  roundRect(w * 0.51, -h * 0.24, w * 0.14, h * 0.06, 3);
  ctx.fill();
  ctx.strokeStyle = "rgba(244,251,248,0.22)";
  ctx.lineWidth = Math.max(1, w * 0.011);
  ctx.beginPath();
  ctx.moveTo(-w * 0.33, -h * 0.08);
  ctx.lineTo(-w * 0.42, h * 0.38);
  ctx.moveTo(w * 0.33, -h * 0.08);
  ctx.lineTo(w * 0.42, h * 0.38);
  ctx.moveTo(-w * 0.22, -h * 0.36);
  ctx.lineTo(w * 0.22, -h * 0.36);
  ctx.stroke();
  ctx.fillStyle = "rgba(5,8,7,0.86)";
  roundRect(-w * 0.23, h * 0.38, w * 0.46, h * 0.065, 4);
  ctx.fill();
  ctx.fillStyle = "rgba(244,251,248,0.74)";
  roundRect(-w * 0.075, h * 0.392, w * 0.15, h * 0.032, 3);
  ctx.fill();
  ctx.fillStyle = "rgba(255,248,214,0.8)";
  roundRect(-w * 0.37, -h * 0.46, w * 0.17, h * 0.055, 3);
  ctx.fill();
  roundRect(w * 0.2, -h * 0.46, w * 0.17, h * 0.055, 3);
  ctx.fill();
  ctx.fillStyle = "rgba(5,8,7,0.84)";
  roundRect(-w * 0.34, h * 0.25, w * 0.68, h * 0.055, 3);
  ctx.fill();
  ctx.fillStyle = "rgba(244,251,248,0.18)";
  roundRect(-w * 0.22, -h * 0.31, w * 0.44, h * 0.045, 3);
  ctx.fill();
  if (raceIsNight(selectedRace)) {
    ctx.save();
    ctx.globalCompositeOperation = "screen";
    ctx.fillStyle = police ? "rgba(70,217,255,0.35)" : "rgba(255,248,214,0.28)";
```

```
    ctx.beginPath();
    ctx.ellipse(-w * 0.285, -h * 0.48, w * 0.18, h * 0.04, -0.08, 0, Math.PI * 2);
    ctx.ellipse(w * 0.285, -h * 0.48, w * 0.18, h * 0.04, 0.08, 0, Math.PI * 2);
    ctx.fill();
    ctx.fillStyle = "rgba(255,51,72,0.34)";
    ctx.beginPath();
    ctx.ellipse(-w * 0.3, h * 0.42, w * 0.14, h * 0.035, 0, 0, Math.PI * 2);
    ctx.ellipse(w * 0.3, h * 0.42, w * 0.14, h * 0.035, 0, 0, Math.PI * 2);
    ctx.fill();
    ctx.restore();
  }
  ctx.restore();
}

function drawLicensePlate(w, h, plate) {
  const text = sanitizePlate(plate || "RACER");
  ctx.save();
  ctx.fillStyle = "rgba(244,251,248,0.88)";
  roundRect(-w * 0.16, h * 0.36, w * 0.32, h * 0.058, 3);
  ctx.fill();
  ctx.fillStyle = "#06100e";
  ctx.font = `900 ${Math.max(6, Math.min(13, w * 0.075))}px system-ui, sans-serif`;
  ctx.textAlign = "center";
  ctx.textBaseline = "middle";
  ctx.fillText(text, 0, h * 0.389, w * 0.28);
  ctx.restore();
}

function drawSpecialVehicleSilhouette(w, h, color, vehicleType, player) {
  const paint = ctx.createLinearGradient(-w * 0.5, -h * 0.48, w * 0.48, h * 0.48);
  paint.addColorStop(0, shade(color, 42));
  paint.addColorStop(0.5, color);
  paint.addColorStop(1, shade(color, -48));
  ctx.fillStyle = paint;

  if (vehicleType === "f1" || vehicleType === "prototype") {
    ctx.fillStyle = "#050807";
    roundRect(-w * 0.62, h * 0.2, w * 0.18, h * 0.2, 6);
    ctx.fill();
    roundRect(w * 0.44, h * 0.2, w * 0.18, h * 0.2, 6);
    ctx.fill();
    ctx.fillStyle = paint;
    ctx.beginPath();
    ctx.moveTo(0, -h * 0.58);
    ctx.lineTo(w * 0.36, -h * 0.2);
    ctx.lineTo(w * 0.26, h * 0.45);
    ctx.lineTo(-w * 0.26, h * 0.45);
    ctx.lineTo(-w * 0.36, -h * 0.2);
    ctx.closePath();
    ctx.fill();
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    ctx.beginPath();
    ctx.ellipse(0, -h * 0.1, w * 0.18, h * 0.16, 0, 0, Math.PI * 2);
    ctx.fill();
    ctx.fillStyle = shade(color, -35);
    roundRect(-w * 0.48, h * 0.36, w * 0.96, h * 0.08, 3);
    ctx.fill();
  } else if (vehicleType === "semi") {
    ctx.fillStyle = "#050807";
    roundRect(-w * 0.72, -h * 0.02, w * 0.18, h * 0.5, 7);
    ctx.fill();
    roundRect(w * 0.54, -h * 0.02, w * 0.18, h * 0.5, 7);
    ctx.fill();
    roundRect(-w * 0.72, -h * 0.42, w * 0.16, h * 0.22, 6);
    ctx.fill();
    roundRect(w * 0.56, -h * 0.42, w * 0.16, h * 0.22, 6);
    ctx.fill();
    ctx.fillStyle = paint;
    roundRect(-w * 0.42, -h * 0.58, w * 0.84, h * 0.38, 6);
    ctx.fill();
    ctx.fillStyle = shade(color, -28);
    roundRect(-w * 0.5, -h * 0.16, w * 1.0, h * 0.68, 4);
    ctx.fill();
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    roundRect(-w * 0.27, -h * 0.5, w * 0.54, h * 0.18, 4);
    ctx.fill();
    ctx.fillStyle = "rgba(244,251,248,0.48)";
    ctx.fillRect(-w * 0.34, -h * 0.1, w * 0.68, h * 0.08);
    ctx.fillRect(-w * 0.34, h * 0.08, w * 0.68, h * 0.08);
  } else if (vehicleType === "tractor") {
    ctx.fillStyle = "#050807";
    roundRect(-w * 0.64, h * 0.12, w * 0.26, h * 0.36, 9);
    ctx.fill();
    roundRect(w * 0.38, h * 0.12, w * 0.26, h * 0.36, 9);
```

```
    ctx.fill();
    roundRect(-w * 0.48, -h * 0.42, w * 0.14, h * 0.2, 5);
    ctx.fill();
    roundRect(w * 0.34, -h * 0.42, w * 0.14, h * 0.2, 5);
    ctx.fill();
    ctx.fillStyle = paint;
    roundRect(-w * 0.3, -h * 0.48, w * 0.6, h * 0.36, 8);
    ctx.fill();
    ctx.fillStyle = shade(color, -28);
    roundRect(-w * 0.42, -h * 0.06, w * 0.84, h * 0.38, 8);
    ctx.fill();
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    roundRect(-w * 0.2, -h * 0.38, w * 0.4, h * 0.16, 4);
    ctx.fill();
    ctx.strokeStyle = "rgba(255,209,102,0.6)";
    ctx.lineWidth = Math.max(2, w * 0.035);
    ctx.beginPath();
    ctx.moveTo(-w * 0.44, h * 0.38);
    ctx.lineTo(w * 0.44, h * 0.38);
    ctx.stroke();
  } else if (vehicleType === "truck" || vehicleType === "monster" || vehicleType === "tank") {
    const tire = vehicleType === "monster" ? 0.24 : 0.17;
    ctx.fillStyle = "#050807";
    roundRect(-w * 0.67, -h * 0.08, w * tire, h * 0.5, 7);
    ctx.fill();
    roundRect(w * (0.67 - tire), -h * 0.08, w * tire, h * 0.5, 7);
    ctx.fill();
    ctx.fillStyle = paint;
    roundRect(-w * 0.48, -h * 0.48, w * 0.96, h * 0.82, vehicleType === "tank" ? 3 : 7);
    ctx.fill();
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    roundRect(-w * 0.27, -h * 0.32, w * 0.54, h * 0.22, 5);
    ctx.fill();
    if (vehicleType === "tank") {
      ctx.fillStyle = shade(color, -25);
      roundRect(-w * 0.22, -h * 0.18, w * 0.44, h * 0.3, 4);
      ctx.fill();
      ctx.fillRect(-w * 0.04, -h * 0.66, w * 0.08, h * 0.5);
    }
  } else if (vehicleType === "boat" || vehicleType === "snowmobile") {
    ctx.beginPath();
    ctx.moveTo(0, -h * 0.58);
    ctx.quadraticCurveTo(w * 0.5, -h * 0.05, w * 0.24, h * 0.48);
    ctx.lineTo(-w * 0.24, h * 0.48);
    ctx.quadraticCurveTo(-w * 0.5, -h * 0.05, 0, -h * 0.58);
    ctx.closePath();
    ctx.fill();
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    roundRect(-w * 0.16, -h * 0.12, w * 0.32, h * 0.24, 5);
    ctx.fill();
    ctx.strokeStyle = vehicleType === "boat" ? "rgba(70,217,255,0.58)" : "rgba(244,251,248,0.58)";
    ctx.lineWidth = Math.max(2, w * 0.04);
    ctx.beginPath();
    ctx.moveTo(-w * 0.44, h * 0.36);
    ctx.lineTo(-w * 0.68, h * 0.5);
    ctx.moveTo(w * 0.44, h * 0.36);
    ctx.lineTo(w * 0.68, h * 0.5);
    ctx.stroke();
  } else if (vehicleType === "helicopter") {
    ctx.fillStyle = "rgba(244,251,248,0.68)";
    roundRect(-w * 0.62, -h * 0.64, w * 1.24, h * 0.05, 2);
    ctx.fill();
    ctx.fillStyle = paint;
    roundRect(-w * 0.28, -h * 0.36, w * 0.56, h * 0.56, 14);
    ctx.fill();
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    roundRect(-w * 0.17, -h * 0.24, w * 0.34, h * 0.18, 5);
    ctx.fill();
    ctx.strokeStyle = shade(color, -30);
    ctx.lineWidth = Math.max(2, w * 0.05);
    ctx.beginPath();
    ctx.moveTo(0, h * 0.18);
    ctx.lineTo(0, h * 0.56);
    ctx.stroke();
  } else if (vehicleType === "airplane") {
    ctx.fillStyle = paint;
    ctx.beginPath();
    ctx.moveTo(0, -h * 0.62);
    ctx.lineTo(w * 0.2, h * 0.48);
    ctx.lineTo(-w * 0.2, h * 0.48);
    ctx.closePath();
    ctx.fill();
    ctx.beginPath();
    ctx.moveTo(-w * 0.68, h * 0.02);
```

```
    ctx.lineTo(w * 0.68, h * 0.02);
    ctx.lineTo(w * 0.22, h * 0.22);
    ctx.lineTo(-w * 0.22, h * 0.22);
    ctx.closePath();
    ctx.fill();
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    roundRect(-w * 0.1, -h * 0.34, w * 0.2, h * 0.26, 4);
    ctx.fill();
  }

  const groundVehicle = !["boat", "snowmobile", "helicopter", "airplane"].includes(vehicleType);
  if (groundVehicle) {
    const rimAccent = vehicleType === "tractor" ? "rgba(255,209,102,0.38)" : "rgba(70,217,255,0.28)";
    const frontY = vehicleType === "f1" || vehicleType === "prototype" ? h * 0.33 : h * 0.18;
    const rearY = vehicleType === "f1" || vehicleType === "prototype" ? h * 0.43 : h * 0.38;
    drawWheelRim(-w * 0.55, frontY, w * 0.075, h * 0.12, rimAccent);
    drawWheelRim(w * 0.55, frontY, w * 0.075, h * 0.12, rimAccent);
    drawWheelRim(-w * 0.5, rearY, w * 0.07, h * 0.105, rimAccent);
    drawWheelRim(w * 0.5, rearY, w * 0.07, h * 0.105, rimAccent);
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    roundRect(-w * 0.26, h * 0.31, w * 0.52, h * 0.075, 3);
    ctx.fill();
    ctx.fillStyle = "rgba(255,248,214,0.86)";
    roundRect(-w * 0.34, -h * 0.5, w * 0.18, h * 0.055, 3);
    ctx.fill();
    roundRect(w * 0.16, -h * 0.5, w * 0.18, h * 0.055, 3);
    ctx.fill();
    ctx.strokeStyle = "rgba(244,251,248,0.2)";
    ctx.lineWidth = Math.max(1, w * 0.012);
    ctx.beginPath();
    ctx.moveTo(-w * 0.32, -h * 0.08);
    ctx.lineTo(-w * 0.36, h * 0.3);
    ctx.moveTo(w * 0.32, -h * 0.08);
    ctx.lineTo(w * 0.36, h * 0.3);
    ctx.stroke();
  } else if (vehicleType === "boat" || vehicleType === "snowmobile") {
    ctx.fillStyle = "rgba(244,251,248,0.18)";
    roundRect(-w * 0.18, -h * 0.4, w * 0.36, h * 0.055, 3);
    ctx.fill();
    ctx.fillStyle = vehicleType === "boat" ? "rgba(70,217,255,0.44)" : "rgba(244,251,248,0.44)";
    roundRect(-w * 0.34, h * 0.35, w * 0.68, h * 0.045, 3);
    ctx.fill();
  } else if (vehicleType === "helicopter") {
    ctx.fillStyle = "rgba(244,251,248,0.22)";
    ctx.beginPath();
    ctx.ellipse(0, -h * 0.62, w * 0.66, h * 0.055, 0, 0, Math.PI * 2);
    ctx.fill();
    ctx.fillStyle = "rgba(5,8,7,0.82)";
    roundRect(-w * 0.1, h * 0.48, w * 0.2, h * 0.08, 3);
    ctx.fill();
  } else if (vehicleType === "airplane") {
    ctx.fillStyle = "rgba(255,248,214,0.82)";
    roundRect(-w * 0.08, -h * 0.54, w * 0.16, h * 0.05, 3);
    ctx.fill();
    ctx.fillStyle = "rgba(70,217,255,0.36)";
    roundRect(-w * 0.48, h * 0.08, w * 0.96, h * 0.035, 3);
    ctx.fill();
  }

  ctx.strokeStyle = "rgba(255,255,255,0.24)";
  ctx.lineWidth = Math.max(1, w * 0.016);
  ctx.stroke();
  if (player) drawVehicleDamageMarks(w, h, raceState.damage || 0);
  if (player) {
    ctx.fillStyle = "rgba(255,209,102,0.55)";
    ctx.beginPath();
    ctx.ellipse(0, h * 0.52, w * 0.24, h * 0.05, 0, 0, Math.PI * 2);
    ctx.fill();
  }
}

function drawDemoPursuitTraffic(w, h) {
  const t = performance.now() / 1000;
  const demo = [
    { lane: -1.15 + Math.sin(t * 0.7) * 0.15, y: h * 0.43, color: "#ff5b6b", police: false, s: 0.58 },
    { lane: 1.05 + Math.sin(t * 0.9) * 0.12, y: h * 0.5, color: "#f4fbf8", police: true, s: 0.68 },
    { lane: 0.08 + Math.sin(t * 1.1) * 0.18, y: h * 0.61, color: "#46d9ff", police: false, s: 0.86 }
  ];
  demo.forEach((car) => {
    const p = objectPos(car.lane, car.y);
    drawTrafficRearCar(p.x, p.y, 78 * car.s * p.scale, 118 * car.s * p.scale, car.color, car.police);
  });
  ctx.save();
  ctx.globalAlpha = 0.18 + Math.max(0, Math.sin(t * 12)) * 0.1;
```

```
  let wash = ctx.createRadialGradient(w * 0.18, h * 0.48, 10, w * 0.18, h * 0.48, w * 0.55);
  wash.addColorStop(0, "rgba(255,51,72,0.44)");
  wash.addColorStop(1, "rgba(255,51,72,0)");
  ctx.fillStyle = wash;
  ctx.fillRect(0, 0, w, h);
  wash = ctx.createRadialGradient(w * 0.82, h * 0.48, 10, w * 0.82, h * 0.48, w * 0.55);
  wash.addColorStop(0, "rgba(70,217,255,0.4)");
  wash.addColorStop(1, "rgba(70,217,255,0)");
  ctx.fillStyle = wash;
  ctx.fillRect(0, 0, w, h);
  ctx.restore();
}

function drawCar(w, h) {
  if (cameraMode === "cockpit") return;
  const vehicle = selectedVehicle();
  const color = selectedVehicleColor(vehicle);
  if (phoneGraphicsActive() && cameraMode === "hood") {
    drawPhoneDriverHood(w, h, Object.assign({}, vehicle, { color }));
    return;
  }
  const carLaneFollowDepth = useWebGLRenderer() && cameraMode === "chase" ? 0.42 : 1;
  let x = w / 2 + (raceState.lane - cameraLaneOffset(carLaneFollowDepth)) * laneWidth();
  let y = cameraMode === "hood" ? h * 0.99 : useWebGLRenderer() ? h * 0.84 : h * 0.86;
  let projectionScale = 1;
  if (useWebGLRenderer() && cameraMode === "chase" && webglRenderer && typeof webglRenderer.projectWorldRoadPoint ===
"function") {
    const anchor = webglRenderer.projectWorldRoadPoint(raceState.lane, 5.2);
    if (anchor && Number.isFinite(anchor.x) && Number.isFinite(anchor.y)) {
      y = Math.max(h * 0.78, Math.min(h * 0.96, anchor.y + h * 0.045));
      projectionScale = Math.max(0.92, Math.min(1.12, anchor.scale || 1));
    }
  }
  const sizeBoost = vehicle.type === "semi" ? 1.28 : vehicle.type === "monster" || vehicle.type === "tank" ? 1.18 :
vehicle.type === "tractor" ? 1.08 : vehicle.type === "f1" || vehicle.type === "snowmobile" ? 0.9 : 1;
  const phoneChaseScale = phoneGraphicsActive() && cameraMode === "chase" ? 0.66 : 1;
  const carWidth = (cameraMode === "hood" ? 220 : 118) * sizeBoost * projectionScale * phoneChaseScale;
  const carHeight = (cameraMode === "hood" ? 190 : 178) * sizeBoost * projectionScale * phoneChaseScale;
  if (cameraMode === "chase") {
    const phoneLift = phoneGraphicsActive() ? h * 0.065 : 0;
    drawPlayerChaseCar(x, y + phoneLift + 18 + roadContactSink(projectionScale, vehicle.type), carWidth * 1.26,
carHeight * 1.02, color, vehicle.type);
  } else {
    drawVehicle(x, y, carWidth, carHeight, color, true, false, vehicle.type);
  }
  if ((input.boost || input.gamepadBoost) && raceState.active) {
    ctx.fillStyle = "rgba(255,209,102,0.82)";
    ctx.beginPath();
    ctx.moveTo(x - carWidth * 0.28, y + carHeight * 0.44);
    ctx.lineTo(x, y + carHeight * 0.95 + Math.random() * 34);
    ctx.lineTo(x + carWidth * 0.28, y + carHeight * 0.44);
    ctx.closePath();
    ctx.fill();
  }
}

function drawPhoneDriverHood(w, h, vehicle) {
  const hoodY = h * 0.84;
  const color = vehicle.color || "#46d9ff";
  ctx.save();
  const hoodShift = Math.max(-w * 0.035, Math.min(w * 0.035, (raceState.steerAngle || 0) * w * 0.018 +
(raceState.lateralVelocity || 0) * w * 0.012));
  ctx.translate(hoodShift, 0);
  const shadow = ctx.createLinearGradient(0, h * 0.68, 0, h);
  shadow.addColorStop(0, "rgba(0,0,0,0)");
  shadow.addColorStop(0.62, "rgba(0,0,0,0.62)");
  shadow.addColorStop(1, "rgba(0,0,0,0.92)");
  ctx.fillStyle = shadow;
  ctx.fillRect(0, h * 0.62, w, h * 0.38);

  const hood = ctx.createLinearGradient(w * 0.28, hoodY, w * 0.72, h);
  hood.addColorStop(0, shade(color, -24));
  hood.addColorStop(0.45, shade(color, 42));
  hood.addColorStop(1, shade(color, -52));
  ctx.fillStyle = hood;
  ctx.beginPath();
  ctx.moveTo(w * 0.18, h + 18);
  ctx.quadraticCurveTo(w * 0.28, hoodY + 4, w * 0.42, hoodY - h * 0.015);
  ctx.lineTo(w * 0.58, hoodY - h * 0.015);
  ctx.quadraticCurveTo(w * 0.72, hoodY + 4, w * 0.82, h + 18);
  ctx.closePath();
  ctx.fill();
  ctx.strokeStyle = "rgba(244,251,248,0.24)";
  ctx.lineWidth = Math.max(2, w * 0.004);
```

```
  ctx.stroke();

  ctx.globalAlpha = 0.72;
  ctx.strokeStyle = "rgba(5,8,7,0.58)";
  ctx.lineWidth = Math.max(3, w * 0.006);
  ctx.beginPath();
  ctx.moveTo(w * 0.5, hoodY);
  ctx.lineTo(w * 0.5, h);
  ctx.stroke();
  ctx.globalAlpha = 1;
  ctx.fillStyle = "rgba(5,8,7,0.82)";
  roundRect(w * 0.36, hoodY + h * 0.08, w * 0.28, h * 0.045, 5);
  ctx.fill();
  ctx.fillStyle = "rgba(244,251,248,0.2)";
  roundRect(w * 0.375, hoodY + h * 0.095, w * 0.25, h * 0.012, 4);
  ctx.fill();
  if ((input.boost || input.gamepadBoost) && raceState.active) {
    ctx.globalCompositeOperation = "screen";
    ctx.fillStyle = "rgba(255,209,102,0.28)";
    ctx.beginPath();
    ctx.moveTo(w * 0.42, h * 0.97);
    ctx.lineTo(w * 0.5, h * 0.82);
    ctx.lineTo(w * 0.58, h * 0.97);
    ctx.closePath();
    ctx.fill();
  }
  ctx.restore();
}

function drawPlayerChaseCar(x, y, width, height, color, vehicleType = "car") {
  ctx.save();
  ctx.translate(x, y);
  const bodyYaw = Math.max(-0.12, Math.min(0.12, (raceState.steerAngle || 0) * 0.055 + (raceState.lateralVelocity ||
0) * 0.018));
  ctx.rotate(bodyYaw);

  drawRoadContactShadow(width, height, vehicleType, 1);

  if (drawPhoneAssetVehicleSprite(width, height, color, vehicleType, false, raceState.damage || 0, true)) {
    ctx.save();
    ctx.translate(0, -height * 0.18);
    drawLicensePlate(width * 0.78, height * 0.78, driverPlate(activeProfile));
    ctx.restore();
    ctx.restore();
    return;
  }

  ctx.translate(0, -height * 0.58);

  if (vehicleType !== "car") {
    drawSpecialVehicleSilhouette(width, height, color, vehicleType, true);
    drawLicensePlate(width * 0.82, height * 0.82, driverPlate(activeProfile));
    ctx.restore();
    return;
  }

  const paint = ctx.createLinearGradient(-width * 0.5, -height * 0.52, width * 0.42, height * 0.5);
  paint.addColorStop(0, shade(color, 56));
  paint.addColorStop(0.22, color);
  paint.addColorStop(0.58, shade(color, -22));
  paint.addColorStop(1, shade(color, -56));

  ctx.fillStyle = "#050807";
  roundRect(-width * 0.56, -height * 0.18, width * 0.16, height * 0.5, 8);
  ctx.fill();
  roundRect(width * 0.4, -height * 0.18, width * 0.16, height * 0.5, 8);
  ctx.fill();
  drawWheelRim(-width * 0.49, -height * 0.01, width * 0.08, height * 0.16);
  drawWheelRim(width * 0.49, -height * 0.01, width * 0.08, height * 0.16);
  drawWheelRim(-width * 0.48, height * 0.27, width * 0.07, height * 0.12, "rgba(255,209,102,0.24)");
  drawWheelRim(width * 0.48, height * 0.27, width * 0.07, height * 0.12, "rgba(255,209,102,0.24)");

  ctx.fillStyle = paint;
  ctx.beginPath();
  ctx.moveTo(-width * 0.34, -height * 0.52);
  ctx.lineTo(width * 0.34, -height * 0.52);
  ctx.quadraticCurveTo(width * 0.54, -height * 0.38, width * 0.5, -height * 0.08);
  ctx.lineTo(width * 0.43, height * 0.36);
  ctx.quadraticCurveTo(width * 0.31, height * 0.55, 0, height * 0.58);
  ctx.quadraticCurveTo(-width * 0.31, height * 0.55, -width * 0.43, height * 0.36);
  ctx.lineTo(-width * 0.5, -height * 0.08);
  ctx.quadraticCurveTo(-width * 0.54, -height * 0.38, -width * 0.34, -height * 0.52);
  ctx.closePath();
  ctx.fill();
```

```
  ctx.strokeStyle = "rgba(255,255,255,0.28)";
  ctx.lineWidth = Math.max(2, width * 0.018);
  ctx.stroke();
  drawVehicleTrimDetails(width, height, color, false);
  drawOpponentRaceIdentity(width, height, activeProfile ? activeProfile.name : "Driver", false, vehicleType,
raceState.damage || 0, false);

  const windshield = ctx.createLinearGradient(0, -height * 0.46, 0, height * 0.08);
  windshield.addColorStop(0, "rgba(222,249,255,0.72)");
  windshield.addColorStop(0.38, "rgba(58,88,94,0.82)");
  windshield.addColorStop(1, "rgba(3,6,7,0.96)");
  ctx.fillStyle = windshield;
  ctx.beginPath();
  ctx.moveTo(-width * 0.28, -height * 0.39);
  ctx.lineTo(width * 0.28, -height * 0.39);
  ctx.lineTo(width * 0.34, -height * 0.08);
  ctx.quadraticCurveTo(0, height * 0.03, -width * 0.34, -height * 0.08);
  ctx.closePath();
  ctx.fill();

  ctx.fillStyle = "rgba(5,8,7,0.9)";
  roundRect(-width * 0.32, height * 0.12, width * 0.64, height * 0.27, 10);
  ctx.fill();
  ctx.strokeStyle = "rgba(244,251,248,0.18)";
  ctx.stroke();

  ctx.strokeStyle = "rgba(5,8,7,0.52)";
  ctx.lineWidth = Math.max(2, width * 0.024);
  ctx.beginPath();
  ctx.moveTo(0, -height * 0.49);
  ctx.lineTo(0, height * 0.55);
  ctx.moveTo(-width * 0.42, -height * 0.05);
  ctx.lineTo(width * 0.42, -height * 0.05);
  ctx.moveTo(-width * 0.38, height * 0.36);
  ctx.lineTo(width * 0.38, height * 0.36);
  ctx.stroke();

  ctx.fillStyle = "rgba(255,255,255,0.18)";
  ctx.beginPath();
  ctx.moveTo(-width * 0.31, -height * 0.48);
  ctx.lineTo(-width * 0.08, -height * 0.43);
  ctx.lineTo(-width * 0.23, -height * 0.3);
  ctx.closePath();
  ctx.moveTo(width * 0.31, -height * 0.48);
  ctx.lineTo(width * 0.08, -height * 0.43);
  ctx.lineTo(width * 0.23, -height * 0.3);
  ctx.closePath();
  ctx.fill();

  ctx.fillStyle = "#ff3348";
  roundRect(-width * 0.36, height * 0.42, width * 0.22, height * 0.055, 4);
  ctx.fill();
  roundRect(width * 0.14, height * 0.42, width * 0.22, height * 0.055, 4);
  ctx.fill();
  ctx.globalAlpha = 0.28;
  ctx.fillStyle = "#ff3348";
  ctx.beginPath();
  ctx.ellipse(-width * 0.25, height * 0.44, width * 0.32, height * 0.08, 0, 0, Math.PI * 2);
  ctx.ellipse(width * 0.25, height * 0.44, width * 0.32, height * 0.08, 0, 0, Math.PI * 2);
  ctx.fill();
  ctx.globalAlpha = 1;

  ctx.fillStyle = "rgba(5,8,7,0.9)";
  roundRect(-width * 0.19, height * 0.49, width * 0.38, height * 0.05, 4);
  ctx.fill();
  ctx.fillStyle = "rgba(244,251,248,0.64)";
  roundRect(-width * 0.08, height * 0.5, width * 0.16, height * 0.03, 3);
  ctx.fill();

  drawVehicleDamageMarks(width, height, raceState.damage || 0);
  ctx.restore();
}

function drawVehicleDamageMarks(width, height, damage) {
  if (!damage || damage < 14) return;
  ctx.save();
  const dents = Math.min(8, Math.floor(damage / 12));
  ctx.strokeStyle = "rgba(5,8,7,0.72)";
  ctx.lineWidth = Math.max(1.5, width * 0.018);
  ctx.lineCap = "round";
  for (let i = 0; i < dents; i += 1) {
    const side = i % 2 ? -1 : 1;
    const x = side * width * (0.12 + (i % 3) * 0.085);
    const y = -height * 0.25 + (i % 5) * height * 0.14;
```

```
    ctx.beginPath();
    ctx.moveTo(x, y);
    ctx.lineTo(x + side * width * 0.16, y + height * (0.04 + (i % 2) * 0.035));
    ctx.stroke();
  }
  ctx.globalAlpha = Math.min(0.45, damage / 160);
  ctx.fillStyle = "rgba(0,0,0,0.7)";
  for (let i = 0; i < dents; i += 1) {
    const x = ((i * 41) % 100 - 50) / 100 * width * 0.62;
    const y = ((i * 29) % 100 - 48) / 100 * height * 0.72;
    ctx.beginPath();
    ctx.ellipse(x, y, width * 0.035, height * 0.025, i * 0.4, 0, Math.PI * 2);
    ctx.fill();
  }
  if (damage > 58) {
    ctx.globalAlpha = Math.min(0.62, damage / 140);
    ctx.fillStyle = "rgba(255,91,107,0.42)";
    roundRect(-width * 0.3, -height * 0.48, width * 0.18, height * 0.055, 3);
    ctx.fill();
  }
  ctx.restore();
}

function drawCameraOverlay(w, h, theme) {
  if (cameraMode === "chase") {
    ctx.save();
    const mirror = ctx.createLinearGradient(0, 0, 0, h * 0.18);
    mirror.addColorStop(0, "rgba(3,6,6,0.78)");
    mirror.addColorStop(1, "rgba(3,6,6,0)");
    ctx.fillStyle = mirror;
    ctx.fillRect(0, 0, w, h * 0.18);
    ctx.fillStyle = "rgba(5,8,7,0.58)";
    roundRect(w * 0.39, h * 0.055, w * 0.22, h * 0.055, 6);
    ctx.fill();
    ctx.strokeStyle = "rgba(244,251,248,0.18)";
    ctx.stroke();
    ctx.fillStyle = theme[1];
    ctx.font = "900 12px system-ui";
    ctx.textAlign = "center";
    ctx.fillText(raceState.chaseActive ? "PURSUIT CAMERA" : "CHASE CAMERA", w * 0.5, h * 0.09);
    ctx.restore();
    return;
  }
  if (cameraMode === "hood") {
    const hood = ctx.createLinearGradient(0, h * 0.72, 0, h);
    hood.addColorStop(0, "rgba(70,217,255,0.05)");
    hood.addColorStop(0.58, "#101817");
    hood.addColorStop(1, "#050807");
    ctx.fillStyle = hood;
    ctx.beginPath();
    ctx.moveTo(w * 0.14, h);
    ctx.quadraticCurveTo(w * 0.5, h * 0.68, w * 0.86, h);
    ctx.closePath();
    ctx.fill();
    ctx.strokeStyle = theme[1];
    ctx.lineWidth = 4;
    ctx.globalAlpha = 0.8;
    ctx.beginPath();
    ctx.moveTo(w * 0.36, h * 0.91);
    ctx.lineTo(w * 0.64, h * 0.91);
    ctx.stroke();
    ctx.globalAlpha = 1;
    return;
  }
  if (cameraMode !== "cockpit") return;
  ctx.save();
  ctx.fillStyle = "rgba(2,5,5,0.64)";
  ctx.fillRect(0, 0, w, h * 0.1);
  const dash = ctx.createLinearGradient(0, h * 0.74, 0, h);
  dash.addColorStop(0, "rgba(6,10,10,0.52)");
  dash.addColorStop(0.42, "#0a0f0e");
  dash.addColorStop(1, "#020403");
  ctx.fillStyle = dash;
  ctx.fillRect(0, h * 0.75, w, h * 0.25);
  ctx.fillStyle = "#080d0c";
  ctx.beginPath();
  ctx.moveTo(0, h);
  ctx.lineTo(w * 0.2, h * 0.78);
  ctx.lineTo(w * 0.8, h * 0.78);
  ctx.lineTo(w, h);
  ctx.closePath();
  ctx.fill();
  ctx.strokeStyle = "rgba(244,251,248,0.28)";
  ctx.lineWidth = 10;
```

```
  roundRect(w * 0.08, h * 0.12, w * 0.84, h * 0.58, 18);
  ctx.stroke();
  ctx.strokeStyle = "rgba(244,251,248,0.16)";
  ctx.lineWidth = 2;
  ctx.beginPath();
  ctx.moveTo(w * 0.08, h * 0.68);
  ctx.quadraticCurveTo(w * 0.5, h * 0.76, w * 0.92, h * 0.68);
  ctx.stroke();
  ctx.strokeStyle = theme[1];
  ctx.lineWidth = 3;
  ctx.globalAlpha = 0.65;
  ctx.beginPath();
  ctx.moveTo(w * 0.5, h * 0.14);
  ctx.lineTo(w * 0.5 + raceState.lane * 22, h * 0.68);
  ctx.stroke();
  ctx.globalAlpha = 1;
  ctx.fillStyle = "#111817";
  ctx.beginPath();
  ctx.arc(w * 0.5, h * 0.9, Math.min(w, h) * 0.13, Math.PI, 0);
  ctx.fill();
  ctx.strokeStyle = "#46d9ff";
  ctx.lineWidth = 8;
  ctx.beginPath();
  ctx.arc(w * 0.5, h * 0.9, Math.min(w, h) * 0.1, Math.PI * 1.06, Math.PI * 1.94);
  ctx.stroke();
  ctx.fillStyle = "#bbf24a";
  ctx.font = "900 18px system-ui";
  ctx.textAlign = "center";
  ctx.fillText(`${Math.round(raceState.speed)} MPH`, w * 0.5, h * 0.9);
  ctx.fillStyle = "rgba(5,8,7,0.74)";
  roundRect(w * 0.11, h * 0.8, w * 0.22, h * 0.1, 8);
  ctx.fill();
  ctx.fillStyle = raceState.chaseActive ? "#ff5b6b" : "#a9bbb5";
  ctx.font = "900 14px system-ui";
  ctx.textAlign = "left";
  ctx.fillText(raceState.chaseActive ? "PURSUIT ACTIVE" : "STREET CLEAR", w * 0.13, h * 0.835);
  ctx.fillStyle = "#f4fbf8";
  ctx.font = "800 12px system-ui";
  ctx.fillText(`HEAT ${Math.round(raceState.heat)}%`, w * 0.13, h * 0.865);
  ctx.fillStyle = "rgba(5,8,7,0.74)";
  roundRect(w * 0.67, h * 0.8, w * 0.22, h * 0.1, 8);
  ctx.fill();
  ctx.fillStyle = theme[1];
  ctx.font = "900 14px system-ui";
  ctx.textAlign = "right";
  ctx.fillText(selectedRace ? selectedRace.mood.toUpperCase() : "STREET RACE", w * 0.87, h * 0.835);
  ctx.fillStyle = "#f4fbf8";
  ctx.font = "800 12px system-ui";
  ctx.fillText(`${Math.round((raceState.distance / raceLength()) * 100)}% ROUTE`, w * 0.87, h * 0.865);
  if (raceState.chaseActive) {
    ctx.globalAlpha = 0.18 + Math.max(0, Math.sin(raceState.elapsed * 16)) * 0.14;
    ctx.fillStyle = "#ff3348";
    ctx.fillRect(0, 0, w * 0.5, h);
    ctx.fillStyle = "#46d9ff";
    ctx.fillRect(w * 0.5, 0, w * 0.5, h);
    ctx.globalAlpha = 1;
  }
  ctx.restore();
}

function drawDamageOverlay(w, h, theme) {
  if (!raceState.active) return;
  const damage = Math.max(0, raceState.damage || 0);
  const resetting = raceState.resetTimer > 0;
  const damageAlert = (raceState.damageAlertTimer || 0) > 0;
  const hazardWarning = (raceState.hazardWarningTimer || 0) > 0;
  if (damage < 10 && !resetting && !damageAlert && !hazardWarning) return;
  ctx.save();
  const danger = Math.min(0.34, damage / 280 + (resetting ? 0.18 : 0));
  const edge = ctx.createRadialGradient(w * 0.5, h * 0.52, h * 0.28, w * 0.5, h * 0.52, h * 0.9);
  edge.addColorStop(0, "rgba(0,0,0,0)");
  edge.addColorStop(0.68, "rgba(0,0,0,0)");
  edge.addColorStop(1, `rgba(255,51,72,${danger})`);
  ctx.fillStyle = edge;
  ctx.fillRect(0, 0, w, h);

  if (hazardWarning || damageAlert) {
    const label = hazardWarning ? raceState.hazardWarningLabel : raceState.damageAlertLabel;
    const panelW = Math.min(phoneGraphicsActive() ? 330 : 420, w - 28);
    const panelH = phoneGraphicsActive() ? 34 : 40;
    const x = (w - panelW) / 2;
    const y = phoneGraphicsActive() ? h * 0.2 : h * 0.16;
    ctx.globalAlpha = hazardWarning ? 0.9 : 0.82;
    ctx.fillStyle = hazardWarning ? "rgba(255,209,102,0.16)" : "rgba(255,91,107,0.18)";
```

```
    roundRect(x, y, panelW, panelH, 7);
    ctx.fill();
    ctx.strokeStyle = hazardWarning ? "rgba(255,209,102,0.82)" : "rgba(255,91,107,0.84)";
    ctx.lineWidth = 2;
    ctx.stroke();
    ctx.globalAlpha = 1;
    ctx.fillStyle = "#f4fbf8";
    ctx.font = `900 ${phoneGraphicsActive() ? 13 : 15}px system-ui`;
    ctx.textAlign = "center";
    ctx.fillText(label || "WATCH TRAFFIC", w / 2, y + panelH * 0.64);
  }

  if (cameraMode === "cockpit" || damage > 48) {
    ctx.strokeStyle = `rgba(244,251,248,${Math.min(0.28, damage / 240)})`;
    ctx.lineWidth = Math.max(1, w * 0.002);
    const cracks = Math.floor(Math.min(8, damage / 12));
    for (let i = 0; i < cracks; i += 1) {
      const anchorX = w * (0.2 + ((i * 0.17) % 0.6));
      const anchorY = h * (0.18 + ((i * 0.13) % 0.38));
      ctx.beginPath();
      ctx.moveTo(anchorX, anchorY);
      ctx.lineTo(anchorX + Math.sin(i * 1.9) * w * 0.08, anchorY + h * (0.05 + (i % 3) * 0.025));
      ctx.lineTo(anchorX + Math.cos(i * 1.4) * w * 0.1, anchorY + h * (0.1 + (i % 2) * 0.03));
      ctx.stroke();
    }
  }

  if (resetting) {
    ctx.fillStyle = "rgba(5,8,7,0.78)";
    roundRect(w * 0.5 - 132, h * 0.36, 264, 78, 8);
    ctx.fill();
    ctx.strokeStyle = theme[2];
    ctx.lineWidth = 2;
    ctx.stroke();
    ctx.fillStyle = "#f4fbf8";
    ctx.font = "900 20px system-ui";
    ctx.textAlign = "center";
    ctx.fillText(`RESET ${Math.ceil(raceState.resetTimer)}`, w * 0.5, h * 0.36 + 32);
    ctx.fillStyle = "#a9bbb5";
    ctx.font = "800 12px system-ui";
    ctx.fillText(raceState.resetReason || "Vehicle recovery", w * 0.5, h * 0.36 + 55);
  }
  ctx.restore();
}

function drawRaceGoalIntro(w, h, theme) {
  if (!raceState.active || raceState.goalIntroTimer <= 0) return;
  const phoneMode = phoneGraphicsActive();
  const total = 5.8;
  const fadeIn = Math.min(1, (total - raceState.goalIntroTimer) / 0.45);
  const fadeOut = Math.min(1, raceState.goalIntroTimer / 0.85);
  const alpha = Math.max(0, Math.min(1, fadeIn, fadeOut));
  if (alpha <= 0) return;
  const route = routeWorldInfo(selectedRace && selectedRace.place ? selectedRace.place : "city");
  const panelW = Math.min(phoneMode ? 390 : 560, w - 28);
  const panelH = phoneMode ? 92 : 108;
  const x = (w - panelW) / 2;
  const y = Math.max(phoneMode ? 48 : 72, h * (phoneMode ? 0.15 : 0.18));
  ctx.save();
  ctx.globalAlpha = alpha;
  ctx.fillStyle = "rgba(5,8,7,0.8)";
  roundRect(x, y, panelW, panelH, 8);
  ctx.fill();
  ctx.strokeStyle = `${theme[1]}aa`;
  ctx.lineWidth = 2;
  ctx.stroke();
  ctx.fillStyle = theme[1];
  ctx.font = `900 ${phoneMode ? 12 : 14}px system-ui`;
  ctx.textAlign = "center";
  ctx.fillText("RACE GOAL", w / 2, y + (phoneMode ? 22 : 26));
  ctx.fillStyle = "#f4fbf8";
  ctx.font = `900 ${phoneMode ? 18 : 24}px system-ui`;
  const goal = selectedRace ? selectedRace.target : "Finish the race";
  ctx.fillText(goal, w / 2, y + (phoneMode ? 48 : 58));
  ctx.fillStyle = "#a9bbb5";
  ctx.font = `800 ${phoneMode ? 11 : 13}px system-ui`;
  ctx.fillText(`${route.country}: ${route.scene} | ${activeScenario().name} | Hideouts + shortcut branches`, w / 2, y
+ (phoneMode ? 72 : 84));
  ctx.restore();
}

function drawCinematicGrade(w, h, theme) {
  const speedLines = Math.max(0, (raceState.speed - 190) / 150);
  if (speedLines > 0) {
```

```
    ctx.save();
    ctx.globalAlpha = Math.min(0.16, speedLines * 0.14);
    const edgeRush = ctx.createLinearGradient(0, h * 0.42, 0, h);
    edgeRush.addColorStop(0, "rgba(255,255,255,0)");
    edgeRush.addColorStop(0.7, "rgba(244,251,248,0.13)");
    edgeRush.addColorStop(1, "rgba(255,255,255,0)");
    ctx.fillStyle = edgeRush;
    ctx.fillRect(0, h * 0.5, w, h * 0.5);
    ctx.restore();
  }
  const slipGlow = Math.min(0.24, (raceState.slip || 0) * 0.18 + (raceState.brakeHeat || 0) * 0.08);
  if (slipGlow > 0.01 && raceState.active) {
    ctx.save();
    const gripWash = ctx.createLinearGradient(0, h * 0.55, 0, h);
    gripWash.addColorStop(0, "rgba(255,255,255,0)");
    gripWash.addColorStop(1, `rgba(244,251,248,${slipGlow})`);
    ctx.fillStyle = gripWash;
    ctx.fillRect(0, h * 0.55, w, h * 0.45);
    ctx.restore();
  }
  const vignette = ctx.createRadialGradient(w * 0.5, h * 0.52, h * 0.22, w * 0.5, h * 0.52, h * 0.9);
  vignette.addColorStop(0, "rgba(0,0,0,0)");
  vignette.addColorStop(1, "rgba(0,0,0,0.58)");
  ctx.fillStyle = vignette;
  ctx.fillRect(0, 0, w, h);
}

function drawRealisticDrivingPass(w, h, theme) {
  const place = selectedRace && selectedRace.place ? selectedRace.place : "city";
  const speed = Math.max(0, raceState.speed);
  const horizon = cameraMode === "cockpit" ? h * 0.28 : cameraMode === "hood" ? h * 0.31 : h * 0.34;
  const webglActive = useWebGLRenderer();
  ctx.save();

  const depthFog = ctx.createLinearGradient(0, horizon - h * 0.08, 0, horizon + h * 0.18);
  depthFog.addColorStop(0, "rgba(244,251,248,0.02)");
  depthFog.addColorStop(0.52, place === "desert" || place === "canyon" ? "rgba(205,178,142,0.08)" :
"rgba(210,230,235,0.08)");
  depthFog.addColorStop(1, "rgba(5,8,7,0)");
  ctx.fillStyle = depthFog;
  ctx.fillRect(0, Math.max(0, horizon - h * 0.12), w, h * 0.36);
  if (webglActive) {
    ctx.restore();
    return;
  }
  if (phoneCleanRoadActive()) {
    ctx.restore();
    return;
  }

  const wetPlaces = ["city", "tokyo", "coast", "harbor", "rainforest"];
  if (wetPlaces.includes(place)) {
    ctx.globalAlpha = place === "tokyo" ? 0.32 : 0.2;
    for (let i = 0; i < 16; i += 1) {
      const y = ((i * 47 + raceState.roadOffset * 0.72) % (h * 0.62)) + horizon;
      const t = Math.min(1, Math.max(0, (y - horizon) / (h - horizon)));
      const ribbonW = w * (0.12 + t * 0.5);
      const x = w * 0.5 + Math.sin(i * 1.7 + raceState.elapsed) * w * 0.04;
      const shine = ctx.createLinearGradient(x - ribbonW, y, x + ribbonW, y);
      shine.addColorStop(0, "rgba(244,251,248,0)");
      shine.addColorStop(0.5, "rgba(244,251,248,0.26)");
      shine.addColorStop(1, "rgba(244,251,248,0)");
      ctx.strokeStyle = shine;
      ctx.lineWidth = 1 + t * 5;
      ctx.beginPath();
      ctx.moveTo(x - ribbonW, y);
      ctx.quadraticCurveTo(x, y + 5 + t * 12, x + ribbonW, y + 2);
      ctx.stroke();
    }
    ctx.globalAlpha = 1;
  }

  if (raceState.active && speed > 35) {
    const carX = w / 2 + raceState.lane * laneWidth();
    const baseY = cameraMode === "cockpit" ? h * 0.76 : cameraMode === "hood" ? h * 0.9 : h * 0.86;
    const slipAlpha = Math.min(0.32, 0.05 + (raceState.slip || 0) * 0.22);
    ctx.strokeStyle = `rgba(8,10,10,${slipAlpha})`;
    ctx.lineWidth = Math.max(2, w * 0.006);
    for (let side = -1; side <= 1; side += 2) {
      const tireX = carX + side * laneWidth() * 0.22;
      ctx.beginPath();
      ctx.moveTo(tireX, baseY);
      ctx.bezierCurveTo(
        tireX - raceState.lateralVelocity * 15,
```

```
        baseY + h * 0.06,
        tireX - raceState.steerAngle * 24,
        h,
        tireX - raceState.steerAngle * 48,
        h + 20
      );
      ctx.stroke();
    }
  }

  if (speed > 105) {
    const blur = Math.min(0.18, (speed - 105) / 900);
    ctx.globalAlpha = blur;
    ctx.strokeStyle = "rgba(244,251,248,0.7)";
    ctx.lineWidth = 2;
    for (let i = 0; i < 34; i += 1) {
      const side = i % 2 ? -1 : 1;
      const y = h * (0.36 + (i % 17) * 0.04);
      const x0 = w * 0.5 + side * w * (0.18 + (i % 5) * 0.05);
      ctx.beginPath();
      ctx.moveTo(x0, y);
      ctx.lineTo(x0 + side * w * 0.16, y + h * 0.018);
      ctx.stroke();
    }
    ctx.globalAlpha = 1;
  }

  if (["city", "tokyo", "airfield", "freight"].includes(place) || cameraMode === "cockpit") {
    const cone = ctx.createRadialGradient(w * 0.5, h * 0.78, w * 0.04, w * 0.5, h * 0.7, w * 0.42);
    cone.addColorStop(0, "rgba(255,246,202,0.16)");
    cone.addColorStop(1, "rgba(255,246,202,0)");
    ctx.fillStyle = cone;
    ctx.fillRect(0, horizon, w, h - horizon);
  }

  if (place === "desert" || place === "canyon" || place === "freight") {
    ctx.globalAlpha = Math.min(0.28, 0.08 + speed / 980);
    ctx.fillStyle = "rgba(255,183,74,0.22)";
    for (let i = 0; i < 26; i += 1) {
      const x = (i * 79 + raceState.roadOffset * 0.11) % w;
      const y = h * 0.38 + ((i * 31 + raceState.elapsed * 55) % (h * 0.5));
      ctx.beginPath();
      ctx.ellipse(x, y, 18 + (i % 4) * 8, 3 + (i % 3), 0.15, 0, Math.PI * 2);
      ctx.fill();
    }
    ctx.globalAlpha = 1;
  }

  if (place === "snow") {
    ctx.globalAlpha = 0.34;
    ctx.fillStyle = "rgba(244,251,248,0.75)";
    for (let i = 0; i < 46; i += 1) {
      const x = (i * 61 + raceState.elapsed * 34) % w;
      const y = (i * 43 + raceState.elapsed * 52 + raceState.roadOffset * 0.05) % h;
      ctx.fillRect(x, y, 2, 2);
    }
    ctx.globalAlpha = 1;
  }

  if (place === "coast" || place === "desert") {
    const sunX = place === "coast" ? w * 0.18 : w * 0.78;
    const sunY = h * 0.16;
    const flare = ctx.createRadialGradient(sunX, sunY, 8, sunX, sunY, w * 0.28);
    flare.addColorStop(0, "rgba(255,226,150,0.35)");
    flare.addColorStop(1, "rgba(255,226,150,0)");
    ctx.fillStyle = flare;
    ctx.fillRect(0, 0, w, h * 0.48);
  }

  ctx.restore();
}

function drawVehicle(x, y, width, height, color, player, police = false, vehicleType = "car") {
  ctx.save();
  ctx.translate(x, y);
  drawRoadContactShadow(width, height, vehicleType, 1);

  if (drawPhoneAssetVehicleSprite(width, height, color, vehicleType, police, player ? raceState.damage || 0 : 0,
true)) {
    if (player) {
      ctx.save();
      ctx.translate(0, -height * 0.18);
      drawLicensePlate(width * 0.78, height * 0.78, driverPlate(activeProfile));
      ctx.restore();
```

```
  }
  ctx.restore();
  return;
}

ctx.translate(0, -height * 0.58);

if (!police && vehicleType !== "car") {
  drawSpecialVehicleSilhouette(width, height, color, vehicleType, player);
  if (player) drawLicensePlate(width * 0.82, height * 0.82, driverPlate(activeProfile));
  ctx.restore();
  return;
}

const tireW = width * 0.17;
const tireH = height * 0.3;
ctx.fillStyle = "#050807";
roundRect(-width * 0.56, -height * 0.29, tireW, tireH, 5);
ctx.fill();
roundRect(width * 0.39, -height * 0.29, tireW, tireH, 5);
ctx.fill();
roundRect(-width * 0.56, height * 0.14, tireW, tireH, 5);
ctx.fill();
roundRect(width * 0.39, height * 0.14, tireW, tireH, 5);
ctx.fill();
drawWheelRim(-width * 0.475, -height * 0.14, width * 0.07, height * 0.12);
drawWheelRim(width * 0.475, -height * 0.14, width * 0.07, height * 0.12);
drawWheelRim(-width * 0.475, height * 0.26, width * 0.07, height * 0.12, "rgba(255,209,102,0.24)");
drawWheelRim(width * 0.475, height * 0.26, width * 0.07, height * 0.12, "rgba(255,209,102,0.24)");

const body = ctx.createLinearGradient(-width * 0.48, -height * 0.52, width * 0.48, height * 0.52);
body.addColorStop(0, shade(color, 42));
body.addColorStop(0.2, color);
body.addColorStop(0.54, shade(color, -18));
body.addColorStop(1, shade(color, -42));
ctx.fillStyle = police ? "#f0f2f0" : body;
ctx.beginPath();
ctx.moveTo(-width * 0.26, -height * 0.54);
ctx.lineTo(width * 0.26, -height * 0.54);
ctx.quadraticCurveTo(width * 0.51, -height * 0.48, width * 0.5, -height * 0.2);
ctx.lineTo(width * 0.45, height * 0.38);
ctx.quadraticCurveTo(width * 0.28, height * 0.57, 0, height * 0.58);
ctx.quadraticCurveTo(-width * 0.28, height * 0.57, -width * 0.45, height * 0.38);
ctx.lineTo(-width * 0.5, -height * 0.2);
ctx.quadraticCurveTo(-width * 0.51, -height * 0.48, -width * 0.26, -height * 0.54);
ctx.closePath();
ctx.fill();
ctx.strokeStyle = "rgba(255,255,255,0.28)";
ctx.lineWidth = Math.max(1, width * 0.018);
ctx.stroke();
drawVehicleTrimDetails(width, height, color, police);
if (player) drawLicensePlate(width, height, driverPlate(activeProfile));

ctx.fillStyle = police ? "#111817" : shade(color, -36);
ctx.globalAlpha = 0.64;
ctx.beginPath();
ctx.moveTo(-width * 0.38, -height * 0.24);
ctx.lineTo(-width * 0.22, -height * 0.48);
ctx.lineTo(width * 0.22, -height * 0.48);
ctx.lineTo(width * 0.38, -height * 0.24);
ctx.closePath();
ctx.fill();
ctx.globalAlpha = 1;

if (police) {
  ctx.fillStyle = "#111817";
  ctx.fillRect(-width * 0.34, -height * 0.16, width * 0.68, height * 0.11);
  ctx.fillStyle = "#f4fbf8";
  ctx.font = `${Math.max(8, width * 0.14)}px system-ui`;
  ctx.textAlign = "center";
  ctx.fillText("POLICE", 0, height * 0.24);
}

const glass = ctx.createLinearGradient(0, -height * 0.42, 0, height * 0.24);
glass.addColorStop(0, "rgba(220,244,255,0.58)");
glass.addColorStop(0.45, "rgba(43,66,70,0.84)");
glass.addColorStop(1, "rgba(7,11,12,0.92)");
ctx.fillStyle = glass;
roundRect(-width * 0.31, -height * 0.39, width * 0.62, height * 0.21, 8);
ctx.fill();
roundRect(-width * 0.29, -height * 0.11, width * 0.58, height * 0.32, 10);
ctx.fill();
ctx.strokeStyle = "rgba(244,251,248,0.28)";
ctx.lineWidth = Math.max(1, width * 0.015);
```

```
  ctx.stroke();
  ctx.fillStyle = player ? "#dce8ef" : "#e6d69b";
  ctx.beginPath();
  ctx.arc(0, -height * 0.02, width * 0.12, 0, Math.PI * 2);
  ctx.fill();
  ctx.fillStyle = "#050807";
  roundRect(-width * 0.09, -height * 0.04, width * 0.18, height * 0.045, 2);
  ctx.fill();
  ctx.strokeStyle = "rgba(255,255,255,0.18)";
  ctx.beginPath();
  ctx.arc(0, -height * 0.02, width * 0.12, Math.PI * 1.04, Math.PI * 1.96);
  ctx.stroke();

  ctx.fillStyle = "rgba(255,248,214,0.94)";
  ctx.beginPath();
  ctx.moveTo(-width * 0.36, -height * 0.49);
  ctx.lineTo(-width * 0.11, -height * 0.45);
  ctx.lineTo(-width * 0.24, -height * 0.36);
  ctx.closePath();
  ctx.moveTo(width * 0.36, -height * 0.49);
  ctx.lineTo(width * 0.11, -height * 0.45);
  ctx.lineTo(width * 0.24, -height * 0.36);
  ctx.closePath();
  ctx.fill();
  ctx.globalAlpha = 0.18;
  ctx.fillStyle = "#fff8d6";
  ctx.beginPath();
  ctx.ellipse(-width * 0.25, -height * 0.42, width * 0.28, height * 0.1, -0.28, 0, Math.PI * 2);
  ctx.ellipse(width * 0.25, -height * 0.42, width * 0.28, height * 0.1, 0.28, 0, Math.PI * 2);
  ctx.fill();
  ctx.globalAlpha = 1;

  ctx.fillStyle = "#ff3348";
  roundRect(-width * 0.32, height * 0.42, width * 0.17, height * 0.055, 3);
  ctx.fill();
  roundRect(width * 0.15, height * 0.42, width * 0.17, height * 0.055, 3);
  ctx.fill();
  if (police) {
    const flash = Math.sin(raceState.elapsed * 18) > 0;
    ctx.fillStyle = flash ? "#ff3348" : "#46d9ff";
    roundRect(-width * 0.21, -height * 0.25, width * 0.42, height * 0.055, 3);
    ctx.fill();
    ctx.globalAlpha = 0.35;
    ctx.fillStyle = flash ? "#ff3348" : "#46d9ff";
    ctx.beginPath();
    ctx.arc(0, -height * 0.25, width * 0.45, 0, Math.PI * 2);
    ctx.fill();
    ctx.globalAlpha = 1;
  }
  ctx.strokeStyle = "rgba(5,8,7,0.5)";
  ctx.lineWidth = Math.max(1.5, width * 0.02);
  ctx.beginPath();
  ctx.moveTo(0, -height * 0.51);
  ctx.lineTo(0, height * 0.5);
  ctx.moveTo(-width * 0.41, -height * 0.16);
  ctx.lineTo(width * 0.41, -height * 0.16);
  ctx.moveTo(-width * 0.37, height * 0.28);
  ctx.lineTo(width * 0.37, height * 0.28);
  ctx.stroke();
  ctx.fillStyle = "rgba(244,251,248,0.22)";
  ctx.fillRect(-width * 0.34, height * 0.19, width * 0.68, height * 0.026);

  ctx.strokeStyle = "rgba(255,255,255,0.22)";
  ctx.lineWidth = Math.max(1, width * 0.012);
  ctx.beginPath();
  ctx.moveTo(-width * 0.43, -height * 0.13);
  ctx.quadraticCurveTo(-width * 0.56, height * 0.02, -width * 0.42, height * 0.33);
  ctx.moveTo(width * 0.43, -height * 0.13);
  ctx.quadraticCurveTo(width * 0.56, height * 0.02, width * 0.42, height * 0.33);
  ctx.stroke();
  if (player) drawVehicleDamageMarks(width, height, raceState.damage || 0);
  ctx.restore();
}

function shade(hex, amount) {
  const raw = hex.replace("#", "");
  const num = parseInt(raw.length === 3 ? raw.split("").map((c) => c + c).join("") : raw, 16);
  const r = Math.max(0, Math.min(255, (num >> 16) + amount));
  const g = Math.max(0, Math.min(255, ((num >> 8) & 255) + amount));
  const b = Math.max(0, Math.min(255, (num & 255) + amount));
  return `rgb(${r}, ${g}, ${b})`;
}

function roundRect(x, y, width, height, radius) {
```

```
  ctx.beginPath();
  ctx.moveTo(x + radius, y);
  ctx.lineTo(x + width - radius, y);
  ctx.quadraticCurveTo(x + width, y, x + width, y + radius);
  ctx.lineTo(x + width, y + height - radius);
  ctx.quadraticCurveTo(x + width, y + height, x + width - radius, y + height);
  ctx.lineTo(x + radius, y + height);
  ctx.quadraticCurveTo(x, y + height, x, y + height - radius);
  ctx.lineTo(x, y + radius);
  ctx.quadraticCurveTo(x, y, x + radius, y);
  ctx.closePath();
}

function drawParticles() {
  raceState.particles.forEach((p) => {
    const fade = Math.max(0, p.life / (p.maxLife || 0.7));
    ctx.globalAlpha = fade;
    ctx.fillStyle = p.color;
    ctx.beginPath();
    ctx.arc(p.x, p.y, (p.radius || 4) * (1.1 - fade * 0.35), 0, Math.PI * 2);
    ctx.fill();
    ctx.globalAlpha = 1;
  });
}

function drawAttract(w, h) {
  ctx.save();
  ctx.textAlign = "center";
  ctx.fillStyle = "rgba(5,8,7,0.38)";
  roundRect(w / 2 - 330, h * 0.36 - 54, 660, 118, 8);
  ctx.fill();
  ctx.fillStyle = "#f4fbf8";
  ctx.font = "900 34px system-ui";
  ctx.fillText("Press Play. Hit The Street.", w / 2, h * 0.36);
  ctx.fillStyle = "#a9bbb5";
  ctx.font = "700 16px system-ui";
  ctx.fillText("Performance cars, police heat, and cinematic routes built for replay.", w / 2, h * 0.36 + 34);
  ctx.restore();
}

function drawPause(w, h) {
  ctx.fillStyle = "rgba(0,0,0,0.56)";
  ctx.fillRect(0, 0, w, h);
  ctx.fillStyle = "#f4fbf8";
  ctx.textAlign = "center";
  ctx.font = "900 48px system-ui";
  ctx.fillText("Paused", w / 2, h / 2);
}

function syncViewportSize() {
  const viewport = window.visualViewport;
  const width = Math.max(320, Math.floor(viewport ? viewport.width : window.innerWidth));
  const height = Math.max(240, Math.floor(viewport ? viewport.height : window.innerHeight));
  document.documentElement.style.setProperty("--vvw", `${width}px`);
  document.documentElement.style.setProperty("--vvh", `${height}px`);
}

function fitCanvas() {
  syncViewportSize();
  const rect = canvas.getBoundingClientRect();
  const racingViewport = document.body.classList.contains("race-live");
  const minWidth = racingViewport ? 320 : 640;
  const minHeight = racingViewport ? 240 : 420;
  const width = Math.max(minWidth, Math.floor(rect.width));
  const height = Math.max(minHeight, Math.floor(rect.height));
  canvas.width = width;
  canvas.height = height;
  if (glCanvas) {
    glCanvas.width = width;
    glCanvas.height = height;
  }
  if (webglRenderer) webglRenderer.resize(width, height);
}

function bindEvents() {
  $$(".age-card").forEach((btn) => {
    btn.addEventListener("click", () => {
      selectedAge = btn.dataset.age;
      $$(".age-card").forEach((item) => item.classList.toggle("selected", item === btn));
    });
  });
  $("#quickPlay").addEventListener("click", () => enterProfile(true));
  $("#startProfile").addEventListener("click", () => enterProfile(false));
  $("#installApp").addEventListener("click", installApp);
```

```
$("#resetProfiles").addEventListener("click", () => {
  if (!confirm("Clear all local Velocity Vault profiles on this device?")) return;
  profiles = {};
  saveProfiles();
  clearSavedRace();
  renderSavedRaceButton();
  showToast("Local profiles cleared.");
});
$("#logoutBtn").addEventListener("click", () => {
  activeProfile = null;
  raceState.active = false;
  $("#modeChip").textContent = "Select Profile";
  $("#missionChip").textContent = "Mission Ready";
  updateHud();
  showView("login");
});
$$(".tab").forEach((btn) => {
  btn.addEventListener("click", () => {
    tab = btn.dataset.tab;
    $$(".tab").forEach((item) => item.classList.toggle("active", item === btn));
    $$(".tab-panel").forEach((panel) => panel.classList.toggle("active", panel.id === `${tab}Tab`));
  });
});
$("#launchRace").addEventListener("click", launchRace);
$("#resumeRace").addEventListener("click", resumeSavedRace);
$("#saveRace").addEventListener("click", saveRace);
$("#resetCar").addEventListener("click", manualResetVehicle);
$("#endRace").addEventListener("click", () => endRace(true));
$("#toGarage").addEventListener("click", () => {
  showView("garage");
  renderHub();
});
$("#nextRace").addEventListener("click", () => {
  const idx = races.findIndex((race) => race.id === selectedRace.id);
  selectedRace = races[Math.min(races.length - 1, idx + 1)];
  showView("garage");
  renderHub();
});
$("#pauseBtn").addEventListener("click", () => {
  input.paused = !input.paused;
  $("#pauseBtn").textContent = input.paused ? "Play" : "Pause";
});
$("#refreshDirector").addEventListener("click", () => {
  renderDirector();
  renderRaces();
  renderUpgrades();
  showToast("AI tuning refreshed for this driver.");
});
$$(".camera-btn").forEach((btn) => {
  btn.addEventListener("click", () => setCameraMode(btn.dataset.camera));
});
$$(".renderer-btn").forEach((btn) => {
  btn.addEventListener("click", () => setRendererMode(btn.dataset.renderer));
});
bindHold($("#leftBtn"), "left");
bindHold($("#rightBtn"), "right");
bindHold($("#gasBtn"), "gas");
bindHold($("#brakeBtn"), "brake");
bindHold($("#boostBtn"), "boost");
$$(".mobile-control").forEach((button) => bindMobileControl(button));
bindDriveStickPointerControl();
bindFloatingPhoneJoystick();
const mobileLayer = $(".mobile-drive-controls");
mobileLayer.addEventListener("touchstart", handleMobileTouchStart, { passive: false });
mobileLayer.addEventListener("touchmove", handleMobileTouchMove, { passive: false });
mobileLayer.addEventListener("touchend", handleMobileTouchEnd, { passive: false });
mobileLayer.addEventListener("touchcancel", handleMobileTouchEnd, { passive: false });
window.addEventListener("keydown", (event) => setKey(event, true));
window.addEventListener("keyup", (event) => setKey(event, false));
const resizeGame = () => {
  syncViewportSize();
  fitCanvas();
};
window.addEventListener("resize", resizeGame);
window.addEventListener("orientationchange", () => setTimeout(resizeGame, 180));
if (window.visualViewport) {
  window.visualViewport.addEventListener("resize", resizeGame);
  window.visualViewport.addEventListener("scroll", resizeGame);
}
window.addEventListener("gamepadconnected", (event) => setController(event.gamepad));
window.addEventListener("gamepaddisconnected", () => clearController());
window.addEventListener("beforeinstallprompt", (event) => {
  event.preventDefault();
  deferredInstallPrompt = event;
```

```
    showToast("App install is available.");
  });
  window.addEventListener("appinstalled", () => {
    deferredInstallPrompt = null;
    showToast("Velocity Vault installed.");
  });
}

function bindHold(button, key) {
  const start = (event) => {
    event.preventDefault();
    input[key] = true;
    if (button.setPointerCapture && event.pointerId !== undefined) {
      try {
        button.setPointerCapture(event.pointerId);
      } catch {
        // Some browsers reject capture after synthetic pointer events.
      }
    }
  };
  const stop = (event) => {
    if (event) event.preventDefault();
    input[key] = false;
    if (button.releasePointerCapture && event && event.pointerId !== undefined) {
      try {
        button.releasePointerCapture(event.pointerId);
      } catch {
        // Capture may already be released by the browser.
      }
    }
  };
  button.addEventListener("pointerdown", start);
  button.addEventListener("pointerup", stop);
  button.addEventListener("pointercancel", stop);
  button.addEventListener("lostpointercapture", stop);
  button.addEventListener("contextmenu", (event) => event.preventDefault());
}

function bindMobileControl(button) {
  const stickButton = button.closest && button.closest(".drive-stick") ? button.closest(".drive-stick") : null;
  if (stickButton) {
    bindDriveStickButtonFallback(button, stickButton);
    return;
  }
  const control = button.dataset.control;
  const start = (event) => {
    if (event.pointerType === "touch" && mobileTouchState.usingTouchEvents) return;
    event.preventDefault();
    startAudio();
    if (audioSystem && audioSystem.ctx.state === "suspended") audioSystem.ctx.resume();
    if (control === "size") {
      setTouchControlSize(touchControlSize === "mini" ? "full" : "mini");
      return;
    }
    if (control === "mode") {
      setTouchDriveMode(touchDriveMode === "toggle" ? "hold" : "toggle");
      return;
    }
    if (touchDriveMode === "toggle") {
      toggleDriveInput(control);
      updateRaceUi();
      return;
    }
    setDriveInput(control, true);
    updateRaceUi();
    if (button.setPointerCapture && event.pointerId !== undefined) {
      try {
        button.setPointerCapture(event.pointerId);
      } catch {
        // Some browsers reject capture after synthetic pointer events.
      }
    }
  };
  const stop = (event) => {
    if (event && event.pointerType === "touch" && mobileTouchState.usingTouchEvents) return;
    if (event) event.preventDefault();
    if (touchDriveMode === "toggle" || control === "mode" || control === "size") return;
    setDriveInput(control, false);
    updateRaceUi();
    if (button.releasePointerCapture && event && event.pointerId !== undefined) {
      try {
        button.releasePointerCapture(event.pointerId);
      } catch {
        // Capture may already be released by the browser.
```

```
      }
    }
  };
  button.addEventListener("pointerdown", start);
  button.addEventListener("pointerup", stop);
  button.addEventListener("pointercancel", stop);
  button.addEventListener("lostpointercapture", stop);
  button.addEventListener("contextmenu", (event) => event.preventDefault());
}

function directStickControl(control) {
  if (control === "gas") return "stick:gas";
  if (control === "brake") return "stick:brake";
  if (control === "left") return "stick:left:steer=-1";
  if (control === "right") return "stick:right:steer=1";
  return "stick:neutral";
}

function bindDriveStickButtonFallback(button, stick) {
  const pointerKey = `drive-stick-button-${button.dataset.control || "neutral"}`;
  const readControl = (event) => driveStickControlAt(stick, event.clientX, event.clientY) ||
directStickControl(button.dataset.control);
  const apply = (event) => {
    if (event.pointerType === "touch" && mobileTouchState.usingTouchEvents) return;
    if (useFloatingStickControls()) return;
    if (touchControlSize !== "mini") return;
    event.preventDefault();
    startAudio();
    if (audioSystem && audioSystem.ctx.state === "suspended") audioSystem.ctx.resume();
    const control = readControl(event);
    if (touchDriveMode === "toggle") mobileTouchState.latchedStickControl = control.includes("neutral") ? "" :
control;
    mobileTouchState.active.set(pointerKey, control);
    applyMobileTouchSnapshot();
    if (button.setPointerCapture && event.pointerId !== undefined) {
      try {
        button.setPointerCapture(event.pointerId);
      } catch {
        // Some browsers reject capture after synthetic pointer events.
      }
    }
  };
  const stop = (event) => {
    if (event && event.pointerType === "touch" && mobileTouchState.usingTouchEvents) return;
    if (event) event.preventDefault();
    if (touchDriveMode !== "toggle") mobileTouchState.active.delete(pointerKey);
    applyMobileTouchSnapshot();
    if (button.releasePointerCapture && event && event.pointerId !== undefined) {
      try {
        button.releasePointerCapture(event.pointerId);
      } catch {
        // Capture may already be released by the browser.
      }
    }
  };
  button.addEventListener("pointerdown", apply);
  button.addEventListener("pointermove", apply);
  button.addEventListener("pointerup", stop);
  button.addEventListener("pointercancel", stop);
  button.addEventListener("lostpointercapture", stop);
  button.addEventListener("contextmenu", (event) => event.preventDefault());
}

function setController(gamepad) {
  controllerState.connected = true;
  controllerState.name = gamepad.id || "Gamepad";
  controllerState.index = gamepad.index;
  $("#controllerChip").textContent = "Controller: On";
  $("#controllerName").textContent = controllerState.name;
  showToast("Controller connected.");
}

function clearController() {
  controllerState.connected = false;
  controllerState.name = "";
  controllerState.index = null;
  input.gamepadSteer = 0;
  input.gamepadGas = false;
  input.gamepadBrake = false;
  input.gamepadBoost = false;
  input.gamepadPauseHeld = false;
  input.gamepadResetHeld = false;
  $("#controllerChip").textContent = "Controller: Off";
  $("#controllerName").textContent = "Bluetooth controller ready";
```

```
  showToast("Controller disconnected.");
}

function pollGamepad() {
  if (!navigator.getGamepads) return;
  const pads = navigator.getGamepads();
  const pad = controllerState.index !== null ? pads[controllerState.index] : Array.from(pads).find(Boolean);
  if (!pad) {
    if (controllerState.connected) clearController();
    return;
  }
  if (!controllerState.connected) setController(pad);
  const stick = Math.abs(pad.axes[0] || 0) > 0.18 ? pad.axes[0] : 0;
  const dpadLeft = pad.buttons[14] && pad.buttons[14].pressed;
  const dpadRight = pad.buttons[15] && pad.buttons[15].pressed;
  input.gamepadSteer = dpadLeft ? -1 : dpadRight ? 1 : stick;
  input.gamepadGas = Boolean((pad.buttons[7] && pad.buttons[7].value > 0.18) || (pad.buttons[0] &&
pad.buttons[0].pressed));
  input.gamepadBrake = Boolean((pad.buttons[6] && pad.buttons[6].value > 0.18) || (pad.buttons[2] &&
pad.buttons[2].pressed));
  input.gamepadBoost = Boolean((pad.buttons[1] && pad.buttons[1].pressed) || (pad.buttons[5] &&
pad.buttons[5].pressed));
  const pausePressed = Boolean((pad.buttons[9] && pad.buttons[9].pressed) || (pad.buttons[8] &&
pad.buttons[8].pressed));
  if (pausePressed && !input.gamepadPauseHeld) {
    input.paused = !input.paused;
    $("#pauseBtn").textContent = input.paused ? "Play" : "Pause";
  }
  input.gamepadPauseHeld = pausePressed;
  const resetPressed = Boolean(pad.buttons[3] && pad.buttons[3].pressed);
  if (resetPressed && !input.gamepadResetHeld) manualResetVehicle();
  input.gamepadResetHeld = resetPressed;
}

function setKey(event, down) {
  if (event.key === "ArrowLeft" || event.key.toLowerCase() === "a") input.left = down;
  if (event.key === "ArrowRight" || event.key.toLowerCase() === "d") input.right = down;
  if (event.key === "ArrowUp" || event.key.toLowerCase() === "w") input.gas = down;
  if (event.key === "ArrowDown" || event.key.toLowerCase() === "s") input.brake = down;
  if (event.key === " ") input.boost = down;
  if (event.key.toLowerCase() === "p" && down) {
    input.paused = !input.paused;
    $("#pauseBtn").textContent = input.paused ? "Play" : "Pause";
  }
  if (event.key.toLowerCase() === "r" && down) manualResetVehicle();
}

bindEvents();
syncViewportSize();
fitCanvas();
updateHud();
applyPhoneGraphicsDefaults();
setCameraMode(cameraMode, true);
setTouchDriveMode(touchDriveMode, true);
setTouchControlSize(touchControlSize, true);
initWebGLRenderer();
setRendererMode(rendererMode, true);
registerOfflineApp();
drawFrame();
```

phone-asset-pipeline.js

```
"use strict";

(function () {
  const spriteCache = new Map();
  const textureCache = new Map();

  function makeCanvas(width, height) {
    const canvas = document.createElement("canvas");
    canvas.width = width;
    canvas.height = height;
    return canvas;
  }

  function hexToRgb(hex) {
    if (!hex || typeof hex !== "string") return [70, 217, 255];
    const clean = hex.replace("#", "");
    const full = clean.length === 3 ? clean.split("").map((c) => c + c).join("") : clean;
    const value = parseInt(full, 16);
    return [(value >> 16) & 255, (value >> 8) & 255, value & 255];
  }

  function shade(hex, amount) {
    const [r, g, b] = hexToRgb(hex);
    return `rgb(${Math.max(0, Math.min(255, r + amount))}, ${Math.max(0, Math.min(255, g + amount))}, ${Math.max(0,
Math.min(255, b + amount))})`;
  }

  function rgba(hex, alpha) {
    const [r, g, b] = hexToRgb(hex);
    return `rgba(${r}, ${g}, ${b}, ${alpha})`;
  }

  function roundRect(ctx, x, y, width, height, radius) {
    const r = Math.min(radius, Math.abs(width) / 2, Math.abs(height) / 2);
    ctx.beginPath();
    ctx.moveTo(x + r, y);
    ctx.lineTo(x + width - r, y);
    ctx.quadraticCurveTo(x + width, y, x + width, y + r);
    ctx.lineTo(x + width, y + height - r);
    ctx.quadraticCurveTo(x + width, y + height, x + width - r, y + height);
    ctx.lineTo(x + r, y + height);
    ctx.quadraticCurveTo(x, y + height, x, y + height - r);
    ctx.lineTo(x, y + r);
    ctx.quadraticCurveTo(x, y, x + r, y);
    ctx.closePath();
  }

  function pathVehicleBody(ctx, w, h, type) {
    const cx = w / 2;
    const top = h * 0.16;
    const bottom = h * 0.86;
    const narrow = type === "f1" || type === "prototype";
    const wide = type === "truck" || type === "monster" || type === "tank" || type === "semi";
    const upper = narrow ? w * 0.21 : wide ? w * 0.31 : w * 0.24;
    const shoulder = narrow ? w * 0.34 : wide ? w * 0.43 : w * 0.38;
    const lower = narrow ? w * 0.27 : wide ? w * 0.4 : w * 0.34;
    ctx.beginPath();
    ctx.moveTo(cx - upper, top);
    ctx.lineTo(cx + upper, top);
    ctx.quadraticCurveTo(cx + shoulder, h * 0.22, cx + shoulder, h * 0.42);
    ctx.lineTo(cx + lower, bottom);
    ctx.quadraticCurveTo(cx + lower * 0.72, h * 0.93, cx, h * 0.94);
    ctx.quadraticCurveTo(cx - lower * 0.72, h * 0.93, cx - lower, bottom);
    ctx.lineTo(cx - shoulder, h * 0.42);
    ctx.quadraticCurveTo(cx - shoulder, h * 0.22, cx - upper, top);
    ctx.closePath();
  }

  function drawWheel(ctx, x, y, rx, ry, accent) {
    ctx.save();
    ctx.fillStyle = "rgba(3,5,5,0.96)";
    ctx.beginPath();
    ctx.ellipse(x, y, rx, ry, 0, 0, Math.PI * 2);
    ctx.fill();
    ctx.strokeStyle = "rgba(244,251,248,0.28)";
    ctx.lineWidth = Math.max(2, rx * 0.14);
    ctx.stroke();
    ctx.strokeStyle = accent;
    ctx.lineWidth = Math.max(1, rx * 0.1);
    ctx.beginPath();
    ctx.moveTo(x - rx * 0.62, y);
    ctx.lineTo(x + rx * 0.62, y);
```

```
  ctx.moveTo(x, y - ry * 0.62);
  ctx.lineTo(x, y + ry * 0.62);
  ctx.moveTo(x - rx * 0.42, y - ry * 0.42);
  ctx.lineTo(x + rx * 0.42, y + ry * 0.42);
  ctx.moveTo(x + rx * 0.42, y - ry * 0.42);
  ctx.lineTo(x - rx * 0.42, y + ry * 0.42);
  ctx.stroke();
  ctx.restore();
}

function drawPaintGrain(ctx, w, h, color) {
  ctx.save();
  ctx.globalAlpha = 0.15;
  for (let i = 0; i < 95; i += 1) {
    const x = (i * 37) % w;
    const y = (i * 61) % h;
    ctx.strokeStyle = i % 2 ? "rgba(255,255,255,0.28)" : rgba(color, 0.35);
    ctx.lineWidth = 1;
    ctx.beginPath();
    ctx.moveTo(x, y);
    ctx.lineTo(x + 18 + (i % 5) * 4, y + 3);
    ctx.stroke();
  }
  ctx.restore();
}

function drawDamage(ctx, w, h, damage) {
  if (!damage || damage < 12) return;
  ctx.save();
  const count = Math.min(9, Math.floor(damage / 12));
  ctx.strokeStyle = "rgba(3,5,5,0.72)";
  ctx.lineWidth = Math.max(2, w * 0.012);
  ctx.lineCap = "round";
  for (let i = 0; i < count; i += 1) {
    const side = i % 2 ? -1 : 1;
    const x = w * 0.5 + side * w * (0.08 + (i % 3) * 0.055);
    const y = h * (0.28 + (i % 5) * 0.1);
    ctx.beginPath();
    ctx.moveTo(x, y);
    ctx.lineTo(x + side * w * 0.11, y + h * (0.02 + (i % 2) * 0.025));
    ctx.stroke();
  }
  if (damage > 58) {
    ctx.fillStyle = "rgba(255,91,107,0.46)";
    roundRect(ctx, w * 0.38, h * 0.18, w * 0.2, h * 0.045, 5);
    ctx.fill();
  }
  ctx.restore();
}

function drawBodyRealismDetails(ctx, w, h, type, color, police) {
  const paint = police ? "#f2f5f2" : color;
  ctx.save();
  pathVehicleBody(ctx, w, h, type === "semi" ? "truck" : type);
  ctx.clip();

  const sideShade = ctx.createLinearGradient(w * 0.12, 0, w * 0.88, 0);
  sideShade.addColorStop(0, "rgba(0,0,0,0.48)");
  sideShade.addColorStop(0.22, "rgba(255,255,255,0.08)");
  sideShade.addColorStop(0.5, "rgba(255,255,255,0.18)");
  sideShade.addColorStop(0.78, "rgba(255,255,255,0.06)");
  sideShade.addColorStop(1, "rgba(0,0,0,0.46)");
  ctx.fillStyle = sideShade;
  ctx.fillRect(0, 0, w, h);

  ctx.globalAlpha = 0.42;
  ctx.strokeStyle = "rgba(255,255,255,0.7)";
  ctx.lineWidth = Math.max(2, w * 0.012);
  ctx.beginPath();
  ctx.moveTo(w * 0.36, h * 0.18);
  ctx.bezierCurveTo(w * 0.42, h * 0.34, w * 0.33, h * 0.56, w * 0.28, h * 0.82);
  ctx.moveTo(w * 0.64, h * 0.18);
  ctx.bezierCurveTo(w * 0.58, h * 0.34, w * 0.67, h * 0.56, w * 0.72, h * 0.82);
  ctx.stroke();

  ctx.globalAlpha = 0.22;
  ctx.strokeStyle = rgba(paint, 0.8);
  ctx.lineWidth = Math.max(1, w * 0.006);
  for (let i = 0; i < 18; i += 1) {
    const y = h * (0.18 + i * 0.038);
    ctx.beginPath();
    ctx.moveTo(w * 0.22, y);
    ctx.lineTo(w * 0.78, y + h * 0.012);
    ctx.stroke();
```

```
  }
  ctx.restore();

  ctx.save();
  ctx.strokeStyle = "rgba(3,5,5,0.68)";
  ctx.lineWidth = Math.max(2, w * 0.012);
  ctx.beginPath();
  ctx.moveTo(w * 0.26, h * 0.34);
  ctx.lineTo(w * 0.2, h * 0.78);
  ctx.moveTo(w * 0.74, h * 0.34);
  ctx.lineTo(w * 0.8, h * 0.78);
  ctx.moveTo(w * 0.37, h * 0.7);
  ctx.lineTo(w * 0.63, h * 0.7);
  ctx.stroke();
  ctx.fillStyle = "rgba(2,4,4,0.88)";
  roundRect(ctx, w * 0.24, h * 0.17, w * 0.1, h * 0.04, 5);
  ctx.fill();
  roundRect(ctx, w * 0.66, h * 0.17, w * 0.1, h * 0.04, 5);
  ctx.fill();
  ctx.restore();
}

function drawAirWaterRealismDetails(ctx, w, h, type) {
  ctx.save();
  ctx.globalAlpha = 0.38;
  ctx.strokeStyle = "rgba(255,255,255,0.78)";
  ctx.lineWidth = Math.max(2, w * 0.01);
  if (type === "airplane") {
    ctx.beginPath();
    ctx.moveTo(w * 0.5, h * 0.12);
    ctx.lineTo(w * 0.5, h * 0.86);
    ctx.moveTo(w * 0.18, h * 0.52);
    ctx.lineTo(w * 0.82, h * 0.52);
    ctx.stroke();
  } else if (type === "helicopter") {
    ctx.beginPath();
    ctx.moveTo(w * 0.5, h * 0.18);
    ctx.lineTo(w * 0.5, h * 0.84);
    ctx.moveTo(w * 0.28, h * 0.38);
    ctx.lineTo(w * 0.72, h * 0.38);
    ctx.stroke();
  } else {
    ctx.beginPath();
    ctx.moveTo(w * 0.5, h * 0.12);
    ctx.bezierCurveTo(w * 0.38, h * 0.38, w * 0.34, h * 0.66, w * 0.42, h * 0.86);
    ctx.moveTo(w * 0.5, h * 0.12);
    ctx.bezierCurveTo(w * 0.62, h * 0.38, w * 0.66, h * 0.66, w * 0.58, h * 0.86);
    ctx.stroke();
  }
  ctx.restore();
}

function drawSpriteContact(ctx, w, h, type) {
  const air = type === "airplane" || type === "helicopter";
  const water = type === "boat";
  const snow = type === "snowmobile";
  ctx.save();
  const baseY = h * (air ? 0.68 : water ? 0.78 : 0.84);
  const contact = ctx.createRadialGradient(w * 0.5, baseY - h * 0.03, w * 0.06, w * 0.5, baseY, w * 0.48);
  contact.addColorStop(0, air ? "rgba(0,0,0,0.22)" : "rgba(0,0,0,0.62)");
  contact.addColorStop(1, "rgba(0,0,0,0)");
  ctx.fillStyle = contact;
  ctx.beginPath();
  ctx.ellipse(w * 0.5, baseY, w * 0.42, h * (air ? 0.045 : 0.075), 0, 0, Math.PI * 2);
  ctx.fill();

  if (!air && !water) {
    ctx.fillStyle = snow ? "rgba(244,251,248,0.42)" : "rgba(3,5,5,0.82)";
    ctx.beginPath();
    ctx.ellipse(w * 0.29, h * 0.4, w * 0.09, h * 0.025, 0, 0, Math.PI * 2);
    ctx.ellipse(w * 0.71, h * 0.4, w * 0.09, h * 0.025, 0, 0, Math.PI * 2);
    ctx.ellipse(w * 0.29, h * 0.8, w * 0.11, h * 0.034, 0, 0, Math.PI * 2);
    ctx.ellipse(w * 0.71, h * 0.8, w * 0.11, h * 0.034, 0, 0, Math.PI * 2);
    ctx.fill();
  }

  ctx.strokeStyle = water ? "rgba(70,217,255,0.48)" : snow ? "rgba(244,251,248,0.52)" : "rgba(3,5,5,0.5)";
  ctx.lineWidth = Math.max(2, w * 0.012);
  ctx.lineCap = "round";
  ctx.beginPath();
  ctx.moveTo(w * 0.3, baseY - h * 0.035);
  ctx.lineTo(w * 0.18, baseY + h * 0.075);
  ctx.moveTo(w * 0.7, baseY - h * 0.035);
  ctx.lineTo(w * 0.82, baseY + h * 0.075);
```

```
  ctx.stroke();
  ctx.restore();
}

function drawCarLike(ctx, w, h, type, color, police, damage) {
  const paint = police ? "#f2f5f2" : color;
  const accent = police ? "rgba(70,217,255,0.45)" : rgba(color, 0.45);
  const wide = type === "truck" || type === "monster" || type === "tank";
  const narrow = type === "f1" || type === "prototype";
  const semi = type === "semi";
  const tractor = type === "tractor";

  const wheelRX = w * (tractor ? 0.09 : narrow ? 0.075 : wide ? 0.095 : 0.078);
  const wheelRY = h * (tractor ? 0.13 : narrow ? 0.08 : wide ? 0.115 : 0.105);
  const wheelX = w * (wide ? 0.2 : 0.18);
  drawWheel(ctx, w * 0.5 - wheelX, h * 0.38, wheelRX, wheelRY, accent);
  drawWheel(ctx, w * 0.5 + wheelX, h * 0.38, wheelRX, wheelRY, accent);
  drawWheel(ctx, w * 0.5 - wheelX, h * 0.76, wheelRX, wheelRY, "rgba(255,209,102,0.34)");
  drawWheel(ctx, w * 0.5 + wheelX, h * 0.76, wheelRX, wheelRY, "rgba(255,209,102,0.34)");

  if (semi) {
    const trailer = ctx.createLinearGradient(w * 0.18, h * 0.36, w * 0.82, h * 0.88);
    trailer.addColorStop(0, shade(paint, 32));
    trailer.addColorStop(0.55, shade(paint, -12));
    trailer.addColorStop(1, shade(paint, -42));
    ctx.fillStyle = trailer;
    roundRect(ctx, w * 0.18, h * 0.36, w * 0.64, h * 0.5, 14);
    ctx.fill();
    ctx.fillStyle = "rgba(244,251,248,0.22)";
    for (let i = 0; i < 3; i += 1) {
      roundRect(ctx, w * 0.24, h * (0.46 + i * 0.11), w * 0.52, h * 0.025, 4);
      ctx.fill();
    }
  }

  const body = ctx.createLinearGradient(w * 0.18, h * 0.14, w * 0.82, h * 0.9);
  body.addColorStop(0, shade(paint, 52));
  body.addColorStop(0.3, paint);
  body.addColorStop(0.68, shade(paint, -26));
  body.addColorStop(1, shade(paint, -58));
  ctx.fillStyle = body;
  pathVehicleBody(ctx, w, h, semi ? "truck" : type);
  ctx.fill();
  ctx.strokeStyle = "rgba(255,255,255,0.28)";
  ctx.lineWidth = Math.max(2, w * 0.01);
  ctx.stroke();
  drawBodyRealismDetails(ctx, w, h, semi ? "semi" : type, color, police);

  if (type === "tank") {
    ctx.fillStyle = shade(paint, -30);
    roundRect(ctx, w * 0.35, h * 0.42, w * 0.3, h * 0.2, 8);
    ctx.fill();
    roundRect(ctx, w * 0.47, h * 0.12, w * 0.06, h * 0.35, 4);
    ctx.fill();
  }

  if (type === "f1" || type === "prototype") {
    ctx.fillStyle = shade(paint, -42);
    roundRect(ctx, w * 0.18, h * 0.8, w * 0.64, h * 0.05, 5);
    ctx.fill();
    ctx.fillStyle = "rgba(4,6,6,0.9)";
    ctx.beginPath();
    ctx.ellipse(w * 0.5, h * 0.42, w * 0.11, h * 0.085, 0, 0, Math.PI * 2);
    ctx.fill();
  }

  if (tractor) {
    ctx.fillStyle = shade(paint, -30);
    roundRect(ctx, w * 0.28, h * 0.48, w * 0.44, h * 0.26, 12);
    ctx.fill();
    ctx.strokeStyle = "rgba(255,209,102,0.55)";
    ctx.lineWidth = Math.max(3, w * 0.018);
    ctx.beginPath();
    ctx.moveTo(w * 0.27, h * 0.79);
    ctx.lineTo(w * 0.73, h * 0.79);
    ctx.stroke();
  }

  const glass = ctx.createLinearGradient(0, h * 0.18, 0, h * 0.58);
  glass.addColorStop(0, "rgba(222,249,255,0.78)");
  glass.addColorStop(0.38, "rgba(53,83,90,0.78)");
  glass.addColorStop(1, "rgba(3,6,7,0.96)");
  ctx.fillStyle = glass;
  roundRect(ctx, w * 0.32, h * 0.22, w * 0.36, h * 0.15, 10);
```

```
  ctx.fill();
  if (!narrow) {
    roundRect(ctx, w * 0.28, h * 0.46, w * 0.44, h * 0.2, 12);
    ctx.fill();
  }

  ctx.strokeStyle = "rgba(244,251,248,0.24)";
  ctx.lineWidth = Math.max(2, w * 0.008);
  ctx.beginPath();
  ctx.moveTo(w * 0.5, h * 0.17);
  ctx.lineTo(w * 0.5, h * 0.86);
  ctx.moveTo(w * 0.31, h * 0.43);
  ctx.lineTo(w * 0.69, h * 0.43);
  ctx.stroke();

  ctx.fillStyle = shade(paint, -48);
  roundRect(ctx, w * 0.18, h * 0.38, w * 0.085, h * 0.035, 5);
  ctx.fill();
  roundRect(ctx, w * 0.735, h * 0.38, w * 0.085, h * 0.035, 5);
  ctx.fill();

  ctx.fillStyle = "rgba(255,248,214,0.95)";
  roundRect(ctx, w * 0.32, h * 0.18, w * 0.12, h * 0.035, 5);
  ctx.fill();
  roundRect(ctx, w * 0.56, h * 0.18, w * 0.12, h * 0.035, 5);
  ctx.fill();
  ctx.fillStyle = police ? "#46d9ff" : "#ff3348";
  roundRect(ctx, w * 0.31, h * 0.78, w * 0.15, h * 0.038, 5);
  ctx.fill();
  ctx.fillStyle = "#ff3348";
  roundRect(ctx, w * 0.54, h * 0.78, w * 0.15, h * 0.038, 5);
  ctx.fill();

  if (police) {
    ctx.fillStyle = "rgba(5,8,7,0.86)";
    roundRect(ctx, w * 0.3, h * 0.42, w * 0.4, h * 0.045, 5);
    ctx.fill();
    ctx.fillStyle = "#f4fbf8";
    ctx.font = `900 ${Math.max(12, w * 0.055)}px system-ui`;
    ctx.textAlign = "center";
    ctx.fillText("POLICE", w * 0.5, h * 0.62);
    ctx.fillStyle = "#ff3348";
    roundRect(ctx, w * 0.4, h * 0.32, w * 0.08, h * 0.032, 4);
    ctx.fill();
    ctx.fillStyle = "#46d9ff";
    roundRect(ctx, w * 0.52, h * 0.32, w * 0.08, h * 0.032, 4);
    ctx.fill();
  }

  ctx.fillStyle = "rgba(244,251,248,0.7)";
  roundRect(ctx, w * 0.42, h * 0.82, w * 0.16, h * 0.03, 5);
  ctx.fill();
  drawPaintGrain(ctx, w, h, paint);
  drawDamage(ctx, w, h, damage);
}

function drawAirOrWater(ctx, w, h, type, color, damage) {
  const paint = ctx.createLinearGradient(w * 0.26, h * 0.12, w * 0.74, h * 0.9);
  paint.addColorStop(0, shade(color, 48));
  paint.addColorStop(0.55, color);
  paint.addColorStop(1, shade(color, -44));
  ctx.fillStyle = paint;
  ctx.beginPath();
  if (type === "airplane") {
    ctx.moveTo(w * 0.5, h * 0.08);
    ctx.lineTo(w * 0.62, h * 0.86);
    ctx.lineTo(w * 0.5, h * 0.92);
    ctx.lineTo(w * 0.38, h * 0.86);
    ctx.closePath();
    ctx.fill();
    ctx.beginPath();
    ctx.moveTo(w * 0.12, h * 0.52);
    ctx.lineTo(w * 0.88, h * 0.52);
    ctx.lineTo(w * 0.62, h * 0.63);
    ctx.lineTo(w * 0.38, h * 0.63);
    ctx.closePath();
    ctx.fill();
  } else if (type === "helicopter") {
    roundRect(ctx, w * 0.3, h * 0.26, w * 0.4, h * 0.42, 24);
    ctx.fill();
    ctx.fillStyle = "rgba(244,251,248,0.72)";
    roundRect(ctx, w * 0.13, h * 0.13, w * 0.74, h * 0.035, 5);
    ctx.fill();
    ctx.fillStyle = shade(color, -34);
```

```
    roundRect(ctx, w * 0.47, h * 0.64, w * 0.06, h * 0.25, 5);
    ctx.fill();
  } else {
    ctx.moveTo(w * 0.5, h * 0.1);
    ctx.quadraticCurveTo(w * 0.82, h * 0.5, w * 0.62, h * 0.88);
    ctx.lineTo(w * 0.38, h * 0.88);
    ctx.quadraticCurveTo(w * 0.18, h * 0.5, w * 0.5, h * 0.1);
    ctx.closePath();
    ctx.fill();
  }
  ctx.fillStyle = "rgba(3,6,7,0.86)";
  roundRect(ctx, w * 0.39, h * 0.35, w * 0.22, h * 0.18, 10);
  ctx.fill();
  ctx.fillStyle = "rgba(255,248,214,0.86)";
  roundRect(ctx, w * 0.43, h * 0.18, w * 0.14, h * 0.035, 5);
  ctx.fill();
  drawAirWaterRealismDetails(ctx, w, h, type);
  drawPaintGrain(ctx, w, h, color);
  drawDamage(ctx, w, h, damage);
}

function drawRearWheel(ctx, x, groundY, rx, ry, accent, heavy = false) {
  ctx.save();
  const cy = groundY - ry;
  ctx.fillStyle = "rgba(2,4,4,0.98)";
  ctx.beginPath();
  ctx.ellipse(x, cy, rx, ry, 0, 0, Math.PI * 2);
  ctx.fill();
  ctx.strokeStyle = "rgba(244,251,248,0.22)";
  ctx.lineWidth = Math.max(2, rx * 0.13);
  ctx.stroke();
  ctx.globalAlpha = 0.42;
  ctx.strokeStyle = accent;
  ctx.lineWidth = Math.max(1, rx * 0.12);
  for (let i = -1; i <= 1; i += 1) {
    ctx.beginPath();
    ctx.moveTo(x - rx * 0.62, cy + i * ry * 0.26);
    ctx.lineTo(x + rx * 0.62, cy + i * ry * 0.26);
    ctx.stroke();
  }
  ctx.globalAlpha = heavy ? 0.52 : 0.34;
  ctx.fillStyle = "rgba(0,0,0,0.92)";
  ctx.beginPath();
  ctx.ellipse(x, groundY + ry * 0.08, rx * 1.12, ry * 0.18, 0, 0, Math.PI * 2);
  ctx.fill();
  ctx.restore();
}

function drawRearGroundVehicle(ctx, w, h, type, color, police, damage) {
  const paint = police ? "#f2f5f2" : color;
  const accent = police ? "rgba(70,217,255,0.56)" : rgba(color, 0.58);
  const semi = type === "semi";
  const tractor = type === "tractor";
  const tank = type === "tank";
  const monster = type === "monster";
  const truck = type === "truck";
  const f1 = type === "f1" || type === "prototype";
  const snow = type === "snowmobile";
  const wide = semi || tractor || tank || monster || truck;
  const groundY = h * (snow ? 0.94 : 0.955);

  ctx.save();
  const contact = ctx.createRadialGradient(w * 0.5, groundY - h * 0.025, w * 0.07, w * 0.5, groundY, w * 0.55);
  contact.addColorStop(0, snow ? "rgba(244,251,248,0.42)" : "rgba(0,0,0,0.72)");
  contact.addColorStop(1, "rgba(0,0,0,0)");
  ctx.fillStyle = contact;
  ctx.beginPath();
  ctx.ellipse(w * 0.5, groundY, w * (wide ? 0.52 : 0.44), h * 0.06, 0, 0, Math.PI * 2);
  ctx.fill();

  if (snow) {
    ctx.strokeStyle = "rgba(244,251,248,0.82)";
    ctx.lineWidth = Math.max(7, w * 0.036);
    ctx.lineCap = "round";
    ctx.beginPath();
    ctx.moveTo(w * 0.26, groundY);
    ctx.lineTo(w * 0.43, groundY - h * 0.02);
    ctx.moveTo(w * 0.57, groundY - h * 0.02);
    ctx.lineTo(w * 0.74, groundY);
    ctx.stroke();
  } else if (tank) {
    ctx.fillStyle = "rgba(1,2,2,0.98)";
    roundRect(ctx, w * 0.11, h * 0.44, w * 0.22, groundY - h * 0.44, 14);
    ctx.fill();
```

```
  roundRect(ctx, w * 0.67, h * 0.44, w * 0.22, groundY - h * 0.44, 14);
  ctx.fill();
  ctx.strokeStyle = "rgba(244,251,248,0.16)";
  ctx.lineWidth = Math.max(2, w * 0.012);
  for (let i = 0; i < 7; i += 1) {
    const y = h * 0.49 + i * h * 0.052;
    ctx.beginPath();
    ctx.moveTo(w * 0.14, y);
    ctx.lineTo(w * 0.3, y);
    ctx.moveTo(w * 0.7, y);
    ctx.lineTo(w * 0.86, y);
    ctx.stroke();
  }
} else {
  const rx = w * (monster ? 0.105 : semi || tractor ? 0.088 : f1 ? 0.074 : 0.082);
  const ry = h * (monster ? 0.155 : semi || tractor ? 0.14 : f1 ? 0.09 : 0.125);
  const left = w * (wide ? 0.2 : 0.24);
  const right = w - left;
  drawRearWheel(ctx, left, groundY, rx, ry, accent, wide);
  drawRearWheel(ctx, right, groundY, rx, ry, accent, wide);
  if (semi || tractor || truck) {
    drawRearWheel(ctx, left + w * 0.08, groundY - h * 0.01, rx * 0.78, ry * 0.8, "rgba(255,209,102,0.28)", true);
    drawRearWheel(ctx, right - w * 0.08, groundY - h * 0.01, rx * 0.78, ry * 0.8, "rgba(255,209,102,0.28)", true);
  }
}

const body = ctx.createLinearGradient(w * 0.18, h * 0.14, w * 0.82, groundY);
body.addColorStop(0, shade(paint, 54));
body.addColorStop(0.34, paint);
body.addColorStop(0.7, shade(paint, -30));
body.addColorStop(1, shade(paint, -62));
ctx.fillStyle = body;

if (f1) {
  ctx.fillStyle = shade(paint, -38);
  roundRect(ctx, w * 0.12, h * 0.73, w * 0.76, h * 0.075, 9);
  ctx.fill();
  ctx.fillStyle = body;
  ctx.beginPath();
  ctx.moveTo(w * 0.5, h * 0.28);
  ctx.lineTo(w * 0.68, h * 0.5);
  ctx.lineTo(w * 0.61, h * 0.82);
  ctx.lineTo(w * 0.39, h * 0.82);
  ctx.lineTo(w * 0.32, h * 0.5);
  ctx.closePath();
  ctx.fill();
  ctx.fillStyle = "rgba(4,6,6,0.92)";
  ctx.beginPath();
  ctx.ellipse(w * 0.5, h * 0.53, w * 0.12, h * 0.09, 0, 0, Math.PI * 2);
  ctx.fill();
} else if (semi) {
  roundRect(ctx, w * 0.19, h * 0.33, w * 0.62, h * 0.43, 18);
  ctx.fill();
  ctx.fillStyle = shade(paint, 18);
  roundRect(ctx, w * 0.24, h * 0.17, w * 0.52, h * 0.26, 16);
  ctx.fill();
  ctx.fillStyle = "rgba(4,7,8,0.9)";
  roundRect(ctx, w * 0.31, h * 0.22, w * 0.38, h * 0.12, 8);
  ctx.fill();
  ctx.fillStyle = "rgba(244,251,248,0.28)";
  for (let i = 0; i < 3; i += 1) {
    roundRect(ctx, w * 0.27, h * (0.48 + i * 0.1), w * 0.46, h * 0.026, 4);
    ctx.fill();
  }
} else if (tractor) {
  roundRect(ctx, w * 0.3, h * 0.25, w * 0.4, h * 0.24, 14);
  ctx.fill();
  ctx.fillStyle = shade(paint, -28);
  roundRect(ctx, w * 0.23, h * 0.48, w * 0.54, h * 0.26, 12);
  ctx.fill();
  ctx.strokeStyle = "rgba(255,209,102,0.58)";
  ctx.lineWidth = Math.max(4, w * 0.022);
  ctx.beginPath();
  ctx.moveTo(w * 0.23, h * 0.77);
  ctx.lineTo(w * 0.77, h * 0.77);
  ctx.stroke();
} else if (tank) {
  roundRect(ctx, w * 0.24, h * 0.32, w * 0.52, h * 0.46, 9);
  ctx.fill();
  ctx.fillStyle = shade(paint, -34);
  roundRect(ctx, w * 0.34, h * 0.38, w * 0.32, h * 0.2, 8);
  ctx.fill();
  ctx.fillRect(w * 0.47, h * 0.18, w * 0.06, h * 0.26);
} else {
```

```
  const topHalf = w * (truck || monster ? 0.24 : 0.2);
  const shoulder = w * (truck || monster ? 0.38 : 0.34);
  const lower = w * (truck || monster ? 0.43 : 0.37);
  const top = h * (snow ? 0.3 : 0.18);
  const belly = groundY - h * 0.055;
  ctx.beginPath();
  ctx.moveTo(w * 0.5 - topHalf, top);
  ctx.lineTo(w * 0.5 + topHalf, top);
  ctx.quadraticCurveTo(w * 0.5 + shoulder, h * 0.26, w * 0.5 + shoulder, h * 0.48);
  ctx.lineTo(w * 0.5 + lower, belly);
  ctx.quadraticCurveTo(w * 0.5 + lower * 0.7, groundY - h * 0.015, w * 0.5, groundY - h * 0.012);
  ctx.quadraticCurveTo(w * 0.5 - lower * 0.7, groundY - h * 0.015, w * 0.5 - lower, belly);
  ctx.lineTo(w * 0.5 - shoulder, h * 0.48);
  ctx.quadraticCurveTo(w * 0.5 - shoulder, h * 0.26, w * 0.5 - topHalf, top);
  ctx.closePath();
  ctx.fill();
}

ctx.strokeStyle = "rgba(255,255,255,0.28)";
ctx.lineWidth = Math.max(2, w * 0.012);
ctx.stroke();

if (snow) {
  ctx.strokeStyle = "rgba(244,251,248,0.92)";
  ctx.lineWidth = Math.max(7, w * 0.034);
  ctx.lineCap = "round";
  ctx.beginPath();
  ctx.moveTo(w * 0.24, groundY);
  ctx.lineTo(w * 0.43, groundY - h * 0.01);
  ctx.moveTo(w * 0.57, groundY - h * 0.01);
  ctx.lineTo(w * 0.76, groundY);
  ctx.stroke();
} else if (tank) {
  ctx.fillStyle = "rgba(1,2,2,1)";
  roundRect(ctx, w * 0.11, groundY - h * 0.12, w * 0.23, h * 0.13, 10);
  ctx.fill();
  roundRect(ctx, w * 0.66, groundY - h * 0.12, w * 0.23, h * 0.13, 10);
  ctx.fill();
} else {
  const rx = w * (monster ? 0.108 : semi || tractor ? 0.092 : f1 ? 0.078 : 0.086);
  const ry = h * (monster ? 0.16 : semi || tractor ? 0.14 : f1 ? 0.092 : 0.128);
  const left = w * (wide ? 0.2 : 0.24);
  const right = w - left;
  drawRearWheel(ctx, left, groundY, rx, ry, accent, true);
  drawRearWheel(ctx, right, groundY, rx, ry, accent, true);
}

ctx.fillStyle = "rgba(5,8,7,0.9)";
roundRect(ctx, w * 0.31, h * (semi ? 0.46 : 0.32), w * 0.38, h * 0.13, 10);
ctx.fill();
if (!f1 && !semi && !tractor && !tank) {
  roundRect(ctx, w * 0.27, h * 0.55, w * 0.46, h * 0.19, 12);
  ctx.fill();
}
ctx.fillStyle = "rgba(222,249,255,0.24)";
roundRect(ctx, w * 0.34, h * (semi ? 0.49 : 0.35), w * 0.14, h * 0.026, 4);
ctx.fill();
roundRect(ctx, w * 0.52, h * (semi ? 0.49 : 0.35), w * 0.14, h * 0.026, 4);
ctx.fill();

if (police) {
  ctx.fillStyle = "rgba(5,8,7,0.88)";
  roundRect(ctx, w * 0.29, h * 0.53, w * 0.42, h * 0.052, 5);
  ctx.fill();
  ctx.fillStyle = "#f4fbf8";
  ctx.font = `900 ${Math.max(16, w * 0.07)}px system-ui`;
  ctx.textAlign = "center";
  ctx.fillText("POLICE", w * 0.5, h * 0.65);
  ctx.fillStyle = "#ff3348";
  roundRect(ctx, w * 0.4, h * 0.27, w * 0.08, h * 0.034, 4);
  ctx.fill();
  ctx.fillStyle = "#46d9ff";
  roundRect(ctx, w * 0.52, h * 0.27, w * 0.08, h * 0.034, 4);
  ctx.fill();
}

ctx.fillStyle = "#ff3348";
roundRect(ctx, w * 0.28, groundY - h * 0.12, w * 0.17, h * 0.048, 5);
ctx.fill();
roundRect(ctx, w * 0.55, groundY - h * 0.12, w * 0.17, h * 0.048, 5);
ctx.fill();
ctx.fillStyle = "rgba(244,251,248,0.68)";
roundRect(ctx, w * 0.43, groundY - h * 0.085, w * 0.14, h * 0.028, 4);
ctx.fill();
```

```
    ctx.strokeStyle = "rgba(5,8,7,0.46)";
    ctx.lineWidth = Math.max(2, w * 0.016);
    ctx.beginPath();
    ctx.moveTo(w * 0.5, h * 0.2);
    ctx.lineTo(w * 0.5, groundY - h * 0.08);
    ctx.stroke();

    drawPaintGrain(ctx, w, h, paint);
    drawDamage(ctx, w, h, damage);
    ctx.restore();
  }

  function getVehicleSprite(type = "car", color = "#46d9ff", options = {}) {
    const police = Boolean(options.police);
    const damage = Math.max(0, Math.min(100, Number(options.damage) || 0));
    const damageBucket = Math.floor(damage / 14);
    const key = `${type}|${color}|${police ? 1 : 0}|${damageBucket}`;
    if (spriteCache.has(key)) return spriteCache.get(key);

    const canvas = makeCanvas(384, 552);
    const ctx = canvas.getContext("2d");
    ctx.imageSmoothingEnabled = true;

    if (["boat", "airplane", "helicopter"].includes(type)) {
      const shadow = ctx.createRadialGradient(192, 422, 26, 192, 422, 176);
      shadow.addColorStop(0, "rgba(0,0,0,0.52)");
      shadow.addColorStop(1, "rgba(0,0,0,0)");
      ctx.fillStyle = shadow;
      ctx.beginPath();
      ctx.ellipse(192, 422, 168, 56, 0, 0, Math.PI * 2);
      ctx.fill();
      drawSpriteContact(ctx, 384, 552, type);
      drawAirOrWater(ctx, 384, 552, type, police ? "#f4fbf8" : color, damage);
    } else {
      drawRearGroundVehicle(ctx, 384, 552, type, color, police, damage);
    }

    spriteCache.set(key, canvas);
    return canvas;
  }

  function getRoadTexture(place = "city", theme = ["#09100f", "#46d9ff", "#ffd166"]) {
    const key = `${place}|${theme.join("|")}`;
    if (textureCache.has(key)) return textureCache.get(key);
    const canvas = makeCanvas(512, 512);
    const ctx = canvas.getContext("2d");
    ctx.fillStyle = place === "snow" ? "rgba(205,222,224,0.14)" : place === "desert" || place === "canyon" ?
"rgba(156,92,42,0.13)" : "rgba(244,251,248,0.08)";
    ctx.fillRect(0, 0, 512, 512);
    for (let i = 0; i < 420; i += 1) {
      const x = (i * 47) % 512;
      const y = (i * 83) % 512;
      const len = 8 + (i % 9) * 7;
      ctx.globalAlpha = 0.08 + (i % 5) * 0.018;
      ctx.strokeStyle = i % 3 ? "rgba(255,255,255,0.55)" : rgba(theme[1], 0.75);
      ctx.lineWidth = 1 + (i % 3);
      ctx.beginPath();
      ctx.moveTo(x, y);
      ctx.lineTo(x + len, y + (i % 2 ? 2 : -2));
      ctx.stroke();
    }
    ctx.globalAlpha = 0.18;
    ctx.strokeStyle = place === "snow" ? "rgba(244,251,248,0.9)" : "rgba(0,0,0,0.85)";
    ctx.lineCap = "round";
    for (let i = 0; i < 54; i += 1) {
      const x = (i * 109) % 512;
      const y = (i * 67) % 512;
      ctx.lineWidth = 1 + (i % 3);
      ctx.beginPath();
      ctx.moveTo(x, y);
      ctx.bezierCurveTo(x + 24, y + 9, x - 18, y + 31, x + 38, y + 45);
      ctx.stroke();
    }
    ctx.globalAlpha = place === "tokyo" || place === "city" || place === "harbor" || place === "rainforest" ? 0.22 :
0.1;
    for (let i = 0; i < 18; i += 1) {
      const x = (i * 131) % 512;
      const y = (i * 191) % 512;
      const glow = ctx.createRadialGradient(x, y, 4, x, y, 92);
      glow.addColorStop(0, rgba(theme[1], 0.9));
      glow.addColorStop(1, "rgba(0,0,0,0)");
      ctx.fillStyle = glow;
      ctx.fillRect(x - 96, y - 96, 192, 192);
```

phone-asset-pipeline.js

```
    }
    ctx.globalAlpha = 1;
    textureCache.set(key, canvas);
    return canvas;
  }

  function isPhoneViewport() {
    return window.matchMedia && (window.matchMedia("(pointer: coarse)").matches || Math.min(window.innerWidth,
window.innerHeight) < 720);
  }

  window.VelocityPhoneAssets = {
    getVehicleSprite,
    getRoadTexture,
    isPhoneViewport,
    ready: true
  };
})();
```

webgl-pipeline.js

```
"use strict";

(function () {
  function hexToRgba(hex, alpha = 1) {
    if (!hex || typeof hex !== "string") return [1, 1, 1, alpha];
    const clean = hex.replace("#", "");
    const value = parseInt(clean.length === 3 ? clean.split("").map((c) => c + c).join("") : clean, 16);
    return [((value >> 16) & 255) / 255, ((value >> 8) & 255) / 255, (value & 255) / 255, alpha];
  }

  function shade(color, factor) {
    return [
      Math.max(0, Math.min(1, color[0] * factor)),
      Math.max(0, Math.min(1, color[1] * factor)),
      Math.max(0, Math.min(1, color[2] * factor)),
      color[3]
    ];
  }

  function perspective(fovy, aspect, near, far) {
    const f = 1 / Math.tan(fovy / 2);
    const nf = 1 / (near - far);
    return [
      f / aspect, 0, 0, 0,
      0, f, 0, 0,
      0, 0, (far + near) * nf, -1,
      0, 0, (2 * far * near) * nf, 0
    ];
  }

  function normalize(v) {
    const length = Math.hypot(v[0], v[1], v[2]) || 1;
    return [v[0] / length, v[1] / length, v[2] / length];
  }

  function cross(a, b) {
    return [
      a[1] * b[2] - a[2] * b[1],
      a[2] * b[0] - a[0] * b[2],
      a[0] * b[1] - a[1] * b[0]
    ];
  }

  function dot(a, b) {
    return a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
  }

  function lookAt(eye, center, up) {
    const z = normalize([eye[0] - center[0], eye[1] - center[1], eye[2] - center[2]]);
    const x = normalize(cross(up, z));
    const y = cross(z, x);
    return [
      x[0], y[0], z[0], 0,
      x[1], y[1], z[1], 0,
      x[2], y[2], z[2], 0,
      -dot(x, eye), -dot(y, eye), -dot(z, eye), 1
    ];
  }

  function multiply(a, b) {
    const out = new Array(16).fill(0);
    for (let col = 0; col < 4; col += 1) {
      for (let row = 0; row < 4; row += 1) {
        out[col * 4 + row] =
          a[0 * 4 + row] * b[col * 4 + 0] +
          a[1 * 4 + row] * b[col * 4 + 1] +
          a[2 * 4 + row] * b[col * 4 + 2] +
          a[3 * 4 + row] * b[col * 4 + 3];
      }
    }
    return out;
  }

  function compile(gl, type, source) {
    const shader = gl.createShader(type);
    gl.shaderSource(shader, source);
    gl.compileShader(shader);
    if (!gl.getShaderParameter(shader, gl.COMPILE_STATUS)) {
      throw new Error(gl.getShaderInfoLog(shader) || "WebGL shader compile failed");
    }
    return shader;
  }
```

```
  function linkProgram(gl) {
    const vertex = compile(gl, gl.VERTEX_SHADER, `
      attribute vec3 aPosition;
      attribute vec4 aColor;
      uniform mat4 uMvp;
      varying vec4 vColor;
      void main() {
        gl_Position = uMvp * vec4(aPosition, 1.0);
        vColor = aColor;
      }
    `);
    const fragment = compile(gl, gl.FRAGMENT_SHADER, `
      precision mediump float;
      varying vec4 vColor;
      void main() {
        gl_FragColor = vColor;
      }
    `);
    const program = gl.createProgram();
    gl.attachShader(program, vertex);
    gl.attachShader(program, fragment);
    gl.linkProgram(program);
    if (!gl.getProgramParameter(program, gl.LINK_STATUS)) {
      throw new Error(gl.getProgramInfoLog(program) || "WebGL program link failed");
    }
    return program;
  }

  function VelocityWebGLPipeline(canvas) {
    const gl = canvas.getContext("webgl", {
      alpha: false,
      antialias: true,
      depth: true,
      powerPreference: "high-performance"
    });
    if (!gl) throw new Error("WebGL is not available in this browser");

    const program = linkProgram(gl);
    const buffer = gl.createBuffer();
    const aPosition = gl.getAttribLocation(program, "aPosition");
    const aColor = gl.getAttribLocation(program, "aColor");
    const uMvp = gl.getUniformLocation(program, "uMvp");
    let vertices = [];
    let lastProjection = null;

    function visualLaneValue(data) {
      const state = data && data.raceState ? data.raceState : {};
      const visualLane = Number(state.visualLane);
      if (Number.isFinite(visualLane)) return visualLane;
      return Number(state.lane) || 0;
    }

    function laneToWorldX(lane) {
      return Number(lane) * 2.08;
    }

    function cameraProjection(data) {
      const laneX = laneToWorldX(visualLaneValue(data));
      const shake = data.raceState.cameraShake || 0;
      const curve = roadCurveValue(data);
      const jitter = shake > 0 ? (Math.random() - 0.5) * shake * 0.012 : 0;
      const phoneFrame = data.width <= 940 || data.height <= 540;
      let eye = phoneFrame ? [laneX * 0.08 - curve * 0.78 + jitter, 1.62, -15.8] : [laneX * 0.06 - curve * 0.9 +
jitter, 2.1, -14.6];
      let target = phoneFrame ? [laneX * 0.015 + curve * 3.45, 1.24, 62] : [laneX * 0.015 + curve * 3.4, 1.96, 50];
      if (data.cameraMode === "hood") {
        eye = phoneFrame ? [laneX * 0.3 - curve * 0.56 + jitter, 1.18, -7.8] : [laneX * 0.34 - curve * 0.62 + jitter,
1.42, -7.4];
        target = phoneFrame ? [laneX * 0.08 + curve * 3.65, 1.02, 64] : [laneX * 0.1 + curve * 3.6, 1.38, 52];
      } else if (data.cameraMode === "cockpit") {
        eye = [laneX * 0.2 - curve * 0.5 + jitter, 1.58, -3.8];
        target = [laneX * 0.05 + curve * 3.8, 1.52, 56];
      }
      const view = lookAt(eye, target, [0, 1, 0]);
      const fov = phoneFrame && data.cameraMode !== "cockpit" ? 2.28 : data.cameraMode === "cockpit" ? 2.25 : 2.55;
      const proj = perspective(Math.PI / fov, data.width / Math.max(1, data.height), 0.1, 220);
      return {
        mvp: multiply(proj, view),
        width: data.width,
        height: data.height,
        raceDistance: data.raceState.distance,
        roadTurn: curve,
        roadOffset: data.raceState.roadOffset || 0,
```

```
    cameraMode: data.cameraMode
  };
}

function projectPoint(projection, x, y, z) {
  const m = projection.mvp;
  const clipX = m[0] * x + m[4] * y + m[8] * z + m[12];
  const clipY = m[1] * x + m[5] * y + m[9] * z + m[13];
  const clipW = m[3] * x + m[7] * y + m[11] * z + m[15];
  if (!Number.isFinite(clipW) || clipW <= 0.02) return null;
  const ndcX = clipX / clipW;
  const ndcY = clipY / clipW;
  if (!Number.isFinite(ndcX) || !Number.isFinite(ndcY)) return null;
  const screenY = (0.5 - ndcY * 0.5) * projection.height;
  const t = Math.max(0, Math.min(1.16, screenY / Math.max(1, projection.height)));
  return {
    x: (ndcX * 0.5 + 0.5) * projection.width,
    y: screenY,
    scale: Math.max(0.34, Math.min(1.24, 0.42 + t * 0.78)),
    clipW,
    z
  };
}

function screenYFromRoadDistance(data, distance) {
  return data.height * 0.68 - (distance - data.raceState.distance) * 0.11;
}

function zFromRoadScreenY(data, y) {
  const t = Math.max(0.32, Math.min(1.02, y / Math.max(1, data.height)));
  return 116 - t * 108;
}

function zFromRoadDistance(data, distance) {
  return zFromRoadScreenY(data, screenYFromRoadDistance(data, distance));
}

function addVertex(point, color) {
  vertices.push(point[0], point[1], point[2], color[0], color[1], color[2], color[3]);
}

function tri(a, b, c, color) {
  addVertex(a, color);
  addVertex(b, color);
  addVertex(c, color);
}

function quad(a, b, c, d, color) {
  tri(a, b, c, color);
  tri(a, c, d, color);
}

function box(x, y, z, w, h, d, color) {
  const x0 = x - w / 2;
  const x1 = x + w / 2;
  const y0 = y - h / 2;
  const y1 = y + h / 2;
  const z0 = z - d / 2;
  const z1 = z + d / 2;
  quad([x0, y1, z0], [x1, y1, z0], [x1, y1, z1], [x0, y1, z1], shade(color, 1.14));
  quad([x0, y0, z1], [x1, y0, z1], [x1, y0, z0], [x0, y0, z0], shade(color, 0.55));
  quad([x0, y0, z0], [x0, y1, z0], [x0, y1, z1], [x0, y0, z1], shade(color, 0.76));
  quad([x1, y0, z1], [x1, y1, z1], [x1, y1, z0], [x1, y0, z0], shade(color, 0.88));
  quad([x0, y0, z1], [x0, y1, z1], [x1, y1, z1], [x1, y0, z1], shade(color, 1.02));
  quad([x1, y0, z0], [x1, y1, z0], [x0, y1, z0], [x0, y0, z0], shade(color, 0.64));
}

function taperedBox(x, y, z, bottomW, topW, h, d, color) {
  const y0 = y - h / 2;
  const y1 = y + h / 2;
  const z0 = z - d / 2;
  const z1 = z + d / 2;
  const b0 = x - bottomW / 2;
  const b1 = x + bottomW / 2;
  const t0 = x - topW / 2;
  const t1 = x + topW / 2;
  quad([t0, y1, z0], [t1, y1, z0], [t1, y1, z1], [t0, y1, z1], shade(color, 1.16));
  quad([b0, y0, z1], [b1, y0, z1], [b1, y0, z0], [b0, y0, z0], shade(color, 0.55));
  quad([b0, y0, z0], [t0, y1, z0], [t0, y1, z1], [b0, y0, z1], shade(color, 0.78));
  quad([b1, y0, z1], [t1, y1, z1], [t1, y1, z0], [b1, y0, z0], shade(color, 0.9));
  quad([b0, y0, z1], [t0, y1, z1], [t1, y1, z1], [b1, y0, z1], shade(color, 1.02));
  quad([b1, y0, z0], [t1, y1, z0], [t0, y1, z0], [b0, y0, z0], shade(color, 0.66));
}
```

```
    function lowMound(x, z, w, h, d, color) {
      taperedBox(x, h * 0.3, z, w, w * 0.72, h * 0.6, d, shade(color, 0.82));
      taperedBox(x, h * 0.78, z - d * 0.05, w * 0.72, w * 0.42, h * 0.44, d * 0.76, color);
      taperedBox(x, h * 1.08, z - d * 0.1, w * 0.42, w * 0.18, h * 0.24, d * 0.48, shade(color, 1.18));
    }

    function signPanel(x, z, textColor, baseColor) {
      box(x, 2.15, z, 3.2, 1.25, 0.14, baseColor);
      box(x, 2.22, z - 0.08, 2.35, 0.14, 0.16, textColor);
      box(x - 0.9, 1.7, z + 0.05, 0.12, 1.0, 0.12, [0.42, 0.44, 0.4, 1]);
      box(x + 0.9, 1.7, z + 0.05, 0.12, 1.0, 0.12, [0.42, 0.44, 0.4, 1]);
    }

    function person(x, z, shirtColor, scale = 1) {
      box(x, 0.18 * scale, z, 0.12 * scale, 0.36 * scale, 0.08 * scale, [0.04, 0.045, 0.04, 1]);
      box(x, 0.62 * scale, z, 0.28 * scale, 0.52 * scale, 0.16 * scale, shirtColor);
      box(x, 1.02 * scale, z, 0.22 * scale, 0.22 * scale, 0.18 * scale, [0.72, 0.52, 0.38, 1]);
      box(x - 0.24 * scale, 0.68 * scale, z, 0.08 * scale, 0.42 * scale, 0.08 * scale, shade(shirtColor, 0.75));
      box(x + 0.24 * scale, 0.68 * scale, z, 0.08 * scale, 0.42 * scale, 0.08 * scale, shade(shirtColor, 0.75));
    }

    function spectatorCluster(x, z, accent, accent2) {
      for (let j = 0; j < 5; j += 1) {
        const row = Math.floor(j / 3);
        const px = x + (j % 3 - 1) * 0.55;
        const pz = z + row * 0.34;
        person(px, pz, j % 2 ? accent : accent2, 0.74);
      }
      box(x, 0.28, z + 0.85, 2.4, 0.14, 0.14, [0.78, 0.82, 0.78, 1]);
    }

    function streetLight(x, z, accent) {
      const side = x < 0 ? 1 : -1;
      box(x, 1.35, z, 0.1, 2.7, 0.1, [0.38, 0.42, 0.39, 1]);
      box(x + side * 0.55, 2.62, z, 1.1, 0.08, 0.08, [0.38, 0.42, 0.39, 1]);
      box(x + side * 1.05, 2.52, z, 0.34, 0.12, 0.32, accent);
      box(x + side * 1.05, 2.43, z, 0.6, 0.03, 0.56, shade(accent, 1.35));
    }

    function roadsideTree(x, z, color, kind = "tree") {
      box(x, 0.74, z, 0.22, 1.45, 0.22, [0.34, 0.2, 0.1, 1]);
      if (kind === "palm") {
        taperedBox(x - 0.45, 1.82, z, 1.6, 0.25, 0.22, 1.0, color);
        taperedBox(x + 0.45, 1.86, z, 1.6, 0.25, 0.22, 1.0, color);
        taperedBox(x, 2.02, z - 0.45, 1.4, 0.2, 0.2, 1.15, color);
      } else {
        taperedBox(x, 1.85, z, 1.8, 0.7, 1.3, 1.6, color);
        taperedBox(x + 0.35, 2.45, z - 0.1, 1.3, 0.45, 1.0, 1.25, shade(color, 1.18));
      }
    }

    function barrierRun(x, z, accent) {
      box(x, 0.58, z, 0.14, 0.75, 0.16, [0.62, 0.66, 0.62, 1]);
      box(x, 0.86, z, 0.16, 0.12, 3.2, [0.78, 0.82, 0.78, 1]);
      box(x, 0.68, z, 0.18, 0.08, 3.0, shade(accent, 0.9));
    }

    function groundContactPad(type, x, z, scale, paint) {
      const air = type === "airplane" || type === "helicopter";
      const water = type === "boat";
      const wide = type === "semi" || type === "monster" || type === "tank" || type === "truck";
      const shadow = water ? [0.02, 0.18, 0.22, 1] : [0.006, 0.008, 0.007, 1];
      const dust = water ? [0.1, 0.5, 0.62, 1] : shade(paint, 0.24);
      const padW = (air ? 1.65 : wide ? 2.45 : 1.95) * scale;
      const padD = (air ? 2.1 : wide ? 3.2 : 2.75) * scale;
      box(x, 0.012 * scale, z, padW, 0.024 * scale, padD, shadow);
      box(x, 0.018 * scale, z + 0.42 * scale, padW * 0.72, 0.014 * scale, padD * 0.32, shade(shadow, 1.4));
      if (air || water) {
        box(x, 0.045 * scale, z + 0.88 * scale, padW * 0.58, 0.018 * scale, 0.36 * scale, dust);
        return;
      }
      const tireX = wide ? 0.86 : 0.64;
      const rearZ = wide ? 1.0 : 0.78;
      const frontZ = wide ? -0.95 : -0.82;
      box(x - tireX * scale, 0.044 * scale, z + rearZ * scale, 0.42 * scale, 0.03 * scale, 0.34 * scale, shadow);
      box(x + tireX * scale, 0.044 * scale, z + rearZ * scale, 0.42 * scale, 0.03 * scale, 0.34 * scale, shadow);
      box(x - tireX * scale, 0.044 * scale, z + frontZ * scale, 0.36 * scale, 0.03 * scale, 0.3 * scale, shadow);
      box(x + tireX * scale, 0.044 * scale, z + frontZ * scale, 0.36 * scale, 0.03 * scale, 0.3 * scale, shadow);
      box(x - tireX * 0.72 * scale, 0.038 * scale, z + (rearZ + 0.54) * scale, 0.13 * scale, 0.025 * scale, 0.9 *
scale, shade(shadow, 1.25));
      box(x + tireX * 0.72 * scale, 0.038 * scale, z + (rearZ + 0.54) * scale, 0.13 * scale, 0.025 * scale, 0.9 *
scale, shade(shadow, 1.25));
    }
```

```
    function wrapZ(index, spacing, offset, speed = 0.07) {
      const total = spacing * 22;
      let z = index * spacing - ((offset * speed * 2.35) % total);
      while (z < 0) z += total;
      return z + 8;
    }

    function roadCurveValue(data) {
      const state = data && data.raceState ? data.raceState : {};
      return Math.max(-1.34, Math.min(1.34, Number(state.roadTurn || state.roadCurve || 0)));
    }

    function roadWorldXFor(turn, offset, x, z) {
      const depth = Math.max(0, Math.min(1.12, Number(z) / 150));
      const sweep = turn * depth * depth * 8.4;
      const surfaceWave = Math.sin((Number(z) || 0) * 0.035 + (Number(offset) || 0) * 0.0028) * Math.abs(turn) * depth
 * 0.72;
      return x + sweep + surfaceWave;
    }

    function roadWorldX(data, x, z) {
      return roadWorldXFor(roadCurveValue(data), data.raceState.roadOffset || 0, x, z);
    }

    function addRoad(data) {
      const place = data.selectedRace.place;
      const theme = data.selectedRace.theme || ["#09100f", "#46d9ff", "#ffd166"];
      const accent = hexToRgba(theme[1], 1);
      const accent2 = hexToRgba(theme[2], 1);
      const turn = roadCurveValue(data);
      const phoneFrame = data.width <= 940 || data.height <= 540;
      const roadColors = {
        snow: [0.34, 0.42, 0.41, 1],
        harbor: [0.12, 0.25, 0.3, 1],
        airfield: [0.24, 0.24, 0.22, 1],
        farm: [0.31, 0.25, 0.16, 1],
        desert: [0.42, 0.31, 0.18, 1],
        tokyo: [0.12, 0.11, 0.18, 1],
        rainforest: [0.12, 0.18, 0.14, 1],
        europe: [0.18, 0.24, 0.25, 1]
      };
      const groundColors = {
        coast: [0.05, 0.2, 0.2, 1],
        city: [0.06, 0.07, 0.08, 1],
        canyon: [0.28, 0.13, 0.08, 1],
        alpine: [0.05, 0.14, 0.12, 1],
        harbor: [0.03, 0.16, 0.2, 1],
        snow: [0.6, 0.68, 0.66, 1],
        airfield: [0.24, 0.18, 0.12, 1],
        freight: [0.2, 0.22, 0.13, 1],
        farm: [0.12, 0.28, 0.12, 1],
        tokyo: [0.04, 0.03, 0.08, 1],
        desert: [0.48, 0.32, 0.16, 1],
        rainforest: [0.03, 0.18, 0.1, 1],
        europe: [0.07, 0.18, 0.16, 1]
      };
      let roadColor = roadColors[place] || [0.14, 0.16, 0.15, 1];
      let groundColor = groundColors[place] || [0.04, 0.08, 0.06, 1];
      let roadMark = [0.78, 0.82, 0.78, 1];
      let roadMarkDim = [0.48, 0.52, 0.5, 1];
      if (phoneFrame) {
        roadColor = shade(roadColor, place === "snow" ? 0.56 : 0.38);
        groundColor = shade(groundColor, 0.34);
        roadMark = [0.34, 0.37, 0.36, 1];
        roadMarkDim = [0.18, 0.2, 0.2, 1];
      }
      quad([-70, -0.05, 0], [70, -0.05, 0], [70, -0.05, 170], [-70, -0.05, 170], groundColor);
      quad([roadWorldX(data, -5.8, 0), 0, 0], [roadWorldX(data, 5.8, 0), 0, 0], [roadWorldX(data, 7.4, 170), 0, 170],
[roadWorldX(data, -7.4, 170), 0, 170], roadColor);
      for (let segment = 0; segment < 12; segment += 1) {
        const z0 = segment * 14.2;
        const z1 = (segment + 1) * 14.2;
        const t0 = z0 / 170;
        const t1 = z1 / 170;
        const half0 = 5.8 + t0 * 1.6;
        const half1 = 5.8 + t1 * 1.6;
        const tone = phoneFrame ? (segment % 2 ? 0.82 : 0.92) : (segment % 2 ? 0.98 : 1.06);
        quad(
          [roadWorldX(data, -half0, z0), 0.006, z0],
          [roadWorldX(data, half0, z0), 0.006, z0],
          [roadWorldX(data, half1, z1), 0.006, z1],
          [roadWorldX(data, -half1, z1), 0.006, z1],
          shade(roadColor, tone)
        );
```

```
      }
      quad([roadWorldX(data, -8.7, 0), -0.02, 0], [roadWorldX(data, -5.9, 0), -0.02, 0], [roadWorldX(data, -7.7, 170),
 -0.02, 170], [roadWorldX(data, -10.2, 170), -0.02, 170], shade(groundColor, 0.72));
      quad([roadWorldX(data, 5.9, 0), -0.02, 0], [roadWorldX(data, 8.7, 0), -0.02, 0], [roadWorldX(data, 10.2, 170),
-0.02, 170], [roadWorldX(data, 7.7, 170), -0.02, 170], shade(groundColor, 0.72));
      for (let i = 0; i < 42; i += 1) {
        const z = wrapZ(i, 4.1, data.raceState.roadOffset, 0.16);
        const x = ((i * 1.37) % 10.6) - 5.3;
        const patch = phoneFrame ? (i % 3 ? shade(roadColor, 0.72) : shade(roadColor, 0.98)) : (i % 3 ?
shade(roadColor, 0.78) : shade(roadColor, 1.18));
        box(roadWorldX(data, x, z), 0.026, z, 0.38 + (i % 4) * 0.18, 0.014, 1.1 + (i % 5) * 0.28, patch);
      }
      for (let i = 0; i < 36; i += 1) {
        const z = wrapZ(i, 5.1, data.raceState.roadOffset, 0.16);
        const color = i % 2 ? roadMarkDim : shade(roadMarkDim, 0.72);
        box(roadWorldX(data, -6.25, z), 0.046, z, 0.16, 0.016, 1.05, color);
        box(roadWorldX(data, 6.25, z), 0.046, z, 0.16, 0.016, 1.05, color);
      }
      for (let i = 0; i < 34; i += 1) {
        const z = wrapZ(i, 5.6, data.raceState.roadOffset, 0.14);
        box(roadWorldX(data, -5.45, z), 0.04, z, 0.14, 0.018, 0.24, roadMark);
        box(roadWorldX(data, 5.45, z), 0.04, z, 0.14, 0.018, 0.24, roadMark);
      }
      for (let lane = -1; lane <= 1; lane += 1) {
        for (let i = 0; i < 24; i += 1) {
          const z = wrapZ(i, 7.2, data.raceState.roadOffset, 0.1);
          box(roadWorldX(data, lane * 2.08, z), 0.034, z, 0.1, 0.016, 1.55, [0.86, 0.9, 0.86, 1]);
        }
      }
      for (let i = 0; i < 28; i += 1) {
        const z = wrapZ(i, 6.5, data.raceState.roadOffset, 0.13);
        const side = i % 2 ? -1 : 1;
        barrierRun(roadWorldX(data, side * 8.4, z), z, i % 3 ? roadMark : roadMarkDim);
        box(roadWorldX(data, side * 6.55, z + 1.9), 0.08, z + 1.9, 0.45, 0.05, 1.0, i % 2 ? [0.85, 0.86, 0.82, 1] :
[0.72, 0.08, 0.06, 1]);
        box(roadWorldX(data, side * 6.55, z - 1.9), 0.08, z - 1.9, 0.45, 0.05, 1.0, i % 2 ? [0.72, 0.08, 0.06, 1] :
[0.85, 0.86, 0.82, 1]);
      }
      if (Math.abs(turn) > 0.22) {
        const side = turn > 0 ? 1 : -1;
        for (let i = 0; i < 12; i += 1) {
          const z = wrapZ(i, 9.2, data.raceState.roadOffset, 0.14);
          box(roadWorldX(data, side * 7.25, z), 0.75, z, 0.18, 1.0, 0.12, roadMarkDim);
          box(roadWorldX(data, side * 7.05, z - 0.18), 1.32, z - 0.18, 1.2, 0.34, 0.12, i % 2 ? accent : accent2);
          box(roadWorldX(data, side * 7.05, z - 0.34), 1.32, z - 0.34, 0.48, 0.08, 0.14, [0.9, 0.94, 0.88, 1]);
        }
      }
      if (place === "city" || place === "tokyo") {
        for (let i = 0; i < 10; i += 1) {
          const z = wrapZ(i, 18, data.raceState.roadOffset, 0.09);
          for (let stripe = -2; stripe <= 2; stripe += 1) {
            box(roadWorldX(data, stripe * 1.15, z), 0.034, z, 0.72, 0.018, 1.2, [0.85, 0.86, 0.82, 1]);
          }
        }
      }
      if (["city", "tokyo", "coast", "harbor", "rainforest"].includes(place)) {
        for (let i = 0; i < 22; i += 1) {
          const z = wrapZ(i, 7.8, data.raceState.roadOffset, 0.12);
          box(roadWorldX(data, Math.sin(i * 1.7) * 1.6, z), 0.032, z, 1.0 + (i % 4) * 0.42, 0.012, 0.16, i % 2 ?
roadMarkDim : roadMark);
        }
      }
      if (["city", "tokyo", "europe", "freight"].includes(place)) {
        for (let i = 0; i < 8; i += 1) {
          const z = wrapZ(i, 22, data.raceState.roadOffset, 0.08);
          box(-6.8, 3.1, z, 0.22, 6.2, 0.22, [0.34, 0.36, 0.34, 1]);
          box(6.8, 3.1, z, 0.22, 6.2, 0.22, [0.34, 0.36, 0.34, 1]);
          box(0, 6.12, z, 13.8, 0.22, 0.3, [0.34, 0.36, 0.34, 1]);
          box(-2.8, 5.72, z - 0.18, 2.4, 0.3, 0.16, i % 2 ? accent : [0.08, 0.1, 0.1, 1]);
          box(2.8, 5.72, z - 0.18, 2.4, 0.3, 0.16, i % 2 ? [0.08, 0.1, 0.1, 1] : accent);
        }
      }
    }

    function addScenery(data) {
      const place = data.selectedRace.place;
      const theme = data.selectedRace.theme || ["#09100f", "#46d9ff", "#ffd166"];
      const accent = hexToRgba(theme[1], 1);
      const accent2 = hexToRgba(theme[2], 1);
      const offset = data.raceState.roadOffset;
      for (let i = 0; i < 30; i += 1) {
        const z = wrapZ(i, 8.4, offset, 0.045);
        const side = i % 2 ? -1 : 1;
        const x = roadWorldX(data, side * (11 + (i % 5) * 3.4), z);
```

```
        if (place === "city" || place === "tokyo") {
          const height = 5 + (i % 7) * 2.2;
          taperedBox(x, height / 2, z, 2.6 + (i % 3), 2.1 + (i % 2), height, 2.8, place === "tokyo" ? [0.08, 0.06,
0.16, 1] : [0.08, 0.1, 0.11, 1]);
          box(x, height * 0.55, z - 1.42, 1.7, 0.14, 0.08, i % 2 ? accent : accent2);
          box(x, height * 0.32, z - 1.45, 0.24, 0.1, 0.1, i % 2 ? accent2 : accent);
          box(x, height * 0.72, z - 1.45, 0.24, 0.1, 0.1, i % 2 ? accent : accent2);
          if (place === "tokyo" && i % 5 === 0) {
            box(side * 6.6, 3.1, z + 1.2, 0.2, 4.5, 0.18, accent2);
            box(side * 6.95, 3.1, z + 1.2, 0.2, 4.5, 0.18, accent);
          }
          if (i % 4 === 0) streetLight(side * 7.55, z + 1.1, place === "tokyo" ? accent2 : accent);
          if (i % 6 === 0) signPanel(x - side * 2.8, z + 1.5, place === "tokyo" ? accent2 : accent, [0.04, 0.05, 0.06,
 1]);
          if (i % 8 === 0) spectatorCluster(side * 8.9, z + 2.4, accent, accent2);
        } else if (place === "farm") {
          box(x, 0.8, z, 2.2, 1.6, 2.4, i % 2 ? [0.5, 0.1, 0.08, 1] : [0.72, 0.66, 0.38, 1]);
          taperedBox(x, 1.95, z - 1.25, 2.45, 1.3, 0.6, 2.55, [0.44, 0.22, 0.12, 1]);
          box(side * 8.5, 0.35, z + 3, 0.12, 0.7, 4, [0.72, 0.56, 0.32, 1]);
          if (i % 4 === 0) roadsideTree(side * 10.8, z + 1.5, [0.12, 0.44, 0.14, 1]);
          if (i % 5 === 0) box(x + side * 3.4, 0.65, z - 1, 2.2, 1.3, 1.8, [0.86, 0.72, 0.34, 1]);
          if (i % 9 === 0) spectatorCluster(side * 9.4, z + 2, accent, accent2);
        } else if (place === "freight") {
          box(x, 0.9, z, 4.8, 1.8, 2.2, i % 2 ? [0.22, 0.25, 0.24, 1] : [0.68, 0.7, 0.72, 1]);
          box(x + side * 3.2, 1.4, z + 1.4, 1.1, 2.8, 1.1, [0.12, 0.14, 0.14, 1]);
          if (i % 4 === 0) signPanel(x - side * 4.1, z + 1.8, accent, [0.04, 0.07, 0.06, 1]);
          if (i % 7 === 0) box(side * 9.4, 0.9, z - 2.4, 3.6, 1.8, 1.9, [0.52, 0.08, 0.06, 1]);
          if (i % 8 === 0) streetLight(side * 7.7, z + 2.1, accent2);
        } else if (place === "desert" || place === "canyon") {
          lowMound(x, z, 4.2 + (i % 3) * 1.4, 1.2 + (i % 5) * 0.28, 3.3, place === "canyon" ? [0.55, 0.22, 0.12, 1] :
[0.74, 0.48, 0.22, 1]);
          if (i % 6 === 0) lowMound(side * 18.5, z + 2.7, 7.5, 2.6, 5.2, place === "canyon" ? [0.64, 0.25, 0.13, 1] :
[0.78, 0.5, 0.22, 1]);
          if (i % 10 === 0) signPanel(side * 10.2, z + 1.2, accent, [0.12, 0.08, 0.04, 1]);
        } else if (place === "rainforest") {
          taperedBox(x, 2.0, z, 0.7, 0.42, 4, 0.7, [0.12, 0.22, 0.12, 1]);
          taperedBox(x, 4.3, z, 3.6, 2.2, 1.8, 2.5, i % 2 ? [0.1, 0.42, 0.22, 1] : [0.18, 0.52, 0.18, 1]);
          if (i % 4 === 0) roadsideTree(side * 8.6, z + 1.8, [0.08, 0.38, 0.18, 1]);
          if (i % 11 === 0) spectatorCluster(side * 8.7, z + 1.4, accent, accent2);
        } else if (place === "snow" || place === "alpine" || place === "europe") {
          lowMound(x, z, 5.8 + (i % 4) * 1.3, 2.3 + (i % 5) * 0.5, 3.8, [0.74, 0.82, 0.84, 1]);
          if (i % 3 === 0) box(x + side * 2.5, 1.2, z + 2.2, 1.8, 2.4, 1.7, [0.08, 0.2, 0.16, 1]);
          if (i % 5 === 0) roadsideTree(side * 9.2, z + 1.5, [0.08, 0.24, 0.18, 1]);
          if (place === "europe" && i % 8 === 0) spectatorCluster(side * 8.8, z + 2.2, accent, accent2);
        } else if (place === "harbor" || place === "coast") {
          box(x, 0.22, z, 5.5, 0.25, 6.5, [0.04, 0.22, 0.28, 1]);
          box(x + side * 1.4, 0.9, z, 0.28, 1.8, 0.28, [0.58, 0.44, 0.26, 1]);
          if (i % 6 === 0) {
            box(side * 14.2, 2.1, z, 0.24, 4.2, 0.24, [0.72, 0.48, 0.2, 1]);
            box(side * 12.4, 4.0, z - 0.6, 3.8, 0.22, 0.24, [0.72, 0.48, 0.2, 1]);
            taperedBox(side * 15.4, 0.5, z + 1.8, 2.4, 1.5, 0.72, 3.2, accent2);
          }
          if (i % 4 === 0) roadsideTree(side * 10.4, z + 0.9, [0.18, 0.48, 0.2, 1], "palm");
          if (i % 5 === 0) taperedBox(x - side * 2.8, 0.65, z + 1.6, 2.2, 1.1, 0.55, 3.1, accent2);
          if (i % 9 === 0) spectatorCluster(side * 9.1, z + 2.8, accent, accent2);
        } else if (place === "airfield") {
          box(x, 1.1, z, 4.4, 2.2, 3.4, [0.18, 0.18, 0.16, 1]);
          taperedBox(x, 2.35, z - 1.8, 4.8, 2.4, 0.6, 3.6, [0.24, 0.24, 0.22, 1]);
          if (i % 6 === 0) box(side * 8.8, 0.35, z + 2, 1.8, 0.7, 2.6, accent);
          if (i % 8 === 0) streetLight(side * 7.6, z + 1.7, accent2);
        }
      }
    }

    function addVehicle(type, x, z, colorHex, scale = 1, police = false, damage = 0) {
      const paint = police ? [0.9, 0.92, 0.9, 1] : hexToRgba(colorHex, 1);
      const dark = [0.01, 0.015, 0.014, 1];
      const red = [1, 0.08, 0.12, 1];
      const blue = [0.1, 0.55, 1, 1];
      const glass = [0.52, 0.72, 0.78, 1];
      const chrome = [0.76, 0.82, 0.78, 1];
      groundContactPad(type, x, z, scale, paint);
      if (type === "semi") {
        taperedBox(x, 0.8 * scale, z + 0.95 * scale, 1.75 * scale, 1.25 * scale, 1.2 * scale, 1.6 * scale, paint);
        taperedBox(x, 0.95 * scale, z - 1.25 * scale, 2.05 * scale, 1.78 * scale, 1.6 * scale, 3.2 * scale,
shade(paint, 0.78));
        box(x, 1.14 * scale, z + 1.45 * scale, 0.95 * scale, 0.16 * scale, 0.08 * scale, [0.72, 0.86, 0.9, 1]);
      } else if (type === "tractor") {
        taperedBox(x, 0.7 * scale, z, 1.45 * scale, 0.95 * scale, 0.8 * scale, 1.55 * scale, paint);
        taperedBox(x, 1.25 * scale, z - 0.2 * scale, 1.0 * scale, 0.72 * scale, 0.9 * scale, 0.8 * scale, shade(paint,
 0.9));
        box(x - 0.75 * scale, 0.45 * scale, z + 0.2 * scale, 0.35 * scale, 0.65 * scale, 0.75 * scale, dark);
        box(x + 0.75 * scale, 0.45 * scale, z + 0.2 * scale, 0.35 * scale, 0.65 * scale, 0.75 * scale, dark);
      } else if (type === "f1" || type === "prototype") {
```

```
        taperedBox(x, 0.36 * scale, z, 0.9 * scale, 0.36 * scale, 0.34 * scale, 2.5 * scale, paint);
        taperedBox(x, 0.54 * scale, z - 0.2 * scale, 0.58 * scale, 0.32 * scale, 0.32 * scale, 0.55 * scale, dark);
        box(x, 0.28 * scale, z + 1.0 * scale, 1.5 * scale, 0.12 * scale, 0.25 * scale, shade(paint, 0.7));
      } else if (type === "truck" || type === "monster" || type === "tank") {
        taperedBox(x, 0.75 * scale, z, 1.85 * scale, 1.35 * scale, type === "tank" ? 0.75 * scale : 1.0 * scale, 2.1 *
 scale, paint);
        taperedBox(x, 1.25 * scale, z - 0.25 * scale, 1.1 * scale, 0.78 * scale, 0.65 * scale, 0.9 * scale,
shade(paint, 0.82));
        if (type === "tank") box(x, 1.55 * scale, z - 1.15 * scale, 0.22 * scale, 0.18 * scale, 1.8 * scale,
shade(paint, 0.7));
      } else if (type === "boat" || type === "snowmobile") {
        taperedBox(x, 0.42 * scale, z, 1.45 * scale, 0.7 * scale, 0.36 * scale, 2.3 * scale, paint);
        taperedBox(x, 0.68 * scale, z - 0.25 * scale, 0.62 * scale, 0.35 * scale, 0.42 * scale, 0.75 * scale, dark);
      } else if (type === "airplane") {
        taperedBox(x, 0.7 * scale, z, 0.68 * scale, 0.38 * scale, 0.45 * scale, 2.9 * scale, paint);
        box(x, 0.66 * scale, z, 2.55 * scale, 0.13 * scale, 0.42 * scale, shade(paint, 0.88));
      } else if (type === "helicopter") {
        taperedBox(x, 0.95 * scale, z, 1.2 * scale, 0.78 * scale, 0.65 * scale, 1.45 * scale, paint);
        box(x, 1.45 * scale, z, 2.3 * scale, 0.08 * scale, 0.1 * scale, [0.9, 0.94, 0.94, 1]);
      } else {
        taperedBox(x, 0.55 * scale, z, 1.46 * scale, 0.9 * scale, 0.55 * scale, 2.1 * scale, paint);
        taperedBox(x, 0.95 * scale, z - 0.25 * scale, 0.88 * scale, 0.55 * scale, 0.45 * scale, 0.82 * scale, dark);
        box(x, 0.78 * scale, z + 0.8 * scale, 0.7 * scale, 0.08 * scale, 0.08 * scale, [0.86, 0.93, 1, 1]);
      }
      const groundVehicle = !["airplane", "helicopter"].includes(type);
      if (groundVehicle) {
        box(x - 0.42 * scale, 0.54 * scale, z + 1.04 * scale, 0.28 * scale, 0.08 * scale, 0.08 * scale, [1, 0.92,
0.62, 1]);
        box(x + 0.42 * scale, 0.54 * scale, z + 1.04 * scale, 0.28 * scale, 0.08 * scale, 0.08 * scale, [1, 0.92,
0.62, 1]);
        box(x, 0.46 * scale, z + 1.08 * scale, 0.58 * scale, 0.06 * scale, 0.08 * scale, shade(dark, 1.5));
        box(x - 0.82 * scale, 0.66 * scale, z - 0.16 * scale, 0.08 * scale, 0.36 * scale, 0.72 * scale, shade(paint,
0.72));
        box(x + 0.82 * scale, 0.66 * scale, z - 0.16 * scale, 0.08 * scale, 0.36 * scale, 0.72 * scale, shade(paint,
0.84));
        box(x - 0.96 * scale, 0.88 * scale, z + 0.34 * scale, 0.18 * scale, 0.1 * scale, 0.2 * scale, shade(paint,
0.64));
        box(x + 0.96 * scale, 0.88 * scale, z + 0.34 * scale, 0.18 * scale, 0.1 * scale, 0.2 * scale, shade(paint,
0.72));
        box(x, 1.02 * scale, z + 0.38 * scale, 0.72 * scale, 0.08 * scale, 0.36 * scale, glass);
        box(x, 0.94 * scale, z - 0.66 * scale, 0.86 * scale, 0.08 * scale, 0.28 * scale, shade(glass, 0.6));
        box(x, 0.52 * scale, z - 1.22 * scale, 0.36 * scale, 0.055 * scale, 0.07 * scale, chrome);
        box(x, 0.42 * scale, z + 1.12 * scale, 0.78 * scale, 0.055 * scale, 0.08 * scale, dark);
        box(x - 0.46 * scale, 0.82 * scale, z - 0.22 * scale, 0.05 * scale, 0.52 * scale, 1.42 * scale, chrome);
        box(x + 0.46 * scale, 0.82 * scale, z - 0.22 * scale, 0.05 * scale, 0.52 * scale, 1.42 * scale, chrome);
        if (type === "f1" || type === "prototype" || type === "car") {
          box(x, 0.8 * scale, z - 1.16 * scale, 1.28 * scale, 0.1 * scale, 0.16 * scale, shade(paint, 0.65));
        }
        if (type === "semi" || type === "truck") {
          box(x, 1.25 * scale, z - 0.86 * scale, 1.55 * scale, 0.06 * scale, 0.12 * scale, chrome);
          box(x, 0.72 * scale, z - 1.84 * scale, 1.45 * scale, 0.08 * scale, 0.1 * scale, shade(paint, 1.18));
        }
      } else {
        box(x - 0.32 * scale, 0.78 * scale, z + 1.12 * scale, 0.18 * scale, 0.08 * scale, 0.08 * scale, [1, 0.92,
0.62, 1]);
        box(x + 0.32 * scale, 0.78 * scale, z + 1.12 * scale, 0.18 * scale, 0.08 * scale, 0.08 * scale, [1, 0.92,
0.62, 1]);
      }
      if (damage > 18) {
        const dent = [0.02, 0.022, 0.02, 1];
        box(x - 0.28 * scale, 1.04 * scale, z - 0.34 * scale, 0.48 * scale, 0.035 * scale, 0.16 * scale, dent);
        box(x + 0.34 * scale, 0.86 * scale, z + 0.18 * scale, 0.42 * scale, 0.035 * scale, 0.14 * scale, dent);
        if (damage > 52) box(x - 0.1 * scale, 0.72 * scale, z + 0.9 * scale, 0.36 * scale, 0.06 * scale, 0.1 * scale,
[1, 0.18, 0.16, 1]);
        if (damage > 76) box(x + 0.18 * scale, 1.2 * scale, z - 0.86 * scale, 0.28 * scale, 0.08 * scale, 0.18 *
scale, [0.05, 0.05, 0.045, 1]);
      }
      box(x - 0.64 * scale, 0.17 * scale, z + 0.72 * scale, 0.24 * scale, 0.34 * scale, 0.5 * scale, dark);
      box(x + 0.64 * scale, 0.17 * scale, z + 0.72 * scale, 0.24 * scale, 0.34 * scale, 0.5 * scale, dark);
      box(x - 0.64 * scale, 0.17 * scale, z - 0.72 * scale, 0.24 * scale, 0.34 * scale, 0.5 * scale, dark);
      box(x + 0.64 * scale, 0.17 * scale, z - 0.72 * scale, 0.24 * scale, 0.34 * scale, 0.5 * scale, dark);
      box(x - 0.64 * scale, 0.2 * scale, z + 0.72 * scale, 0.1 * scale, 0.055 * scale, 0.2 * scale, chrome);
      box(x + 0.64 * scale, 0.2 * scale, z + 0.72 * scale, 0.1 * scale, 0.055 * scale, 0.2 * scale, chrome);
      box(x - 0.64 * scale, 0.2 * scale, z - 0.72 * scale, 0.1 * scale, 0.055 * scale, 0.2 * scale, chrome);
      box(x + 0.64 * scale, 0.2 * scale, z - 0.72 * scale, 0.1 * scale, 0.055 * scale, 0.2 * scale, chrome);
      box(x - 0.32 * scale, 0.62 * scale, z - 1.15 * scale, 0.36 * scale, 0.08 * scale, 0.08 * scale, police ? blue :
red);
      box(x + 0.32 * scale, 0.62 * scale, z - 1.15 * scale, 0.36 * scale, 0.08 * scale, 0.08 * scale, police ? red :
red);
    }

    function addMovingObjects(data) {
      const defs = data.vehicleDefs || [];
      const getVehicle = (id) => defs.find((vehicle) => vehicle.id === id) || defs[0] || { type: "car", color:
```

```
"#46d9ff" };
      const screenYFromRoadDistance = (distance) => data.height * 0.68 - (distance - data.raceState.distance) * 0.11;
      const distanceFromScreenY = (y) => {
        const clampedY = Math.max(data.height * 0.34, Math.min(data.height * 0.98, Number(y) || data.height * 0.34));
        return data.raceState.distance + (data.height * 0.68 - clampedY) / 0.11;
      };
      const zFromRoadDistance = (distance) => {
        const y = screenYFromRoadDistance(distance);
        const t = Math.max(0.32, Math.min(1.02, y / Math.max(1, data.height)));
        return 116 - t * 108;
      };
      const projectRoadObject = (object) => {
        const distance = Number.isFinite(Number(object.distance)) ? Number(object.distance) :
distanceFromScreenY(object.y);
        const y = screenYFromRoadDistance(distance);
        if (y < data.height * 0.32 || y > data.height * 1.08) return null;
        return { distance, z: zFromRoadDistance(distance) };
      };
      data.raceState.opponents.forEach((opponent) => {
        const projected = projectRoadObject(opponent);
        if (!projected) return;
        const z = projected.z;
        if (z < 4 || z > 132) return;
        const vehicle = getVehicle(opponent.vehicleId);
        addVehicle(vehicle.type, roadWorldX(data, laneToWorldX(opponent.lane), z), z, opponent.color || vehicle.color,
 opponent.wrecked ? 0.9 : 0.95, false, opponent.damage || 0);
      });
      data.raceState.rivals.forEach((rival) => {
        const projected = projectRoadObject(rival);
        if (!projected) return;
        const z = projected.z;
        if (z < 4 || z > 132) return;
        addVehicle(rival.type || "car", roadWorldX(data, laneToWorldX(rival.lane), z), z, rival.color, rival.type ===
"semi" ? 1.08 : 0.9, false, rival.damage || 0);
      });
      data.raceState.police.forEach((unit) => {
        const projected = projectRoadObject(unit);
        if (!projected) return;
        const z = projected.z;
        if (z < 4 || z > 132) return;
        addVehicle("car", roadWorldX(data, laneToWorldX(unit.lane), z), z, "#f4fbf8", 0.98, true, unit.damage || 0);
      });
      data.raceState.coinsOnRoad.forEach((coin) => {
        const projected = projectRoadObject(coin);
        if (!projected) return;
        const z = projected.z;
        if (z < 4 || z > 132) return;
        box(roadWorldX(data, laneToWorldX(coin.lane), z), 0.52, z, 0.52, 0.72, 0.12, [1, 0.77, 0.26, 1]);
      });
      if (data.cameraMode === "chase") {
        addVehicle(data.selectedVehicle.type, roadWorldX(data, laneToWorldX(data.raceState.lane), 5.2), 5.2,
data.selectedVehicle.color, data.selectedVehicle.type === "semi" ? 1.2 : 1.05, false, data.raceState.damage || 0);
      } else if (data.cameraMode === "hood") {
        const hoodX = roadWorldX(data, laneToWorldX(data.raceState.lane), 1.2);
        box(hoodX, 0.22, 1.2, 2.2, 0.28, 2.3, hexToRgba(data.selectedVehicle.color, 1));
        if ((data.raceState.damage || 0) > 25) box(hoodX - 0.5, 0.39, 1.68, 0.5, 0.04, 0.16, [0.02, 0.022, 0.02, 1]);
      }
    }

    function render(data) {
      vertices = [];
      const theme = data.selectedRace.theme || ["#09100f", "#46d9ff", "#ffd166"];
      const clear = hexToRgba(theme[0], 1);
      const phoneFrame = data.width <= 940 || data.height <= 540;
      gl.viewport(0, 0, gl.canvas.width, gl.canvas.height);
      gl.clearColor(clear[0] * 0.72, clear[1] * 0.72, clear[2] * 0.72, 1);
      gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
      gl.enable(gl.DEPTH_TEST);
      gl.disable(gl.CULL_FACE);

      const projection = cameraProjection(data);
      const mvp = projection.mvp;
      lastProjection = {
        ...projection,
        data
      };

      addRoad(data);
      if (!phoneFrame) addScenery(data);

      gl.useProgram(program);
      gl.uniformMatrix4fv(uMvp, false, new Float32Array(mvp));
      gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
      gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(vertices), gl.DYNAMIC_DRAW);
```

```
      gl.enableVertexAttribArray(aPosition);
      gl.vertexAttribPointer(aPosition, 3, gl.FLOAT, false, 28, 0);
      gl.enableVertexAttribArray(aColor);
      gl.vertexAttribPointer(aColor, 4, gl.FLOAT, false, 28, 12);
      gl.drawArrays(gl.TRIANGLES, 0, vertices.length / 7);
    }

    function resize(width, height) {
      if (canvas.width !== width) canvas.width = width;
      if (canvas.height !== height) canvas.height = height;
    }

    function projectRoadPoint(lane, distance) {
      if (!lastProjection || !lastProjection.data) return null;
      const data = lastProjection.data;
      const z = zFromRoadDistance(data, Number(distance));
      return projectPoint(lastProjection, roadWorldXFor(lastProjection.roadTurn || 0, lastProjection.roadOffset || 0,
laneToWorldX(lane), z), 0, z);
    }

    function projectWorldRoadPoint(lane, z) {
      if (!lastProjection) return null;
      return projectPoint(lastProjection, roadWorldXFor(lastProjection.roadTurn || 0, lastProjection.roadOffset || 0,
laneToWorldX(lane), z), 0, z);
    }

    return {
      ready: true,
      render,
      resize,
      projectRoadPoint,
      projectWorldRoadPoint
    };
  }

  window.VelocityWebGLPipeline = VelocityWebGLPipeline;
}());
```

manifest.webmanifest


```
{
  "name": "Velocity Vault",
  "short_name": "Velocity",
  "description": "A standalone racing game with local profiles, upgrades, missions, controller support, and offline
play.",
  "id": "/velocity-vault/",
  "start_url": "./index.html",
  "scope": "./",
  "display": "standalone",
  "orientation": "any",
  "background_color": "#09100f",
  "theme_color": "#09100f",
  "categories": ["games", "entertainment"],
  "icons": [
    {
      "src": "app-icon.svg",
      "sizes": "any",
      "type": "image/svg+xml",
      "purpose": "any maskable"
    }
  ],
  "shortcuts": [
    {
      "name": "Race Hub",
      "short_name": "Hub",
      "description": "Open Velocity Vault at the driver gate.",
      "url": "./index.html"
    }
  ]
}
```

sw.js

```
"use strict";

const cacheName = "velocity-vault-v66";
const appShell = [
  "./",
  "./index.html",
  "./styles.css",
  "./phone-asset-pipeline.js",
  "./webgl-pipeline.js",
  "./game.js",
  "./manifest.webmanifest",
  "./app-icon.svg",
  "./README.md"
];

self.addEventListener("install", (event) => {
  event.waitUntil(
    caches.open(cacheName)
      .then((cache) => cache.addAll(appShell))
      .then(() => self.skipWaiting())
  );
});

self.addEventListener("activate", (event) => {
  event.waitUntil(
    caches.keys()
      .then((keys) => Promise.all(keys.filter((key) => key !== cacheName).map((key) => caches.delete(key))))
      .then(() => self.clients.claim())
  );
});

self.addEventListener("fetch", (event) => {
  if (event.request.method !== "GET") return;
  event.respondWith(
    caches.match(event.request).then((cached) => {
      if (cached) return cached;
      return fetch(event.request).then((response) => {
        const copy = response.clone();
        caches.open(cacheName).then((cache) => cache.put(event.request, copy));
        return response;
      }).catch(() => caches.match("./index.html"));
    })
  );
});
```

app-icon.svg

```
<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 512 512" role="img" aria-label="Velocity Vault icon">
  <rect width="512" height="512" rx="96" fill="#09100f"/>
  <path d="M80 358 206 110h100L180 358Z" fill="#bbf24a"/>
  <path d="M220 358 346 110h86L306 358Z" fill="#46d9ff"/>
  <path d="M116 374h286c18 0 32 14 32 32v8H78v-8c0-18 14-32 38-32Z" fill="#f4fbf8"/>
  <path d="M148 276c0-26 21-47 47-47h122c26 0 47 21 47 47v68H148Z" fill="#172220"/>
  <path d="M184 250h144l-22-50H206Z" fill="#46d9ff"/>
  <circle cx="164" cy="382" r="34" fill="#050807"/>
  <circle cx="348" cy="382" r="34" fill="#050807"/>
  <circle cx="164" cy="382" r="15" fill="#bbf24a"/>
  <circle cx="348" cy="382" r="15" fill="#bbf24a"/>
</svg>
```

README.md

```
# Velocity Vault

A standalone browser racing game built with local-only profiles, age-band selection, upgrades, missions, and canvas
racing.

## Play

Open `index.html` in a browser.

Use `Play` to launch straight into a race, or `Enter Garage` to review vehicles, upgrades, missions, and AI tuning
first.

Controls:

- `Left/Right` or `A/D`: steer
- `Up` or `W`: accelerate
- `Down` or `S`: brake, then reverse when stopped
- `Space`: boost while accelerating
- `P`: pause
- `R`: reset the vehicle after a crash or bad position
- On touch screens, use the on-screen controls. Phone Mini mode now uses a floating analog joystick: touch the driving
 screen, drag left/right to change lanes, and the car keeps accelerating while you steer. Pull the stick down only
when you want brake/reverse. Full controls are still available from the `Size` button.
- Landscape scrolling stays available in menus and the garage; the full-screen no-scroll layout only turns on while a
race is live.
- Bluetooth/gamepad controllers: pair the controller with the device, press any controller button in the browser, then
 use left stick or D-pad to steer, RT / A to accelerate, LT / X to brake or reverse, B / RB to boost, Y to reset, and
Start / Select to pause

Driver views:

- `Full Car`: third-person chase camera with the full performance car visible.
- `Half Car`: low hood-style view with the front of the car filling the lower screen.
- `Windshield`: cockpit-style view looking out through the windshield with dashboard framing.
- `WebGL 3D`: hardware-accelerated 3D racing pipeline with a classic 2D fallback button for older browsers.

## How To Play

1. Choose `Play` to start immediately, or `Enter Garage` to pick a vehicle, route, upgrades, and missions.
2. Read the goal card at the start of each race. Goals can ask you to collect route markers, keep focus high, dodge
rivals, or reach a target score.
3. Hold gas to build speed. On phones, use Mini mode by touching the driving screen; the floating joystick appears
under your thumb and keeps gas active while you drag left or right.
4. Steer smoothly to pass opponents. Watch the lane radar, distance badges, and opponent names to see where rivals are
 sitting on the road.
5. Pull the phone joystick down, press `Down/S`, or use controller LT to brake. Once stopped, brake becomes reverse.
6. Use boost only while accelerating. Boost helps pass rivals, escape police heat, and recover speed after corners.
7. Crashes do not end the race. Heavy damage starts a reset countdown, then the car is stabilized so you can continue.
8. Save a race from the live race controls, then use `Resume Saved Race` in the Race Hub to continue that driver
profile later.

Real-world style visuals:

- Race routes use real-world inspired locations across the country and world: Pacific Coast Highway, Chicago lakefront
 skyline, Sedona red rock canyon, Rocky Mountain pass, Miami harbor, Aspen snowfields, Nevada airfield, Texas freight
interstate, Iowa farm roads, Tokyo expressway, Sahara desert, Amazon rainforest, and Swiss Alps.
- Scenery includes skyline buildings, bridge cables, water, canyon walls, mountain silhouettes, street lamps, wet
asphalt glare, roadside route signs, farmland, big-rig freight stops, neon towers, desert dunes, rainforest canopy,
alpine villages, and pass-specific colors.
- The v57 local GenAI scene director adds original route-specific racing scenes at runtime: trackside crowds, camera
crews, brake boards, barriers, gantries, cranes, hangars, barns, cliffs, dunes, snowbanks, neon towers, tunnel frames,
 and cleaner phone-safe route atmosphere.
- Drifting now has stronger style cues with tire smoke, dust or snow wash by surface, darker skid arcs, racing-line
marks, and a `DRIFT` callout during bigger slides.
- Vehicles are original generic classes with shaded 3D-style silhouettes: street supercars, F1 open-wheel racers,
grand prix prototypes, performance trucks, semi truck racers, racing tractors, monster trucks, armored tanks,
snowmobiles, race boats, helicopters, and sport airplanes. No real automaker, military, or aircraft trademarks or
licensed models are used.
- Opponents now get generated race identity overlays with invented race numbers, livery stripes, driver/helmet
silhouettes in glass areas, damage glow, and clearer name/position cues so rivals feel more like named racers instead
of anonymous traffic.
- Engine, tire, boost, shield, and magnet upgrades apply across the whole roster, so semis and tractors can be
upgraded into faster and more agile racing builds too.
- A local WebGL game-engine-style renderer (`webgl-pipeline.js`) draws the 3D road, route scenery, opponents, traffic,
 and vehicle models. It uses browser WebGL directly so the app stays standalone and secure without CDN scripts.
- A phone-focused local asset pipeline (`phone-asset-pipeline.js`) now generates cached high-resolution vehicle
sprites, road texture maps, paint grain, glass, rim detail, light clusters, damage marks, and bloom overlays directly
on the device. This gives the installed phone app richer assets without downloading licensed models or relying on
remote servers.
- Phone graphics now add an extra cinematic realism pass with denser asphalt texture, rubber tire lanes, wet/oil
reflections, stronger foreground depth, subtle film grain, route-tinted lighting, and higher-resolution vehicle
paint/glass highlights. The harsh angular road marks were softened so the road reads like asphalt instead of spikes.
```

On phones, the app uses the clean mobile canvas renderer plus the lower hood camera so the first race stays readable and avoids stacked road layers; the camera buttons still let players switch back to full-car or windshield views.
- Phone graphics now add a console-style chase pass inspired by modern open-world and street-racing presentation: lower camera framing, darker asphalt, softer horizon scenery, wet reflection ribbons, peripheral speed blur, headlight wash, and cleaner road markings so the phone view feels less like a flat arcade lane game.
- Phone clean-road mode now removes the remaining spike-like road artifacts on phones by disabling texture scratches, racing-line glow, tire/skid curves, wet streak ribbons, diagonal speed marks, extra contrast dashes, and diagonal snow/rain strokes while keeping flat asphalt, shoulders, and grounded lane dashes.
- The track now has stronger sweeping bends and route-country presentation for the phone/browser app. The same turn model drives the WebGL camera, road mesh, lane projection, traffic projection, road-edge chevrons, and phone overlay so the course feels like it is bending through cities, highways, coast roads, mountains, and world routes instead of staying on a flat straight strip.
- Road paint is now ground-locked with short dash plates and low pavement bands instead of long rising guide lines, so lane markings stay visually attached to the road while vehicles remain planted on the pavement.
- WebGL phone mode no longer draws the older transparent canvas road sheet over the 3D road. The actual WebGL pavement stays visible, while the overlay is limited to sky, scenery, route signs, and low flat pavement bands so nothing reads like a see-through ramp.
- Tokyo/city atmosphere markers now render as low horizontal neon glints instead of tall diagonal guide strokes, removing the last see-through marker lines that appeared to angle upward over the road.
- Phone WebGL mode now covers the pale see-through road sheet with an opaque, low asphalt surface while keeping WebGL projection for the planted vehicles. Mini-toggle mode also latches drive-stick direction until neutral is tapped, and the opponent grid is spread across longer gaps and separate lanes for clearer passing.
- Phone WebGL mode now removes the remaining mid-screen transparent road wash and moves the opaque asphalt lock higher under the horizon, so the race view no longer shows a see-through angled ramp. The compact 4-way stick now owns all mini-stick input by itself, and the rival grid starts much farther apart across separate lanes so you can chase, pass, and be passed instead of running into a bunched pack.
- Phone WebGL mode now removes the angled pavement trapezoid entirely on small screens and uses only horizontal low-ground shading, so the extra invisible road plane is no longer drawn over the real WebGL road. Phone scenery also avoids sharp floating triangle shapes, and the compact stick now has direct button fallback so pressing the up arrow always accelerates even on browsers that do not deliver the parent stick drag event.
- Phone racing now uses one clean mobile road stack: the old desktop road/scenery layer is skipped on phones, asphalt is opaque instead of see-through, texture is clipped inside the road, yellow chevron markers are replaced with flat road pips, and nearby opponents show distance/lane badges plus a compact lane radar so passing and collision avoidance are easier to read.
- Phone racing now uses a stronger v51 readability override: the mobile road gets high-contrast shoulders and brighter lane paint, opponents are forced into larger race gaps, and phone-only projection spreads far vehicles vertically so they no longer visually collapse into one cluster at the horizon.
- Phone racing now has a v52 passability fix: the AI field actively leaves an open passing lane near the player, random traffic is capped on small screens, overlapped cars are not drawn under the hood, hood view shows only the lower vehicle nose instead of a full top-down car, and phone scenery has a dedicated skyline/roadside backdrop.
- Phone racing now has a v58 fair-damage pass: hidden or unreadable traffic can no longer damage the player, close hazards are forced visible, road-edge drift warns and slows instead of silently destroying the vehicle, and the HUD names the cause whenever damage happens.
- Phone racing now has a v59 lane-feel pass: hood/cockpit/phone cameras follow the player lane, so lane stripes, markers, traffic, and rivals slide across the screen during steering instead of feeling like the car is only changing an invisible number.
- Online/browser racing now has a v62 lane-feel pass: desktop canvas and WebGL chase views use the same smoothed lane camera anchor as phone play, while the WebGL overlay keeps the player car and passable traffic moving visibly across the lane during right/left steering.
- The WebGL route pass now uses smoother world landmarks, neon signs, mounds, hangars, barns, freight stops, streetlights, trees, barriers, spectators, weather overlays, speed streaks, and detailed vehicle bodies instead of the first sharp spike scenery and plain box vehicles.
- The WebGL route pass also adds overhead highway gantries, neon vertical signs, harbor cranes, extra wet-road reflection strips, bridge-style structures, route curve camera targets, and route-specific foreground details so the world feels less empty and more like a real drive.
- Roads now include darker asphalt patches, reflective studs, shoulders, rumble strips, center paint, city crosswalk-style stripes, signs, spectator groups, guardrail details, and route-specific landmarks so each place reads more like a real location.
- Vehicles now have extra WebGL body details such as glass, headlights, grilles, side panels, mirrors, plates, rim highlights, rear wings on fast cars, larger stance cues for trucks, visible damage panels, and route-appropriate traffic types.
- WebGL mode now layers the high-detail canvas vehicle renderer over the 3D road, so player cars, opponents, police, and traffic keep readable wheels, glass, mirrors, trim, labels, brake lights, and damage marks instead of looking like simple boxes.
- Phone WebGL mode now prefers the generated asset sprites over the older simple procedural silhouettes, giving the app a more model-like vehicle read while staying light enough for mobile browsers.
- Race opponents now line up against the player, change pace during the route, show names on the track, show visible damage bars, take collision damage, limp when wrecked, and affect finish position, rewards, and wins.
- Opponent pack racing now uses a longer staggered grid, side-lane passing behavior, draft/catch-up pressure, and overtake tracking. You can pass opponents, they can draft back and pass you, and the phone HUD/finish screen show overtake progress.
- The starting grid now splits the pack with three opponents ahead and two behind, wider side-lane behavior, less early traffic clutter, and close-pass collision tuning so the player can actually overtake named rivals and climb position instead of staring at one grouped wall of vehicles.
- The v63 progression pass adds vehicle ownership, reputation unlocks, higher vehicle/upgrade pricing, per-vehicle saved builds, paint cycling, player license plates based on callsigns, and route rules so boats stay on water, aircraft race sky routes, tanks use military-style routes with a boost-button cannon, tractors stay on farm rallies, semis run freight, and monster trucks get off-road/arena-style routes.
- The v63 race pass tightens player hitboxes so hidden traffic should not damage the car from far ahead, spreads online/browser opponents much farther apart, applies the same passing-lane behavior outside phone mode, makes route markers harder to collect, lowers police frequency outside true pursuit conditions, adds civilian penalties, adds selected-route oncoming traffic, and adds stronger day/night lighting with street lamps and vehicle light glow.
- The v64 scenario pass adds more vehicle models and classes, including rally, drift, buggy, heavy tank, hydroplane, patrol boat, attack helicopter, and jet plane builds. It also adds local ghost/crew multiplayer-style race scenarios,

longer hot-pursuit escalation, route hideouts such as garages, caves, hangars, barns, mountain tunnels, and marina cover, plus shortcut branches that open different road, water, sky, farm, freight, military, and world-route decisions during races.
- The v65 close-pass pass widens side-contact detection and makes nearby rivals steer apart faster, so passing reads as side-by-side racing or a scrape instead of the player car visually floating over another vehicle.
- The v66 visibility pass tightens desktop/WebGL hitboxes to a short road-distance window around the player car, adds route-stage scenery that changes during each race, and draws an actual finish gate/checkered road line so the route has a visible end instead of feeling endless.
- Opponents can now make wheel-to-wheel side contact when the player catches them, causing both vehicles to slow, push sideways, smoke, spin, lose focus, and keep damaged state instead of phasing through.
- Traffic uses rear-view performance silhouettes instead of overhead arcade sprites, with realistic road dashes and route markers replacing blocky lane pickups. Traffic and police no longer disappear on contact; damaged vehicles remain on the road until they move out of view.
- Driving now uses more realistic throttle, brake, reverse, steering inertia, lateral velocity, grip, road-edge slowdown, tire slip, corrected steering direction across touch, keyboard, and gamepad input, vehicle body yaw, damage-based acceleration loss, and damage-based handling loss instead of automatic forward motion or lane-snapping movement.
- Collision damage now appears in the HUD, finish stats, damage bar, car scuffs, smoke, cracked windshield overlays, impact flashes, and reset countdowns. At critical damage the vehicle enters a timed reset, and the manual `Reset Car` button or `R` key can recover from a bad crash position.
- Police pursuit mode adds heat, sirens, flashing light bars, interceptor units, contact penalties, escape scoring, and cockpit pursuit alerts.
- Sound is generated locally with Web Audio: engine tone, boost feel, pickup chimes, collision hits, and police siren sweeps. No external audio files or network calls are used.
- Driving effects include tire smoke, launch haze, boost exhaust, brake/slip wash, speed streaks, road reflections, depth haze, headlight wash, dust/snow/weather layers, camera shake, off-road drag, and car body lean during hard steering.
- Phone landscape racing moves touch controls to a transparent driver-pad overlay so steering, center, gas, brake/reverse, boost, pause, and touch mode switching stay usable without covering the driving screen.
- Touch controls use a multi-touch tracker plus pointer fallback so gas, brake, boost, and steering can stay active together until finger-up/cancel even if both thumbs are down at once.
- Phone mode has a separate in-game control layer fixed to the visible app window in both portrait and landscape. The default Mini layout is now a floating one-thumb joystick that appears under your touch, keeps gas active while you steer, releases cleanly when you lift your thumb, and leaves more of the driving screen open. Tapping `Full` expands the larger separate controls.
- The Mini floating joystick sends analog steering into the driving physics, keeps throttle active while steering sideways, supports brake/reverse only after a deliberate downward pull, and visually follows the player's touch instead of staying locked to a fixed D-pad position.
- Phone collision detection now uses a smaller distance-and-lane overlap check that matches the phone road projection, so traffic should only count as a crash when it is actually in your lane and close to your vehicle. Phone impacts also do less damage and push vehicles apart instead of repeatedly resetting the player.
- Race starts now show a clear goal card on the driving screen with the current challenge, route, longer-race pacing, and crash-recovery status.
- Crashes and total focus loss no longer end the race. Critical damage now starts a reset countdown, stabilizes the vehicle, and lets the player continue the same run.
- Generated vehicle sprites now draw with stronger tire contact shadows, dark tire contact patches, short road streaks, and lower pavement anchoring so cars, trucks, semis, tractors, and opponents read as driving on the road instead of floating above it.
- Phone and browser race views now generate rear/chase-view road vehicle sprites with the wheel bottoms used as the tire-contact anchor, so cars, trucks, semis, tractors, tanks, and snowmobiles sit on the pavement instead of reading like overhead stickers.
- Vehicle tire anchors are now pushed to the bottom of the generated sprite, and every grounded vehicle draws a hard pavement clamp directly across the tire line so there is no transparent gap that can read as floating.
- The player chase camera now places the car lower on the road plane and draws stronger vertical tire contact streaks, so the vehicle reads as planted on the foreground pavement instead of hovering over it.
- Grounded vehicle sprites now use the true generated wheel line as their contact anchor, then sink that anchor into the road by vehicle weight. This makes tires overlap the asphalt instead of hovering just above it.
- Race-view vehicles now draw wider and shorter, with neutral road markings replacing the yellow accent streaks that cluttered the screen. The chase camera favors a planted rear-view car shape instead of a tall angled sprite.
- Browser and phone vehicles now get heavier road grounding with lowered sprite anchoring, black tire sidewall pins, hard road-contact pads, and WebGL ground shadows under every moving vehicle.
- Traffic, police, opponents, and route markers now use road-distance projection instead of raw screen-height movement, so vehicles enter from the road horizon and stay locked to the pavement instead of falling down from the sky.
- Vehicle drawing now uses the tire/contact line as the screen anchor, so generated sprites grow upward from the pavement instead of hovering around a center point.
- Road motion now has an explicit speed-linked pass with moving asphalt bands, lane dashes, shoulder streaks, texture drift, and faster WebGL track markers so the pavement visibly moves when the player accelerates.
- Road paint now uses flat perspective quads and lower WebGL ground markers, while vehicle sprites use a lower tire-contact anchor and stronger contact shadows so tires and road lines sit on the pavement instead of floating above it.
- WebGL mode now projects every canvas vehicle sprite from the WebGL road plane, then pins the generated vehicle art to a tire-contact origin with visible tire pads and contact shadows. This keeps opponents, police, traffic, and the player car visually attached to the same pavement perspective as the 3D road.
- WebGL chase, hood, and cockpit cameras now use a flatter forward-looking road angle, plus a stronger foreground asphalt pass with horizontal pavement bands. The near road reads as flat ground under the tires instead of a ramp climbing up the screen.
- Vehicle sprites in WebGL mode now use a visible-road projection instead of the steep camera projection, with stronger spacing by distance and far-to-near draw ordering. This keeps cars from clustering high in the scene or appearing to climb into the sky while the moving pavement stays underneath them.
- Opponent, police, route marker, and traffic visibility now use the same visible road projection as their final draw position, keeping the far pack on the actual road horizon instead of letting older screen-height math place them above the road.
- WebGL mode now keeps road paint in the 3D ground layer and draws the high-detail vehicle sprites on top, preventing 2D road streaks or duplicate low-poly vehicles from appearing to float in front of the race.
- The Vehicles garage now paints real generated preview images for every vehicle class, including cars, F1,

prototypes, trucks, semis, tractors, monster trucks, tanks, snowmobiles, boats, helicopters, and airplanes.
- `Save Race` stores the current run on the device, and `Resume Saved Race` appears in the Race Hub for the same driver profile.
- New and existing local profiles now get a starter garage budget and early reputation so vehicle switching, upgrades, freight racing, farm racing, and Tokyo-style routes are available quickly instead of feeling locked.
- Race routes now run more than seven times longer than the original short sprint format and cannot finish before the opening minute-plus race window, so events land closer to a real 1-3 minute phone race instead of a quick arcade burst.

Visual direction references:

- Need for Speed-style street-racing energy: heat, police pressure, risk/reward pursuit decisions, expressive speed effects.
- Forza Horizon-style scenic variety: coast roads, city streets, canyon routes, and mountain passes.
- Tokyo night-highway feel: neon signage, wet road reflections, tunnel/highway rhythm, and rival traffic tension.
- Sim/cockpit cues: windshield framing, steering wheel, dashboard speed readout, heat alert, and road-focused forward camera.

Graphics note: this is a practical standalone browser build with both WebGL 3D rendering and high-detail canvas fallback. It is not using licensed console-game assets, but the rendering pipeline is now structured so deeper 3D models, tracks, lighting, and physics can be added over time.

Install As An App

Velocity Vault is now a Progressive Web App. To play it everywhere without this computer being on, put this folder on an HTTPS web host, open the hosted link once on the phone or computer, then install it:

- Android / Chrome / Edge: tap `Install / Add to Phone` or use the browser menu and choose install.
- iPhone / iPad Safari: tap Share, then `Add to Home Screen`.
- Windows / Edge / Chrome: click `Install / Add to Phone` or use the browser install icon.

After installation, the service worker caches the game files for offline play. The app can then launch from the phone or computer home screen without your PC being awake.

Good hosting options include GitHub Pages, Netlify, Cloudflare Pages, Vercel, or any HTTPS web host. The temporary local download link from this PC is useful for transfer, but it cannot keep the phone app available after this computer turns off.

Put It On This Computer

You can use Velocity Vault on this computer in three ways:

- Open `index.html` directly from this folder.
- Use the `Velocity Vault` desktop shortcut if one has been created.
- After publishing to GitHub Pages, open the secure hosted link in Edge or Chrome and click `Install / Add to Phone`. That creates a real app-style launcher in Windows.

The installed browser app is the cleanest computer version because it opens in its own window and caches files for offline play.

Send It To Kids Or Testers

The best way to send the game to kids' phones is to publish the `publish` folder to GitHub Pages and send them the secure link:

`https://YOUR-GITHUB-USERNAME.github.io/velocity-vault/`

They can open the link on their phone and install it:

- iPhone / iPad: open in Safari, tap Share, tap `Add to Home Screen`.
- Android: open in Chrome or Edge, tap `Install / Add to Phone` or `Add to Home screen`.

Once installed, the game appears like an app icon and can play offline after the first load. Each phone keeps its own local driver profiles and PINs; profiles do not sync between devices.

You can also send `VelocityVault-Phone-Download.zip`, but most phones do not run zipped web apps as cleanly as a hosted secure link. The GitHub Pages link is the smoother option.

Local AI Director

Velocity Vault includes a local agentic tuning director in the Garage AI tab. It reviews only the current on-device profile stats and rotates daily bonus events, reward multipliers, rival pacing, coin density, and upgrade recommendations.

This keeps the game feeling fresh without sending player data to any server. Because the game is fully standalone, it does not fetch online trend updates; new content can be added by editing the race, mission, event, and upgrade lists in `game.js`.

Security Notes

- No network calls.
- No third-party scripts, fonts, or assets.
- Strict Content Security Policy in `index.html`.
- Offline app caching uses `sw.js` and same-origin files only.

- WebGL rendering is local-only and does not load shaders, models, scripts, or textures from the network.
- Phone graphics assets are generated locally at runtime and cached in memory; no third-party model, texture, or image files are fetched.
- Profiles and optional PIN hashes stay in `localStorage` on this device.
- Controller support uses the browser Gamepad API only.
- The PIN is only for casual local separation between players; it is not a replacement for operating-system account security.

.nojekyll

```
# Serve Velocity Vault as plain static files.
```